

Engineer On a Disk

Overview: This note set is part of a larger collection of materials available at <http://claymore.engineer.gvsu.edu>. You are welcome to use the material under the license provided at <http://claymore.engineer.gvsu.edu/eod/global/copyright.html>. As always any feedback you can provide will be welcomed.

Copyright © 1993-2001, Hugh Jack

email: jackh@gvsu.edu

phone: (616) 771-6755

fax: (616) 336-7215

1. TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
AN INTRODUCTION TO UNIX	4
OVERVIEW - - - - -	4
UNIX - - - - -	5
THE NETWORK - - - - -	7
GOOD MANNERS - - - - -	9
THE INTERNET	10
NETWORKS - - - - -	10
NETWORK TYPES - - - - -	13
NETWORK PROTOCOLS - - - - -	14
DATA FORMATS - - - - -	17
PULLING ALL THE PROTOCOLS AND FORMATS TOGETHER WITH	
BROWSERS	21
OTHER STUFF - - - - -	21
TEACHING WITH THE INTERNET	26
LECTURES - - - - -	30
ON-LINE NOTES - - - - -	31
ON-LINE MARKING - - - - -	32
The Time-Line For My First On-Line Course (Fall 1996) - - - - -	33
WWW and HTML	36
Why Bother? - - - - -	36
Where to Find Netscape - - - - -	36
How to Get Your Own Home Page - - - - -	36
How to Create a file - - - - -	37
Resources - - - - -	39
A BASIC INTRODUCTION TO 'C'	40
WHY USE 'C'? - - - - -	40
BACKGROUND - - - - -	40
PROGRAM PARTS - - - - -	40
HOW A 'C' COMPILER WORKS - - - - -	48
STRUCTURED 'C' CODE - - - - -	49
ARCHITECTURE OF 'C' PROGRAMS (TOP-DOWN) - - - - -	50
CREATING TOP DOWN PROGRAMS - - - - -	51
HOW THE BEAMCAD PROGRAM WAS DESIGNED - - - - -	52
PRACTICE PROBLEMS - - - - -	58
GUI DESIGN.....	60
PRACTICE PROBLEMS - - - - -	60
AN EXAMPLE - BEAMCAD	60
PROGRAMMING IN JAVA.....	78
OVERVIEW - - - - -	78
THE LANGUAGE - - - - -	78
OBJECT ORIENTED PROGRAMMING - - - - -	81
REFERENCES/BIBLIOGRAPHY - - - - -	84
DATABASES	85

MESSAGE PASSING ON NETWORKS	86
MATHEMATICAL ELEMENTS OF COMPUTER GRAPHICS	127
INTRODUCTION - - - - -	127
PIXELS - - - - -	127
LINE DRAWING - - - - -	129
POLYGON DRAWING - - - - -	132
SHADED POLYGONS - - - - -	134
COLORS - - - - -	135
DITHERING - - - - -	140
RAY TRACING - - - - -	144
RADIOSITY - - - - -	153
ADVANCED GRAPHICS TECHNIQUES - - - - -	153
REFERENCES - - - - -	154
PRACTICE PROBLEMS - - - - -	155
NEW TOPICS.....	156
VIRTUAL REALITY - - - - -	156
MULTIMEDIA - - - - -	156
VISIONS SYSTEMS.....	157
OVERVIEW - - - - -	157
APPLICATIONS - - - - -	158
LIGHTING AND SCENE - - - - -	158
CAMERAS - - - - -	159
FRAME GRABBER - - - - -	162
IMAGE PREPROCESSING - - - - -	163
FILTERING - - - - -	163
EDGE DETECTION - - - - -	164
SEGMENTATION - - - - -	165
RECOGNITION - - - - -	168
PRACTICE PROBLEMS - - - - -	170
SIMULATION.....	175
MODEL BUILDING - - - - -	176
ANALYSIS - - - - -	178
DESIGN OF EXPERIMENTS - - - - -	179
RUNNING THE SIMULATION - - - - -	182
DECISION MAKING STRATEGY - - - - -	182
PLANNING - - - - -	184
NEURAL NETWORK THEORY - - - - -	199
ARTIFICIAL INTELLIGENCE (AI).....	218
OVERVIEW - - - - -	218
EXPERT SYSTEMS - - - - -	219
FUZZY LOGIC - - - - -	219
NEURAL NETWORKS - - - - -	220

Software Topics

2. AN INTRODUCTION TO UNIX

2.1 OVERVIEW

- UNIX was developed in the 70s at AT&T labs by Ken Thompson, and Dennis Ritchie.
- 'C' is the language of choice for UNIX, in fact about 90% of UNIX is written in 'C'.
- UNIX has provided the basis for many simpler operating systems like MS-DOS. Therefore IBM PC users will notice many commands are similar
- Two major releases of UNIX currently popular are AT&T UNIX, and BSD UNIX from Berkeley, these are being replaced by Solaris.
- Popular Concepts
 - Kernel** - A set of hardware services subroutines which talk to all software programs running on the system. This helps provide device independence for programs.
 - Shell** - An interface that accepts user input, executes and tracks user and system programs, executes batch files. Many shells are often run at the same time, this is like having many PC's available at once.
- A very large supply of public domain software has been developed for UNIX over the last 25 years.
- Most popular languages, and packages are now available for UNIX machines.
- Most computer hardware vendors are making UNIX (or a scaled down version) fit into their computers. Some spin off versions are, SCO UNIX, XENIX, MACH, etc.
- Some of the many features of UNIX are,
 - Multitasking (many programs at once)
 - Multi-User (Many users at once)
 - Networking fully integrated
 - Shared Resources (disks, printers, etc)
- The features of many UNIX workstations,
 - Graphics done in hardware
 - Built in Sound synthesizers
 - Video Cameras
 - approximately 1 Gig.. local disks for swap space
 - 32 Meg RAM
 - Virtual memory (uses disk like RAM)
 - Fully Windowed interface

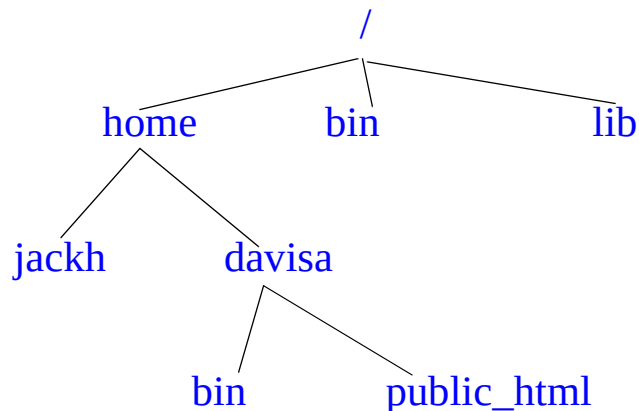
2.2 UNIX

2.2.1 Using UNIX Workstations in general:

- UNIX is case sensitive, which means don't confuse upper and lower cases. Almost everything uses lower case.
- You DO NOT TURN OFF OR RESET THESE MACHINES.
- To start using a UNIX system you must identify yourself by entering your login name and your password.. If using telnet you must know the machine name also.
- Use the '**passwd**' command to change your password. If you use a simple password, it will be easy to find, and somebody will break into your account, and wipe out all of your work. A good rule of thumb is that if it could not be found in a dictionary or index of a book it will generally be safer.
- Don't be afraid to try something new, you can only wipe out your files, all others are protected.

2.2.2 Directories, Files, Etc.

- All of the computers in the lab share disks. Your directory will be available at any computer.
- Note: the UNIX slash is '\', not the '/' used on PCs.



- You can use standard UNIX commands to look around, and manipulate your directory, and files,

cd	Change directory
pwd	Show present working directory
ls	List a directory
ls -la	List directories in full form
rm	remove a file (WARNING DON'T DO THIS IF YOU ARE NOT SURE)
mkdir	make a directory
rmdir	remove a directory
mv	moves a file/directory from the first name to the second
cp	copy a file/directory from the first name to the second
more	type out the contents of a file
man	bring up a manual page for a command
man -k	brings up information on a subject
vi	a text editor (some commands given below) 'dd' - delete a line (edit mode) 'x' - delete a character (edit mode) 'r' - replace a character (edit mode) 'R' - replace a string (command mode -> edit mode) 'a' - append to a line (command mode -> edit mode) 'i' - insert a string (command mode -> edit mode) ':w' - write to a file (command mode) ':q' - quit from a file (command mode) ESC - move from edit to command mode cursor key - move the cursor
ps -ef	Print a list of processes running on the computer
kill -9	Kill a process running on the computer (uses the PID # from ps -ef)
passwd	Change your password (protect yourself)
du	Check your personal disk usage
df .	Check total disk space available
quota -v	Check how much quota you have left. Being over quota for a few days will result in your account being cut off automatically.
date	get system date and time
who	show who is logged into your machine
textedit	A mouse based editor
pine	A simple interface for mail usage
mail	A somewhat bothersome mail tool (see pine).

- You can use wildcards when filenames. The characters used are,

*	Any string
?	Any Character
..	The directory above
.	this directory
~	your home directory

- Some examples of valid wildcard strings are,

Ad*	Advertise Advent.not Ad
Ad?	Ad. Ade

Ad?..?	Ade.d
??e.*	ape.exe eee.thisisanother
../hi.*	hi.there (in directory above)
~/*.there	hi.there (in your home directory)

- There are no real limits to file name lengths, but users should be careful with nonalphabetic characters.
- Files that begin with ‘.’ are often system files (like ‘**.login**’), and should not be deleted, moved, or created unless you are sure that is what you want to do.

2.2.3 Advanced Concepts

- Besides the normal keyboard operation, some keys have special meanings. Erasing a character can be done with delete/rubout or backspace /control-h (^h), the actual key varies from machine to machine. Control-s (^s) and control-q (^q) are for flow control, typing a ^s will stop output to the screen, which can then be continued by typing ^q. ^d represents end-of-file, and is used to exit from several programs like mail, or possibly to log off the computer.
- Most workstations run the UNIX operating system, which comes in two varieties: 4.3BSD (Sun) and System V (SGI), whose differences are minor for the average user (except for printing). When one logs into a UNIX system, you are actually running a program (shell) this is in some ways similar to DOS. In the standard shell you are given a prompt, where you type your command. If it is not a built-in command, it searches on the disk according to a user-specified search path, for an executable program of that name. Almost all commands are programs that are run in this manner. There are also executable shell scripts, similar to command files on DOS. UNIX is limited to running a program of a size equal to the sum of its memory, and swap space. As the system is multi-tasking, any program (or part thereof) that is not currently being run when extra memory is required, is swapped (moved) out to the disk, until it is ready to run again.

2.3 THE NETWORK

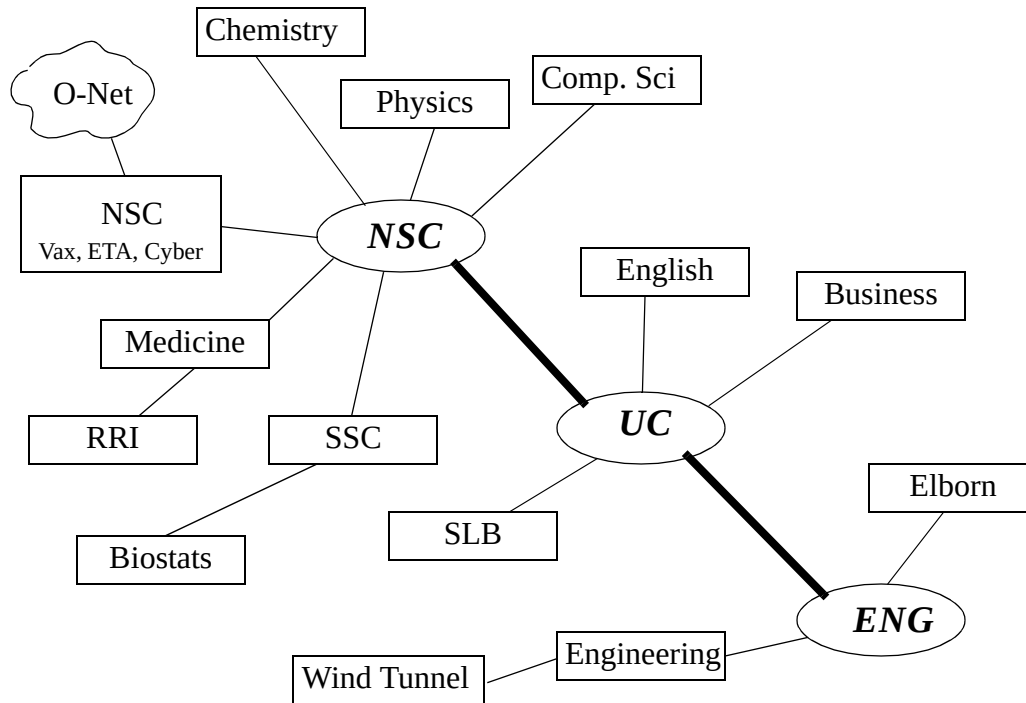
- Because the computers are on a network, they each have a unique identification.
- Each machine has a name, for example one of the machines is called ‘**claymore.engi-neer.gvsu.edu**’. The short form of this name is ‘**claymore**’. The machine also has an equivalent number ‘**148.61.36.215**’.
- you can log into other machines. Try ‘**rlogin ws1**’. This allows you to sit at one machine, and use many others, even around the world.
- you can also use a number of other commands which will allow network access,

rusers	look at other users on the network
---------------	------------------------------------

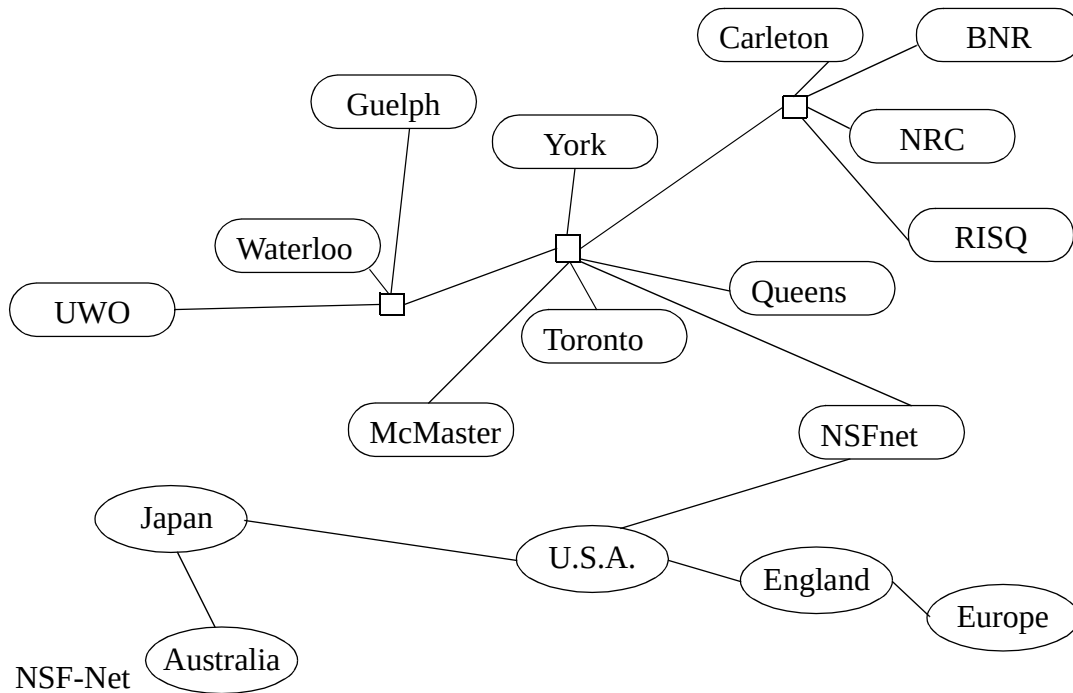
who look at who is logged into your machine
last a list of the last users logged in

- You can also access other computers with public access directories using the 'ftp' command. For example try '**ftp 152.2.22.81**'. This will connect you to a computer some place in the U.S. When it asks you for your '**login name**' type '**anonymous**'. When it asks for a '**password**', enter your name. You may now move around using **ls**, **pwd**, **cd**, etc. If you want to get a file from some directory, type '**binary**', then type '**get filename**'. '**quit**' ends everything.

• LOCAL UWO NETWORK(1989)



• **Wide-Area Ontario and on (1989)**



2.4 GOOD MANNERS

- Check your mail on a regular basis - your e-mail address is **your_login_name@river.it.gvsu.edu**
- Do not run big programs on computers somebody else is using
- Keep your directory nicely structured
- DELETE ALL FILES THAT ARE NO LONGER NEEDED. We only have 200 Megabytes, and it will be used up faster than you think. Check your own disk usage with '**du ~**'. Check the total area available with '**df .**'. Check your quota with '**quota -v**'
- Compress all your big files using '**compress ***'. Uncompress them with '**uncompress ***'

3. THE INTERNET

3.1 NETWORKS

- What does a network do - It lets us communicate.
- Start by thinking of a telegraph machine.
 - a talker hits a key and a pulse is sent along a wire to a receiving station
 - at the receiving station a listener hears the pulses
 - the sequence of the pulses (morse code) determine the message
 - the two take turns talking and listening to communicate
- Computer networks work exactly the same way (just a bit faster)
 - instead of keying stations we have network cards and cables
 - there are many talkers and listeners
 - instead of morse code we use ASCII to transmit letters, numbers, etc.
 - there are certain methods for determining who is listening and who is talking (eg, FTP)

EXERCISE: Have a look at the back of your computer. There should be a network cable, and a cable. Look for status lights, there should be one red and one green. The green light indicates that your computer is communicating, and there are no problems. If the red light flashes, your computer was trying to communicate, but too many others were talking, and it got confused (this is OK, networks are designed that way). Don't disconnect the network cable while the computer is on, but, if you did the lights would both stay off because no communication is occurring.

EXERCISE: Start a program such as Netscape, and point to a site. Look at the light while nothing is happening. Now, access a site, and quickly look to see what the lights are doing. You should see more activity.

- A small network is called a Local Area Network (LAN) and only connects a few computers for fast communication.
- We can connect smaller networks to larger networks that may just go across campus. These are Wide Area Networks (WAN).

EXERCISE: Follow the cables on the back of the computers back to the hub. Notice the status lights on the hub flash as communication occurs. Also notice that one wire on the hub goes elsewhere to another hub for the building or campus - It is taking another step towards the Internet.

- The Internet is just a lot of LANs and WANs connected together. If your computer is on one LAN that is connected to the Internet, you can reach computers on other LANs.
- The information that networks typically communicate includes,
 - email - text files, binary files (MIME encoded)
 - programs - binary, or uuencoded
 - web pages - (HTML) HyperText Markup Language
- To transfer this information we count on access procedures that allow agreement about when computers talk and listen, and what they say.
 - email - (SMTP) Simple Mail Transfer Protocol, POP3, IMAP
 - programs - (FTP) File Transfer Protocol
 - login sessions - Telnet
 - web access - (HTTP) HyperText Transfer Protocol

EXERCISE - Basic: Open a Dos window and type in 'ftp'. You can then say 'open sun-site.unc.edu. When it asks for your name type in 'anonymous', and for the password type in your email address. You should now be connected to a computer in North Carolina. you can look in the directories with 'ls' or 'dir', and you can move to new directories using 'cd DIRECTORY_NAME'. If you see a file that you want type 'get FILE_NAME'. Next, type 'close' to end the connection. Quit the Dos window, and use the Windows based FTP program to access the same computer.

EXERCISE - Intermediate: Open a Dos window and type in 'telnet river.it.gvsu.edu'. This connects you to the main student computer on campus. If you have an account, enter your user name and password. You can the use this computer as if you are using another dos window on your computer. Type 'exit' and quit the Dos window. Run the Windows Telnet program to access river again.

EXERCISE - Advanced: Open a Dos window and type 'telnet river.it.gvsu.edu 25'. this will connect you to the main student computer. But instead of the normal main door, you are talking to a program that delivers mail. Type the following to send an email message.

```
ehlo northpole.com
mail from: santa
rcpt to: jackh
data
Subject: Bogus mail
this is mail that is not really from santa
.
```

3.1.1 Computer Addresses

- Computers are often given names, because names are easy to remember.
- In truth the computers are given numbers.

Machine Name:	claymore.engineer.gvsu.edu
Alternate Name:	www.eod.gvsu.edu
IP Number:	148.61.104.215

- When we ask for a computer by name, your computer must find the number. It does this using a DNS (Domain Name Server). On campus we have two '148.61.1.10' and '148.61.1.15'.

EXERCISE: In netscape go to the location above using the name, and using the IP number (148.61.104.215).

- The number has four parts. The first two digits '148.61' indicate to all of the internet that the computer is at 'gvsu.edu', or on campus here (we actually pay a yearly fee of about \$50 to register this internationally). The third number indicates what LAN the computer is located on (Basically each hub has its own number). Finally the last digit is specific to a machine.

EXERCISE: Run the program 'winipcfg'. You will see numbers come up, including an IP number, and gateway. The IP number has been temporarily assigned to your computer. The gateway number is the IP address for the router. The router is a small computer that controls traffic between local computers (it is normally found in a locked cabinet/closet).

3.2 NETWORK TYPES

- The network connection type has an impact on the effectiveness and cost of the connection.

3.2.1 Permanent Wires

- These networks are fast, but require a permanent connection
- For the campus network the peak data transfer rate is about (4 GB/hour)
- These types of networks include,
 - Ethernet
 - ATM
 - Fast Ethernet

3.2.2 Phone Lines

- The merit dialup network is a good example. It is an extension of the internet that you can reach by phone.
- The phone based connection is slower (about 5 MB/hour peak)
- There are a few main types,
 - SLIP - most common
 - PPP - also common
 - ISDN - an faster, more expensive connection, geared to permanent connections
- You need a modem in your computer, and you must dial up to another computer that has a modem and is connected to the Internet. The slower of the two modems determines the speed of the connection. Typical modem speeds are,
 - 52.4 kbps - very fast
 - 28.8/33.3 kbps - moderate speed, inexpensive
 - 14.4 kbps - a bit slow for internet access
 - 2.4, 9.6 kbps - ouch
 - 300 bps - just shoot me

3.3 NETWORK PROTOCOLS

- What are protocols - sequences that computers must follow when sending and receiving information. These agreed methods make sure that information is sent and received correctly.
- Why do we need protocols - Without some agreement about what information is arriving over the network, it would just seem like garbage. This would be like somebody suddenly sending stock market numbers by morse code without telling us what it is.

3.3.1 Mail Transfer Protocols

- Popular email methods include,
 - SMTP (Simple Mail Transfer Protocol) - for sending mail
 - POP3 - for retrieving mail
 - IMAP - for retrieving mail

EXERCISE: In netscape go to the 'edit-preferences' selection. Choose the 'mail and groups' option. Notice how there is a choice for mail service type under 'Mail Server'. It should be set for 'POP3' and refer to 'mailhost.gvsu.edu'. This is where one of the campus mail servers lives. Set it up for your river account, and check to see if you have any mail.

- Note that the campus mail system 'ccmail' is not standard. It will communicate with other mail programs using standard services, but internally special software must be used. Soon ccmil will be available using the POP3 standard, so that you will be able to view your ccmil using Netscape, but some of the features of ccmil will not be available.
- Listservers allow you to send mail to a single address, and it will distribute it to many users (IT can set this up for you).

3.3.1.1 - Attachments

- We can attach files to our email. It is normally stored using the MIME format.
- These formats will allow the files to travel with the text of the message, and then be extracted by the receiver.

EXERCISE: In Netscape create a mail message, and attach some file to it. Mail it to your river account, and then read your mail. Click on the attachment. You should see it open, or be asked where to save it.

3.3.1.2 - Mail Lists

- We can set up mail servers that will accept a mail message addressed to the group, and then distribute it to the individual users.

EXERCISE: Send a message to the mailgroup 'ftlc_seminar'. Read your mail in Netscape to see what has come back.

3.3.2 FTP - File Transfer Protocol

- This is a method for retrieving or sending files to remote computers.

EXERCISE: In Netscape ask for the location 'ftp://sunsite.unc.edu' This will connect you via ftp the same way as with the windows and the dos software.

3.3.3 News

- These are informal groups (from local to international). The groups allow ongoing discussions between many individuals.

EXERCISE: Open the 'edit-preferences' option, and set the news server to 'news.it.gvsu.edu'. Next, open 'window-collabra-discussion', and open the newsgroups line. There may be newsgroups present. You will be able to subscribe to new newsgroups. Add a gvsu group, and add one international group in your area of interest. Open the groups and see the messages there. You can post to the gvsu FTLC group.

- You can get newsgroups set up for your courses to allow student discussions (just ask the IT people). This is a good alternative to chat groups.
- Notice that newsgroups have messages spanning a period of time. If the time period for message

storage is the length of the term, the group will act as a permanent record of discussions.

3.3.4 HTTP - Hypertext Transfer Protocol

- This is the protocol used for talking to a web server.

3.3.5 Chat

- chat groups allow on-line conversations. If we compare these to newsgroup - these are realtime by nature.

3.3.6 Novell

- Allows us to share files stored on a server.

EXERCISE: Look at the 'my computer' icon. The drives from 'F' and up are shared by network, and files are brought to the computer as you request them.

3.3.7 Security

- Security problems usually arise through protocols. For example it is common for a hacker to gain access through the mail system.
- The system administrator is responsible for security, and if you are using the campus server, security problems will normally be limited to a single user.
- Be careful with passwords, this is your own protection against hacking. General rules include,
 1. Don't leave yourself logged in when somebody else has access to your computer.
 2. Don't give your password to anybody (even the system administrator).
 3. Pick a password that is not,
 - in the dictionary
 - some variation of your name
 - all lower case letters
 - found in television
 - star trek, the bible

- pet/children/spouse/nick names
 - swear words
 - colloquial phrases
 - birthdays
 - etc.
4. Watch for unusual activity in you computer account.
 5. Don't be afraid to call information technology and ask questions.
 6. Don't run software that comes from suspect or unknown sources.
 7. Don't write your password down or give it to others.

3.4 DATA FORMATS

- The format of the data is important so that other programs may interpret it correctly.

3.4.1 HTML - Hyper Text Markup Language

- This is a format that is invisible to the user on the web. It allows documents to be formatted to fit the local screen.

EXERCISE: While looking at a home page in Netscape select 'View - Page Source'. You will see a window that includes the actual HTML file - This file was interpreted by Netscape to make the page you saw previously. Look through the file to see if you can find any text that was on the original page.

- Editors are available that allow users to update HTML documents the same way they use word processors.

EXERCISE: Find a home page in Netscape. Use the 'File - Edit Page' button to start the editor. Notice the buttons along the top for font sizes, colors, etc. Play with page and add your own name.

EXERCISE: Type in two new line of text. Name these lines 'sunsite' and 'other'. highlight 'sunsite' first, and select the small chain link at the top of the page. type in the link 'http://sunsite.unc.edu'. Accept this and then highlight the 'other' line. Enter a new link again using 'other.html'. (don't close the edit window, we will use it again shortly)

3.4.1.1 - Publishing Web Pages

- Once a web page has been modified it is necessary to put it back on the web server.
- When publishing a page with a browser 'FTP' will be used.
- The web page called 'index.html' is the first one to be returned. If you are publishing a main page your main page should be called 'index.html'.

EXERCISE: Using the web page that you modified before, publish the results to your home page. You can do this using the 'publish' option. You will have to provide a site name 'http://www2.gvsu.edu/~YOURNAME', a user name, and a password, and call the file 'index.html'. Use Netscape to view your updated home page. Note: You may have to hit reload, as Netscape will keep old copies, and does not automatically reload web pages if it has a recent copy is stored.

EXERCISE: Edit the file again and add a link to 'other.html'. You can do this by highlighting text, and then clicking on the 'link' icon.

- Keep in mind that the website is just another computer. You have directories and files there too. To create a web site that has multiple files we need to create other files or directory names.

EXERCISE: Create a new web page, and add something to it. Publish this page as before, except call it 'other.html'. Call up the browser, and load in the 'index.html' page that you created. Click on the links and see what happens.

- Note that some web servers do not observe upper/lower case and cut the 'html' extension to 'htm'. Microsoft based computers are notorious for this, and this will be the most common source of trouble.

EXERCISE (Basic): Use the Windows ftp program to access your remote account that your web page is set up in. Look at the files and file names. Transfer the files on the web site back to your local computer.

EXERCISE (Advanced): You can open these files in Netscape, edit them, save them back to the disk, and then publish them using the ftp program.

3.4.2 URLs

- In HTML documents we need to refer to resources. To do this we use a label to identify the type of resource, followed by a location.
- Universal Resource Locators (URLs)
 - http:WEB_SITE_NAME
 - ftp:FTP_SITE_NAME
 - mailto:USER@MAIL_SERVER
 - news:NEWSGROUP_NAME

EXERCISE: In netscape type in 'mailto:YOUR_NAME@river.it.gvsu.edu'. After you are done try 'news:gvsu'.

3.4.3 Hints

- Below is a list of hints for publishing web pages
 - Windows will not allow multiple applications to open the same file at the same time. If you seem to be having trouble opening a file, make sure it is not open in another application.
 - As you add other files to your homepage, put them in the 'temp' directory. This will make all of the procedures simpler.
 - Try to make your web pages small, and link them together. This will decrease download time and make browsers happier.
 - Avoid using excessive images. Anything over 10K will make it very slow downloading over modem. Anything over 100K makes modem downloading painfully slow.
 - When putting images on the web page use 'jpg' for photographic images, and 'gif' for line images. 'jpg' images can be compressed more than 'gif', but lines will become blurred.
 - To link to other files or web pages there will be a 'link' command. If you want to add a file that is in your 'temp' directory, just put the name of the file in the 'URL' field.
 - Watch upper/lower case. This is a major cause of web page problems.

3.4.4 Specialized Editors

- There are a variety of editors that will allow us to edit single web pages or entire sites.
- These programs include,
 - Microsoft Word

- Powerpoint
- WebCT
- Frontpage

EXERCISE: Start Microsoft Word and create a new document. Save this document as HTML on the hard drive. Use notepad to open the file and see how it relates to the original file.

3.4.5 PDF

- A format proposed by Adobe. This is not a 'standard', but is very widely accepted.
- When documents are presented in pdf format their original layout is preserved (HTML will actually change the look/layout of a document), but the files become hard/impossible to work with.
- A special plug-in is required to view these files.

EXERCISE: Point Netscape to '<http://claymore.engineer.gvsu.edu/~sahlis/214>' and look under handouts for a PDF file.

3.4.6 Encryption

- Allows some degree of privacy, but this is not guaranteed.
- Basically, if you have something you don't want seen, don't do it on the computer.

3.4.7 Compression

- We can make a file smaller by compressing it (unless it is already compressed, then it gets larger)
- File compression can make files harder to use in Web documents, but the smaller size makes them faster to download. A good rule of thumb is that when the file is MB in size, compression will have a large impact.
- Many file formats have compression built in, including,
 - images - JPG, GIF
 - video - MPEG, AVI

programs - installation programs are normally compressed

- Typical compression formats include,
 - zip - zip, medium range compression
 - gz - g-zip - good compression
 - Z - unix compression
 - Stuffit - A Mac compression format
- Some files, such as text, will become 1/10 of their original size.

3.5 PULLING ALL THE PROTOCOLS AND FORMATS TOGETHER WITH BROWSERS

- As you have already seen, the browser (ie, Netscape) helps pull these resources together.
- When we want to do things that are not part of the standard browser, we can use plug-ins.
- Plug-ins are small programs that can be used by Browsers to deal with different Protocols and Formats.

EXERCISE: Go to the Netscape home page, and call up the plug-in directory. Look for a plug-in you would be able to use. You may also want to try (www.autodesk.com) to find a DWF viewer plugin.

3.6 OTHER STUFF

3.6.1 Clients and Servers

- Some computers are set up to serve others as centers of activity, sort of like a campus library. Other computers are set up only as users, like bookshelves in a closed office. The server is open to all, while the private bookshelf has very limited access.
- A computer server will answer requests from other computers. These requests may be,
 - to get/put files with FTP
 - to send email
 - to provide web pages
- A client does not answer requests.

- Both clients and servers can generate requests.

EXERCISE: Using Netscape try to access the IP number of the machine beside you. You will get a message that says the connection was refused. This is because the machine is a client. You have already been using servers to get web pages.

- Any computer that is connected to the network Client or Server must be able to generate requests. You can see this as the Servers have more capabilities than the Clients.
- Microsoft and Apple computers have limited server capabilities, while unix and other computer types generally have more.
 - Windows 3.1 - No client or server support without special software
 - Windows 95 - No server support without special software
 - Windows NT - Limited server support with special versions
 - MacOS - Some server support with special software
 - Unix - Both client and server models built in
- In general you are best advised to use the main campus servers. But in some cases the extra effort to set up and maintain your own server may also be useful.
- To set up your own server machine you might,
 1. Purchase a computer and network card. A Pentium class machine will actually provide more than enough power for a small web site.
 2. Purchase of copy of Windows NT server version.
 3. Choose a name for your computer that is easy to remember. An example is 'artsite'.
 4. Call the Information technology people on campus, and request an IP address. Also ask for the gateway number, netmask, and nameserver numbers. They will add your machine to the campus DNS so that others may find it by name (the number will always work if chosen properly).
 5. Connect the computer to the network, then turn it on.
 6. Install Windows NT, and when asked provide the network information. Indicate that web serving will be permitted.
 7. Modify web pages as required.

3.6.2 Java

- This is a programming language that is supported on most Internet based computers.
- These programs will run on any computer - there is no need for a Mac, PC and Unix version.
- Most users don't need to program in Java, but the results can be used in your web pages

EXERCISE: Go to 'www.javasoft.com' and look at some sample java programs.

3.6.3 Javascript

- Simple programs can be written as part of an html file that will add abilities to the HTML page.

3.6.4 CGI

- CGI (Common Gateway Interface) is a very popular technique to allow the html page on the client to run programs on the server.
- Typical examples of these include,
 - counters
 - feedback forms
 - information requests

3.6.5 Searches

- There are major search engines on the Internet.

EXERCISE: Go to 'www.yahoo.com' and search for something you are interested in.

- You can also install local search engines for your site.

EXERCISE: Go to 'claymore.engineer.gvsu.edu' and use the search function to find a word/term you are interested in.

3.6.6 ActiveX

- This is a programming method proposed by Microsoft to reduce the success of Java - It has been part of the antitrust suit against Microsoft by the Justice Department.
- It will only work on IBM PC computers running the 'Internet Explorer' browser from Microsoft.

- One major advantage of ActiveX is that it allows users to take advantage of programs written for Windows machines.
- Note: Unless there is no choice avoid this technique. If similar capabilities are needed, use Java instead.

3.6.7 Graphics

- Two good formats are,
 - GIF - well suited to limited color images - no loss in compression. Use these for line images, technical drawings, etc
 - JPG - well suited to photographs - image can be highly compressed with minimal distortion. Use these for photographs.
- Digital cameras will permit image capture and storage - images in JPG format are best.
- Scanners will capture images, but this is a poor alternative as the image sizes are larger and image quality is poorer
 - Photographs tend to become grainy when scanned.
 - Line drawings become blurred.
- Screen captures are also possible, but do these with a lower color resolution on the screen (256 color mode).

3.6.8 Animation

- These are not video, but moving drawings/cartoons.
- Animations are limited, but are best done with animated gif files.
- Other options include,
 - java programs
 - special plug-ins such as shockwave

EXERCISE: Find an animation on a student page at 'claymore.engineer.gvsu.edu/students.html'.
--

3.6.9 Video

- Streaming built into Netscape for real-time video.

EXERCISE: Point Netscape to 'www.aml.gvsu.edu'. Select and watch the video stream.

- We can also get special plug-ins that will allow us to see video files,
 - MPEG very popular, good compression, and fault tolerant.
 - AVI popular on PC platforms, but limited drivers elsewhere.
 - Apple Quicktime.
- Real-time video conferencing is possible, but not yet practical.

EXERCISE: Start the Netscape Conference software.

3.6.10 Sounds

- Sound files are poorly supported, and most require special plugins,
 - real audio
 - wav audio
 - etc

EXERCISE: (If there is a sound card in the computer) Go to www.mtv.com and try to play a

3.6.11 Other Program Files

- We can connect any type of non-standard computer file to our pages, such as a Microsoft Excel file '.xls'.
- To do this we need to,
 1. Put the file in our web directory
 2. Link it to one or more HTML pages
 3. Have the system administrator add this as a MIME type to the system.
 4. When you click on the link to the file in Netscape it will ask you to choose an application. For an excel file you would want to choose 'program files\office97\excel'. This will automatically start when you choose the file next time. If you did not do step 3, or did not choose an application you would be asked to save the file.

- This is an excellent way to extend the capabilities of a web browser.

EXERCISE: Create a Word or Powerpoint file, and put it in your web directory using FTP. Add a link to it in your home page, and test to see if the link works.

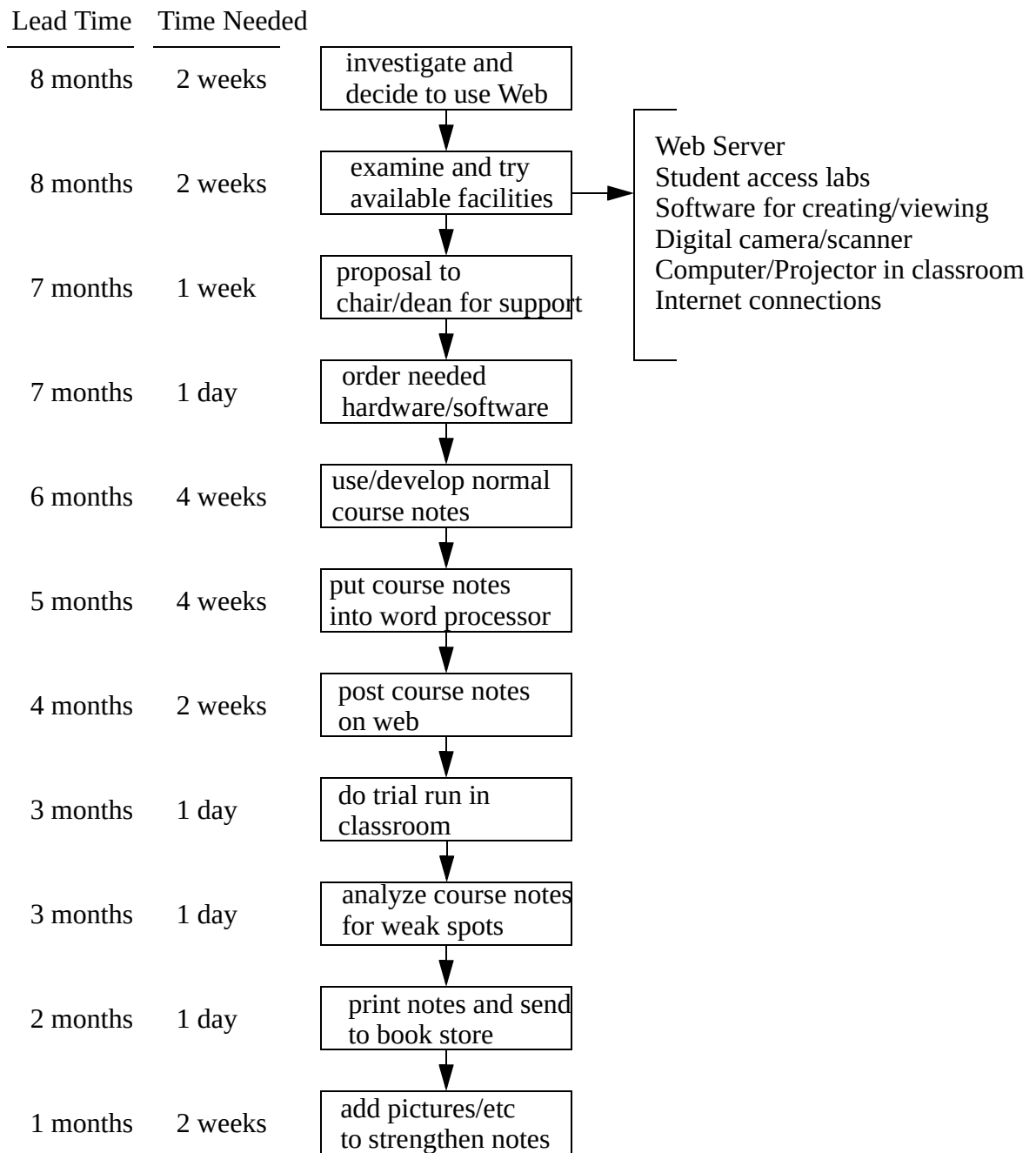
3.6.12 Fancy Stuff

- Compact Discs (CDs) are becoming a common way to transfer information. These will allow you to store up to 650 MB of data. When used they look exactly like another disk drive that cannot be overwritten. To make a CD the writers cost about \$400, and writable CDs can be purchased for about \$1-2 each. It will take 30-60 minutes for a CD writer to create a CD that is completely filled.

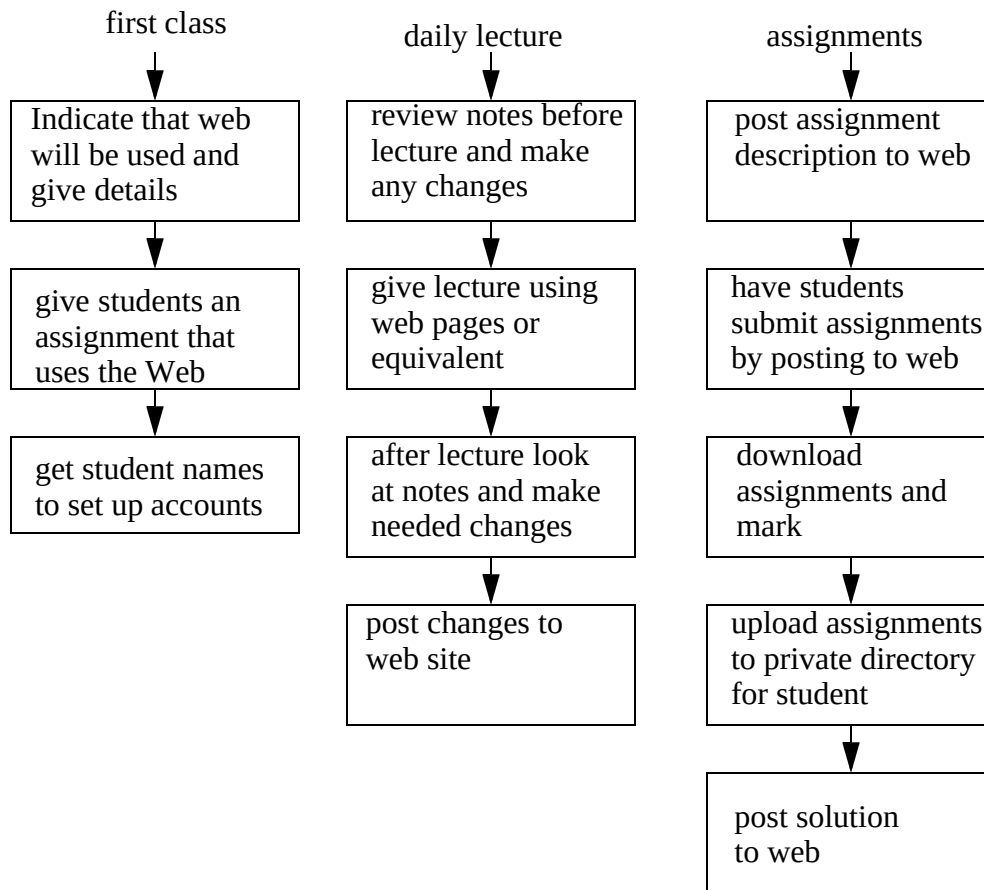
4. TEACHING WITH THE INTERNET

- The impact that web based material has on the students is,
 - easy access to course note correction and additions.
 - easy access to 'bulletin-board' type information.
 - students can add their own assignments to the course notes.
 - software and application specific files can be easily downloaded by the students.
 - the students can have a copy of the lecture material, and so the lecture focus shifts from dictation, to discussion of the course content.
 - homework can be submitted and returned without leaving home.
- In general the students strongly favor the Web based approach, although there are a number of problems that face some of them,
 - different levels of computer support. Some students have no Internet access from home.
 - access problems in campus computer labs.
 - the learning curve - this is quickly decreasing.

- There are some strong benefits for the faculty member who plans to use the Web for course materials,
 - the preparation before each lecture is decreased.
 - each lecture can be more consistent, if the web pages are used for presentation.
 - software and files are easy to distribute.
 - paperless marking can be done by having students upload files to their accounts. The instructor can then download these files and use software (e.g. Word, Photoshop, MathCAD and Working Model) to mark and leave comments. These files are then uploaded to a private directory where the student can retrieve them.
 - homework submitted with software such as Mathcad forces students to be exact in their calculations. It makes it harder to be obtuse, verbose, or vague.
 - notices can be posted on the web, saving trips to a designated board.
 - individual students can be reached with email.
 - changes in the course notes can be easily distributed.
 - students can post their work publicly and compare to others.
- There are some additional demands placed on the instructor,
 - before the course is offered the first time the instructor must spend a large amount of time developing solid course materials.
 - support is needed from the department/faculty to provide needed release time and equipment support.
 - mistakes on-line can become very public.
 - the course may become less flexible after the beginning of the term.
- You can start small by developing materials for part of a course or a single lecture.
- Before the course starts you will probably want to follow a time line like that shown below.



- The approach to the lectures should generally follow the form below,



- Strategies for success include,
 - Know the software and hardware.
 - Keep a bit of 'MTV' style in mind. Videos, sounds and other moving things help.
 - Do a 'dress rehearsal' well before the first class - small details such as fonts can ruin all the other efforts. Ask somebody to sit through a short trial run.
 - Try to make computer use unavoidable for some aspects - If not some students will 'get it from their friends' and you never get good contact.
 - Provide a good tutorial guide for all software students use, and offer assistance when possible.
 - Try to avoid commiserating with students when they have software problems. They need your confidence. If necessary find the answer and get back to them.
 - Get full administrative support.
 - Give the students something to do during the class - discussions or drill problems.
 - Turn on the lights and solve problems on the board frequently, it will wake up students going to sleep in the dark.

4.1 LECTURES

4.1.1 Equipment

- Essential

Web Server - Your campus will have at least one web server, and this can act as a useful place to leave files. Another (not recommended) alternative is to set up your own web server. I have set mine up using Linux (Unix for PCs) on a standard desktop computer, this can also be done using Windows NT (TM) produced by Microsoft. Between your files, and the students work, you should expect to fill tens of megabytes conservatively or up to gigabytes for large files and/or classes.

Faculty Computers - You will need at least one computer. A laptop is best, and can often be taken to the classroom. Networked desktop machines in the office and in the classroom can be set up with appropriate software.

Data Projector - If possible get a high quality data projector that will connect to your computer. If this is not possible, use an LCD panel on an overhead projector, this will appear a bit dark. Worst case use a computer to TV converter, this will be the least expensive, but the graphic quality is very poor. (in many classrooms and on rolling carts - ask IT)

Word Processor/Publishing Software - You will need a word processor of some description to prepare web pages. This is actually somewhat difficult for lecture material. The word processor typically needs to be able to include figures and possibly equations. The format used in this package must then be able to be converted to HTML. At the time of writing the variety of option is limited, but the market will soon be flooded. At present the best solution that I have been able to find is Adobe Framemaker. This runs on PCs, Macintosh, and Unix platforms.

Browser - There are a number of excellent browsers available today, but the two best are available from Netscape and Microsoft (use Netscape when possible). Both can be obtained at no charge under certain conditions. Either will do for the students, and this software can be used as a presentation tool in class. (www.netscape.com)

- Optional

Application Software - I have used packages such as Working Model, and Mathcad to support lectures. Anybody using the browsers for course notes will need copies of application software to included files. These files will allow the notes to become interactive, visual, experimental, etc. This could include other software such as Photoshop for modifying pictures.

Digital Camera/Scanner - A scanner can be very useful for capturing images on paper. But when possible this should be discouraged. Scanned documents are very large, and can be very slow when downloaded for viewing. Scanned photographs also tend to have a poor quality. A very good option is to buy a digital camera that captures images directly to digital format. These cameras come in a variety of prices, but a good midrange camera can be purchased for \$600 that will give good quality photographs. Within a short period of time these costs will drop quickly, and real time

video capture will be an option. Other poor options include camcorders with image capture hardware in a PC. This gives grainy pictures or low resolution. (available from IT and in many departments)

4.1.2 Techniques

- The computer based presentation can be quite similar to overheads, therefore a sequence that helps improve the attention level is,
 1. Some overview that sets the reason and objective for the section. (a good place for pictures)
 2. An overview of relevant theory.
 3. A simple example problem - with solution and notes about steps, notations, etc.
 4. A simple unsolved drill problem.
 5. Additional notes on technique and methods.
 6. An advanced problem with solution.
 7. An unsolved complex problem, done on the board.
- The lecture style needs to be dynamic, some ideas are,
 - Use the computer to present new material, then turn on the lights and discuss or do problems on the board.
 - If the computer screen is in front of a whiteboard, pull up the screen, and add notes using the screen underneath.
 - Ask students questions about what was just covered.
 - Get the students to calculate numbers, write, draw, etc..
 - Say something ridiculous to get a response.
 - Tell a joke.
 - Walk into the seats.
 - Borrow something for an example.

4.2 ON-LINE NOTES

- Typical tips for making on-line notes are,
 - have references that will automatically link to the reference source.
 - have different levels of material - the students can then skim the material they understand, and explore challenging topics.
 - use other media to explain topics.
 - allow links to fundamental materials for students needing review.
 - written text is bulky and hard to read, point form or highly condensed text is better.
 - the material should be cut up into small sections, a single printed page is a good size, but more or less can be used as required.
 - a good directory structure is needed to organize the information. This should be done

before doing any large scale development of the Web site. Cut the information up and organize by major and minor topic areas. (don't worry if these are not perfect, you can adjust this later when setting up tables of content.)

- the need to worry about fonts and layout (e.g. pagination) is not an issue as the readers viewer will take care of these. But, each section/subsection should be in a separate file to decrease delays.
 - we can add links to other documents that allow direct connection to references. This frees us from having to fully explain every point (e.g. go straight to an earlier proof).
 - tables of contents can be used, as in normal books, but they can be broken into smaller parts.
 - when there are enough small segments of information, we can put together different books. Each of these can put the material in different sequences, or as reference. I typically produce one of these for each course.
 - a date must be added to sections. The document is living, and corrections and additions can be made frequently. The reader needs a date or version number to know when these changes have occurred.
 - various miscellaneous items need to be added. A set of icons are needed for navigating through the document (next, previous, up one level). A contact email address. An identifying icon that marks the page. A copyright notice is optional, but should appear on each web page.
 - line drawing figures convert well to Web format.
 - photographs can be added easily but are best left as links. Viewing documents with a large number of photographs on the page slows down viewing, and makes the document very large.
 - linking to video and sound can cause problems, as no format is supported universally.
 - linking to application files requires that the reader have specific application software (e.g. Mathcad or Photoshop).
 - we can add hypertext links to other sites outside of our documents, but these are highly unreliable, and so they may be separated into a separate file.
- Issues include,
 - password protection is possible when you don't want notes to be shared publicly. Student then need to have the password to access the notes on-line.

4.3 ON-LINE MARKING

- On-line marking requires the transmission of student work to the instructor, and back to the student.
- Work can be submitted and returned using,
 - email - student email files. These are marked and returned by email.
 - web server - students post solutions to the web. These files are marked and returned by email or

put in a private student directory.

4.3.1 Web Pages

- Students are given web pages at the start of the term, and encouraged to use them quickly.
- This has turned out to be an excellent method for transmitting information, and it also allows students to build a portfolio.
- Students submit work by linking their homework to their page.
- The student controlled pages have lead to a great deal of 'ownership' and pride.
- On-line marking with the Web involves,
 1. go to the students home page.
 2. find the particular assignment to be marked.
 3. select the file of interest.
 4. the file is examined for technical detail, comments and a grade are added in red text.
 5. the file is saved, and copied to the students directory.
 6. the mark is recorded in a spreadsheet program.

4.3.2 email

- I use email, but not as an integral part of the course. If marking is done this way then it would become essential
- Many students use this as an alternative to finding me for simple questions.
- If you were to mark by email,
 1. When the assignment is completed/due the student emails you with the assignment file attached. This might be a Word file containing an essay.
 2. You receive the email, and open the attached file. You grade the file directly, and add comments. Comments can be put in a different font/color to make them stand out.
 - 3 Record the grade in a spreadsheet.
 4. email the marked file back to the student.

4.4 The Time-Line For My First On-Line Course (Fall 1996)

- These notes were made from a log of experiences with my first computer based course.
- Since then many improvements have been made (see PAD 168 as an example) by IT based on feedback - If you have any needs, talk to them.

Pre-August - Notes entered into computer.

Aug., 12 - Notebook computer arrives.

Aug.,13 - Notes handed to printer.

Aug.,20 - Investigate computer projection equipment in assigned classrooms, the overhead projectors are only for TV, but there are 10baseT network sockets in rooms. There are data projectors in a locked hallway cabinet. Keys for cabinet are ordered.

Aug., 23 - Start up software to test run, it. Results are OK.

Aug., 26 - Classes start. The keys are not ready yet, I need to arrive early to ask somebody to unlock the closet. I test data projector, it only displays 640x480. The setup time was under five minutes. The last ten minutes of first lecture done with computer and data projector. When the lights are turned out the room is too dark to see paper (no side scatter of light as with overhead). The lights only go on in half of room. The trackpoint on the computer is very awkward to control scroll bars, etc. Even though part of the screen is chopped, it is mainly scrollbars. Using the word processor software, only half a printed page appears, and page breaks can fill a screen. The second section had windows with screens, and the light levels were fine. - Lesson Learned: do an early test run before the first lecture.

Aug.,28 - Arrived ten minutes early to get cabinet unlocked and equipment set up. In the dark room I decided to use overhead projector pointed at wall for ambient light - seems to work well, but proper lighting would be better. Took a regular mouse (instead of trackpoint) to help control problems. Used working model to run a simulation, seemed to clarify conceptual difference between statics and dynamics.

Aug., 30 - Settled on 10 minutes early as the expected pre-class preparation time. Noticed dark made some students sleepy. Ran entire lecture in dark, including questions. In second section I tried to do problems on board to parallel what was on the computer, had to remove fluorescent bulbs at front of room to get reasonable lighting. Show-of-hands survey indicated that; they liked not having to copy down notes - more time to think, no problems seeing screen, or following along. Projector cabinet key arrives today. - Lesson Learned: Be aware that the dark room puts students to sleep.

Sept., 4 - Arrived a bit late, having to pick up the data projector cost lecture time. Students seemed a bit sluggish in dark, tried turning on lights to solve a problem on the board, this seemed to help a bit. - Lesson Learned: Change the mode of presentation to keep them active.

Sept., 6 - The bulk of books, computer, etc. is somewhat annoying - needed to take the power supply because of the extended time (3 hours) of continuous lectures.

Sept., 9 - The notebook power manager was shutting down the laptop every couple of minutes, went to lecture on blackboard. Took power supply to second lecture.

Sept., 13 - Power manager acted up in first class, PC was almost unusable - Lesson Learned: decided to take power supply each time, or disable power manager software. Picked up digital camera Apple Quicktake 100 - the camera was relatively easy to setup and use, but battery life was too short (<5 minutes) so the power supply was needed when downloading to computer. The camera took 8 or 32 pictures. The lower resolution of 320x240 was too poor for sharp detail, so the camera could really only hold 8 pictures.

Sept., 16 - Web Site almost ready.

Sept., 17 - Introduced students to Web pages, reception was good. Worked on getting web pages up, including hypertext links to gifs. This was only possible by tricking WebWorks software into passing it through without converting characters. Lesson Learned: plan for the document conversion process, the software available still limits abilities. Made some graphics to go with pages - gives the pages a polished look. Took pictures with digital camera - it was very sensitive to light levels, and distance/close detail was blurry.

Sept., 18 - Presented in class using Netscape off local disk files - increased font sizes, and DPI for gifs (up to 120 from 74) to better fill screen. The text was more viewable on screen. By contrast the normal screen font is too small. Lesson Learned: Use Netscape instead of Framemaker to present.

Sept., 20 - Presented with netscape again, this time using the network and the Web site - this ran well, but as the morning progressed the transfer rates slowed, especially as the hour approached.

Sept., 23 - Presented off local disk again, ran much faster. Added course info to web site, including sample problems and syllabus. Prepared first midterm, and had answers prepared, and ready to post after test.

Sept.,24 - Got camera and started taking more pictures. Got copies of Working Model quick start guide, plan to give to students tomorrow, and will suggest a problem to try next week.

Sept., 25 - Demonstrated Working model in class by building a ball between wedges, a robot, and a block hung by two cables. Also gave out tutorial manual, and indicated that next week some problems will be assigned to be done on working model. Student reaction was 'WOW'. Lesson Learned: Visual impact should be used whenever possible.

Sept., 26 - Fixing up things, but realize that pictures collected in future must be to fulfill need. there are too many pictures to take, and not enough disk space. Got some email questions about textbook problems. Did not have textbook. It would have been nice if all questions were on-line for easy reference, and so that notes could be added to the problems.

Sept., 27 - Exam day. Showed mpeg movie of Tacoma Narrows bridge - a good visual part. More videos would be great. Solutions posted on line, and students seemed to have them printed shortly after the exam. Lesson Learned: When possible post exam solutions on-line.

Sept., 29 - Some Working Model examples created for classes this week. Misalignment was a small problem - but more practice expected to reduce problems. A 3D problem is not possible using the current version.

Sept., 30 - Presented in class using Netscape only. This included working model links, and photographs. The models were Static, and a loss of interest was apparent. First working model assignment discussed as bonus. Student interest picks up. The lab version is not working well, (it later turns out that some computer labs don't support the package) I load a copy of the demo software to my website for local download. Find transcription error in the exam solution posted on-line, fix problem on-line and mention to students. Lesson Learned: Even though we

can change things on line, if it is printed mistakes are permanent. Lesson Learned: Just because your computer will do it doesn't mean the students computers will.

- Oct., 4 - The network locked up, had to present using framemaker - considered keeping copy of course on hard drive. Lesson Learned: The Internet is not reliable, have alternate solutions. SVGA to NTSC converter arrived - tried in class, very poor with high resolution graphics. Went back to data projector. Other problems, cables too short, and s-video connector only on ceiling mounted projector. Good rule of thumb, count on a loss of 50% of resolution, and avoid thin lines.
- Oct., 7 - tried using the SVGA converter in class again, very poor appearance, but as okay with larger fonts, but the unit seemed to shut down after a few minutes.
- Oct., 11 - Had classes, and then working model contest at the end. This seems to have set a fire in some students. Most were impressed. In general the interest level was higher in the smaller class. One very good technique with the data projector is to project the image onto a whiteboard and draw over the image using markers. This allows the students to add/refer to their notes.
- Oct., 18 - A very effective use of working model to show the students slip/tip case for a block, and then calculate to verify.
- Oct., 22 - Tried a test of SVGA to TV converter in class with large fonts. The students seemed to be happy with presentation styles, and agreed to an in class trial. Helvetica font seemed to be perceived as slightly better, by show of hands. Took a while and changed some of the notes to helvetica, and posted to net. Put some problem solutions on the net that were missing in the text.
- Oct., 23 - Went to present in class with SVGA to TV and the network was misbehaving, so could not connect to site. The network was down today, so the students could not get access to the web pages, and I could not update.
- Oct., 25 - Data projector did not work in first class, used the board instead. In second class found that the data projector was set for the wrong input. Lesson Learned: Even when the equipment works well, figure out why it is working well just in case. In the conversion process the multiplication sign was changed to a 'yen' sign. I also noted that in printing some of the equations were not printed properly, but the students were able to copy off the screen (or could use the web pages).
- Oct., 28 - Data projector stopped working, and sent to get repaired. presented on board in class.
- Oct., 29 - Presented in morning class using SVGA to TV converter. Did not seem to cause many problems.
- Oct., 30 - Exam day. Did not have time to put exam solutions on net.
- Nov., 1 - Returned exams - solutions not yet on net, but meeting students face to face to return papers (one at a time). Did the first survey in class. Used the SVGA to TV converter to present, the second class was seeing it for the first time, and complained of the poor quality.
- Nov., 4 - Presented in class as normal using board mainly. projector is still missing.
- Nov., 5 - Computer did not work.
- Nov., 6 - Data projector did not work again. Will use computer and projector next class.
- Nov., 8 - Data projector fixed, lecture proceeded as normal.

5. WWW and HTML

5.1 Why Bother?

Although decades old, the Internet has only recently come of age. This is easily illustrated by the prevalent use of World Wide Web (WWW) addresses in advertisements and business cards. It could be argued that the current public awareness is rooted in the over hyped 'Information Superhighway'. The tool that has made the Internet accessible to many users is Netscape. As with many Internet tools, this software is available free, and is supported for UNIX, Macintosh, and IBM PC platforms. Even more important is the fact that Netscape supersedes previously popular Internet interfaces (i.e., gopher, netnews, ftp, archie, etc.). Within minutes of your first exposure to Netscape, you can retrieve pictures, sounds, movies, and software from around the world.

5.2 Where to Find Netscape

Netscape software can be obtained free, but you will need an Internet connection directly, or a PPP connection through the phone lines. On campus the computers in all labs are already connected. In widows open an internet or similar program group. The netscape icon should be visible. After picking this icon the program should start and some home page will be displayed. If you are at home and you have an Internet connection you should have netscape, or an equivalent.

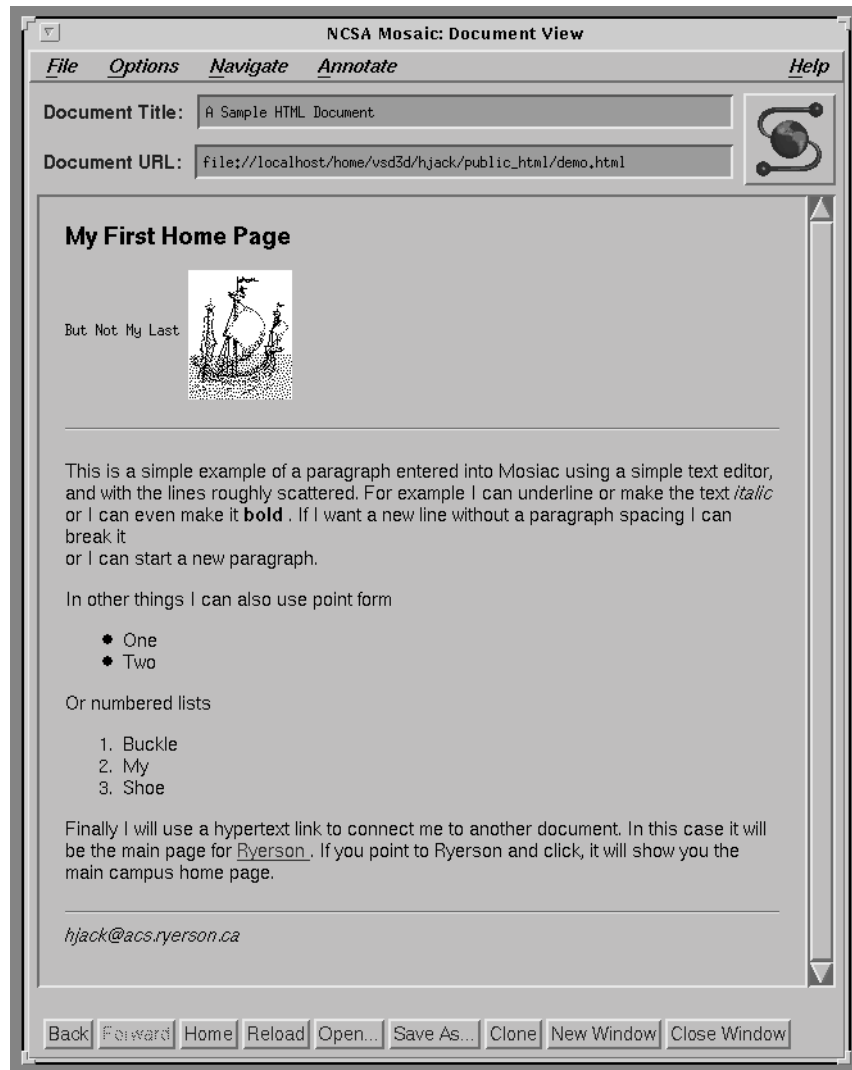
5.3 How to Get Your Own Home Page

You must create an HTML file (HyperText Markup Language) using a word processor, such as Wordperfect. Some points to consider.

1. To edit files, you must first download them using 'ftp' or netscape gold. They will exist in the 'public_html' directory in your directory. The 'index.html' file in this directory is the default start file.
2. Edit the file using Netscape Gold, Microsoft Word, or with a text editor.
3. Upload the modified files, and any new pictures, sounds, etc. using 'ftp' or netscape gold.
4. If necessary 'login' to the server and modify file permissions. Type the command,
`chmod 755 ~/. ~/public_html ~/public_html/*`
5. You can get to your home page with addresses. Use a browser to check your HTML files. My main home page is given below,
<http://claymore.engineer.gvsu.edu/~jackh>

5.4 How to Create a file

This section will present a sample html file that can be used as an example when creating your own html file (this must be typed into a word processor). The example begins with an example homepage displayed in Mosaic. The Mosaic file that created this page is given after.



<HTML>

```
<HEAD> <TITLE> A Sample HTML Document </TITLE> </HEAD>
```

```
<BODY>
```

```
  <H1> My First Home Page </H1>
```

```
  <H2>
```

```
    But Not My Last
```

```
    <IMG SRC = "picture.gif" ALIGN=MIDDLE>
```

```
</H2>
```

```

<HR>
<P> This is a simple example of a paragraph entered into
      Mosaic using a simple text editor, and with the lines
      roughly scattered. For example I can <U> underline </U>
      or make the text <I> italic </I> or I can even make
      it <B> bold </B>. If I want a new line without a
      paragraph spacing I can break it <BR> or I can start a
      new paragraph.

</P>
<P> In other things I can also use point form </P>
<UL>
    <LI> One
    <LI> Two
</OL>
<P> Or numbered lists </P>
<OL>
    <LI> Buckle
    <LI> My
    <LI> Shoe
</OL>
<P> Finally I will use a hypertext link to connect me to
      another document. In this case it will be the main
      page for <A HREF="http://acs.ryerson.ca"> Ryerson </A>.
      If you point to Ryerson and click, it will show you
      the main campus home page.

</P>
<HR>

<ADDRESS> hjack@acs.ryerson.ca </ADDRESS>

</BODY>
</HTML>

```

In this file the instructions basically indicate where typesetting should occur (much like viewing the control codes in Wordperfect). There are some commands that should always be used. First the <HTML> and </HTML> pair should always be used at the beginning and end of the document. Within this there should be a <HEAD> and </HEAD> to define overall information. Notice that in the <HEAD> section there is a <TITLE> _____ </TITLE> pair that define the name of the document in the top of the Mosaic window. After the <HEAD> section comes the <BODY> section (this is followed by </BODY>). Within the section we define the Mosaic document. In this example there are defined heading (i.e., <H1>____ </H1> and <H2>____ </H2>). There is also a picture file included (i.e.,). The preferred file format is GIF, but others are sometimes supported. A rule line is included to visually separate parts of the page (i.e., <HR> - Note this command does not need a second mate such as </HR>). Paragraphs are defined using <P> and </P>, so that the text will be kept in a block, and extra lines are added between paragraphs. A nice feature of Mosaic is the ability to create indented lists. Using the commands unordered lists with bullets can be made. Numbered lists can be made using and . Each point in any type of list begins with . At the end of the document is an <ADDRESS>_____ </ADDRESS> definition where you can put your email address, or other contact information.

There are some control codes buried inside the HTML document that be used for various purposes. For example `<I>__</I>` gives italics, `<U>__</U>` gives underlines, and `<B>__</B>` gives bold text. The most powerful tool in Mosaic is the anchor `<A>__</A>`. In general this is how hypertext links are created. In the example given above the command used is `<A HREF="____">____</A>`. The effect this has is that the work displayed (i.e. Ryerson) will appear differently on the screen, and if a user clicks on it the Mosaic viewer will now try to open the new document (i.e. `http://acs.ryerson.ca`, this will call up Ryerson's home page). These anchors can also refer to other locations in the same document.

5.5 Resources

Etherware:

- <http://www.ncsa.uiuc.edu/General/Internet/WWW/HTMLPrimer.html>
- <http://www.ncsa.uiuc.edu/demoweb/url-primer.html>
- http://kuhttp.cc.ukans.edu/lynx_help/HTML_quick.html
- <http://info.cern.ch/hypertext/WWW/MarkUp/MarkUp.html>
- <http://info.cern.ch/hypertext/WWW/Provider/Style/Overview.html>
- <http://info.cern.ch/hypertext/WWW/Addressing/Addressing.html>
- <http://info.cern.ch/hypertext/WWW/TheProject.html>
- <http://info.cern.ch/hypertext/WWW/MarkUp/ISOlat1.html>

Software:

- <ftp://ftp.ncsa.uiuc.edu/Web/html/hotmetal/Windows/hotmetal.exe>
- <ftp://ftp.einet.net/einet/pc/winweb/winweb.zip>
- <ftp://ftp.ncsa.uiuc.edu/PC/MOSAIC>
- <ftp://ftp2.cc.ukans.edu/pub/www/lynx>
- <http://www.netscape.com>

Sites:

- <http://www.yahoo.com>
- <http://gnn.com/GNNhome.html>
- <http://rubens.anu.edu.au/>
- <http://cybersight.com/cgi-bin/cs/s?main.gmml>

6. A BASIC INTRODUCTION TO 'C'

6.1 WHY USE 'C'?

- 'C' is commonly used to produce operating systems and commercial software. Some examples of these are UNIX, Lotus-123, dBase, and some 'C' compilers.
- *Machine Portable*, which means that it requires only small changes to run on other computers.
- *Very Fast*, almost as fast as assembler.
- Emphasizes *structured programming*, by focusing on functions and subroutines.
- You may easily customize 'C' to your own needs.
- Suited to Large and Complex Programs.
- Very Flexible, allows you to create your own functions.

6.2 BACKGROUND

- Developed at Bell Laboratories in the Early 70's, and commercial compilers became available in the Late 70's. Recently has become more popular because of its ties to UNIX. Over 90% of UNIX is written in 'C'. AT&T originally developed 'C' with the intention of making it an in-house standard.

6.3 PROGRAM PARTS

- /* is the start of a comment.
- */ is the end of comment.
- The main program is treated like a function, and thus it has the name main().
- lower/UPPER case is crucial, and can never be ignored.
- Statements are separated by semi-colons ;

- Statements consist of one operation, or a set of statements between curly brackets {, }
- There are no line numbers.

```

Program to Add two Numbers:
/* A simple program to add two numbers and print the results */

main()
{
int x, y = 2, z; /* define three variables and give one a value */
x = 3; /* give another variable a value */
z = x + y; /* add the two variables */
printf("%d + %d = %d\n", x, y, z); /*print the results */
}

Results (output):
3 + 2 = 5
    
```

- lines may be of any length.
- A very common function in 'C' is printf(). This function will do a formatted print. The format is the first thing which appears between the brackets. In this case the format says print an integer %d followed by a space then a '+' then another space, another integer, another space, '=', and another space, another integer, then a line feed '\n'. All variables that follow the format statement are those to be printed. x, y, and z are the three integers to be printed, in their respective orders.
- Major Data Types for variables and functions are (for IBM PC):
 - int (2 byte integer),
 - short (1 byte integer),
 - long (4 byte integer),
 - char (1 byte integer),
 - float (4 byte IEEE floating point standard),
 - double (8 byte IEEE floating point standard).
- int, short, long, char can be modified by the addition of unsigned, and register. An unsigned integer will not use 1 bit for number sign. A register variable will use a data register in the micro-processor, if possible, and it will speed things up (this is only available for integers).

Example of Defining Different Data Types:

```
main()
{
  unsigned int i;
  register j;
  short k;
  char l;
  double m;
  etc
```

- A *function* consists of a sub-routine or program, which has been assigned a name. This function is capable of accepting an argument list, and returning a single value. The function must be defined before it is called from within the program. (e.g. sin() and read()).

Program to add numbers with a function:

```
/* A simple program to add two numbers and print the results */
```

```
int add(); /* Declare a integer function called 'add' */
```

```
main()
{
  int x = 3, y = 2, z; /* define three variables and give values */
  z = add(x, y); /* pass the two values to 'add' and get the sum */
  printf("%d + %d = %d\n", x, y, z); /* print the results */
}
```

```
int add(a, b) /* define function and variable list */
int a, b; /* describe types of variable lists */
{
  int c; /* define a work integer */
  c = a + b; /* add the numbers */
  return(c); /* Return the number to the calling program */
}
```

- Every variable has a *scope*. This determines which functions are able to use that variable. If a variable is *global*, then it may be used by any function. These can be modified by the addition of static, extern and auto. If a variable is defined in a function, then it will be local to that function, and is not used by any other function. If the variable needs to be initialized every time the subroutine is called, this is an auto type. static variables can be used for a variable that must keep the value it had the last time the function was called. Using extern will allow the variable

types from other parts of the program to be used in a function.

```

Program example using global variables:
/* A simple program to add two numbers and print the results */

int x = 3, /* Define global x and y values */
y = 2,
add(); /* Declare an integer function called 'add' */

main()
{
printf(“%d + %d = %d\n”, x, y, add()); /*print the results */
}

int add() /* define function */
{
return(x + y); /* Return the sum to the calling program */
}

```

- Other variable types of variables are union, enum, struct, etc.
- Some basic control flow statements are while(), do-while(), for(), switch(), and if(). A couple of example programs are given below which demonstrate all the 'C' flow statements.

```

Program example with a for loop:
/* A simple program to print numbers from 1 to 5 */

main()
{
int i;
for(i = 1; i <= 5; i = i + 1){
printf(“number %d \n”, i); /*print the number */
}
}

```

or example with a while loop:

```
main()
{
int i = 1;
while(i <= 5){
printf("number %d \n", i);
i = i + 1;
}
}
```

or example with a do while loop

```
main()
{
int i = 1;
do{
printf("number %d \n", i);
i = i + 1;
}while(i <= 5)
}
```

or example with a do until loop:

```
main()
{
int i = 1;
do{
printf("number %d \n", i);
i = i + 1;
}while(i > 5)
}
```

Example Program with an if else:

```
main()
{
int x = 2, y = 3;
if(x > y){
printf("Maximum is %d \n", x);
} else if(y > x){
printf("Maximum is %d \n", y);
} else {
printf("Both values are %d \n", x);
}
}
```

Example Program using switch-case:

```
main()
{
int x = 3; /* Number of People in Family */
switch(x){ /* choose the numerical switch */
case 0: /* Nobody */
printf("There is no family \n");
break;
case 1: /* Only one person, but a start */
printf("There is one parent\n");
break;
case 2: /* You need two to start something */
printf("There are two parents\n");
break;
default: /* critical mass */
printf("There are two parents and %d kids\n", x-2);
break;
}
}
```

- #include <filename.h> will insert the file named filename.h into the program. The *.h extension is used to indicate a header file which contains 'C' code to define functions and constants. This almost always includes "stdio.h". As we saw before, a function must be defined (as with the 'add' function). We did not define printf() before we used it, this is normally done by using #include <stdio.h> at the top of your programs. "stdio.h" contains a line which says 'int printf();'. If we needed to use a math function like y = sin(x) we would have to also use

#include <math.h>, or else the compiler would not know what type of value that sin() is supposed to return.

- #define CONSTANT TEXT will do a direct replacement of CONSTANT in the program with TEXT, before compilation. #undef CONSTANT will undefine the CONSTANT.

A Sample Program to Print Some sin() values
(using defined constants)

```
#include "stdio.h"
#include "math.h"
#define TWO_PI 6.283185307
#define STEPS 5

main()
{
    double x; /* Current x value*/

    for(x = 0.0; x <= TWO_PI; x = x + (TWO_PI / STEPS)){
        printf("%f = sin(%f) \n", sin(x), x);
    }
}
```

- #ifdef, #ifndef, #if, #else and #endif can be used to conditionally include parts of a program. This is used for including and eliminating debugging lines in a program.
- #define, #include, #ifdef, #ifndef, #if, #else, /* and */ are all handled by the Preprocessor, before the compiler touches the program.
- Matrices are defined as shown in the example. In 'C' there are no limits to the matrix size, or dimensions. Arrays may be any data type. Strings are stored as arrays of characters.
- i++ is the same as i = i + 1.

A Sample Program to Get a String Then Print its ASCII Values (with matrix)

```
#include "stdio.h"
#define STRING_LENGTH 5

main()
{
    int i;
    char string[STRING_LENGTH]; /* character array */
    gets(string); /* Input string from keyboard */
    for(i = 0; i < STRING_LENGTH; i++){
        printf("pos %d, char %c, ASCII %d \n", i, string[i], string[i]);
    }
}
```

INPUT:
HUGH<return>

OUTPUT:
pos 0, char H, ASCII 72
pos 0, char U, ASCII 85
pos 0, char G, ASCII 71
pos 0, char H, ASCII 72
pos 0, char , ASCII 0

- Pointers are a very unique feature of 'C'. First recall that each variable uses a real location in memory. The computer remembers where the location of that variable is, this memory of location is called a pointer. This pointer is always hidden from the programmer, and uses it only in the background. In 'C', the pointer to a variable may be used. We may use some of the operations of 'C' to get the variable that the pointer, points to. This allows us to deal with variables in a very powerful way.

A Sample Program to Get a String
Then Print its ASCII Values (with pointers):

```
#include "stdio.h"
```

```
main()
{
    int i;
    char *string; /* character pointer */
    gets(string); /* Input string from keyboard */
    for(i = 0; string[i] != 0; i++){
        printf(" pos %d, char %c, ASCII %d \n", i, string[i], string[i]);
    }
}
```

INPUT:

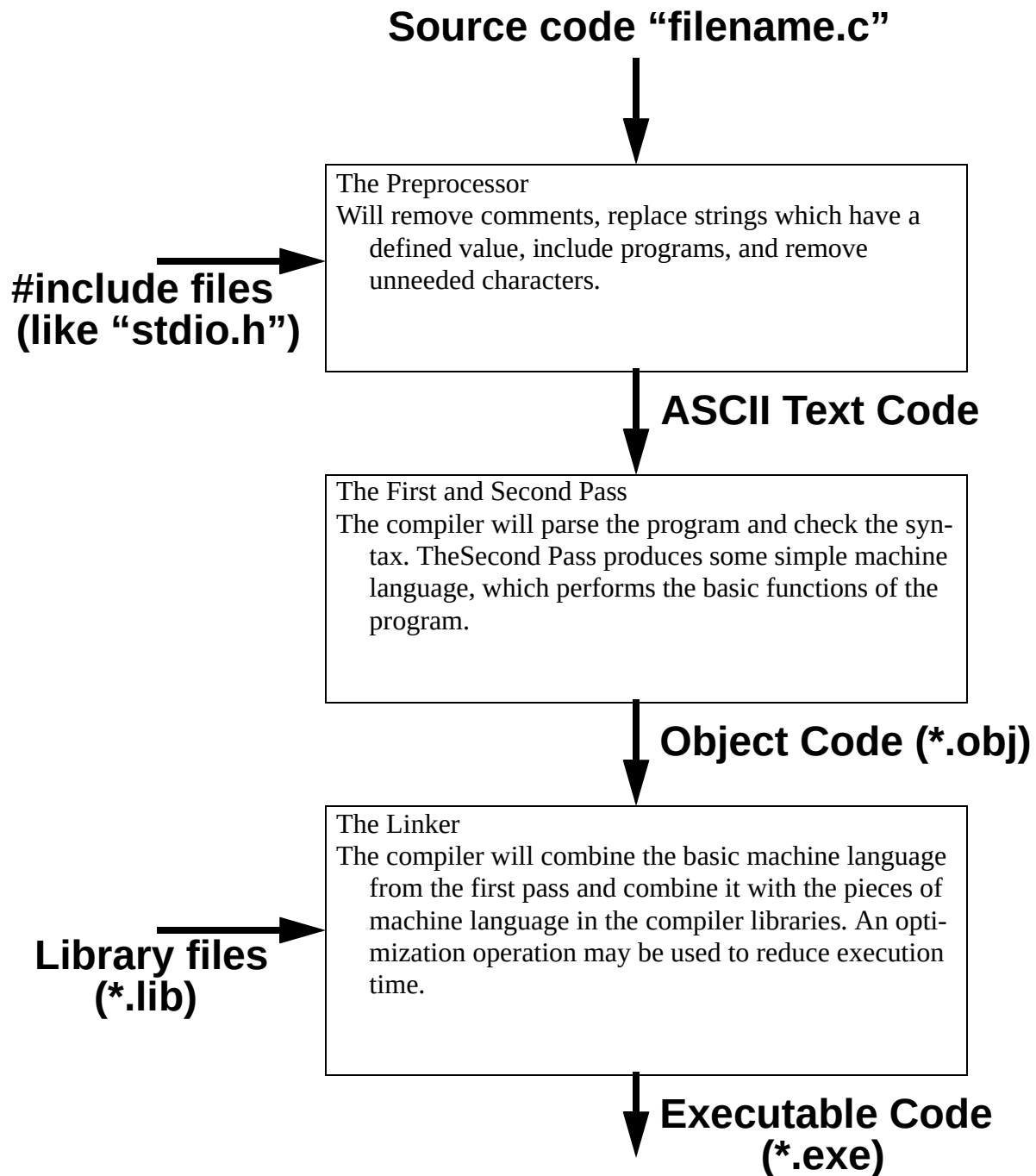
HUGH<return>

OUTPUT:

pos 0, char H, ASCII 72
pos 0, char U, ASCII 85
pos 0, char G, ASCII 71
pos 0, char H, ASCII 72

6.4 HOW A 'C' COMPILER WORKS

- A 'C' compiler has three basic components: Preprocessor, First and Second Pass Compiler, and Linker.



6.5 STRUCTURED 'C' CODE

- A key to well designed and understandable programs.

- Use indents, spaces and blank lines, to make the program look less cluttered, and give it a block style.
- Comments are essential to clarify various program parts.
- Descriptive variable names, and defined constants make the purpose of the variable obvious.
- All declarations for the program should be made at the top of the program listing.

A Sample of a Bad Program Structure:

```
main(){int i;for(i<10;i++)printf("age:%d\n",i);}
```

A Good Example of the same Program:

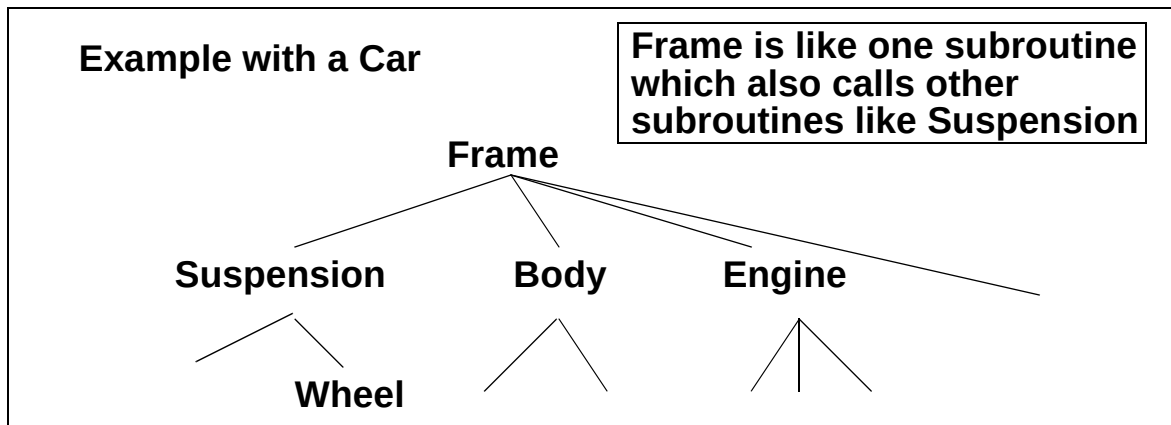
```
#include <stdio.h>
#define COUNT 10 /* Number of counts in loop */

main()
{
    int i; /* counter */
    for(i = 0; i < COUNT; i++){ /* loop to print numbers */
        printf("age:%d\n", i);
    }
    exit(0);
}
```

6.6 ARCHITECTURE OF ‘C’ PROGRAMS (TOP-DOWN)

6.6.1 How?

- A program should be broken into fundamental parts (using functions for each part) and then assembled using functions. Each function consists of programs written using the previous simpler functions.



- A Clear division should be maintained between program levels.
- Never use goto's, they are a major source of logic errors. Functions are much easier to use, once written.
- Try to isolate machine specific commands (like graphics) into a few functions.

6.6.2 Why?

- A top-down design allows modules to be tested as they are completed. It is much easier to find an error in a few lines of code, than in a complete program.
 - When programs are complete, errors tend to be associated with modules, and are thus much easier to locate.
 - Updates to programs are much easier, when we only need to change one function.
 - It is just as easy to change the overall flow of a program, as it is to change a function.
- Application of 'C' to a CAD Program

6.7 CREATING TOP DOWN PROGRAMS

1. *Define Objectives* - Make a written description of what the program is expected to do.
2. *Define Problem* - Write out the relevant theory. This description should include variables, calculations and figures, which are necessary for a complete solution to the problem. From this we make a list of required data (inputs) and necessary results (output).
3. *Design User Interface* - The layout of the screen(s) must be done on paper. The method of data entry must also be considered. User options and help are also considered here. *(There are numerous factors to be considered at this stage, as outlined in the course notes.)*

4. *Write Flow Program* - This is the main code that decides when general operations occur. This is the most abstract part of the program, and is written calling dummy 'program stubs'.
5. *Expand Program* - The dummy 'stubs' are now individually written as functions. These functions will call another set of dummy 'program stubs'. This continues until all of the stubs are completed. *After the completion of any new function, the program is compiled, tested and debugged.*
6. *Testing and Debugging*- The program operation is tested, and checked to make sure that it meets the objectives. If any bugs are encountered, then the program is revised, and then retested.
7. *Document* - At this stage, the operation of the program is formally described. For Programmers, a top-down diagram can be drawn, and a written description of functions should also be given.

Golden Rule: If you are unsure how to proceed when writing a program, then work out the problem on paper, before you commit yourself to your programmed solution.

Note: Always consider the basic elements of Software Engineering, as outlined in the ES488 course notes.

6.8 HOW THE BEAMCAD PROGRAM WAS DESIGNED

6.8.1 Objectives:

- The program is expected to aid the design of beams by taking basic information about beam geometry and material, and then providing immediate feedback. The beam will be simply supported, and be under a single point load. The program should also provide a printed report on the beam.

6.8.2 Problem Definition:

- The basic theory for beam design is available in any good mechanical design textbook. In this example it will not be given.
- The inputs were determined to be few in number: Beam Type, Beam Material, Beam Thickness, Beam Width, Beam Height, Beam Length, Load Position, Load Force.
- The possible outputs are Cross Section Area, Weight, Axial Stiffness, Bending Stiffness, and Beam Deflection, a visual display of Beam Geometry, a display of Beam Deflection.

6.8.3 User Interface:

6.8.3.1 - Screen Layout (also see figure):

- The small number of inputs and outputs could all be displayed, and updated, on a single screen.
- The left side of the screen was for inputs, the right side for outputs.
- The screen is divided into regions for input(2), input display and prompts(1), Beam Cross section(3), Numerical Results(4), and Beam Deflection(5).

6.8.3.2 - Input:

- Current Inputs were indicated by placing a box around the item on the display(1).
- In a separate Prompt Box(2), this input could be made.
- The cursor keys could be used to cursor the input selector up or down.
- Single keystroke operation.
- Keys required: UP/DOWN Cursors, F1, F2, F4, numbers from '0' to '9', '.', '-', and <RETURN>. In the spirit of robustness it was decided to screen all other keys.

6.8.3.3 - Output:

- Equations, calculations, material types, and other relevant information were obtained from a text.
- Proper textual descriptions were used to ensure clarity for the user.
- For a printed report, screen information would be printed to a printer, with the prompt area replaced with the date and time.

6.8.3.4 - Help:

- A special set of help information was needed. It was decided to ensure that the screen always displays all information necessary(2).

6.8.3.5 - Error Checking:

- Reject any input which violates the input limits.
- A default design was given, which the user could modify.
- An error checking program was created, which gives error messages.

6.8.3.6 - Miscellaneous:

- The screen was expressed in normalized coordinates by most sub-routines.
- Colors were used to draw attention, and highlight areas.

6.8.4 Flow Program:

```

main()
/*
 * EXECUTIVE CONTROL LEVEL
 *
 * This is the main terminal point between the
 * various stages of setup, input, revision
 * and termination.
 *
 * January 29th, 1989.
 */
{
    static int error;

    if((error = setup()) != ERROR) {
        screen(NEW);
        screen(UPDATE);
        while((error = input()) != DONE) {
            if(error == REVISED) {

screen(NEW);

screen(UPDATE);

                                }
                            }
        error = NO_ERROR;
    }
    kill();
    if(error == ERROR) {
        printf("EGA Graphics Driver Not Installed");
    }
}

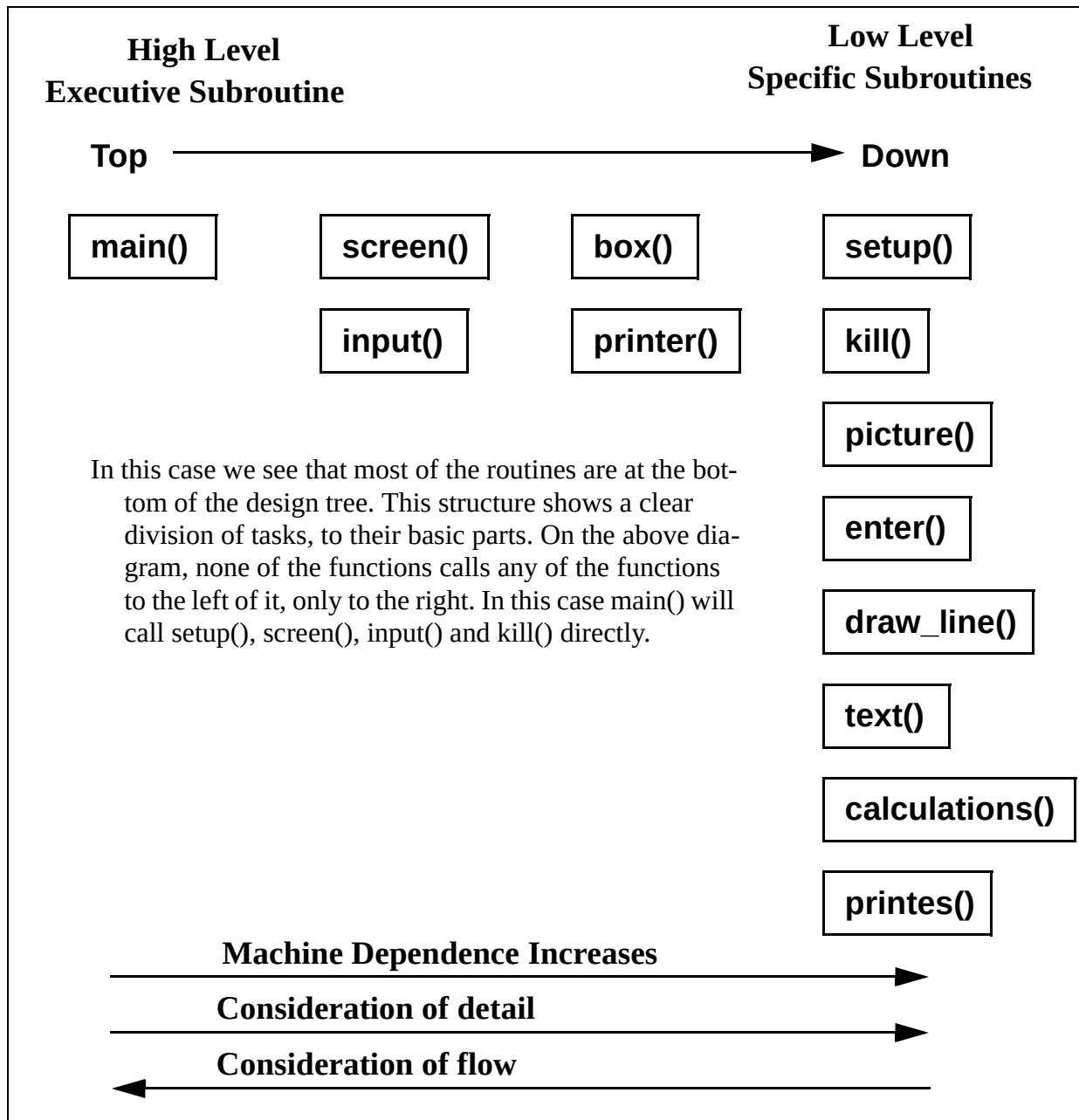
```

6.8.5 Expand Program:

- The routines were written in a top down fashion, in a time of about 30 hours. These routines are listed below.

Routines Used In Package:

- **main()** - to be used as the main program junction.
 - **setup()** - to set up graphics mode and printer.
 - **screen()** - A function to draw, or refresh part of the screen. In the interest of program speed, this function uses some low level commands.
 - **calculations()** - perform the calculations of outputs from the inputs
 - **picture()** - draws the beam cross section and deflection of beam. For the sake of speed, this section will use low level commands.
 - **input()** - A function which controls the main input loop for numbers, controls, error screening, and any calls to desired routines. Input uses both higher and lower level commands for the sake of speed.
 - **printes()** - A function to print the EGA screen.
 - **printer()** - A function to remove help messages from the screen, and then dumps the screen to the printer.
 - **enter()** - checks for entry error, against a set of limits for any input.
 - **text()** - A function to print text on the screen at normalized coords.
 - **draw_line()** - A Function to draw a line in normalized coords.
 - **box()** - A Function to draw a double lined box anywhere on screen.
 - **kill()** - deinitialize anything initialized in **setup()**.
-
- Condition and error flags were used to skip unnecessary operations, and thus speed up response. A response of more than 0.5 seconds will result in loss of attention by the user.



6.8.6 Testing and Debugging:

- The testing and debugging was very fast, with only realignment of graphics being required. This took a couple of hours.

6.8.7 Documentation

6.8.7.1 - Users Manual:

- The documentation included an Executive Summary of what the Program does.
- The Objectives of the program were described.
- The theory for beam design was given for the reference of any program user, who wanted to verify the theory, and possible use it.
- A manual was given which described key layouts, screen layout, basic sequence of operations, inputs and outputs.
- Program Specifications were also given.
- A walk through manual was given. This allowed the user to follow an example which displayed all aspects of the program.

6.8.7.2 - Programmers Manual:

- Design Strategy was outlined and given.
- A complete program listing was given (with complete comments).
- Complete production of this Documentation took about 6 hours.

6.8.8 Listing of BeamCAD Program.

- Written for turbo 'C'

6.9 PRACTICE PROBLEMS

1. What are the basic components of a 'C' compiler, and what do they do?
2. You have been asked to design a CAD program which will choose a bolt and a nut to hold two pieces of sheet metal together. Each piece of sheet metal will have a hole drilled in it that is the size of the screw. You are required to consider that the two pieces are experiencing a single force. State your assumptions about the problem, then describe how you would produce this program with a Top Down design.
3. What are some reasons for using 'C' as a programming language?

4. Describe some of the reasons for Using Top-Down Design, and how to do it.

7. GUI DESIGN

7.1 PRACTICE PROBLEMS

1. Draw a simple window based GUI for a plant floor layout viewer. Also describe the functions of the various buttons, windows, etc.

8. AN EXAMPLE - BEAMCAD

```
/*
 * BeamCAD
 *
 * By Hugh Jack,
 *
 * An Automated approach to beam design. This is an interactive program
 * Designed to take input about a beam and then determine beam para-
 * meters, which are displayed both graphically and numerically.
 * After on screen design a hardcopy may be made on an Epson compatible
 * printer. The program requires an EGA quality monitor, with an IBM
 * PC compatible system.
 *
 * February, 9th, 1989.
 *
 */

/* header files which define functions and constants */
#include "stdio.h"
#include "graphics.h"
#include "math.h"
#include "time.h"
#include "conio.h"
#include "bios.h"

/* define internal operation and error codes */
#define DONE -1
#define NO_ERROR 0
#define ERROR 1
#define REVISED 2
#define NEW 3
#define UPDATE 4
#define PRINTER_ERROR 5
```

```

/* define material types */
#define STEEL 0
#define ALUMINUM 1
#define END_MATERIALS 2

/* define beam types */
#define I_BEAM 0
#define CHANNEL_BEAM 1
#define SOLID_BEAM 2
#define HOLLOW_BEAM 3
#define L_BEAM 4
#define END_BEAMS 5

/* set corners of user input box */
#define INPUT_LEFT .03
#define INPUT_RIGHT .45
#define INPUT_BOTTOM .97
#define INPUT_TOP .85

/* miscellaneous definitions */
#define EXTENSION 20 /* standoff for dimensions */
#define DOUBLE_LINE_GAP .005 /* normalized gap between double lines */
#define X_Y_RATIO 1.408 /* Screen x to y aspect ratio */
#define PAGE_OFFSET 80 /* Output offset for printer */
#define ERROR_BIT 8 /* Printer error bit */

/* define some key codes */
#define DOWN 1
#define UP -1
#define F_1 59
#define F_3 61
#define F_4 62
#define UP_ARROW 72
#define DOWN_ARROW 80
#define ENTER 13

/* declare functions in this listing */
void box(),
      calculations(),
      draw_line(),
      kill(),
      picture(),
      screen(),
      text();
int enter(),
     input(),
     printer(),
     printes(),
     setup();

/* set up formats for user inputs, prompts and help prompts */
struct entry { char name[25];
               double posx;
               double posy;
               char help1[40];
               char help2[40]; };

struct entry list[8] = {{"Beam Type :"}, .08, .18,
                       {"1. I-Beam 2. Channel "},

```

```

        {"3. Solid 4. Hollow 5. L-Beam"}},
{{"Beam Material :", .08, .23,
{"1. Steel 2. Aluminum"}},
{"Beam Thickness :", .08, .33,
{"Metal Plat thickness for beam "},
{"Construction"}},
{"Beam Height :", .08, .38,
{"Total height of beam from top to bottom"}},
{"Beam Width :", .08, .43,
{"Total width of beam from left to right"}},
{"Span Length :", .08, .53,
{"Length of unsuported span to be"},
{"considered"}},
{"Load Position Offset :", .08, .58,
{"Distance from left side to the Point"},
{"at which a load force is applied"}},
{"Load Force :", .08, .63,
{"Total load force applied at load"},
{"position"}}}};

/* set up format and structure for material description */
struct mats { char    name[15];
              double weight;
              double modulus; };

struct mats    materials[10] = {{{"Steel"}, .283, 10300000.0},
                                {"Aluminum"}, .0995, 29000000.0}};

/* set up work strings and various beam names */
char    work[80],
        work2[20],
        beams[10][20] = {{{"I Beam"},
                            {"Channel Beam"},
                            {"Solid Beam"},
                            {"Hollow Beam"},
                            {"L Beam"}}};

/* set up global integers */
int    beam_type = I_BEAM, /* beam cross section type */
        beam_material = STEEL, /* beam material used */
        entry_number = 0; /* Current choice on input menu */

/* set up global double precision numbers */
double beam_width = 3.0, /* Width of beam cross section */
        beam_height = 6.0, /* Height of Beam cross section */
        beam_thick = 0.5, /* Thickness of metal plates */
        beam_length = 120.0, /* Length of beam span */
        beam_load = 0.0, /* Load force applied to beam */
        beam_point = 0.0, /* displacement of applied load */
        beam_area, /* calculated cross section area */
        x_centroid, /* calculated x centroid */
        y_centroid, /* calculated y centroid */
        moment_inertia, /* Calculated moment of inertia */
        x_axis, /* calculated x principal axis */
        y_axis, /* calculated y principal axis */
        axial_stiffness, /* calculated axial stiffness */
        bending_stiffness, /* calculated bending stiffness */
        beam_deflect, /* calculated beam deflection */
        deflect_point, /* point of maximum deflection */
        out_distance, /* calculated distance from fibres */
        beam_weight, /* calculated total beam weight */
        xmax, /* absolute width of screen x */

```

```

        ymax; /* absolute height of screen y */

main()
/*
 * EXECUTIVE CONTROL LEVEL
 *
 * This is the main terminal point between the various stages of setup,
 * input, revision and termination.
 *
 * January 29th, 1989.
 */
{
    static int error;

    if((error = setup()) != ERROR) {
        screen(NEW);
        screen(UPDATE);
        while((error = input()) != DONE) {
            if(error == REVISED) {
                screen(NEW);
                screen(UPDATE);
            }
        }
        error = NO_ERROR;
    }
    kill();
    if(error == ERROR) {
        printf("EGA Graphics Driver Not Installed");
    }
}

int setup()
/*
 * SYSTEM INITIALIZATION
 *
 * This will initialize an EGA monitor and then initialize, and collect
 * video parameters, and initialize the printer.
 *
 * January 29th, 1989.
 */
{
    static int driver,
              mode,
              error;

    /* Detect EGA driver and initialize */
    detectgraph(&driver, &mode);
    error = ERROR;
    if(driver == EGA){
        error = NO_ERROR;
        initgraph(&driver, &mode, "");
    }

    /* Initialize printer port */
    biosprint(1, 7, 0);

    /* get maximum x and y screen pixel coordinates */
    xmax = getmaxx();
    ymax = getmaxy();

    return(error);
}

```



```

void screen(int option)
/*
 * SCREEN LAYOUT/UPDATE
 *
 * This section will set up the screen with basic titles and headings.
 * The section will also update the current values of beam parameters.
 *
 * January 29th, 1989.
 */
{
    /* If instructed then clear screen and add titles */
    if(option == NEW){
        /* Clear screen */
        cleardevice();

        /* Add titles and instructions */
        draw_line(.50, 0.0, .50, 1.0-DOUBLE_LINE_GAP);
        draw_line(.5 + DOUBLE_LINE_GAP, 0.0, .5 + DOUBLE_LINE_GAP, 1.0-DOUBLE_LINE_GAP);
        settxtstyle(SANS_SERIF_FONT, HORIZ_DIR, 4);
        text(.02, .014, "Beam C.A.D.");
        settxtstyle(SMALL_FONT, HORIZ_DIR, 6);
        text(.31, .06, "by Hugh Jack");
        settxtstyle(SMALL_FONT, HORIZ_DIR, 4);
        setcolor(GREEN);
        text(INPUT_LEFT, INPUT_TOP - .10, "F1 - QUIT F3 - SCREEN PRINT F4 - HARDCOPY");
        text(INPUT_LEFT, INPUT_TOP - .07, "Use UP and DOWN Cursors to select Parameter");
        text(INPUT_LEFT, INPUT_TOP - .04, "enter new paramter and press RETURN to accept");
        setcolor(WHITE);
        box(INPUT_LEFT, INPUT_BOTTOM, INPUT_RIGHT, INPUT_TOP, GREEN, NEW);
    }

    /* If instructed then update beam information */
    if(option == UPDATE){
        /* Calculate new output parameters from input parameters */
        calculations();

        /* Show input data */
        text(.05, .13, "Beam");
        sprintf(work, "Type : %s", beams[beam_type]);
        text(.08, .18, work);
        sprintf(work, "Material : %s", materials[beam_material].name);
        text(.08, .23, work);
        text(.05, .28, "Cross Section");
        sprintf(work, "Thickness : %.3f in.", beam_thick);
        text(.08, .33, work);
        sprintf(work, "Height : %.3f in.", beam_height);
        text(.08, .38, work);
        sprintf(work, "Width : %.3f in.", beam_width);
        text(.08, .43, work);
        text(.05, .48, "Span");
        sprintf(work, "Length : %.3f in.", beam_length);
        text(.08, .53, work);
        sprintf(work, "Load Posit.: %.3f in.", beam_point);
        text(.08, .58, work);
        sprintf(work, "Load : %.3f lbs.", beam_load);
        text(.08, .63, work);

        /* Print Calculated Parameters */
        sprintf(work, "Beam Cross Sectional Area in^2: %.3f", beam_area);
        text(.56, .55, work);
        sprintf(work, "Weight lbs.: %.3f", beam_weight);
        text(.56, .59, work);
    }
}

```

```

        sprintf(work, "Axial Stiffness lb. / in: %.3g", axial_stiffness);
        text(.56, .63, work);
        sprintf(work, "Bending Stiffness lbs / in^2: %.3g", bending_stiffness);
        text(.56, .67, work);
        sprintf(work, "Maximum Beam Deflection in.: %.3g", beam_deflect);
        text(.56, .71, work);

        /* Draw cross section and loading diagrams */
        picture(beam_type);
    }
}

void calculations()
/*
 * CALCULATE BEAM PARAMETERS
 *
 * The global values of the beam parameters are use to calculate
 * the other output variables of the beam parameters.
 *
 * February, 9th, 1989.
 */
{
    /* locate general centroid */
    x_centroid = x_axis = beam_width / 2.0;
    y_centroid = y_axis = beam_height / 2.0;

    /* find moment of inertia for solid beam and later subtract
    hollow sections for various beam types. */
    moment_inertia = beam_width*beam_height*beam_height*beam_height/12.0;

    if(beam_type == I_BEAM) {
        beam_area = beam_thick*(2*beam_width + beam_height - 2*beam_thick);
        moment_inertia = moment_inertia - (beam_width-beam_thick)*pow((beam_height-
beam_thick), 3.0)/12.0;
    }

    if(beam_type == CHANNEL_BEAM) {
        beam_area = beam_thick*(2*beam_width + beam_height - 2*beam_thick);
        x_centroid =(pow(beam_width, 2.0) + (beam_height - 2*beam_thick)*beam_thick)/
(beam_width + (beam_height - 2*beam_thick))/2.0;
        moment_inertia = moment_inertia - (beam_width-beam_thick)*pow((beam_height-
beam_thick), 3.0)/12.0;
    }

    if(beam_type == SOLID_BEAM) {
        beam_area = beam_width*beam_height;
    }

    if(beam_type == HOLLOW_BEAM) {
        beam_area = beam_thick*(2*beam_width + 2*beam_height - 4*beam_thick);
        moment_inertia = moment_inertia - (beam_width - 2*beam_thick)*pow((beam_height-
beam_thick), 3.0)/12.0;
    }

    if(beam_type == L_BEAM) {
        beam_area = beam_thick*(beam_width + beam_height - beam_thick);
        x_centroid = (beam_width*beam_width +(beam_height-beam_thick)*beam_thick)/2.0/
(beam_height + beam_width - beam_thick);
        y_centroid = (beam_height*beam_height +(beam_width-beam_thick)*beam_thick)/2.0/
(beam_height + beam_width - beam_thick);
        moment_inertia = (beam_width*pow(y_centroid, 3.0) - (beam_width-
beam_thick)*pow((y_centroid-beam_thick),3.0)+beam_thick*pow((beam_height-y_centroid), 3.0))/3.0;
    }
}

```

```

axial_stiffness = beam_area * materials[beam_material].modulus;
bending_stiffness = materials[beam_material].modulus * moment_inertia;

beam_weight = beam_area * materials[beam_material].weight * beam_length;

/* find deflection of beam under loading */
deflect_point = pow((beam_point*(2 * beam_length-beam_point)/3.0), .5);
beam_deflect = beam_load*(beam_length - beam_point)/
(3.0*bending_stiffness*beam_length)*pow(deflect_point, 3);
}

int input()
/*
 * INPUT MANAGER
 *
 * User input functions are handled through this subroutine. These
 * functions include numerical input and checking, watching function
 * keys, and issuing proper feedback for executive control.
 *
 * January 28th, 1989.
 */
{
    int    up_down,
           error,
           flagger,
           count;

    double x,
           y,
           inpoft;
    char   in;

    error = 0;
    /* set flag for cursor movement */
    up_down = 0;

    /* Do boxes and prompts for current input */
    x = list[entry_number].posx;
    y = list[entry_number].posy;
    box(x-.01, y+.045, x+.30, y-.015, GREEN, NEW);
    text(INPUT_LEFT+2*DOUBLE_LINE_GAP, INPUT_TOP+4*DOUBLE_LINE_GAP, list[entry_number].name);
    text(INPUT_LEFT+2*DOUBLE_LINE_GAP, INPUT_TOP+.05, list[entry_number].help1);
    text(INPUT_LEFT+2*DOUBLE_LINE_GAP, INPUT_TOP+.08, list[entry_number].help2);
    inpoft = textwidth(list[entry_number].name)/xmax+.01;
    work[0] = 0;
    count = 0;

    /* Loop while numbers are being entered */
    setcolor(WHITE+BLINK);
    /* cursor marker */
    text(INPUT_LEFT+inpoft+(count+1)/100.0, INPUT_TOP+.02, "_");
    setcolor(WHITE);
    /* screen for too many chars, and no control keys */
    while((count < 12) && ((in = getch()) != ENTER) && (in != 0)) {
        if(((in >= '0') && (in <= '9')) || (in == '.') || ((in == '-') && (count == 0))) {
            /* update input string */
            work[count] = in;
            count++;
            work[count] = 0;

            /* update screen input line */
            sprintf(work2, "%c", in);
            text(INPUT_LEFT+inpoft+count/100.0, INPUT_TOP+.02, work2);
        }
    }
}

```

```

        /* Cursor update */
        setcolor(WHITE+BLINK);
        text(INPUT_LEFT+inpoff+(count+1)/100.0, INPUT_TOP+.02, "_");
        setcolor(WHITE);
        setcolor(BLACK);
        text(INPUT_LEFT+inpoff+(count)/100.0, INPUT_TOP+.02, "_");
        setcolor(WHITE);
    }
    /* beep in case of input error */
    else { printf("\007");}
}

/* look for control and cursor keys */
if(in == 0) {
    in = getch();
    if(in == DOWN_ARROW) up_down = DOWN;
    if(in == UP_ARROW) up_down = UP;
    if(in == F_1) error = DONE;
    if(in == F_3) {
        if(printes() == ERROR) {error = PRINTER_ERROR;}
        else {error = REVISED;}
    }
    if(in == F_4) {
        if(printer() == ERROR) {error = PRINTER_ERROR;}
        else {error = REVISED;}
    }
    count = 0;
}

/* Check for numerical input error */
if((count > 0) && (count < 12)) {
    error = enter();
    up_down = DOWN;
}
if(count == 12) error = ERROR;

/* Clear input box and indicate error if they have occurred */
box(x-.01, y+.045, x+.30, y-.015, BLACK, NEW);
box(INPUT_LEFT, INPUT_BOTTOM, INPUT_RIGHT, INPUT_TOP, GREEN, UPDATE);
if((error == ERROR) || (error == PRINTER_ERROR)) {
    printf("\007");
    if(error == ERROR) {
        error = ERROR;
        text(INPUT_LEFT+2*DOUBLE_LINE_GAP, INPUT_BOTTOM-.05, "INPUT ERROR - (Hit
any key to clear)");
    }
    if(error == PRINTER_ERROR) {
        error = REVISED;
        text(INPUT_LEFT+2*DOUBLE_LINE_GAP, INPUT_BOTTOM-.05, "PRINTER ERROR - (Hit
any key to clear)");
    }
    getch();
    box(INPUT_LEFT, INPUT_BOTTOM, INPUT_RIGHT, INPUT_TOP, GREEN, UPDATE);
}

/* update to new menu choice */
entry_number = entry_number + up_down;
if(entry_number < 0) entry_number = 7;
if(entry_number > 7) entry_number = 0;

return(error);
}

```

```

int printer()
/* BEAM DUMPER
 *
 * This routine will prepare the screen and dump it to an EPSON type
 * printer. This will not do the printing, but will use the routines
 * PRINTES() to do the printing.
 *
 * February 2nd, 1989.
 */
{
    static double xx,yy;

    /* Do the time sort of thing */
    time_t timet;
    struct tm *times;
    time(&timet);
    times = localtime(&timet);

    /* Clear input boxes on current screen */
    xx = list[entry_number].posx;
    yy = list[entry_number].posy;
    box(xx-.01, yy+.045, xx+.30, yy-.015, BLACK, NEW);
    box(INPUT_LEFT-2*DOUBLE_LINE_GAP, INPUT_BOTTOM, INPUT_RIGHT+2*DOUBLE_LINE_GAP, INPUT_TOP-
.15, BLUE, UPDATE);
    box(INPUT_LEFT-2*DOUBLE_LINE_GAP, INPUT_BOTTOM, INPUT_RIGHT+2*DOUBLE_LINE_GAP, INPUT_TOP-
.15, BLACK, NEW);
    box(DOUBLE_LINE_GAP, 1 - DOUBLE_LINE_GAP, 1 - DOUBLE_LINE_GAP, DOUBLE_LINE_GAP, BLUE,
NEW);

    /* Add design date to screen */
    text(INPUT_LEFT, INPUT_TOP-.05, "Design Date :");
    text(INPUT_LEFT, INPUT_TOP, asctime(times));

    return(printes());
}

```

```

int printes()
/* SCREEN DUMPER
 *
 * This routine will copy the screen to an EPSON type
 * printer. This will constitute a hard copy output. This routine was
 * prepared with the help of a basic routine written by Peter Budgell and
 * Bernard Julien. The basic operation is to read a pixel from the
 * screen and then print it.
 *
 * February 2nd, 1989.
 */
{
    static int x, y, z, count, bit, error;

    /* Continue if no error */
    if((error = ((int)(biosprint(2, 7, 0)) & ERROR_BIT)) == 0) {
        /* Prepare printer */
        biosprint(0, 27, 0);
        biosprint(0, 'A', 0);
        biosprint(0, 8, 0);
        for(y = 0; y < (int)xmax; y = y + 8) {
            biosprint(0, 27, 0);
            biosprint(0, 'L', 0);
            biosprint(0, (int)(2*(ymax+PAGE_OFFSET))%256, 0);
            biosprint(0, (int)(2*(ymax+PAGE_OFFSET))/256, 0);

```

```

        for(x = 0; x < PAGE_OFFSET; x++) {
            biosprint(0, 0, 0);
            biosprint(0, 0, 0);
        }
        for(x = (int)ymax; x > 0; x--) {
            count = 0;
            bit = 1;
            for(z = 7; z >= 0; z--) {
                count = count + bit*(getpixel(y+z, x) != BLACK);
                bit = bit + bit;
            }
            biosprint(0, count, 0);
            biosprint(0, count, 0);
        }
        biosprint(0, 13, 0);
        biosprint(0, 10, 0);
    }
    error = NO_ERROR;
}
else error = ERROR;

return(error);
}

int enter()
/*
 * ENTRY CHECKER
 *
 * This routine will use the previous input value to determine input
 * validity, and update valid inputs. The value will be taken from
 * the WORK string and then considered for the current input.
 *
 * January 28th, 1989.
 */
{
    double f_result;
    int    i_result,
           error;

    /* Prepare for analysis */
    f_result = atof(work);
    i_result = atoi(work);
    error = ERROR;

    /* check for no value */
    if(work[0] == 0) error = NO_ERROR;

    if(error == ERROR) {
        /* Update beam type */
        if((entry_number == 0) && (i_result > 0) && (i_result < L_BEAM+2)) {
            error = NO_ERROR;
            if(beam_type != i_result-1) error = REVISED;
            beam_type = i_result-1;
        }

        if((entry_number == 1) && (i_result > 0) && (i_result < ALUMINUM+2)) {
            error = NO_ERROR;
            if(beam_material != i_result-1) error = REVISED;
            beam_material = i_result-1;
        }

        if((entry_number == 2) && (f_result > 0.025) && (f_result < beam_width/2) &&

```

```

(f_result < beam_height/2)) {
    error = NO_ERROR;
    if(beam_thick != f_result) error = REVISED;
    beam_thick = f_result;
}

if((entry_number == 3) && (f_result > beam_thick*2.0) && (f_result < 60.0)) {
    error = NO_ERROR;
    if(beam_height != f_result) error = REVISED;
    beam_height = f_result;
}

if((entry_number == 4) && (f_result > beam_thick/2.0) && (f_result < 60.0)) {
    error = NO_ERROR;
    if(beam_width != f_result) error = REVISED;
    beam_width = f_result;
}

if((entry_number == 5) && (f_result > 0.0) && (f_result < 200.0) && (f_result >=
beam_point)) {
    error = NO_ERROR;
    if(beam_length != f_result) error = REVISED;
    beam_length = f_result;
}

if((entry_number == 6) && (f_result >= 0.0) && (f_result <= beam_length)) {
    error = NO_ERROR;
    if(beam_point != f_result) error = REVISED;
    beam_point = f_result;
}

if((entry_number == 7) && (abs(f_result) < 10000000.0)) {
    error = NO_ERROR;
    if(beam_load != f_result) error = REVISED;
    beam_load = f_result;
}

}
return(error);
}

```

```

void text(double x, double y, char *text)
/*
 * Text Utility
 *
 * A routine to make text statements more readable, by allowing entry in
 * normalized coordinates.
 *
 * January 28th, 1989.
 */
{
    outtextxy(x * xmax, y * ymax, text);
}

```

```

void draw_line(double x1, double y1, double x2, double y2)
/*
 * Line Utility
 *

```

```

* A routine to simplify line statements by allowing normalized calls.
*
* January 30th, 1989.
*/
{
    line((int)(x1*xmax), (int)(y1*ymin), (int)(x2*xmax), (int)(y2*ymin));
}

void box(double x1, double y1, double x2, double y2, int color, int refresh)
/*
* Box Drawer
*
* This routine will simply draw a box with double lines , or erase
* the contents.
*
* January 28th, 1989.
*/
{
    int    temp_color;
    temp_color = getcolor();
    setcolor(color);

    /* Draw outer box */
    draw_line(x1, y1, x2, y1);
    draw_line(x2, y1, x2, y2);
    draw_line(x2, y2, x1, y2);
    draw_line(x1, y2, x1, y1);

    /* Draw inner box */
    draw_line(x1 + DOUBLE_LINE_GAP, y1 - DOUBLE_LINE_GAP, x2 - DOUBLE_LINE_GAP, y1 -
DOUBLE_LINE_GAP);
    draw_line(x2 - DOUBLE_LINE_GAP, y1 - DOUBLE_LINE_GAP, x2 - DOUBLE_LINE_GAP, y2 +
DOUBLE_LINE_GAP);
    draw_line(x2 - DOUBLE_LINE_GAP, y2 + DOUBLE_LINE_GAP, x1 + DOUBLE_LINE_GAP, y2 +
DOUBLE_LINE_GAP);
    draw_line(x1 + DOUBLE_LINE_GAP, y2 + DOUBLE_LINE_GAP, x1 + DOUBLE_LINE_GAP, y1 -
DOUBLE_LINE_GAP);

    /* Erase box contents if flag set */
    if(refresh == UPDATE) {
        setfillstyle(SOLID_FILL, BLACK);
        floodfill((int)((x2 + x1)/2.0)*xmax, (int)((y1 + y2)/2.0) * ymin, color);
    }

    setcolor(temp_color);
}

void picture(int draw_beam)
/*
* Beam Drawer
*
* This section will produce all beam graphics and dimensions
*
* January 30th, 1989.
*/
{
    static int x_s,
               x_e,

```



```

        y_s,
        y_e,
        cx1,
        cx2,
        cx3,
        cx4,
        cx5,
        cx6,
        cy1,
        cy2,
        cy3,
        cy4,
        cy5,
        cy6,
        x_offset,
        y_offset,
        x_c_offset,
        y_c_offset;
static double scale_x,
        scale_y,
        px,
        py,
        de,
        offset;

x_s = xmax/2 + 3*EXTENSION + 40;
y_s = ymax/2 - 15;
x_e = xmax - 5;
y_e = 22 + EXTENSION;
if(x_axis != x_centroid) y_e = y_e + EXTENSION;
if(y_axis == y_centroid) x_s = xmax/2 + 2*EXTENSION + 40;
if(beam_width < beam_height){
    scale_x = (x_e - x_s) / beam_height / X_Y_RATIO;
    scale_y = (y_e - y_s) / beam_height;
};
if(beam_width >= beam_height){
    scale_x = (x_e - x_s) / beam_width / X_Y_RATIO;
    scale_y = (y_e - y_s) / beam_width;
};

cx1 = x_s;
cx2 = x_s + (int)(scale_x * beam_thick);
cx3 = x_s + (int)(scale_x * (beam_width - beam_thick) / 2);
cx4 = x_s + (int)(scale_x * (beam_width + beam_thick) / 2);
cx5 = x_s + (int)(scale_x * (beam_width - beam_thick));
cx6 = x_s + (int)(scale_x * beam_width);

cy1 = y_s;
cy2 = y_s + (int)(scale_y * beam_thick);
cy3 = y_s + (int)(scale_y * (beam_height - beam_thick) / 2);
cy4 = y_s + (int)(scale_y * (beam_height + beam_thick) / 2);
cy5 = y_s + (int)(scale_y * (beam_height - beam_thick));
cy6 = y_s + (int)(scale_y * beam_height);

x_c_offset = cx1 + scale_x * x_centroid;
y_c_offset = cy1 + scale_y * y_centroid;
x_offset = cx1 + scale_x * x_axis;
y_offset = cy1 + scale_y * y_axis;

setcolor(WHITE);

/* Title for cross section */
if(x_axis != x_centroid)
    {outtextxy(cx1 - EXTENSION, cy6 - 2*EXTENSION-15, "BEAM CROSS SECTION");}
else {outtextxy(cx1 - EXTENSION, cy6 - EXTENSION-15, "BEAM CROSS SECTION");}

```

```

/* Title for beam deflection */
text(.54, .76, "EXAGGERATED BEAM DEFLECTION (simple supports)");

setcolor(CYAN);
if(draw_beam == I_BEAM) {
    moveto(cx1, cy1);
    lineto(cx6, cy1);
    lineto(cx6, cy2);
    lineto(cx4, cy2);
    lineto(cx4, cy5);
    lineto(cx6, cy5);
    lineto(cx6, cy6);
    lineto(cx1, cy6);
    lineto(cx1, cy5);
    lineto(cx3, cy5);
    lineto(cx3, cy2);
    lineto(cx1, cy2);
    lineto(cx1, cy1);
}

if(draw_beam == CHANNEL_BEAM) {
    moveto(cx1, cy1);
    lineto(cx6, cy1);
    lineto(cx6, cy2);
    lineto(cx2, cy2);
    lineto(cx2, cy5);
    lineto(cx6, cy5);
    lineto(cx6, cy6);
    lineto(cx1, cy6);
    lineto(cx1, cy1);
}

if(draw_beam == SOLID_BEAM) {
    moveto(cx1, cy1);
    lineto(cx1, cy6);
    lineto(cx6, cy6);
    lineto(cx6, cy1);
    lineto(cx1, cy1);
}

if(draw_beam == HOLLOW_BEAM) {
    moveto(cx1, cy1);
    lineto(cx6, cy1);
    lineto(cx6, cy6);
    lineto(cx1, cy6);
    lineto(cx1, cy1);
    moveto(cx2, cy2);
    lineto(cx2, cy5);
    lineto(cx5, cy5);
    lineto(cx5, cy2);
    lineto(cx2, cy2);
}

if(draw_beam == L_BEAM) {
    moveto(cx1, cy1);
    lineto(cx6, cy1);
    lineto(cx6, cy2);
    lineto(cx2, cy2);
    lineto(cx2, cy6);
    lineto(cx1, cy6);
    lineto(cx1, cy1);
}

/* Draw beam for bending display */

```

```

line((int)(.60*xmax), (int)(.90*ymax), (int)(.60*xmax), (int)(.88*ymax));
line((int)(.90*xmax), (int)(.90*ymax), (int)(.90*xmax), (int)(.88*ymax));
if(fabs(beam_deflect/beam_length) > 0.00001) {
    de = beam_length/6.0*beam_deflect/fabs(beam_deflect);
    for(px = 0.0; px <= deflect_point; px = px + beam_length/200){
        py = ((de-de*pow(((deflect_point -px)/deflect_point), 2.0))/beam_length) *
(.1) + .88;
        putpixel((int)((.6+px/beam_length*.3)*xmax), (int)(py*ymax), CYAN);
        putpixel((int)((.6+px/beam_length*.3)*xmax), (int)((py+.02)*ymax), CYAN);
    }
    for(px = deflect_point; px <= beam_length; px = px + beam_length/200){
        py = ((de-de*pow(((px-deflect_point)/(beam_length - deflect_point)), 2.0))/
beam_length) * (.1) + .88;
        putpixel((int)((.6+px/beam_length*.3)*xmax), (int)(py*ymax), CYAN);
        putpixel((int)((.6+px/beam_length*.3)*xmax), (int)((py+.02)*ymax), CYAN);
    }
}
else {
line((int)(.60*xmax), (int)(.90*ymax), (int)(.90*xmax), (int)(.90*ymax));
line((int)(.60*xmax), (int)(.88*ymax), (int)(.90*xmax), (int)(.88*ymax));
}

/* Fill Beam Cross section if thick enough */
setfillstyle(LTSLASH_FILL, 7+beam_material);
if(((cx2-cx1) > 2) && ((cy1 - cy2) > 2)){
    floodfill(cx1 + 1, cy1 - 1, CYAN);
    floodfill(cx1 + 1, cy6 + 1, CYAN);
}

setcolor(RED);

/* Beam Length Dimension */
line(xmax*.60, ymax*.90 + 3, xmax*.60, ymax*.90 + EXTENSION);
line(xmax*.90, ymax*.90 + 3, xmax*.90, ymax*.90 + EXTENSION);
line(xmax*.60, ymax*.90 + 10, xmax*.90, ymax*.90 + 10);
sprintf(work, "%.3f in.", beam_length);
text(.61, .94, work);

/* Force Position Location */
offset = .30 * (beam_point / beam_length) + .60;
line(xmax*.6, ymax*.88 - 3, xmax*.6, ymax*.88 - EXTENSION);
line(xmax*offset, ymax*.88 - 3, xmax*offset, ymax*.88 - EXTENSION);
line(xmax*.6, ymax*.88 - 10, xmax*offset, ymax*.88 - 10);
line(xmax*offset, ymax*.88 - 3, xmax*offset -3, ymax*.88 - 8);
line(xmax*offset, ymax*.88 - 3, xmax*offset +3, ymax*.88 - 8);
sprintf(work, "%.3f in.", beam_point);
text(offset+.01, .84, work);
sprintf(work, "%.3f lb.", beam_load);
text(offset, .79, work);

/* Beam Width Dimension */
line(cx1, cy1+3, cx1, cy1 + EXTENSION);
line(cx6, cy1+3, cx6, cy1 + EXTENSION);
line(cx1, cy1 + 10, cx6, cy1 + 10);
sprintf(work, "%.3f in.", beam_width);
outtextxy(cx1 + 10, cy1+15, work);

/* Height and Thickness Dimensions */
settextstyle(SMALL_FONT, VERT_DIR, 4);
line(cx6+3, cy1, cx6+EXTENSION, cy1);
line(cx6+3, cy6, cx6+EXTENSION, cy6);
line(cx6+10, cy1, cx6+10, cy6);
sprintf(work, "%.3f in.", beam_height);
outtextxy(cx6 + EXTENSION, cy1-10-textwidth(work), work);

```

```

/* Draw Thickness Dimension for all but solid beam */
if(beam_type != SOLID_BEAM) {
    line(cx1-EXTENSION, cy1, cx1-3, cy1);
    line(cx1-EXTENSION, cy2, cx1-3, cy2);
    line(cx1-10, cy1, cx1-10, cy1+15);
    line(cx1-10, cy2, cx1-10, cy2+15);
    sprintf(work, "%.3f in.", beam_thick);
    outtextxy(cx1-15, cy2-10-textwidth(work), work);
}

/* Principal and centroid y axis dimensions */
line(cx1-EXTENSION, cy1, cx1-2*EXTENSION, cy1);
line(cx1-EXTENSION-3, y_offset, cx1-2*EXTENSION, y_offset);
line(cx1-EXTENSION-10, cy1, cx1-EXTENSION-10, cy1+15);
line(cx1-EXTENSION-10, y_offset, cx1-EXTENSION-10, y_offset+15);
sprintf(work, "%.3f in.", y_axis);
outtextxy(cx1-EXTENSION-15, y_offset-10-textwidth(work), work);
if(y_axis != y_centroid) {
    line(cx1-2*EXTENSION, cy1, cx1-3*EXTENSION, cy1);
    line(cx1-2*EXTENSION-3, y_c_offset, cx1-3*EXTENSION, y_c_offset);
    line(cx1-2*EXTENSION-10, cy1, cx1-2*EXTENSION-10, cy1+15);
    line(cx1-2*EXTENSION-10, y_c_offset, cx1-2*EXTENSION-10, y_c_offset+15);
    sprintf(work, "%.3f in.", y_centroid);
    outtextxy(cx1-2*EXTENSION-15, y_c_offset-10-textwidth(work), work);
}

settextstyle(SMALL_FONT, HORIZ_DIR, 4);

/* Principal and centroid x axis dimensions */
line(cx1, cy6-3, cx1, cy6 - EXTENSION);
line(x_offset, cy6-3, x_offset, cy6-EXTENSION);
line(cx1-EXTENSION, cy6-10, cx1, cy6-10);
line(x_offset, cy6-10, x_offset+EXTENSION, cy6-10);
sprintf(work, "%.3f in.", x_axis);
outtextxy(x_offset + 10, cy6-EXTENSION, work);
if(x_axis != x_centroid) {
    line(cx1, cy6-EXTENSION-3, cx1, cy6 - 2*EXTENSION);
    line(x_c_offset, cy6-EXTENSION-3, x_c_offset, cy6 - 2*EXTENSION);
    line(cx1-EXTENSION, cy6-EXTENSION-10, cx1, cy6-EXTENSION-10);
    line(x_c_offset, cy6-EXTENSION-10, x_c_offset+EXTENSION, cy6-EXTENSION-10);
    sprintf(work, "%.3f in.", x_centroid);
    outtextxy(x_c_offset + 10, cy6 - 2*EXTENSION, work);
}

/* Draw Center Line */
setcolor(YELLOW);
setlinestyle(CENTER_LINE, 0, NORM_WIDTH);
line(x_offset, cy1+5, x_offset, cy6-5);
line(cx1-5, y_offset, cx6+5, y_offset);

/* Draw Centroid Line */
setcolor(MAGENTA);
setlinestyle(DASHED_LINE, 0, NORM_WIDTH);
line(x_c_offset, cy1+5, x_c_offset, cy6-5);
line(cx1-5, y_c_offset, cx6+5, y_c_offset);
setlinestyle(SOLID_LINE, 0, NORM_WIDTH);

/* Restore original Color */
setcolor(WHITE);
}

```

```
void kill()
/*
 * Close Graphics Routines
 *
 * This will deinitialize the graphics routines initialized earlier.
 *
 * January 20th, 1989.
 */
{
    closegraph();
}
?
```


9. PROGRAMMING IN JAVA

9.1 OVERVIEW

- Basically a compact, object oriented language that is compiled to a machine independent form. This is then translated before execution for specific machines. The language has similarities to C/C++, and has embedded graphics and networking functions.
- This language uses the concept of a virtual machine to compile code. This provides some consistency when translating from the machine independent code into locally executable programs. This makes the code machine independent, and it also allows various protection mechanisms for the local machine, such as virus corruption, and read/write protection of local files.
- Java was originally conceived as a language for embedded controllers (and in a way still is). But more recently it has been adapted to the World Wide Web allowing an interactive extension to the normally reactive HTML.
- When used with HTML, Java programs are referred to as applets. These applets are binary coded (in a machine language for the virtual machine) and are translated to local machine language after downloading. The applets are also checked for corruption using code check values, and checked for any operations that might be illegal locally.

9.2 THE LANGUAGE

- The programs are constructed out of objects - in a simplistic explanation each object is like a small subroutine, the main difference is that the object structure defines a hierarchy of functions, and allows functions to share and hide data and other functions in creative ways.
- Variables in Java are scoped to their declaration, and are static, but objects can be referred to dynamically.
- The Java source is compiled into 'byte codes', or a generic machine language, and then stored in '.class' files. The byte codes are then translated into local machine code, with local extras added in.
- The Java applets are architecture neutral, and can be recompiled to other architectures. Machine specific details are considered using libraries. (using the prefix 'java.'). An author may include none, some, or all the libraries in their programs. Although not consistent with the objectives of the language, customized libraries can also be added for special functions.
- Java uses "threads" to allow a program to split into parallel streams of execution. On a simple

machine this won't speed up execution, but will simplify programming.

- In Java there are no pointers, as a result the memory can be rearranged at times (stored variables and arrays can be moved in memory to be more compact. This “garbage collection” process is done automatically.
- Local code written in other languages can be used with the ‘native’ command.
- A simple example of an applet embedded in an HTML document is given (HTML is not the only way to run an applet)

```
<HTML><HEAD>
<TITLE=“A Java Applet”>
</HEAD>
<BODY>
<H1>Java Applet Below</H1>
<APPLET code=applet_name.class width=200 height=200>
<PARAM> name=file value=.cshrc>
<PARAM name=read_write value=read>
</APPLET>
</BODY></HTML>
```

- Applets can be organized using subdirectories, this is a good idea if there are many pieces. A good idea is have a subdirectory for each class/package. Under these have subdirectories for images, sounds, etc.
- Like most languages Java requires a compiler “javac” and then a separate viewer.

- Basic data types are,
 - boolean - a 1 bit true or false value
 - char - a 2 byte character
 - byte - a 1 byte integer
 - short - a 2 byte integer
 - int - a 4 byte integer
 - long - an 8 byte integer
 - float - a 4 byte floating point number
 - double - an 8 byte floating point number

- arrays (Note: No pointers) are defined as shown below, and have bounds checking when used.

```
int a[] = new int[10];
int[] a = new int[10];
int a[][] = new int[10][10];
int[][] a = new int[10][10];
int a[10] = {0,1,2,4,8,16,32,64,128,256};
int a[3][4] = {{0,1,2,4},{8,16,32,64},{128,256}};
```

- Basic syntax is like C/C++,

- comments use `/*...*/`, or `//`
 - statements end in semicolons `;`
 - curly brackets start and end program blocks `{.....}`
 - `if` statements use boolean arguments (but no others), for example,
 - `if(A && B)` is OK
 - `if(A == 0)` is OK
 - `if(A = 0)` is NOT OK
 - `if...else` statements are allowed
 - `while` and `do.....while` statements are allowed
 - `for` loops are used
 - `switch` statements are used
- classes are required for all code,
 - `abstract` - for a class that is not directly implemented
 - `final` - not allowed to be a subclass
 - `public` - the code can be used by other classes outside the package - the file. This class name must also be the name of the file `<ClassName>.java`
 - `private` - can only be used by code in the same file.
 - `<empty>` - if no access specifier is given (public or private), the class can be accessed from within the current package.
 - `synchronizable` - this function allows arguments that are instances of itself to allow synchronization.
 - Functions in classes (Methods) can be modified using,
 - `public` - this function can be called by any other.
 - `protected` - this function can only be called by subclasses.
 - `private` - this function can only be called by peers in the same class.
 - `<blank>` - can be called by functions in the same package.
 - `final` - no other methods can be used to override.
 - `static` - a generally shared function by all subclasses started with `<class-name>.method_name`.
 - `synchronized` - a lock will be applied and released on exit to prevent deadlock situations.
 - `native` - allows non-Java code to be used (e.g., a `'C'` function)
 - exceptions are one function not found in C/C++ and are intended to help trap errors. These exceptions are generated with `'throw'` statements when a problem occurs. The results of a throw statement is then considered in another function by a `'try...catch'` block,


```
try{
    // executed fully if no exception thrown
    // If exception occurs run best match catch
} catch (exception e1){
    //the best match for the exceptions will be found
} catch (exception e3){

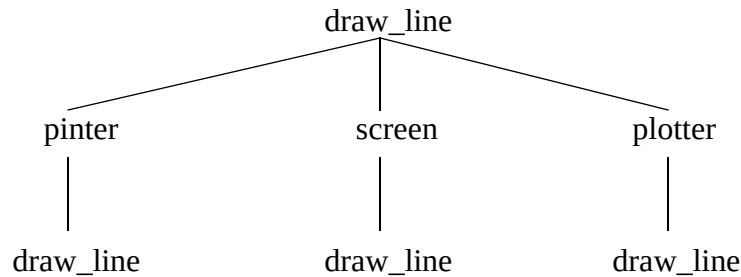
} finally {
    // do this whether or not there was an exception
```

}

- Recall, for class definitions, there is one function with the same name as the class (called the constructor) that is executed whenever a new instance of the class is created.
- functions may have the same name, but different arguments, this allows function overloading.
- the 'this' variable can be used to get a self reference to a function.
- There are no header files to include, but other classes or packages can be added using the 'import' function.
 - java.lang - the basic language classes (automatically imported)
 - java.io - allows input and output to other devices
 - java.util - utilities such as hash tables and vectors
 - java.net - network read/write functions
 - java.awt - GUI toolkit (e.g. windows)
 - java.applet - interfaces for WWW or other browsers
 - java.* - imports all
- 'interface' is a special command that allows inheritance without writing a new class. An interface can also be used to 'extend' a class.
- we can define classes that 'implement' one or more interfaces.
- Threads allow multiple streams of execution in parallel. Basically, if a program has to do more than one task, the traditional approach would be to do one for a while, then the other. But if at some point the program execution can be split (into threads) then execution will continue without paying attention to task switching. To do this a 'Thread' is initiated.
- If a class implements 'Runnable' it should have a function called run (needed for threads among other things).
- If running parallel threads, and they share common resources/data, we might take precautions to ensure that the two don't read/write at the same time. We can do this by declaring potentially conflicting functions as 'synchronized'. Care must be taken to avoid deadlock when a 'synchronized' function causes a monitor to lock a device/data needed by another 'synchronized' function. (these two or more functions would have to try to lock two or more resources).

9.3 OBJECT ORIENTED PROGRAMMING

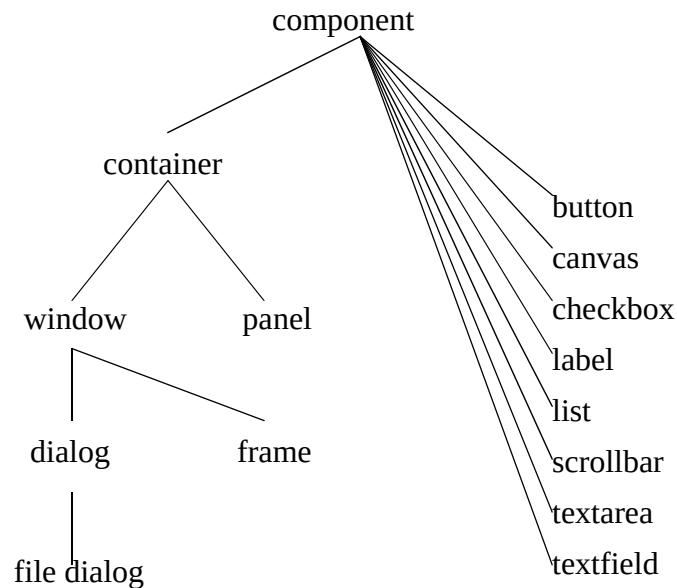
- Inheritance allows us to write one generic class (function) that depends on other functions to deal with special cases (this can be done in other languages with function calls, but this is more consistent)



- an applet (note a special type of Java program) needs to have four functions,
 - init() - called when the applet is first loaded
 - start() - called when an applet becomes visible in the browser
 - stop() - called when the applet is no longer visible
 - destroy() - called when leaving a browser page
 - * init() is always called first, and then start() and stop() will be called while the document using the applet is alive. If the document is exited, then stop() will be called if not already, and destroy() will be called last before exiting.
- We can also call out to the parent HTML document. Generally, these calls deal with URL's and parameters.
 - URL getDocumentBase() - returns the URL for the HTML document
 - URL getCodeBase() - returns the URL of the applet
 - String getParameter(String name) - for the parameter with the 'name', the string value is returned
 - URL A = new URL("http://www.computer.edu"); - define a URL
 - URL B = new URL(A, "directory/file.ext"); - define a new URL by extending the existing URL A
- Various types of media can also be included using simple functions.
 - Image getImage(URL) - will set up a link to an image at a remote site to be used when the image is drawn.
 - Image getImage(URL, filename) - an overloaded version that allows the filename string to be separated from the URL.
 - AudioClip getAudioClip(URL) - will retrieve (but not play yet) an audio file. Functions available include,
 - play() - play the clip
 - loop() - play the clip continuously
 - stop() - stop playing the current clip.
 - AudioClip getAudioClip(URL, file_name)
 - void play(URL) - plays an audio file directly
 - void play(URL, file_name)
- Applets can interact with the browser that has called it,
 - AppletContext getAppletContext() - gets a structure that is required by other functions.
 - Applet getApplet(String filename) - retrieves an applet using an HTML parameter.

Enumeration `getApplets()` - returns applets on the current page.
`void show Document(URL)` - allows a new document to be loaded in the browser.
`void showStatus(String)` - the string is printed to the screen.
`String get AppletInfo()` - Can be used to return information about the applet.
`String[][] getParameterInfo()` - used to inform the browser what parameters are allowed.

- GUIs can be constructed using the 'awt' toolkit. This allows the user to work with a variety of interface tools. The hierarchy is shown below.



Component - the basic class to represent the position & size of objects

Container - a class that can hold other classes

Panel - container in a container for organizing objects

Window - a rectangular area on the GUI

Frame - a window with a border added

Dialog - will receive input from the user

FileDialog - a file selection box

Button - when clicked on with a mouse will cause an action

Canvas - an area generally for free form I/O like lines and mouse events

Checkbox - will allow toggled or checked inputs

Label - a printed string

List - a scrolling list of strings

Scrollbar - can be attached to canvases to scroll

TextArea - a text editing window

TextField - a single text input line

- Components, and all objects in the hierarchy, can use the following functions.
 - Dimension Size() - returns width and height
 - Rectangle bounds() - return x, y, width, and height values
 - void enable() - sets a component to accept user input
 - void disable() - turns off user input to a component
 - void show() - makes a component visible
 - void paint(Graphics g) - a function called to redraw a component
 - void repaint() - requests that a component be redrawn
 - void update(Graphics g) - a level above a call to paint, might allow other function to be added
 - boolean mouseEnter(Event e, int x, int y) - called when the mouse has entered a component
 - boolean mouseExit(Event e, int x, int y) - called when the mouse leaves a component
 - boolean mouseMove(Event e, int x, int y) - reports a move inside a component with no buttons pressed
 - boolean mouseDrag(Event e, int x, int y) - called when a mouse button is held down, and the pointer dragged inside a component
 - boolean keyDown(Event e, int key) - reports a keyboard event inside the component
 - boolean handleEvent(Event e) - The normal event handler for component events.
 - Dimension preferredSize() - returns the idea component size
 - Dimension minimumSize() - returns the smallest size that a component can be.
- The container class also has a variety of associated functions (as well as its children)
 - void add(Component) - add a component
 - void add(String name, Component) - a component is added, with a name as well.
 - void setLayout(Layout Manager) - this automatically positions components

9.4 REFERENCES/BIBLIOGRAPHY

vanHoff, A., Shaio, S., Starbuck, O., Hooked on Java, Addison Wesley Developers Press, 1996

10. DATABASES

- Basically databases are used to store information.
- A primitive form of 'database' is a filing cabinet.
- The fundamental types of databases include,
 - Relational - Data is kept in fixed tables with defined relationships
 - Object Oriented - Dissimilar data can be stored in a single list (relational databases are a sort of subset of these databases)
- Databases allow information stored on one computer to be shared with other computers.
- Data stored in relational databases is commonly access using SQL (Structured Query Language)
- Databases handle problems of,
 - data locking (only allow one user to modify at once)
 - data sharing (other users can view)
 - searching
 - etc

11. MESSAGE PASSING ON NETWORKS

A Manual for Interprocess Communication with the MPS (Message Passing System)

Prepared by Hugh Jack
The University of Western Ontario
London, Ontario, Canada
hugh@engrg.uwo.ca

Keywords: Distributed Processing, Concurrent, Interprocess Communication, Message Passing, Course Grain, Asynchronous, Client-Server, Heterogenous, Network

Abstract

An asynchronous message passing system has been developed to allow integration of many dissimilar tasks. In particular, the tasks run on a number of different hardware platforms, using a number of different software packages and languages. The system is based on a single server which handles communication between all clients.

A new concept of groups and priorities is used to organize message flow. This structure also allows clients to sign on and off in an ad-hoc manner while isolating disturbances to other clients. The groups and priorities concept makes it possible to explicitly define a number of concurrent and parallel execution methods. Thus, the method described in this paper addresses the needs for integrating tasks in a typical computing environment.

1.0 Introduction

The Message Passing System (MPS) is intended to simplify communication between more than two programs. The programs may be resident on many computers, and are not necessarily writ-

ten in C or for a UNIX machine. In real terms, the MPS communication structure can be seen in Figure 1 below. The normal method of communication is also shown below.

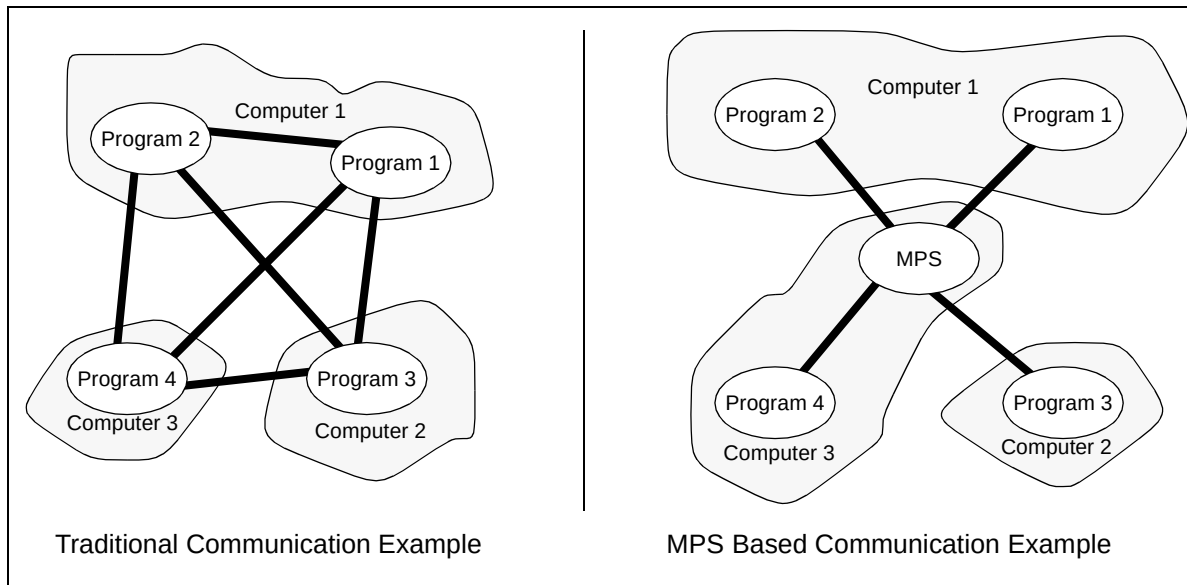


Figure 1 - An Example of Network Based Interprocess Communications

While the traditional method lends itself to speed, it requires that each program have sophisticated functions for communication and an interface to each of the other programs. The MPS scheme allows the sophisticated functions to be present in only the central server (i.e., MPS), and the same interface subroutines to be reused for each program.

1.1 The Objectives of MPS

The MPS system has been developed with a number of idealized objectives. When the system was developed, most of the objectives were met (or will be in the near future). The list below summarizes the objectives.

- Near real time response, in the range of one second or less. (This approach is not intended to compete with real-time systems).
- Buffering of messages between processes.
- General knowledge of processes, but no specifics, or direct access to remote data.
- Maintenance major methods of concurrent and distributed processing.
- Fault tolerance.

- Allows insertion into communication loop for testing analysis, and other processes.
- Allows message broadcast to unknown program.
- Keeps running record of transactions for forensics, and restarting, in case of failure.
- Does not try to replace existing communication methods, but allows coexistence.
- Independent of machine type, language used, and the details of the network.

The objectives above also double as advantages. But, as a result of these functional requirements, we were forced to use a central server for the system. Otherwise, a decision to incorporate these functions into each program would have been a very difficult (if not impossible) task.

The decision to use a central communication server is not ideal. Using a central server builds a bottleneck into the system. It means that only one program may be served at once, and the speed of the server will limit the communication rate. Alternately, if each program was to have its own interface, the programs could communicate simultaneously, and only be limited by their own speeds, and network transmission delays.

This paper will first attempt to put the work in perspective, and then will develop the MPS model by explaining each feature individually.

1.2 A Short Note on Concurrent and Parallel Processes

There are two approaches to splitting up large programming problems; concurrently and parallelism. The major difference between the approaches is in how they divide problems. Concurrent solutions involve many similar processes performing the same operation. In parallel solutions various processes are used to solve different parts of a problem.

An example of a concurrent solution to a problem is using a secretarial pool to have each secretary type one chapter of a book, concurrently. An example of a parallel solution to a problem is to give one copy of the book to a fact checker, and another to a proof reader, so that they may work in parallel. The use of parallel and concurrent solutions should be viewed as complementary approaches. In the MPS, concurrency is automatic when two or more clients with the same group and priority are run. Parallelism is established by groups, with various functions per-

formed by each group. It is up to the user to determine how the programming problem can be split up and coordinated.

1.3 Related Material

Standard programming methods have focussed on sequential programming techniques. This is understandable because of the ease of hardware design, and the appeal of linear logic to the programmer. As computers become more popular as a tool, the bottleneck of a single instruction stream becomes more critical. Thus, researchers began to examine computing with multiple instruction and data streams [Sikorski, 1991]. In particular computing with multiple instruction streams has become very interesting. Researchers have tended to split into two sections, those which develop multi-processor computers for single large problems, and networked computers for multiple and distributable problems. At present many computers are connected by networks on a world wide scale. The enormous potential for computing power makes the value of the network obvious, but without the proper programming techniques the power is lost.

When discussing a concurrent process (distributed on two or more computers), there are two main branches of discussion. In a tightly coupled system the processes use a technique such as shared memory. In a loosely coupled system message passing is used. Passing information and requests by messages allows isolation between processes which might have interaction problems, but this often results in more complex hardware. When message passing, the messages may be characterized in the form [Andrews, 1991],

- asynchronous
- synchronous (good for real time systems)
- RPC or Rendezvous

Each of these has various merits, but in particular the asynchronous methods have great advantage. These were the only methods that would make it easy to cut the connections between two processes.

There are many methods which use asynchronous message passing, but the majority are in languages such as Actors [Agha et. al., 1987], and NIL [Strom et. al., 1983]. The language approach has some inherent problems with the flow of control. The entire concurrent system needs one start point, and will not tolerate two competing main programs, such as user interfaces. This also poses a greater problem when a process is started or stopped during the system execution. The largest advantage to the language based approach is speed, ease of use for one task, and reliability (excluding faults). This may readily be seen in work by Buhr et. al. [1991], Otto [1991], Grimshaw [1991], and Birman [1991]. These researchers use a variety of techniques for programming and communication, but essentially provide embedded tools for distribution and parallelization. Another interesting approach was developed by Singh et. al. [1991] in their Frameworks system. Although their system is based on an RPC method they allow a user to write fairly independent tasks, then fit them together with a graphical tool. The interesting part of their system was a process group structure, which treated all processes as concurrent or non-concurrent, and allowed input and output conditions to be defined. If the reader looks beyond the superficial differences, they will see an underlying philosophy which is common between their work, and the work presented here.

1.4 A Historical Note on MPS

During development of the workcell controller in 1990 (By H.Jack) it was discovered that the control of the workcell was difficult to do with a single program. This set the tone for exploring concurrent programs and communication. The first attempts were done with files, with a degree of success. This approach was basically a file driven semaphore system [Jack and Buchal, 1992]. This had weaknesses, and depended heavily on a common disk. Also, the communications were very slow, difficult to implement, and hard to change. As a result, other methods, like sockets, were examined for networked communication. It was decided that these had many of the same problems of the file transfer mechanism. The concept of MPS was developed in a crude form in the late summer of 1990.

A crude implementation of MPS was developed in early 1991. This version only had simple communication. After proof of this implementation it was discussed with other researchers. It was

discovered that it would have applications to the CAPP project (described in Section 6), in addition to the original Workcell control. The MPS model was improved, and implementation continued in the workcell. At the same time Jimmy Chien was examining the use of the MPS for the CAPP project, along with Hugh Jack. Some of the issues raised by the CAPP project helped bring depth to the MPS implementation. By the Fall of 1991 the MPS system had been fully developed, and had been used in a ray tracing application, and the workcell controller. These implementations were both successful, and lead to the evolution of the MPS model. Another Ray Tracing Application was done by Patrick Surry, when he tied ACIS, and PV-Ray together. He again exposed some short falls in the system, and new features were added. By this time the system was well developed, although some features were (and are) still in planning stages.

This moves to the final application of the MPS to the CAPP/PPC Integration project. There were three competing methods presented for the integration of CAPP and PPC. Two methods were based on using message passing systems which utilize databases (one relational, and the other object-oriented), and the third method used the MPS. The original project decision was to pursue the use of an existing Oracle Database message passing system provided by the German team from IPA, Stuttgart, Germany. After some time in development the problems involved with using Oracle became obvious, and MPS was selected as the method event transfer between CAPP and PPC. This was encouraging from the perspective that the MPS was originally intended for control only, but was so appealing that it replaced an existing (and proven) approach to handle events.

The future of this software is that it will be developed to be one of the primary mechanisms in systems development for a new Design Engine Being developed at UWO, as part of a Strategic NSREC Grant.

2.0 The MPS Model of Interprocess Connection

The actual connections between programs is of little importance in the MPS approach, therefore the reader should ignore it. The graph below contains an example of interprocess communica-

tion based on an MPS approach.

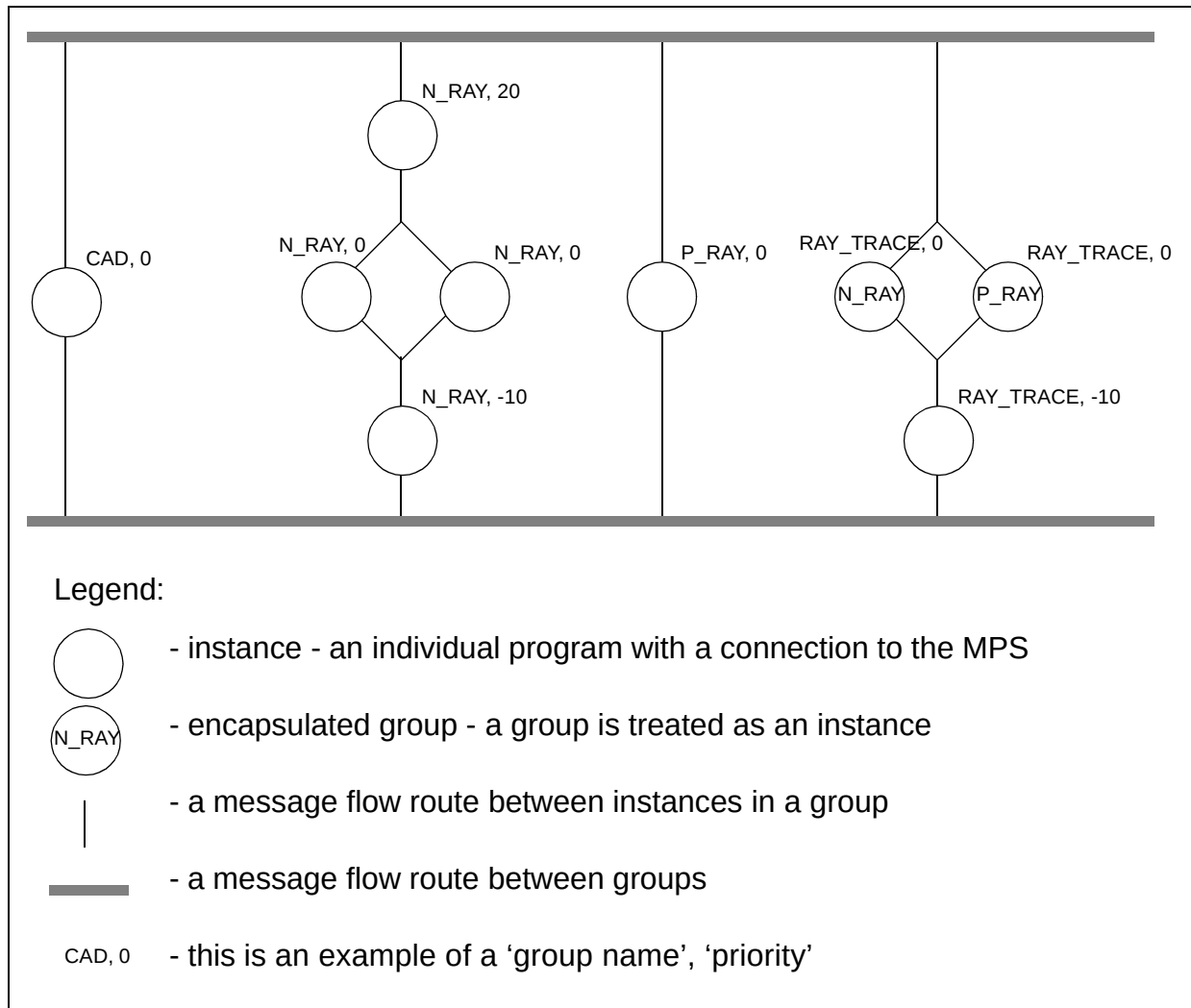


Figure 2 - An Example of an MPS Graph for CAD Driven Ray Tracing

Each instance (a program connected to the MPS) belongs to a group. And, within the group each instance has a priority. For example, Figure 2 shows four groups, one of which is 'N_RAY'. In the group 'N_RAY' there are four instances, one at priority 20, two at priority 0, and one at priority -10. Therefore, when a message is passed to this group, the priority 20 client will be the first to receive it. This client may pass the message unchanged, or change it, or not pass it at all. If any message comes out of 'N_RAY, 20', then it will be available for pickup by 'N_RAY, 0'. Since two programs have the same identification, they are concurrent, and either one can pick up the message (on a first come first served basis). A message written out by

either 'N_RAY, 0' will then go to 'N_RAY, -10'. After a message leaves 'N_RAY, -10' it will be free to be passed to another group. Each message is given a source and destination group as it travels through the graph (and will always have these, although they may be changed).

This graph is updated and changes as new instances are initialized and deinitialized. For example, if 'P_RAY, 0' has not been able to keep up with the demand, the user only needs to start the same program on another computer. There would now be two 'P_RAY, 0' programs running, and processing. If the bottleneck goes away, either program may be stopped, and the MPS graph would again appear as in Figure 2.

In Figure 3 an example program is shown which initializes itself as 'N_RAY, 0', finds the reference number for the 'CAD' group, and then sends a message to it. The program then deinitializes from the MPS and exits.

```

#include <stdio.h>      /* The usual assortment of headers */
#include <stdlib.h>
#include <string.h>

#include "MPS.h"        /* Message board headers */

main()
{
    static int          client_N_RAY,
                       client_CAD;
    static client_info  CLIENT_N_RAY;
    static int          from,
                       to,
                       cmd,
                       error;
    static char          string[MAX_STRING_LENGTH];

    /* Set up data structure for MPS connection */
    error = mb_clear_client(&CLIENT_N_RAY, "/usr/people/hugh/software/host.IP");
    /* Make connection to MPS */
    if(error == NO_ERROR) error = mb_init_client(&CLIENT_N_RAY, "N_RAY");

    /* Get reference numbers for this group and the CAD group */
    if(error == NO_ERROR){
        while((client_N_RAY = mb_get_client_group(&CLIENT_N_RAY, "N_RAY")) == ERROR);
        while((client_CAD = mb_get_client_group(&CLIENT_N_RAY, "CAD")) == ERROR);
    }

    from = client_N_RAY; /* Identify that the message has come from N_RAY */
    to = client_CAD;     /* Flag that the message is going to group CAD */
    cmd = 101;           /* An arbitrary command number */
    strcpy(string, "Testing"); /* A string intended for the remote instance */

    /* Send the message */
    if(error != ERROR) error = mb_send_message(&CLIENT_B, from, to, cmd, string);

    /* Shut down the connection */
    mb_deinit(&CLIENT_B);
}

```

Figure 3 - An Instance 'N_RAY, 0' Passes a Message to Group 'CAD'

In the example above, the instance originates a message. In the example below, the instance 'N_RAY, -10' receives, alters and then passes the message along.

```

main()
{
    static client_info    CLIENT_N_RAY;
    static int            from,
                        to,
                        cmd,
                        error;
    static char           string[MAX_STRING_LENGTH];

    /* Set up data structure for MPS connection */
    error = mb_clear_client(&CLIENT_N_RAY, "/usr/people/hugh/software/host.IP");
    /* Set the priority of the instance to -10 */
    if(error == NO_ERROR) error = mb_set_priority(&CLIENT_N_RAY, -10);
    /* Make connection to MPS */
    if(error == NO_ERROR) error = mb_init_client(&CLIENT_N_RAY, "N_RAY");

    while(error == NO_ERROR){
        /* Wait for a message to arrive */
        if(mb_get_message(&CLIENT_N_RAY, &from, &to, &cmd, string) == NO_ERROR){
            /* change command with a simple increment */
            cmd++;
            /* Change the message string */
            strcpy(string, "Intercepted");
            /* Send the message */
            mb_send_message(&CLIENT_N_RAY, from, to, cmd, string);
            /* force program to stop */
            error = ERROR;

            } else {
                sleep(1);
            }
        }

        /* Shut down the connection */
        mb_deinit(&CLIENT_N_RAY);
    }
}

```

Figure 4 - An Instance for Message Interception and Replacement of Message from 'N_RAY, 0' (Note: Sleep() functions are used because such simple programs tend to overrun the MPS server with requests. The MPS has been able to serve from 5 to 100 requests a second depending upon platform and options used)

The reader should note at this point that each message is an arbitrary combination of integer and string. Their use is entirely decided by the instances which receive and send them. The reader should also note that the message passing mechanism between groups is only one mode available. This will be discussed more in the following pages.

2.1 Instances

Each instance must belong to a group, and have a priority. This determines when a message may be picked up by an instance. We encourage the use of a 0 level priority as the target in each group. By this it is implied that all instances with positive priorities act as preprocessors, and all instances with negative priorities act as post-processors. Moreover, the users are encouraged to leave divisions between priority numbers for easy insertion of new instances. This leads to a naturally modular system.

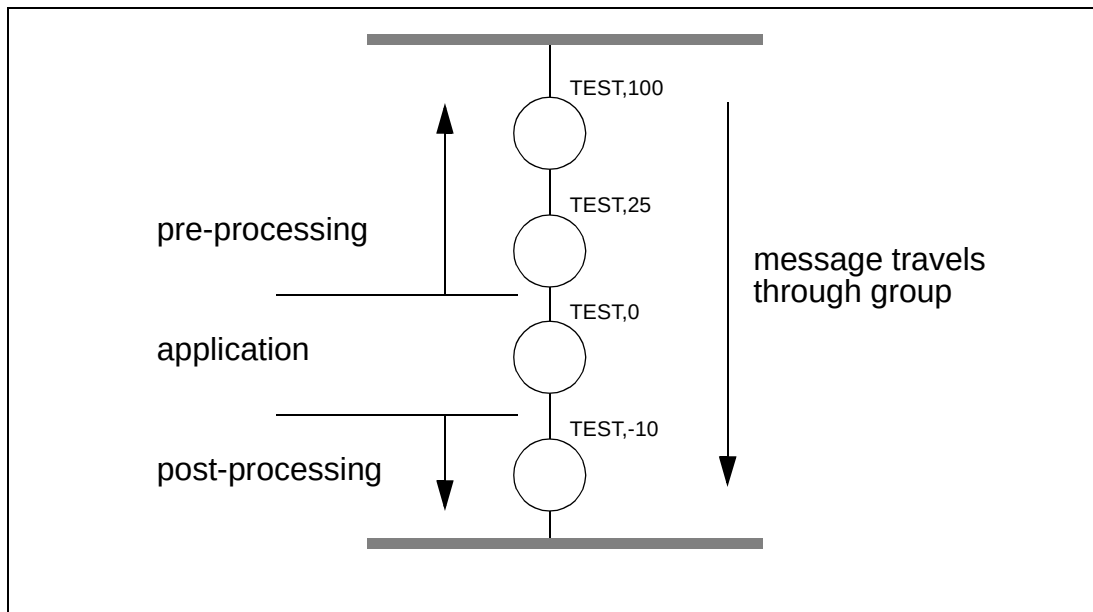


Figure 5 - An Example of Instances in a Group

2.2 Concurrent Instances

When two (or more) instances are run concurrently, they will receive and process messages as fast as possible. If the order of the message output from the concurrent instances does not have to match the order of message input, then there is no problem. In some cases instances it is necessary to maintain message sequence. Figure 6 illustrates the nature of this problem.

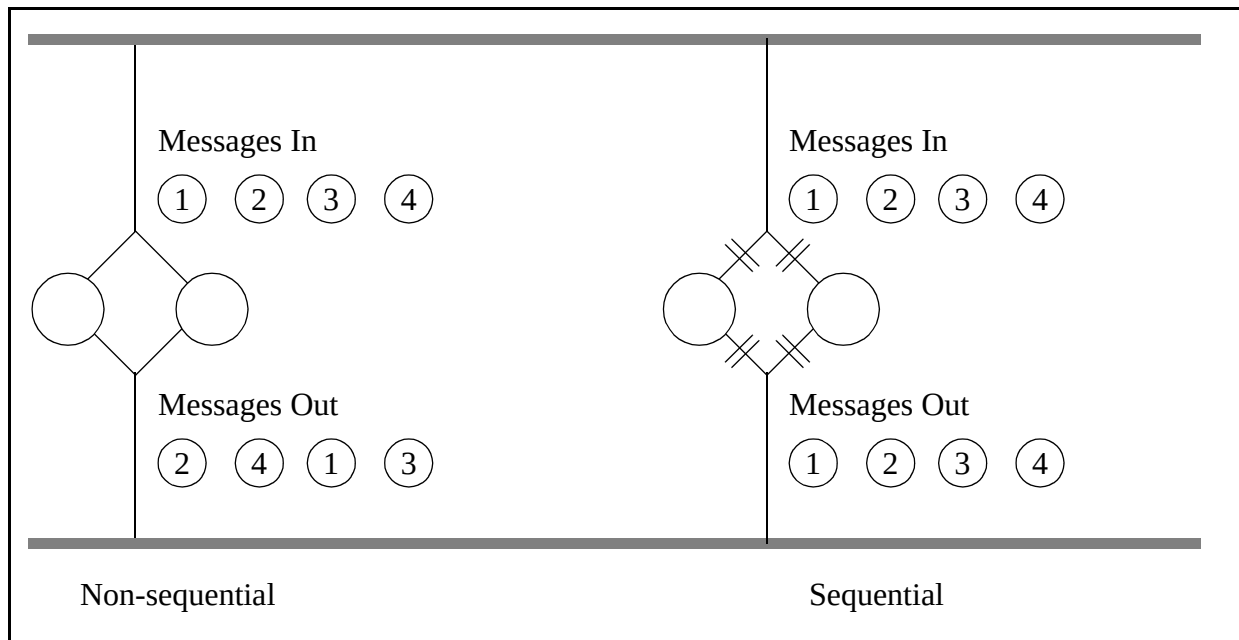


Figure 6 - The Problem of Sequence Preservation in Concurrent Processes

If the user does not require a sequential concurrency, then they need not do anything. The default is for all instances to be considered non-sequential (as sequential instances are more expensive in terms of MPS computation). If an instance (being one of a 'group, priority') must be sequential, the user may turn this on as seen in Figure 7.

```

main()
{
    static client_info    CLIENT_N_RAY;
    static int            from,
                        to,
                        cmd,
                        error;
    static char           string[MAX_STRING_LENGTH];

    /* Set up data structure for MPS connection */
    error = mb_clear_client(&CLIENT_N_RAY, "/usr/people/hugh/software/host.IP");
    /* Set the priority of the instance to -10 */
    if(error == NO_ERROR) error = mb_set_priority(&CLIENT_N_RAY, -10);
    /* Set the client to be a Sequential message passing instance */
    if(error == NO_ERROR) error = mb_set_sequence(&CLIENT_N_RAY, SEQUENT);
    /* Make connection to MPS */
    if(error == NO_ERROR) error = mb_init_client(&CLIENT_N_RAY, "N_RAY");

    while(error == NO_ERROR){
        /* Wait for a message to arrive */
        if(mb_get_message(&CLIENT_N_RAY, &from, &to, &cmd, string) == NO_ERROR){
            /* Both of the two choices below will allow a message to be */
            /* released when the MPS is waiting for a response so that it may */
            /* release messages which are waiting. Note that the read above */
            /* will have the same effect */
            if(cmd != 1){
                /* change command with a simple increment */
                cmd++;
                /* Change the message string */
                strcpy(string, "Intercepted");
                /* Send the message */
                error = mb_send_message(&CLIENT_N_RAY, from, to, cmd, string);
                /* force program to stop */
            } else {
                /* This will release the last message without having */
                /* to write out a message */
                error = mb_release_last_message(&CLIENT_N_RAY);
            }
        } else {
            sleep(1);
        }
    }

    /* Shut down the connection */
    mb_deinit(&CLIENT_N_RAY);
}

```

Figure 7 - Example of An Instance Which Has Declared Itself Sequential (and by default, its concurrent neighbors). For this instance to be run concurrently, the user just runs multiple copies of the program.

When a concurrent case is sequential, there are three events to consider. If one instance gets a message, and passes it on (by sending a message using “mb_send_message()”, then the MPS will keep the messages queued until it is their turn to be passed. This is transparent to the user.

If the instance decides not to pass the message, then it must release the message so that it does not block the other waiting messages. This is also shown in Figure 7 above. There are three actions which may release a message: writing a message; reading a new message; releasing the message. The main reason for having a sequential concurrency is when the messages must be maintained in order for programs which are incapable of sorting order, or time based messages. An example of this is a robot driver which must receive messages in order, or the robot will make the right motions in the wrong order, and thus fail.

2.3 - Direct Instance Connection

In some cases a message will have to pass directly between two instances. This has been allowed, although it violates the assumption that instances be ignorant of each other. As a compromise there have been three modes adopted, which are discussed below.

2.3.1 - General Broadcasts to All Instances

In the most general mode, any instance can issue messages to every other client on the message board. The same message is issued to every client, and takes priority over messages passed between groups. An example of this is seen below in Figure 8. The message is read by each process in a normal manner (although messages directly to an instance are read before messages between groups).

```

static client_info  CLIENT_N_RAY;
static int          to,
                   cmd,
                   error;

static char         string[MAX_STRING_LENGTH];

/* Set up data structure for MPS connection */
error = mb_clear_client(&CLIENT_N_RAY, "/usr/people/hugh/software/host.IP");
/* Make connection to MPS */
if(error == NO_ERROR) error = mb_init_client(&CLIENT_N_RAY, "N_RAY");

to = ALL_INSTANCES;
cmd = 57;
strcpy(string, "Hi there");
if(error != ERROR) error = mb_send_message_ins(&CLIENT_N_RAY, to, cmd, string);

/* Shut down the connection */
mb_deinit(&CLIENT_N_RAY);

```

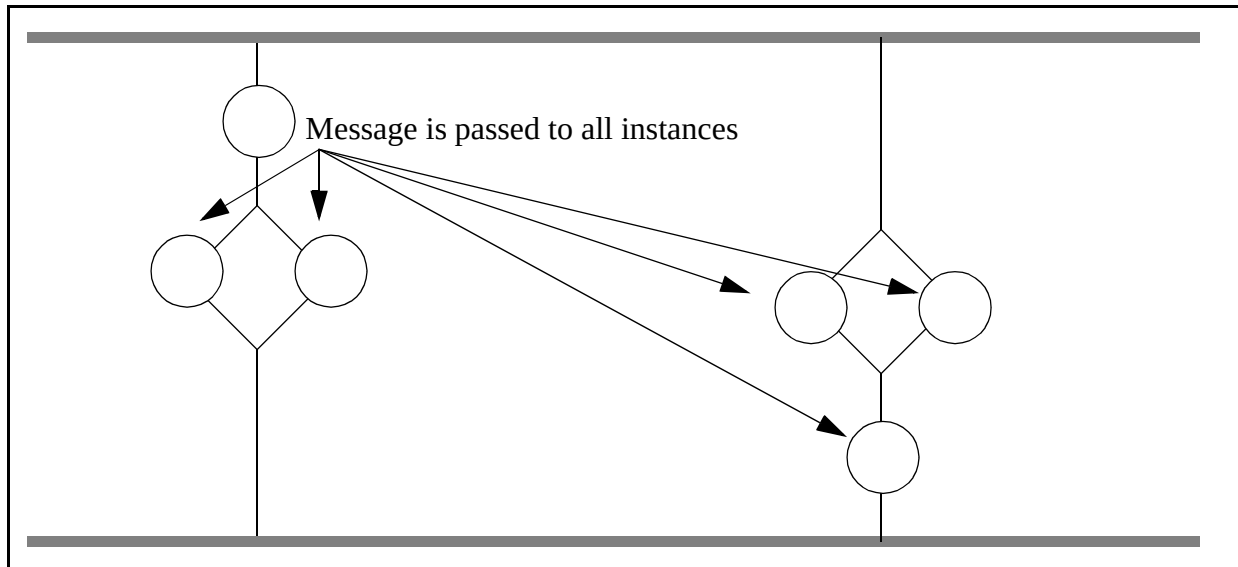


Figure 8 - An Immediate and General Broadcast of a Message to All Clients

This approach is somewhat naive, and the messages may be sent to unexpected instances (except for the sending instance).

2.3.3 - General Broadcast of Messages to All Instances in a Group

Another approach is to send the messages to all instances in a group. In this case the message trav-

els normally until it approaches the first instance in the destination group (this mode is not immediate, and thus allows post-processing). Upon reaching the destination group, a copy is sent to each member of the group. These messages are immediate, and have a priority over messages passed between groups. Figure 9 shows one such example of sending a message of this type.

```

static int      client_N_RAY,
                client_CAD;
static client_info CLIENT_N_RAY;
static int      from,
                to,
                type,
                cmd,
                error;
static char      string[MAX_STRING_LENGTH];

/* Set up data structure for MPS connection */
error = mb_clear_client(&CLIENT_N_RAY, "/usr/people/hugh/software/host.IP");
/* Make connection to MPS */
if(error == NO_ERROR) error = mb_init_client(&CLIENT_N_RAY, "N_RAY");

/* Get reference numbers for this group and the CAD group */
if(error == NO_ERROR){
    while((client_N_RAY = mb_get_client_group(&CLIENT_N_RAY, "N_RAY")) == ERROR);
    while((client_CAD = mb_get_client_group(&CLIENT_N_RAY, "CAD")) == ERROR);
}

type = ALL_INSTANCES_IN_GROUP;
from = client_N_RAY;
to = client_CAD;
cmd = 42;
strcpy(string, "Hi again");
if(error != ERROR)
    error = mb_send_message_special(&CLIENT_N_RAY, type, from, to, cmd, string);

/* Shut down the connection */
mb_deinit(&CLIENT_N_RAY);

```

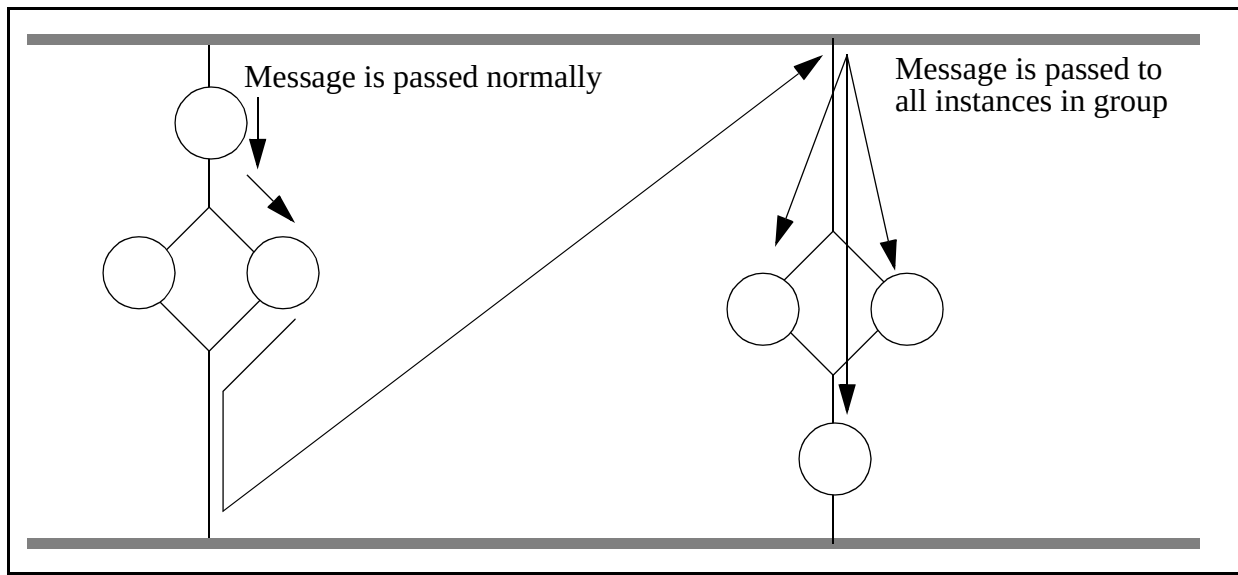


Figure 9 - A Broadcast to All Instances in a Group

This method works well when a group does not contain messages. If a message is between instances in a group when this message arrives, then it may result in a conflict, or the message may be lost. For example, a message to kill all instances would result in the loss of a message which is still active.

2.3.3 - Sequential Broadcast of Messages to All Instances in a Group

As described in the last section, when messages continue to flow through groups, the general broadcasts may not be desired. An alternative is to force the message to travel normally through the graph, but to be passed to all concurrent instances. This feature has not been implemented yet, but there are two cases which have been identified for implementation of this approach. In the illustration below, the two cases are shown comparatively (see Figure 10). The message is still addressed to a group, except that a message may be either blocking or non-blocking.

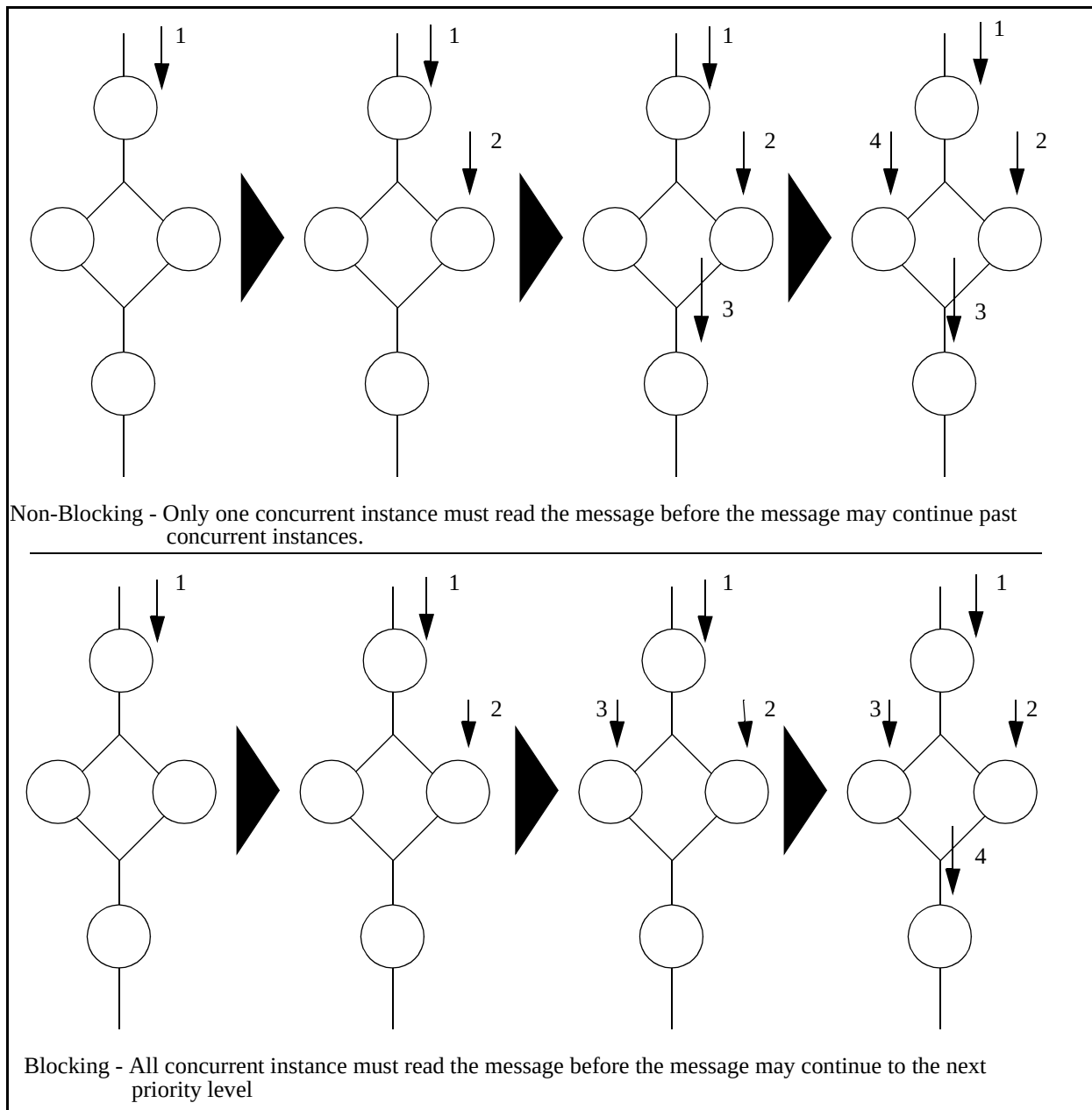


Figure 10 - An Example of Blocking and Non-blocking Broadcasts to all Instances in a Group (The message cannot be altered by any client in the group)

As seen in Figure 10, blocking broadcasts force the message to wait until all concurrent instances have received the message before the next lower priority instance may receive it. In both cases the same message is passed to all instances, only the order of reception may vary (by the blocking/non-blocking constraints). Here, higher priority is guaranteed to see at least one copy of the message before lower priority, while “broadcast to all instances” doesn’t guarantee this.

2.3.4 - Direct Broadcast of a Message Between the Clients

The last direct communication mode is the simplest to do. It mimics many existing methods. In this mode one instance sends a message directly to another instance. For this to happen, the sending instance must have a reference to the remote instance. This reference is a unique integer assigned to each instance. Each instance is only allowed to access its own instance number. Therefore, the sending instance must first get the destination instance number from the destination instance (Using normal group based communication techniques). While this is somewhat complicated, it ensures that integrity is maintained. (Note: An instance number is randomly assigned to each instance and therefore cannot be assumed to be the same between runs of a program.) Figure 11 below shows two programs which will perform a direct message pass. This is done after one gets the instance number of the other.

This Program sends out its instance number, and waits for messages

```
static client_info  CLIENT_B;
static int          client_a,
                  client_b,
                  from,
                  to,
                  cmd,
                  error;

static char        string[MAX_STRING_LENGTH];

error = mb_clear_client(&CLIENT_B, "/usr/people/hugh/software/host.IP");
if(error == NO_ERROR) error = mb_init_client(&CLIENT_B, "B");

if(error == NO_ERROR){
    while((client_a = mb_get_client_group(&CLIENT_B, "A")) == ERROR);
    while((client_b = mb_get_client_group(&CLIENT_B, "B")) == ERROR);
}

error = mb_send_message(&CLIENT_B, client_b, client_a,
    mb_instance_number(&CLIENT_B), "CONNECT");
while(error == NO_ERROR){
    if(mb_get_message(&CLIENT_B, &from, &to, &cmd, string) != ERROR){
        printf("from %3d -- to %3d -- cmd %3d -- string %s\n",
            from, to, cmd, string);
    } else {
        sleep(1);
    }
}

mb_deinit(&CLIENT_B);
```

This program waits for a message containing an instance number, then sends messages to it

```
static client_info  CLIENT_A;
static int          instance,
                  from,
                  to,
                  cmd,
                  i,
                  error;

static char        string[MAX_STRING_LENGTH];

error = mb_clear_client(&CLIENT_A, "/usr/people/hugh/software/host.IP");
if(error == NO_ERROR) error = mb_init_client(&CLIENT_A, "A");

while(mb_get_message(&CLIENT_A, &from, &to, &cmd, string) == ERROR) sleep(1);
instance = cmd;
for(i = 0; i < 10; i++) mb_send_message_ins(&CLIENT_A, instance, i, "DIRECT");

mb_deinit(&CLIENT_A);
```

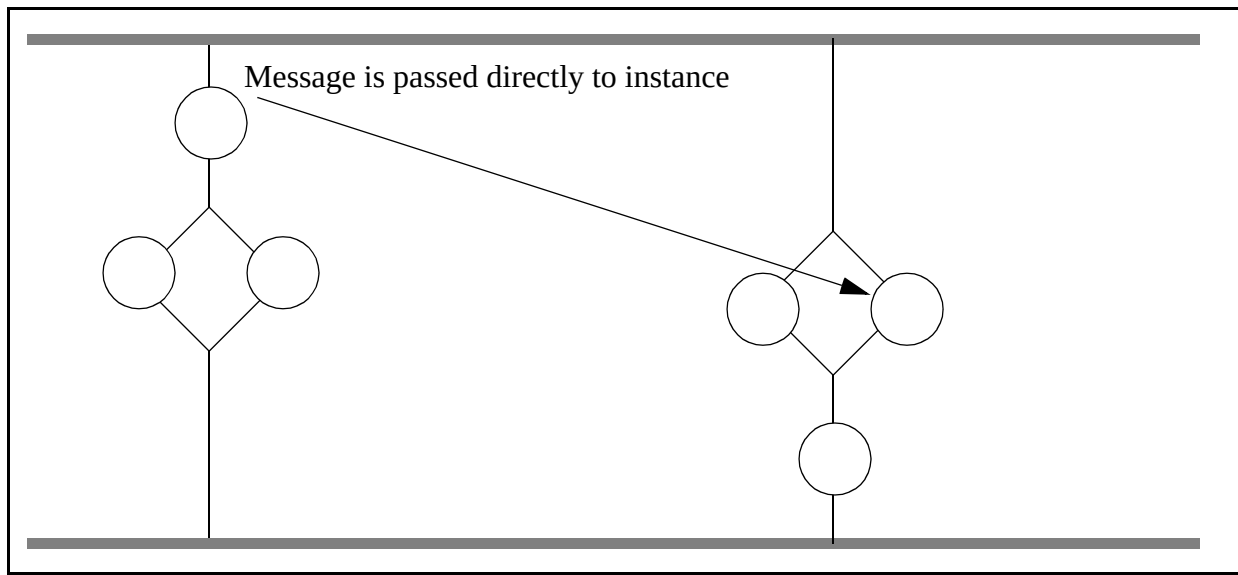


Figure 11 - Two programs which pass messages directly between each other (not via the MPS graph)

2.4 - Message Broadcasts To All Groups

While section 2.3 outlined a number of approaches which address messages to instances, it did not discuss a method for splitting messages to all groups. If a message must be sent to unknown (or all) groups, a broadcast to all groups is ideal. The example in Figure 12 shows an application of this technique.

```

static client_info  CLIENT_N_RAY;
static int          my_number,
                   from,
                   to,
                   cmd,
                   error;

static char         string[MAX_STRING_LENGTH];

/* Set up data structure for MPS connection */
error = mb_clear_client(&CLIENT_N_RAY, "/usr/people/hugh/software/host.IP");
/* Make connection to MPS */
if(error == NO_ERROR) error = mb_init_client(&CLIENT_N_RAY, "N_RAY");

if(error == NO_ERROR)
    while((my_number = mb_get_client_group(&CLIENT_N_RAY, "N_RAY")) == ERROR);

from = my_number;
to = ALL_GROUPS;
cmd = 555;
strcpy(string, "Beam me up");
if(error != ERROR) error = mb_send_message(&CLIENT_N_RAY, from, to, cmd, string);

/* Shut down the connection */
mb_deinit(&CLIENT_N_RAY);

```

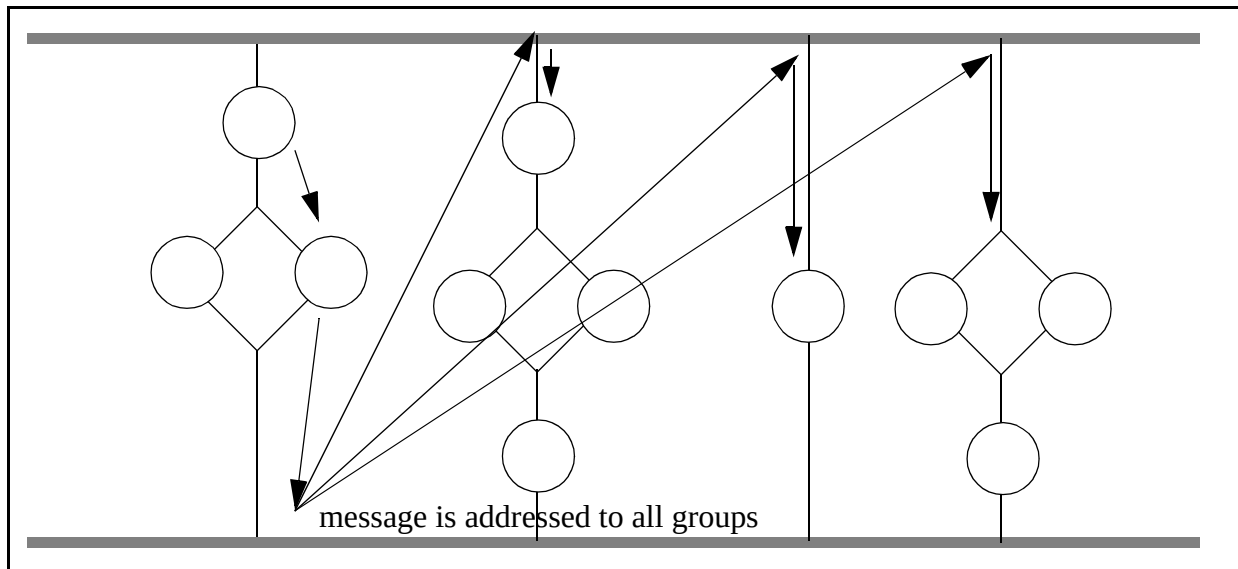


Figure 12 - A General Message Broadcast to All Groups

The reader will note that the message passes normally through all instances as if an individual message has been passed to each group individually.

2.5 Filtering

An instance may decide to limit messages accepted by filtering its input. This ability has been provided by allowing the user to select a list of accepted groups. When an instance has created this list, it will call for messages. If the list is set to 'ALL_GROUPS', all messages will be accepted normally. If one or more groups are put on the filter list, messages from all other groups will be held until the 'ALL_GROUPS' filter status is set again. While the filter is in use, messages from selected groups will be passed normally, but the other messages will be queued up. When the filter is removed, all the queued messages will be available to be read. The example shown in Figure 13 shows a program which sets a filter, then clears it.

```
static client_info  CLIENT_E;
static int          from,
                   to,
                   cmd,
                   i,
                   count = 0,
                   error;

static char        string[MAX_STRING_LENGTH];

error = mb_clear_client(&CLIENT_E, "/usr/people/hugh/software/host.IP");
if(error == NO_ERROR)
    error = mb_init_client(&CLIENT_E, "EXCLUSIVE_RESOURCE");

do{
    if(mb_get_message(&CLIENT_E, &from, &to, &cmd, string) !=ERROR){
        printf("mes # %3d -- fr %3d -- t%3d -- cmd %3d -- %s\n",
               count, from, to, cmd, string);
        count++;
        if(cmd == -100){
            mb_set_filter(&CLIENT_E, from);
        } else if(cmd == -200){
            mb_set_filter(&CLIENT_E, ALL_GROUPS);
        }
    } else {
        sleep(1);
    }
}while(cmd != 1000);

mb_deinit(&CLIENT_E);
```

Figure 13 - An Instance Which Sets and Clears a Group Based Message Filter

The reader will find this feature useful when attempting to solve the mutual exclusion problem. This problem arises when two or more groups share a resource, but they must have exclusive use when they are accessing it. This feature makes it possible to set up a locking mechanism. The robot driver is an excellent example. While one group is using the robot, all other groups

should be locked out. If two groups send a message to the robot at the same time, conflicting requests for motion are sure to happen. Thus, if one group is locked out, then the robot group will continue to receive a continuous and sensible stream of requests.

2.6 - Encapsulation

While the groups in the MPS are vertically symmetrical, this is not always a desirable thing. The approach chosen to overcome this is to make it possible to encapsulate groups as an instance. If the reader refers back to Figure 2, they will notice that groups 'N_RAY' and 'P_RAY' are encapsulated in group 'RAY_TRACE'. In this case the effect would be identical to replacing the instance with the group, as illustrated in Figure 14.

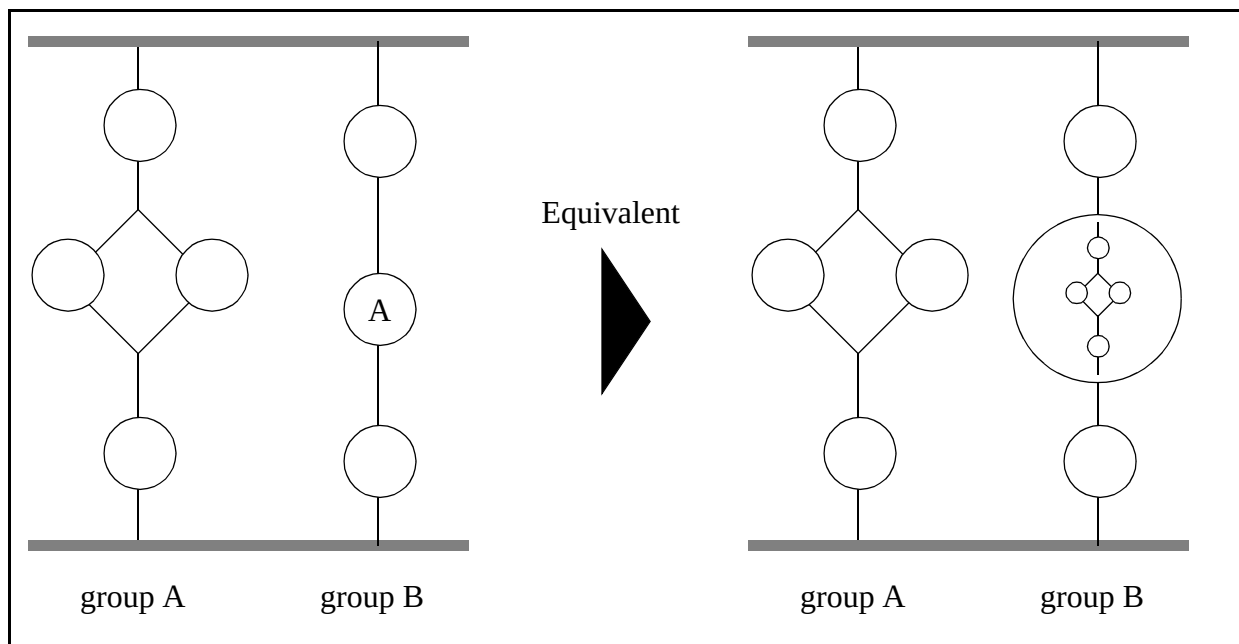


Figure 14 - Equivalent Representation of an Encapsulated Group

Noting that group 'A' is still available for use, making this approach that much more versatile. In Figure 15 below, there is an example of how to declare an encapsulated group, assuming the group exists on the MPS graph.

```

static client_info  CLIENT_ENCAP;
static int          from,
                  to,
                  cmd,
                  error;

static char         string[MAX_STRING_LENGTH];

error = mb_clear_client(&CLIENT_ENCAP, "/usr/people/hugh/software/host.IP");
if(error == NO_ERROR) error = mb_set_priority(&CLIENT_ENCAP, 10);
if(error == NO_ERROR) error = mb_init_client(&CLIENT_ENCAP, "CLIENT_ENCAP");

if(error != ERROR) error = mb_encapsulate(&CLIENT_ENCAP, 0, "CLIENT_ENCAP", "CLIENT_A");

while(cmd != 1000){
    if(mb_get_message(&CLIENT_ENCAP, &from, &to, &cmd, string) == NO_ERROR){
        printf("from %3d to %3d cmd %5d str %s\n", from, to, cmd, string);
        strcpy(string, "ENCAPED");
        mb_send_message(&CLIENT_ENCAP, from, to, cmd, string);
    } else {
        sleep(1);
    }
}

mb_deinit(&CLIENT_ENCAP);

```

Figure 15 - A Coded Example of Declaring an Encapsulated Group

The various general broadcasts will elicit different responses for encapsulated groups. If this is a general broadcast to members of a group, then the encapsulated group will be considered. In the case of a general broadcast to all instances, the encapsulated group instances will only receive one copy of the message.

3.0 Additional Features of MPS

The centralization of MPS allows some features not normally available in distributed systems. In particular the user may create a log file of transactions. When the MPS is run, the normal method is to begin logging all transactions. The user may then switch this off, if desired (see Figure 16). At present it only serves as a tool for forensic evaluations of activity. In the future it may be the basis for reconstructing the MPS if failure occurs. The user is encouraged to disable this feature when not debugging because long disk access times cause this feature to slow the MPS.

```

static client_info    CLIENT_N_RAY;
static int            error;

/* Set up data structure for MPS connection */
error = mb_clear_client(&CLIENT_N_RAY, "/usr/people/hugh/software/host.IP");
/* Make connection to MPS */
if(error == NO_ERROR) error = mb_init_client(&CLIENT_N_RAY, "N_RAY");

if(error == NO_ERROR) error = mb_set_flag(&CLIENT_N_RAY, MESSAGE_LOG_FILE, FALSE);

/* Shut down the connection */
mb_deinit(&CLIENT_N_RAY);

```

Figure 16 - Turning MPS Log Files Off

The MPS also keeps times of transactions. The default is on, and is useful for log file transactions.

Since this is not time consuming, it is not worth the effort to disable, but if the user is inclined, the code in Figure 17 will do so. At present there is no user access to messages, in the future we may consider access to the messages, including time stamp information.

```

static client_info    CLIENT_N_RAY;
static int            error;

/* Set up data structure for MPS connection */
error = mb_clear_client(&CLIENT_N_RAY, "/usr/people/hugh/software/host.IP");
/* Make connection to MPS */
if(error == NO_ERROR) error = mb_init_client(&CLIENT_N_RAY, "N_RAY");

if(error == NO_ERROR) error = mb_set_flag(&CLIENT_N_RAY, MESSAGE_TIME_STAMP, FALSE);

/* Shut down the connection */
mb_deinit(&CLIENT_N_RAY);

```

Figure 17 - Turning Off Time Logging for Messages

In some cases there may not be a 0 level client (as in a pre and post-processing model). If this is the case, there may not be another client to change the destination group of the message, and as a result will cycle through the group indefinitely. To counter this, a flag may be set which will discard messages passing priority level 0, when there is no client present at level 0. The example in Figure 18 shows how to disable this feature (the default is on).


```

static client_info  CLIENT_N_RAY;
static int          error;

/* Set up data structure for MPS connection */
error = mb_clear_client(&CLIENT_N_RAY, "/usr/people/hugh/software/host.IP");
/* Make connection to MPS */
if(error == NO_ERROR) error = mb_init_client(&CLIENT_N_RAY, "N_RAY");

if(error != ERROR) error = mb_set_flag(&CLIENT_N_RAY, ZERO_PRIORITY_BLOCKING, FALSE);

/* Shut down the connection */
mb_deinit(&CLIENT_N_RAY);

```

Figure 18 - Turning Off The Zero Level Priority Message Blocking Feature

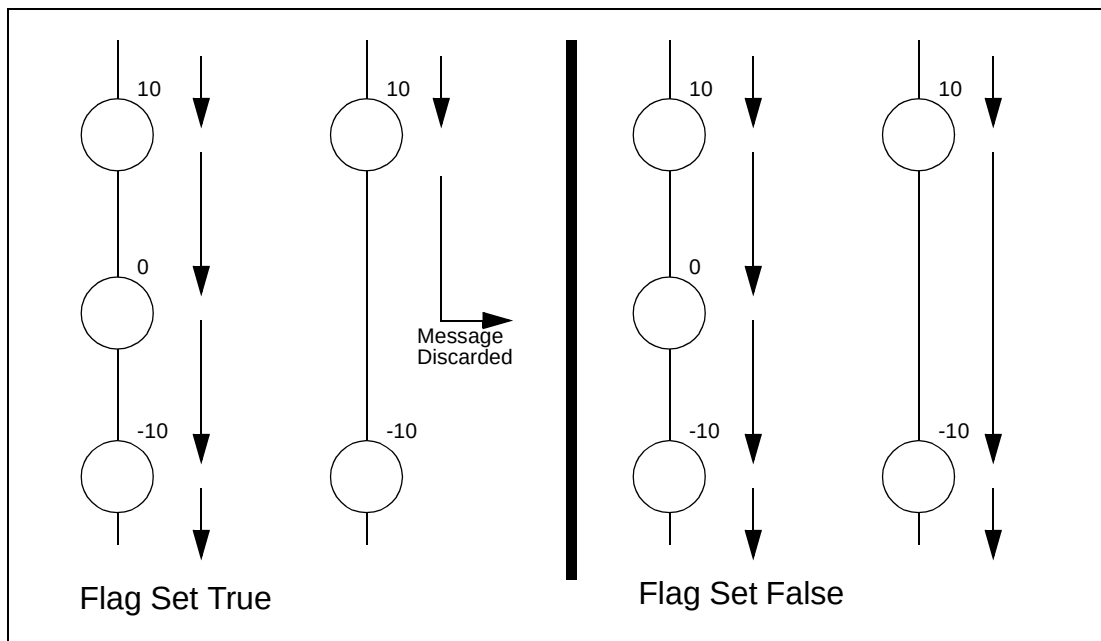


Figure 19 - The Effect of Turning off the Zero Level Priority Message Blocking Feature

4.0 Future Features of MPS

There are a number of functions of MPS which are under development, or in planning. These are listed below with some descriptions.

- i) Blocking/Non-Blocking General Broadcasts to a group - this has been discussed at the end of Section 2.3 - Direct Instance Connection.

- ii) Recovery from log file in the event of an MPS failure (i.e., crash).
- iii) A method for detecting and removing connections to clients which have crashed (and thus are not present). It is possible that a client may not deinitialize properly. If this is the case, messages may wait indefinitely for the client.
- iv) Message time stamps - A message may be stamped with an expiry time, after which it is discarded if not read. The other case is a message which is to be held until a specific time, at which it will be made available for reading.
- v) The deinitialization function will allow the connection between the MPS and the instance to be closed, but the internal data structures of MPS are not adjusted. Until this is fixed, missing clients may block messages.
- vi) Getting lists of all groups on the message board.
- vii) Interfaces written in other languages, and on other operating systems.
- viii) Having concurrently running MPS servers. This would help overcome the bottleneck problem.
- ix) Give the user access to waiting, and written messages to examine content and time stamps. This feature might require some examination of security issues.

5.0 - Non-MPS Communication (Point - to - Point)

For transmitting blocks of data (large objects) it is desirable to go around the MPS, and directly between clients. To do this a set of subroutines are available to the instance program. The example below shows two programs which use the MPS to trade specific information about socket numbers and host names, and then pass data directly (see Figure 20)

```

static client_info  CLIENT_A,
                    DIRECT;
static int          client_a,
                    client_b,
                    from,
                    to,
                    cmd,
                    error,
                    local_socket;
static char         string[MAX_STRING_LENGTH];

error = mb_clear_client(&CLIENT_A, "/usr/people/hugh/software/host.IP");
local_socket = 3999;
if(error == NO_ERROR) error = mb_init_client(&CLIENT_A, "A");

while((client_a = mb_get_client_group(&CLIENT_A, "A")) == ERROR);
while((client_b = mb_get_client_group(&CLIENT_A, "B")) == ERROR);

/* Send connection information */
mb_send_message(&CLIENT_A, client_a, client_b, local_socket, "mr_bun.engrg.uwo.ca");
/* Wait for remote instance to connect */
error = mb_con_wait(&DIRECT, local_socket);
while(error == NO_ERROR){
    /* Get a string from the socket */
    if(mb_con_receive(&DIRECT, string, strlen(string)) != ERROR){
        printf("%s\n", string);
        /* Send response to remote instance */
        mb_con_send(&DIRECT, "RESPONSE");
    } else {
        sleep(1);
    }
}

mb_con_close(&DIRECT);
mb_deinit(&CLIENT_A);

```

Figure 20a - Direct Socket Connection Between Programs Using the MPS

```

static client_info  CLIENT_B,
                    DIRECT;
static int          from,
                    to,
                    cmd,
                    i,
                    error,
                    local_socket,
                    remote_socket;
static char         string[MAX_STRING_LENGTH],
                    remote_host[MAX_STRING_LENGTH];

error = mb_clear_client(&CLIENT_B, "/usr/people/hugh/software/host.IP");
local_socket = 3888; /* The local socket number used */
if(error == NO_ERROR) error = mb_init_client(&CLIENT_B, "B");

while(mb_get_message(&CLIENT_B, &from, &to, &cmd, string) == ERROR) sleep(1);
remote_socket = cmd; /* get the remote socket number */
strcpy(remote_host, string);
/* Establish connection with the waiting client */
mb_con_make(&DIRECT, remote_host, remote_socket, local_socket);
for(i = 0; i < 20; i++){
    sprintf(string, "INITIAL:%d", i);
    /* Send string and wait for response */
    mb_con_loop(&DIRECT, string);
    printf("%s\n", string);
}

mb_con_close(&DIRECT);
mb_deinit(&CLIENT_B);

```

Figure 20b - Direct Socket Connection Between Programs Using the MPS

6.0 - MPS for CAPP/PPC Integration

The description in the earlier sections are best illustrated with an application. A practical example which is being developed at UWO is the integration of CAPP (Computer Aided Process Planning) and PPC (Production Planning and Control). To do this we are using a program called RPE (Reactive Planning Environment) developed at McMaster University. CAPP will issue process plans for individual parts and products. These process plans contain a number of alternative operations. What RPE will do is allow the process planner to choose the optimum set of operations in the process plan. These choices are often dependant on available resources. The problem which arises between CAPP and PPC is that when a collection of process plans are produced with CAPP and RPE, all of the resources are assumed to be available. In reality PPC will have an ongoing revision of resource availability. Quite often PPC can resolve scheduling

problems, but when it cannot, a replanning process must occur. Thus, the existing integration problem is a question of how to include the machine availability in process planning.

The purpose of the integrator is to resolve the feedback problem by using RPE to try short term replanning. The integrator will have two functions. The first function is to store process plans, and resource data. The second function is to receive events (demands, requests, and reports) and as a result send data and requests/reports to other programs in the system. The reader should note that the software is cleanly divided into 4 sections: CAPP; PPC; RPE; The Integrator. To bring this software together in a networked environment we have used the MPS. Each of these programs has been given its own group. An MPS graph is shown below in Figure 21, and as can be seen, the structure is very simple at this point.

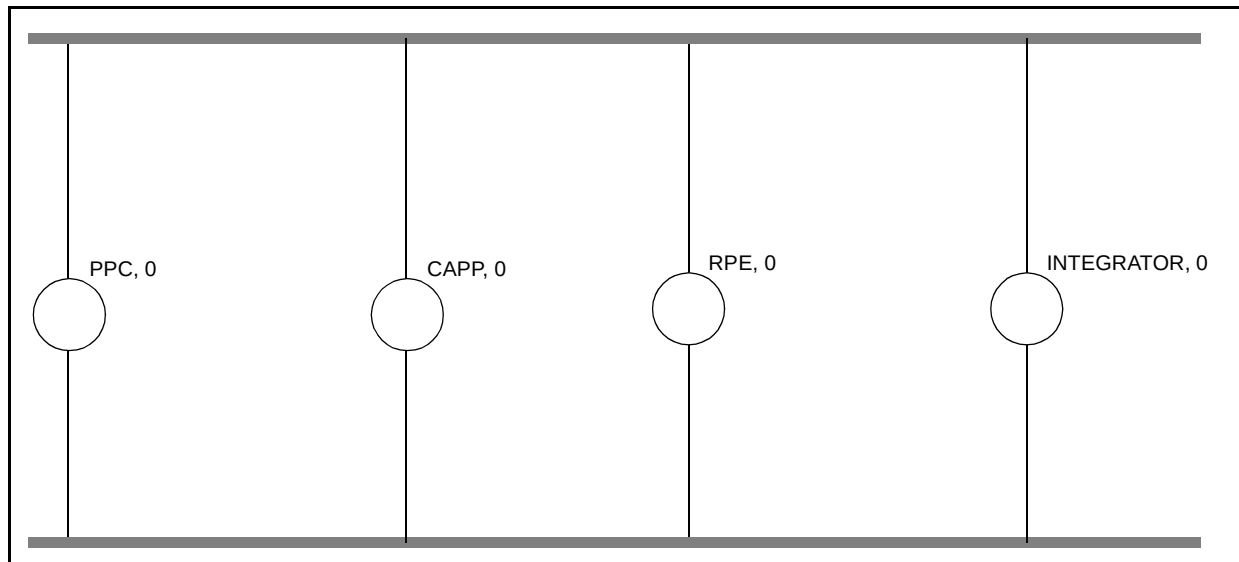


Figure 21 - The fundamental MPS Graph for CAPP/PPC Integration (Keeping in mind that the structure above is intended for event reporting only)

The PPC group should only be an interface to an individual PPC package. While on the other hand, the CAPP group is suited to a large number of concurrent instances. The information used by RPE will be provided by the integrator, therefore the RPE group could also contain a large number of concurrent instances. Finally, the integrator must be run by itself, because it acts as a gateway for all transactions, and is not suited to concurrency.

7.0 Discussion and Conclusions

This message passing system provides an excellent tool for developing distributed applications.

Because of its structure it may be independent of language, platform, and user interface. The cost of all of the advantages discussed is a decrease in speed of communication. Thus, for applications with time constraints, a synchronous system is much more advisable.

If a distributed system has a number of largely independent tasks, MPS will provide an excellent interface environment. This is largely the result of the original effort to use MPS for,

- manufacturing workcell control
- design and production function integration
- distributed graphical applications such as ray tracing and design.

The MPS approach also has an approach which focuses on event flow, instead of data flow. This makes the system more intuitive for the designer and more like a traditional user interface to the programmer. MPS also allows the ad-hoc addition of foreign processes for diagnosis and observation, which is difficult or impossible in the traditional point-to-point systems. As stated before, MPS does not attempt to replace existing distributed processing methods, but instead provide a method which can coexist with them, and allow distributed computing functions not previously available.

Appendix A - Function Names/Definitions

These functions have been labelled with a note which indicates which are optional, and in which order they may be used.

- 1 - first
- 2 - before initialization with the MPS
- 3 - initialization with the MPS
- 4 - during run
- 5 - end of run

(required 1) `int mb_clear_client(client_info *CLIENT_INFO, char`

`*file_name)`

Clear Data Structure - This declaration will clear all of the fields in the `CLIENT_INFO` structure. 'file_name' is the name of an ASCII file contains a machine name, or IP number on the first line, and a socket number on the second line. This is for the MPS. This function does not initialize a client, but must be called before any other functions are called. Returns `ERROR` or `NO_ERROR`.

(optional 2, 4) `int mb_set_timeout_interval(client_info *CLIENT_INFO, int duration)`

Set Timeout - If any read or write to the MPS has failed, this function will determine the number of seconds to wait before quitting. The default is 30 seconds. This only has an effect if the socket connection to the MPS has failed. Returns `ERROR` or `NO_ERROR`.

(optional 2) `int mb_set_priority(client_info *CLIENT_INFO, int priority)`

Set Priority - Must be called before 'mb_init_client()' to set the priority of the client. If not called, the default is 0. Returns `ERROR` or `NO_ERROR`.

(optional 2) `int mb_set_sequence(client_info *CLIENT_INFO, int sequence)`

Set Sequence - This will declare that a concurrent set of instances is sequential (`sequence = SEQUENT`) or non-sequential (`sequence = NON_SEQ`). The default is non-sequential, but if this is called, it over-rides all settings for the other concurrent instances (at the same group and priority level). Returns `ERROR` or `NO_ERROR`.

(required 3) `int mb_init_client(client_info *CLIENT_INFO, char *name)`

Initialize Client - This function establishes a connection to the MPS. The group is determined by the string in 'name'. Returns `ERROR` or `NO_ERROR`.

(option 4) `int mb_get_client_group(client_info *CLIENT_INFO, char`

*name)

Get Group Number - Each group has an index (or key reference number). To find the number of another group this function is called with the name of the remote group 'name'. If the group is found the integer value is returned, or else ERROR is returned.

```
(option 4) int mb_set_flag(client_info *CLIENT_INFO, int type, int
                        state)
```

Set A Flag - This allows a remote flag to be set on the MPS. This returns ERROR or NO_ERROR. The flag types and settings are explained below.

- MESSAGE_LOG_FILE - If set TRUE (the default is TRUE) the MPS will save a copy of every message received, every message read, and every message deleted. This feature will slow the MPS because of disk access times, the user is encouraged to turn this off.
- MESSAGE_TIME_STAMP - Will turn on a function that logs the time when messages arrive, are read, and are deleted. The default is TRUE.
- ZERO_PRIORITY_BLOCKING - The default is TRUE. If set all messages will be discarded if there is no 0 level priority instance when they pass 0 level priority.

```
(option 4) int mb_encapsulate(client_info *CLIENT_INFO, int
                        priority, char *group, char
                        *encapsulated_group)
```

Encapsulate a Group - This function will cause the named 'encapsulated_group' to be encapsulated as an instance in 'group' at 'priority'. The reader should note that this encapsulated group is created automatically as if it is a different instance than represented by 'CLIENT_INFO', so the calling routine does not have to perform any setup. Returns ERROR or NO_ERROR.

```
(option 4) int mb_send_message(client_info *CLIENT_INFO, int
                        from_group, int to_group, int cmd, char
```


*string)

Send Message to Group - This function will post a message on the MPS which will appear to come from 'from_group', and will travel to 'to_group'. The message content is 'cmd' and 'string'. Returns ERROR or NO_ERROR.

(option 4) int mb_get_message(client_info *CLIENT_INFO, int
*from_group, int *to_group, int *cmd, char
*string)

Get Message - This function will get any message waiting on the MPS. If any messages are waiting, the messages directly from other instances will be retrieved first. If the message is from an instance, the 'from_group' value will contain the instance number, if the message comes from a group the 'from_group' number will be a group number. 'to_group' is the destination group or instance of the message (if the message was to an instance, this will be an instance number, otherwise it will be a group number). The 'cmd' and 'string' values have been set by a remote instance. If no message was available, ERROR is returned, otherwise NO_ERROR is returned. The length of string will never be longer than MAX_STRING_LENGTH.

(required 5) int mb_deinit(client_info *CLIENT_INFO)

Deinitialize Client Instance - This breaks the socket connection to the MPS, but does not yet deinitialize the data structures on the MPS. Returns ERROR or NO_ERROR.

(optional 4) int mb_unset_filter(client_info *CLIENT_INFO, int group)

Remove Group From Filter List - If a group is on the filter list, and the client wants to remove it, this function may be called. Refer to mb_set_filter() for details of filters. Returns ERROR or NO_ERROR. (Not implemented yet)

(optional 4) int mb_set_filter(client_info *CLIENT_INFO, int group)

Set Input Filter - If called it will set a filter on messages which may be received. The mechanism is list based. If the list is empty, messages from all groups may be

accepted. To clear the list, set 'group = ALL_GROUPS' When 'group' is a real group number, that group will be appended to the list. If the list has any groups on it, messages from those groups will be the only ones accepted, until the list is cleared. Returns ERROR or NO_ERROR.

(optional 4) `int mb_instance_number(client_info *CLIENT_INFO)`

Find Local Instance Number - Will return the instance number, or ERROR. The user should recall that each instance is given a distinct reference integer number, which other instances are not given access to. Returns ERROR if instance number not available.

(optional 4) `int mb_send_message_ins(client_info *CLIENT_INFO, int to_instance, int cmd, char *string)`

Send a Message to Instance - Will send a message to instance number 'to_instance'. The user is reminded that the destination must first be found from the remote instance.

Point to Point Connections:

These functions are more sophisticated than the previous routines. The routines require a synchronous connection between processes. What this means is that one process must be waiting for a connection. Another process must be waiting for a connection. Another process must connect, and then all communication must occur in an agreed sequence. The function descriptions below are preceded with an indication of which process will use them. In this terminology the server will be the process which has initiated a connection. Order numbers are also provided in this section.

(Server 1) `int mb_con_wait(client_info *CONNECT, int local_socket)`

Wait for Connection - This function will be called, and will not return until a remote client has connected. If the remote client has connected successfully NO_ERROR will be returned, else ERROR will be returned. The 'local_socket' number is arbitrarily chosen by the user/instance. The user is

responsible for informing the remote client about the socket number, and host machine name being used.

```
(client 2) int mb_con_make(client_info *CONNECT, char
                        *remote_server_name, int
                        remote_server_socket, int
                        local_client_socket)
```

Make Connection With Waiting Client - Once a server is waiting for a connection, this function may be called to make the connection. The user must know the 'remote_server_name' and the 'remote_server_socket'. The 'local_client_socket' is arbitrarily chosen by the user. This function returns NO_ERROR if the connection was successfully made, else ERROR is returned.

```
(client/server 3) int mb_con_send(client_info *CONNECT, char *string)
```

Send Data Through Direct Connection - Will send 'string' to the remote client/server. The remote party must be waiting. If the 'string' was not sent ERROR will be returned.

```
(client/server 3) int mb_con_receive(client_info *CLIENT_INFO, char
                        *string)
```

Get Data From Direct Connection - Is used when a string is expected to be received. This function must be called before a 'string' is sent. If a string was received this function will return NO_ERROR.

```
(client/server 3) int mb_con_loop(client_info *CONNECT, char *string)
```

Send and Receive Data From Direct Connection - This uses the send and receive functions above to send a message to a remote connection and then wait for a reply. If the value of 'string' was sent, and a new 'string' has replaced it, NO_ERROR will be returned.

```
(server/client 4) int mb_con_close(client_info *CONNECT)
```

Close Direct Connection - This will break the connection made and used by the previous functions. This function should be called by both the client and server to close their connections. The function returns **ERROR**, or **NO_ERROR**.

Appendix B - Software Structure

The structure of the software may be illustrated with a simple diagram showing the major software modules, and where they fit in.

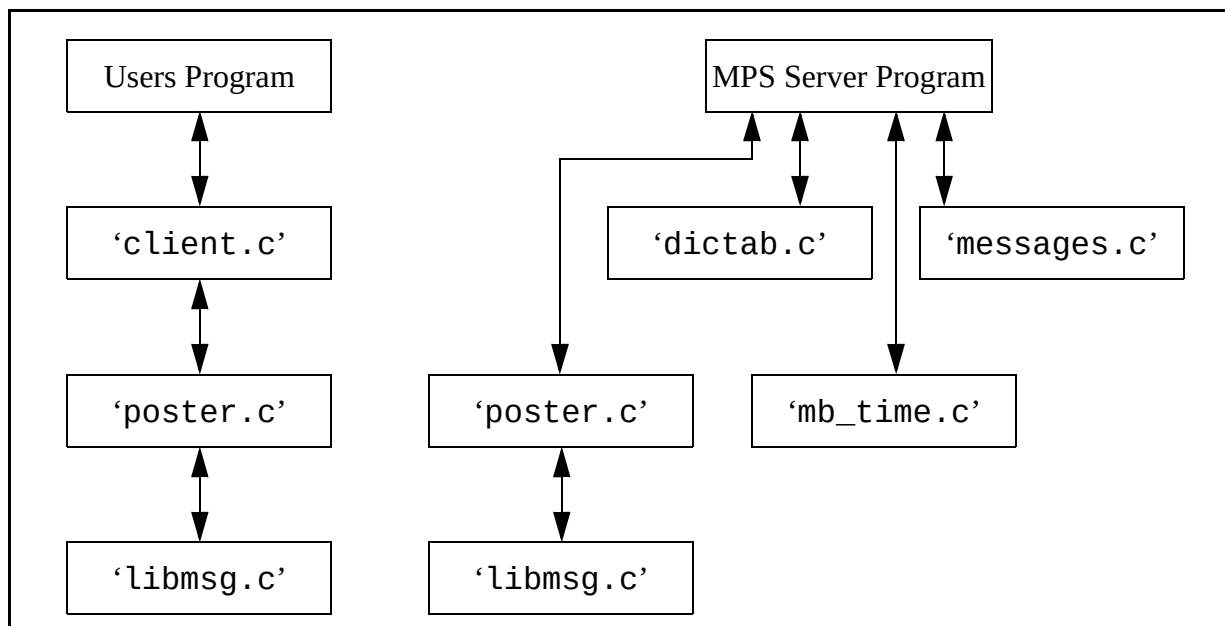


Figure b.1 - A picture of the software structure of the MPS (note that some software routines are shared)

All of these modules use the `define.c` functions for common definitions of messages. A quick description of each module is given below.

`define.c` - Some basic information and functions for message handling.

`libmsg.c` - The low level socket drivers, with no protocol.

`poster.c` - The protocol for message passing using the routines in `libmsg.c`.

`client.c` - Converts the users function calls into message board requests, and interprets the responses.

`dictab.c` - Stores and allows manipulation of information about instances using the MPS.

`messages.c` - Stores messages and recalls them upon request.

`board.c` - Handles all executive functions for coordinating clients, messages, and requests.

`mb_time.c` - Creates time strings from system.

Appendix C - Linking and Running MPS

Linking:

The example UNIX ‘`makefile`’ shows how to link the MPS utilities to an application (see Figure C.1 below).

```
#
# MPS example makefile
#

CFLAGS = -O
LIBS =

all:      test

#
# Message Board interface is linked here
#

MPS_DRIVERS = /usr/people/hugh/software/TOOLS/mps/define.o /usr/people/hugh/software/TOOLS/
             mps/libmsg.o /usr/people/hugh/software/TOOLS/mps/poster.o /usr/people/hugh/software/TOOLS/
             mps/client.o crs_client_decode.o

test:      test.c $(MPS_DRIVERS)
           cc $(CFLAGS) crs_test.c -o crs_test $(MPS_DRIVERS) $(LIBS)
```

Figure c.1 - A Sample UNIX `makefile` for a test program which uses the message board

Running:

The MPS program is called ‘`board`’, and must be run on a particular machine before the application programs. The application programs and MPS must be directed to the same file containing the host IP number, and socket number, which MPS will run from. If the file already exists, and MPS finds a problem with it, it will attempt to correct the problem. If no file is found MPS will

fail to start correctly, and communications will fail. This file may be specified to MPS when it is run,

```
board /usr/people/joe/host.IP
```

The user then runs their program which also refers to the same file.

Glossary of Terms

Client - Used interchangeably with instance.

Group A group is a collection of instances which perform parts of a function. The order in which each instance operates is determined by their priority. The suggested mode of addressing messages is to a group.

InstanceA program which has a connection to the MPS. If a program has more than one connection to the MPS will to represent more than one instance. Each instance is described with a group name and priority number.

MessageAn integer and string combination passed between instances. The contents of the integer value and the string are left to the instances.

Message BoardUsed interchangeably with MPS.

MPS Message Passing Secretary - A central server which communicates with each instance, and handles all message passing between instances. The communications between instances is determined by an ad-hoc graph structure set up when the instances are initialized and deinitialized.

PriorityA numerical order in which instances will receive a message. If the two or more instances have the same priority in the same group, they become concurrent.

References

- Agha, G. and Hewitt, C., 1987, "Concurrent Programming Using Actors", in the book "Object Oriented Concurrent Programming", edited by A. Yonezawa and M. Tokoro, The MIT Press, pp. 37-53.
- Andrews, G. R., 1991, "Concurrent Programming Principles and Practice", The Benjamin/Cummings Publishing Company.
- Birman, K. P., 1991, "The Process Group Approach to Reliable Distributed Computing", July 1991, Department of Computer Science, Cornell University.
- Buhr, P.A. and MacDonald, H. I., 1991, "uSystem Annotated Reference Manual", Department of Computer Science, The University of Waterloo, Waterloo, Ontario, Canada.
- Grimshaw, A. S., 1991, "An Introduction to Parallel Object-Oriented Programming with Mentat", Computer Sciences Report No. TR-91-07, April 4, 1991, University of Virginia.
- Jack, H. and Buchal, R. O., 1992, "An Automated Workcell for Teaching and Research", to be published in The Western Journal of Graduate Research, The University of Western Ontario.
- Otto, A., 1991, "MetaMP: A Higher Level Abstraction for Message-Passing Programming", Department of Computer Sciences and Engineering, Oregon Graduate Institute of Sciences and Technology.
- Sikorski, J., 1991, "Sifting Through Silicon; Features of Parallel Computing Architectures", Sunworld, April 1991, pp. 39-45.
- Singh, A., Schaeffer, J. and Green, M., 1991, "A Template-Based Approach to the Generation of Distributed Applications Using a Network of Workstations", IEEE Transactions on Parallel and Distributed Systems, Vol. 2, No. 1, Jan., 1991, pp. 52-67.
- Strom, R. E. and Yemini, S., 1983, "NIL: An integrated language and system for distributed programming", SIGPLAN Notices 18, 6 (June), pp. 73-82.

12. MATHEMATICAL ELEMENTS OF COMPUTER GRAPHICS

- To put geometries on the computer screen we depend on basic mathematical tools and methods.

12.1 INTRODUCTION

- After a geometric model is constructed, it must be displayed (rendered).
- Rendering is mainly limited by computer hardware, and the geometric model.
- The main methods for doing computer graphics are,
 - Wire Frame
 - Wire frame with hidden line removal
 - Polygon drawing (backface, and clipping)
 - Shaded polygons
 - Raytracing
- As the scene becomes more complicated, the computing time becomes longer, but the picture becomes more realistic.
- The basic history of research on geometric modeling can be summarized as,
 - 2D computer drafting - Mid 60s
 - 2.5 D - Late 60s
 - 3D Wire Frame Systems - Early 70s
 - 3D Surface Systems - Mid 70s
 - 3D Primitive Solids - Early to Mid 70s
 - 3D Arbitrary Solids - Mid to Late 70s

12.2 PIXELS

- The fundamental task is converting lines, points and surfaces in 3D space, to be depicted on a 2D screen using colored pixels, or printed on paper with dots, or plotted with pens.
- A computer screen is made up of an square array of points (pixels). The points can be lit up. When viewed as a whole these points make a picture.
- One major problem is making a map between a geometry model (a collection of points) and what we see on the screen. This is accomplished with the perspective transform.

12.2.1 The Perspective Transform

- A set of basic viewing parameters may be defined (variations are also common),
 - The point the Eye is looking at, and from which direction
 - The focal distance to the viewing plane
 - The size of the viewing plane being focused on
 - Which direction is up for the eye

$$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -(VRP_x + VPN_x V_{dist}) & -(VRP_y + VPN_y V_{dist}) & -(VRP_z + VPN_z V_{dist}) & 1 \end{bmatrix}$$

$$V = \sqrt{VPN_y^2 + VPN_z^2} \quad RUP = \sqrt{VUP_x^2 + VUP_y^2}$$

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{-VPN_z}{V} & \frac{-VPN_y}{V} & 0 \\ 0 & \frac{-VPN_y}{V} & \frac{-VPN_z}{V} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_y = \begin{bmatrix} V & 0 & -VPN_x & 0 \\ 0 & 1 & 0 & 0 \\ VPN_x & 0 & V & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_z = \begin{bmatrix} \frac{VPN_y}{RUP} & \frac{VUP_x}{RUP} & 0 & 0 \\ \frac{-VUP_x}{RUP} & \frac{VPN_y}{RUP} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$S = AP$$

where,

A = Transform Matrix

P = Point in real/model space

S = Point on screen

VRP = The perspective viewing point

VPN = The view plane normal

VUP = The view up vector

- As seen above the viewing parameters can all be combined using simple matrix multiplication which will convert a point in 3D space to a point on the screen.
- The process of drawing an object is merely applying this transformation to each point in the 3D model, then using the resulting (x, y) point on the 2D screen. (Note: If this transformation is

done properly then z = depth in the view plane.)

- The point mapped to the computer screen can then be converted to a single pixel using a simple scaling calculation. (Note: It is not shown, but if a point is off the screen, then it cannot be drawn.)

$$S_x = \text{int}\left(V_x\left(\frac{\text{pixels}_x}{\text{width}}\right)\right) \quad S_y = \text{int}\left(V_y\left(\frac{\text{pixels}_y}{\text{height}}\right)\right)$$

where

S = The screen pixel location

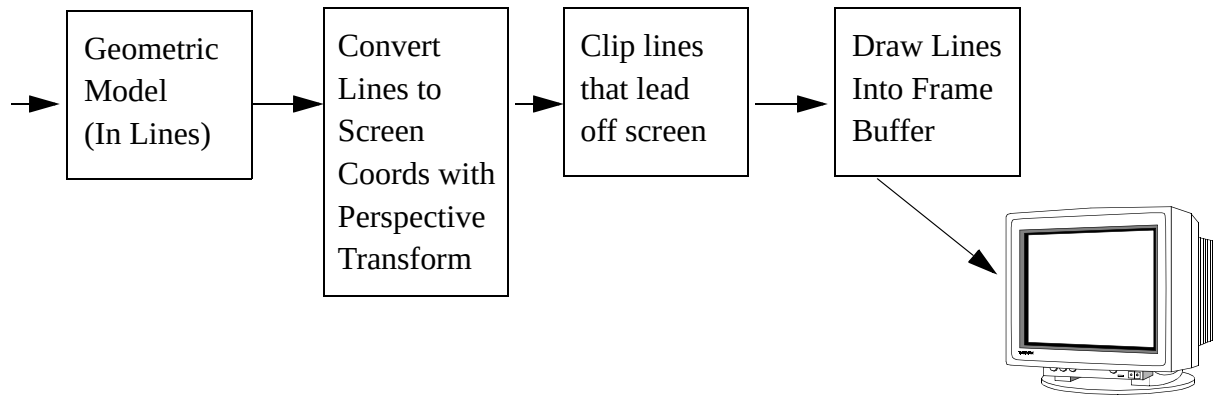
pixels = the number of pixels on the screen

$\text{width}, \text{height}$ = the area of the transformed screen to view

- Visual display can be done using,
 - CRT monitors with Frame Buffer memory to store the image.
 - plotters which draw one line at a time
 - printers using special and proprietary graphics languages
- For the sake of simplicity, the remaining graphics methods ignore some trivial operations such as screen coordinates, line clipping at edge of screen, etc.
- The 'z' value after the perspective transform gives a relative depth of a point. This can be used later for depth sorting, or to set light intensity to cue the user to view depth.

12.3 LINE DRAWING

- A wireframe object is simply rendered using the lines (straight, and curved) in a geometric model, then it is converted to screen coordinates using the Perspective Transformation.
- The process is as pictured below,

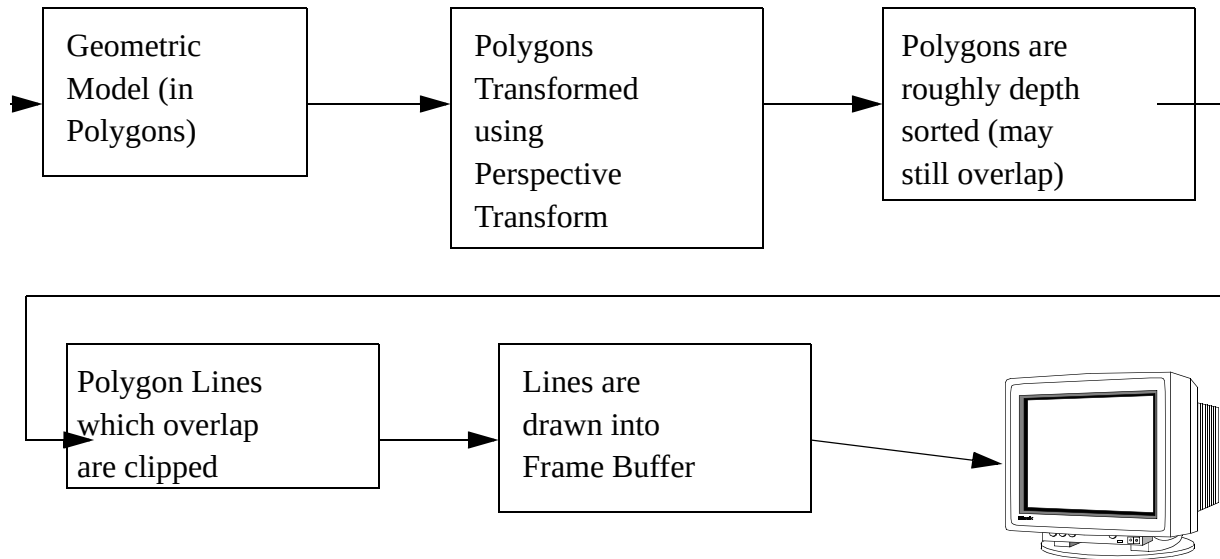


- If the geometric model is in some other form, such as CSG (Constructive Solids Geometry), it may have to be converted to lines before it may be used.
- Advantages,
 - Very fast, allows real time manipulation
 - Easy to see features normally hidden
 - Supported by almost all computer technology
 - This display method works with most displays
- Disadvantages,
 - Drawings can become very crowded, very fast
 - Hard to visualize solid properties
- Almost all computer graphics packages offer a wireframe display option.

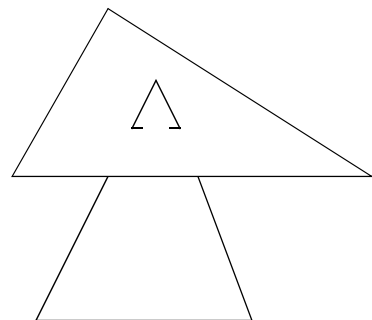
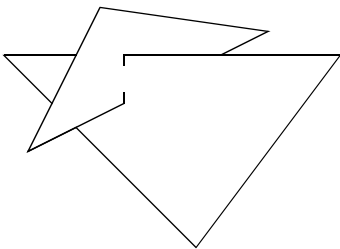
12.3.1 Hidden Lines

- While the results seem similar, this method requires more sophisticated algorithms.
- Polygons must be used for surface representation.

- The diagram below shows the basic steps in the method of rendering

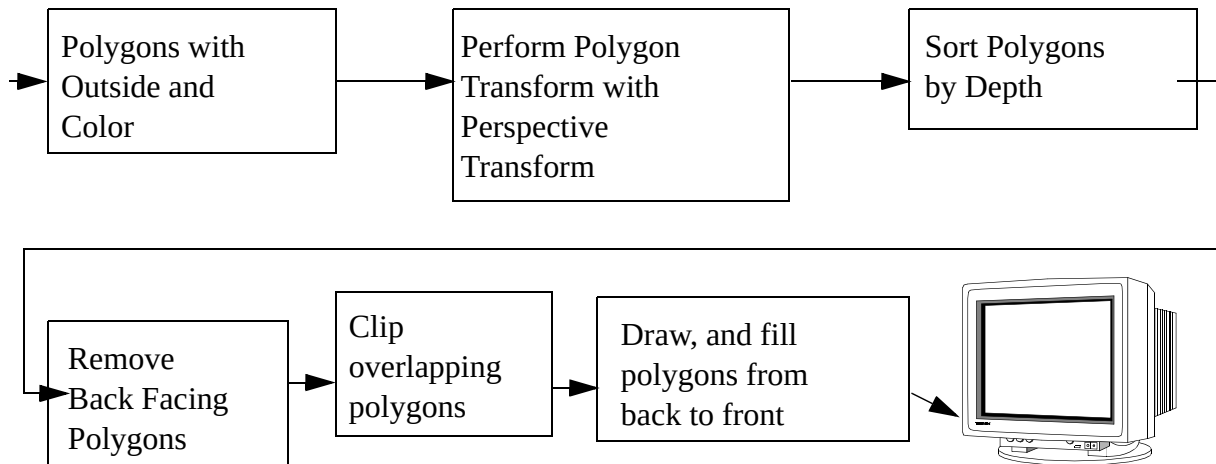


- Advantages,
 - similar advantages of wireframe
 - overcomes the drawing crowding
 - can be simplified if polygons do not overlap
- Disadvantages,
 - Consumes more computer time
 - Does not support simple elemental geometric models
- Clipping algorithms may be difficult, and often use tricks like subdividing polygons into triangles. (There are only 5 cases to consider with overlapping triangles)



12.4 POLYGON DRAWING

- This method bears similarities to the hidden line method. The polygons are still depth sorted, but now polygons are filled, and back facing polygons are removed.
- The basic method is as shown below,



- Sorting polygons by depth is basically a function of finding the general center, and then drawing the rear most polygons first.

$$depth_j = \frac{\sum_{i=1}^{n_j} S_{j,iz}}{n_j}$$

where,

$depth_j$ = depth of center of polygon j

n_j = number of vertices in polygon j

$S_{j,iz}$ = Screen depth of vertex i in polygon j

- A backfacing polygon can be eliminated by calculating its surface normal. The normal is then compared to the viewing axis. If there is more than 90deg between them the polygon doesn't need to be drawn. (Note: this method assumes that vertices are defined in a counter clockwise order for the outside surface).

$$N_j = (V_{2,j} - V_{1,j}) \times (V_{0,j} - V_{1,j})$$

$$view = P - VRP$$

$$proj_j = N_j \bullet view$$

where,

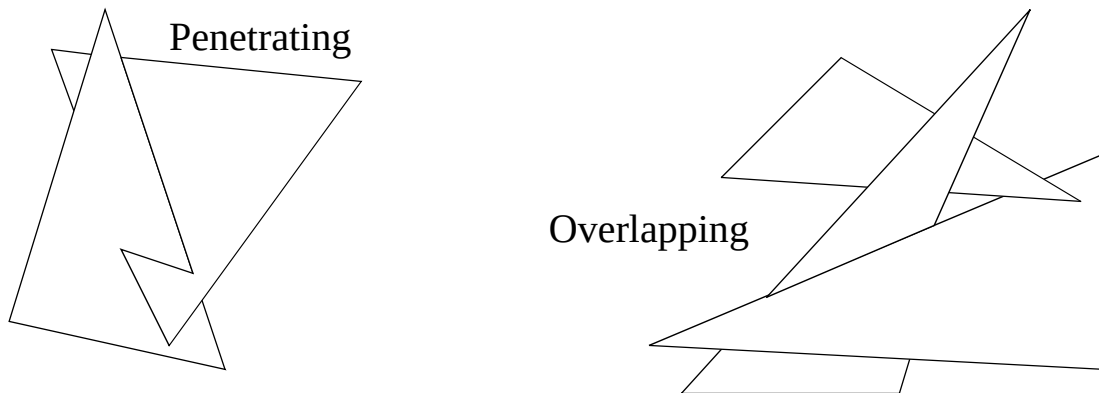
N_j = Normal to outside face of polygon

$view$ = view direction vector

$proj_j$ = The projection of the two vectors

Note: when the proj value is negative, the polygon is back facing and doesn't need to be drawn.

- The above diagram only talks about overlapping polygons, but penetration is an equivalent problem.



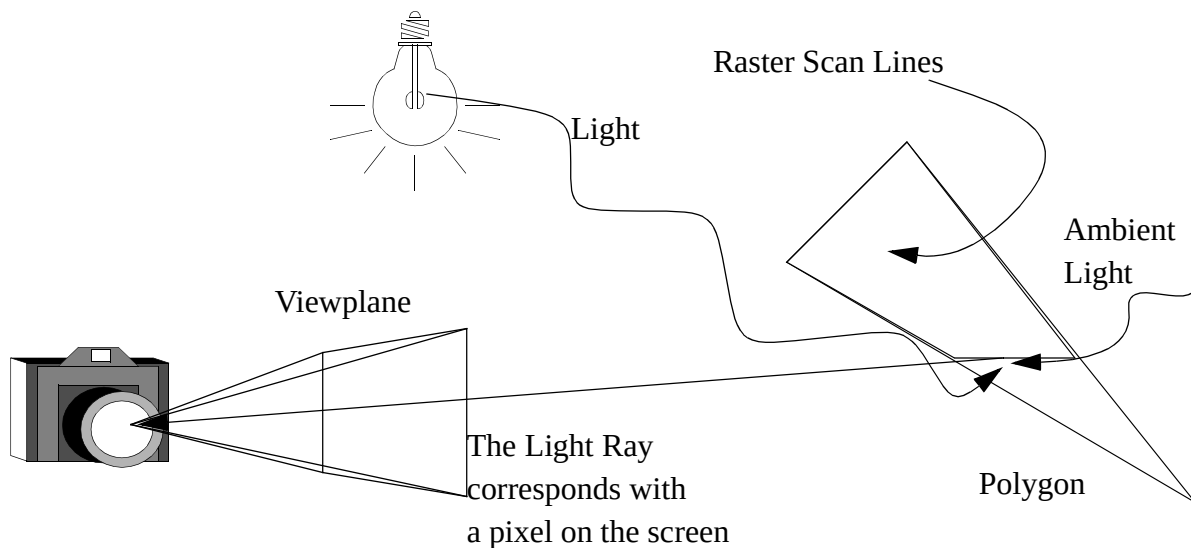
- Advantages,
 - with color added, objects look more real.
 - still relatively easy to implement, and run quickly.
- Disadvantages,
 - doesn't suit all computer equipment (eg. laser printers).
 - surfaces must have outside defined and surfaces must be closed.
- Fill Algorithms generally look at a polygon on the screen, and fill the inside with pixels of a specific color.
- Clipping can also be done by a technique called Z-Buffering, using extra graphics memory. An extra byte is used to store the depth of a pixel when it is calculated. If a new pixel is chosen, it

overwrites the last.

***** Include figure for inside/outside test

12.5 SHADED POLYGONS

- This method is identical to the previous method, except a polygon will be filled with pixels of varying color, which makes the object appear more real.
- A Light Source(s) is defined for a picture. The light has a specific color, direction, distance, etc.
- Ambient light (background, non-direct) also plays a part.



- The color of a pixel will change with the angle of the polygon to the light, the distance from the light, the color of the light, etc.
- Advantages,
 - the solid begins to look like cartoons, or paintings
 - still simple to understand and implement
- Disadvantages,
 - can be slow
 - requires shaded image display capabilities
- Many Shading methods can be used to give different approximations of light,
 - Gouraud shading
 - Phong shading

12.6 COLORS

- It has been long known that the eye can be tricked into seeing a wide range of colors by blending three different colors in different intensities.
- In the additive color scheme we add red blue and green (RGB). We start with black and add shades of these colors.
- In the subtractive color scheme we start with a white pixel. The intensity is reduced by filtering the colors magenta, cyan and yellow.
- Most computers use the RGB scheme, but the subtractive color scheme is popular in printing and photographic reproduction techniques.
- Some of the techniques used when limited numbers of colors or shades are available are,
 - color maps
 - dithering

12.6.1 Color Maps

- A color map is a list of colors that the computer can use to draw with.
- The eye is very sensitive and can sense millions of different colors. And current trends are to go to 24 bit color systems that have 8 bits for each primary color. This gives the ability to display different colors so close that the human eye cannot detect the difference.
- For various reasons we will use machines that have limited numbers of colors available (256 is common).
- When this occurs colors that should look like smooth transitions tend to look more like bands of color.
- One approach to providing colors is to construct a well distributed pallet (a fixed set of colors) that the user can select from. They must always find the best match to their desired colors.
- The eye tends to be more sensitive to certain colors, and so one approach is to map the colors into a pallet using color bit assignments. For example for a pallet of 256 (8 bits) we may choose to assign 3 bits to blue, 3 bits to green and only 2 bits to red. This means that there will be 8 intensity levels for both green and blue, but only four for red.

- When using a pallet the some colors will be overused, and other may never be used.
- Another useful approach is to quantize the color map so that it has the best coverage of the desired colors, and lower coverage of unused colors. This process is called quantization.

12.6.1.1 - Quantization with an Octree RGB Cube

- In the octree quantization method [Graphic Gems by Andrew Glassner] the RGB color information is read sequentially from an input data file. The first k different colors are used as initial entries to the color table. If another color is added so that there are now $k+1$ colors, some very closely related colors are merged into one and their mean is used as the entry in the colormap. This procedure is repeated for each additional color.
- The RGB cube can be represented by an octree of depth eight by recursive subdivision of the RGB cube. The RGB values (0 - 255) are coordinates into the RGB cube. The bit pattern, corresponding to the same level in the octree as the bit position, can be used as the index for the branch into the octree.

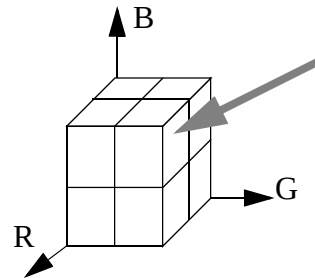
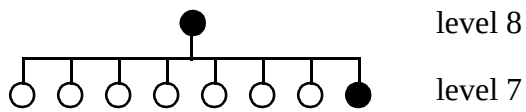
The index gives the position of the color in the RGB cube as well as the branch of octree.
 RGB = 140, 200, 255

Level 8 in Octree

RGB vector

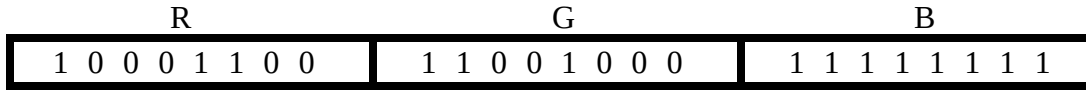


Index = 111

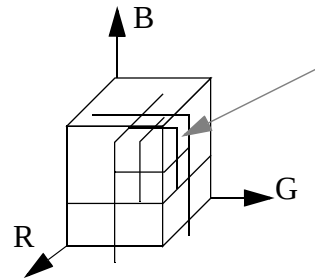
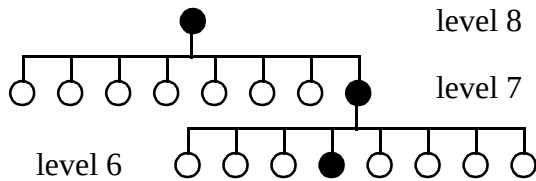


Level 7 in Octree

RGB vector



Index = 011



etc.

Figure 20 - Mapping an RGB Value into the RGB Cube/Octree

12.6.1.1.1 - Algorithm and Implementation

- The algorithm has the following three phases:
 - Phase 1. Evaluation of Representative Colors

The first step to read the input data and insert colors into leaves of the octree.

```

InsertTree(Tree: Octree; RGB: Color);
begin
    if Tree = NIL
        then Make_and_Initialize_New_Node;
    if Leaf
    then
        begin
            {Update the number of represented pixels}
            inc(Tree^.ColorCount);
            {Sum the color values}
            AddColors(Tree^.RGB,RGB)
        end
    else
        InsertTree(Next[Branch(RGB)],RGB);
    end
end

```

The next step is to combine successors of an intermediate node to one leaf node if there are more than the number of colors permitted in the colormap. Closest neighbors are found by putting the position of each leaf node in the octree into a linear list in sorted order and then finding leaves belonging to the same parent node. This is done by masking out the bits below a given parent node level and then finding a parent node with two or more children.

```

ReduceTree(Tree);
begin
    {Find a reducible node}
    GetReducible(Tree);
    Sum := (0,0,0);
    Children := 0;
    for i : integer := 0,7 do
        if Next[i] <> NIL
            {There is an ith successor}
            then
                begin
                    Children := Children + 1;
                    AddColors(Sum,Tree^.Next[i],RGB);
                end;
    Tree^.Leaf := True;
    {Cut the tree at this level}
    Tree^.RGB := Sum;
    {The node represents the sum of all color values of its children}
    Size := Size - Children + 1;
end

```

end

- Filling the color table

The k leaves of the octree contain the colors for the color table (RGB/Color Count). The colors are written to the table by recursively examining the octree and the index to the colormap is stored in the node.

- Mapping the original colors onto the representatives

The image is read a second time using the RGB values stored in an array. For each color in the input, the representative color in the octree is found and its index to the color table is returned.

```
Quant(Tree: Octree; Original_Color: Color): index;
begin
    if Leaf
    then
        return Tree^.ColorIndex
    else
        Quant(Tree^.Next[Branch(Original_Color)],Original_color);
    end
end
```

12.6.1.1.2 - Color Quantization Data Structures

- The following structure for an octnode is the basic structure used for mapping RGB values into the octree:

```
int          structure octnode{
              leaf,
              colorcnt,
              colorindex,
              level,
              sumR,
              sumG,
              sumB;
struct octnode *next[8];
}
```

- level is the level of the node in the octree,
- leaf indicates whether the node is an interior node in the octree or a leaf node,
- If the node is a leaf node,
 - colorcnt counts the number of input colors that have been mapped to the node,

- sumR, sumG, and sumB are the sums of the color components of the colors that have been mapped to the node,
 - colorindex is the index assigned to the color in the node when the colormap is made.
- Each node holds pointers to eight children. If the node is a leaf node, these pointers must be NULL pointers.
 - A list is used to hold octree positions for the representative colors in the colormap. The list is sorted in ascending order to facilitate finding the most closely related colors when it becomes necessary to reduce the number of colors to the maximum size allowed in the colormap.

`list[256]`

- The colormap is a 3 by 256 array to hold the red, green, and blue color intensities.

`colormap[3][256]`

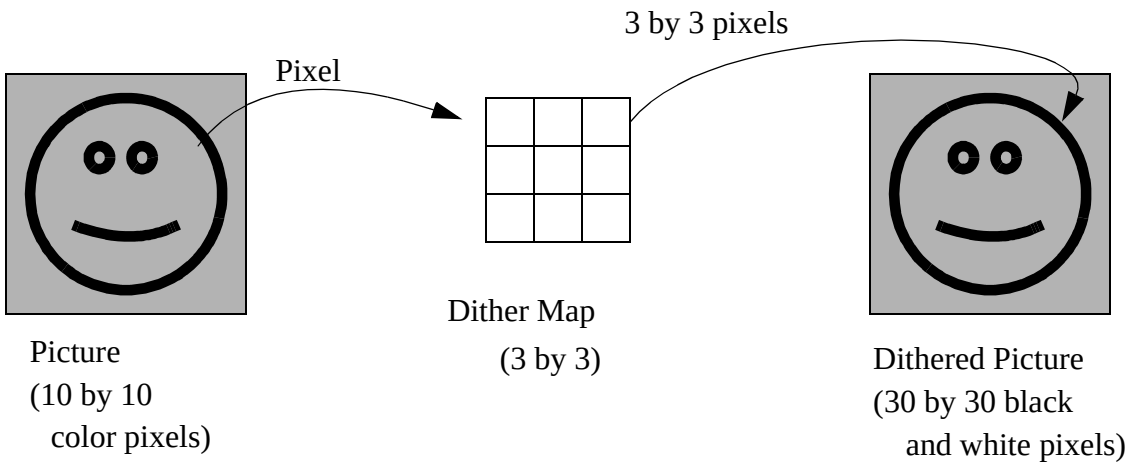
- When colors are to be displayed, the color intensities for the pixels are obtained with an index into the colormap. The module which displays the ray tracer image also uses this data structure.
- The RGB color vectors formed when reading the color intensities from the input file into the color quantizer are stored in an array:

`pict[]`

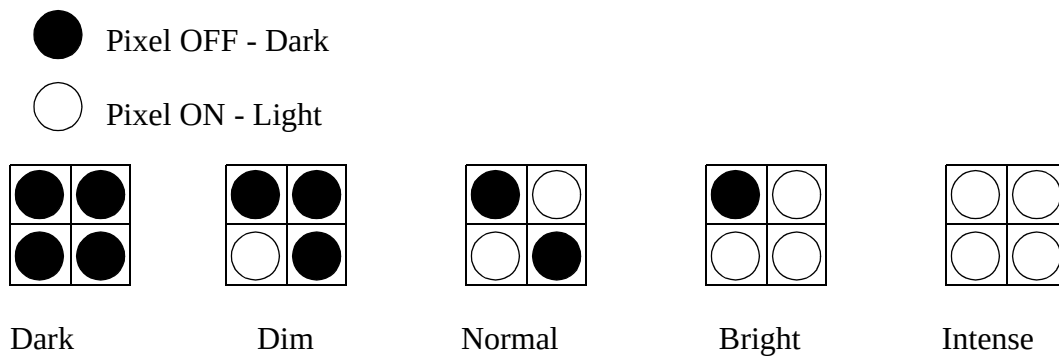
- The color vectors are used initially as representative colors to establish the colormap and a second time to obtain the indices for the original colors.

12.7 DITHERING

- Sometimes we want to display color images in black and white. (a very common use is when printing shaded images).
- The average pixel is composed of the Additive colors Red-Green-Blue (RGB)
- We assume that the intensity (brightness) of the RGB value can be used as a reasonable approximation. Unfortunately, colors like RED and BLUE may appear to be the same with this method.
- The intensity is then used to set a few pixels ON or OFF in a grid. This is assuming a Black, and White device.



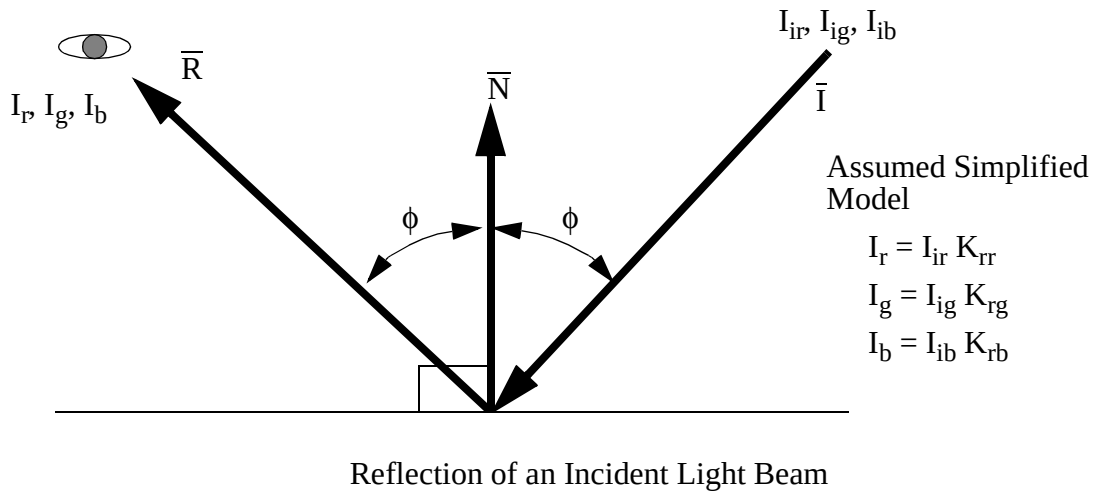
- Say the Dither Map is 2 by 2, In this scheme there are 5 different intensity levels.



- In dithering the dot patterns are made to be at 45 degree angles. This is because if the pattern were horizontal, or vertical, the human eye would detect the repeated pattern easily (this is also done with newspaper pictures).

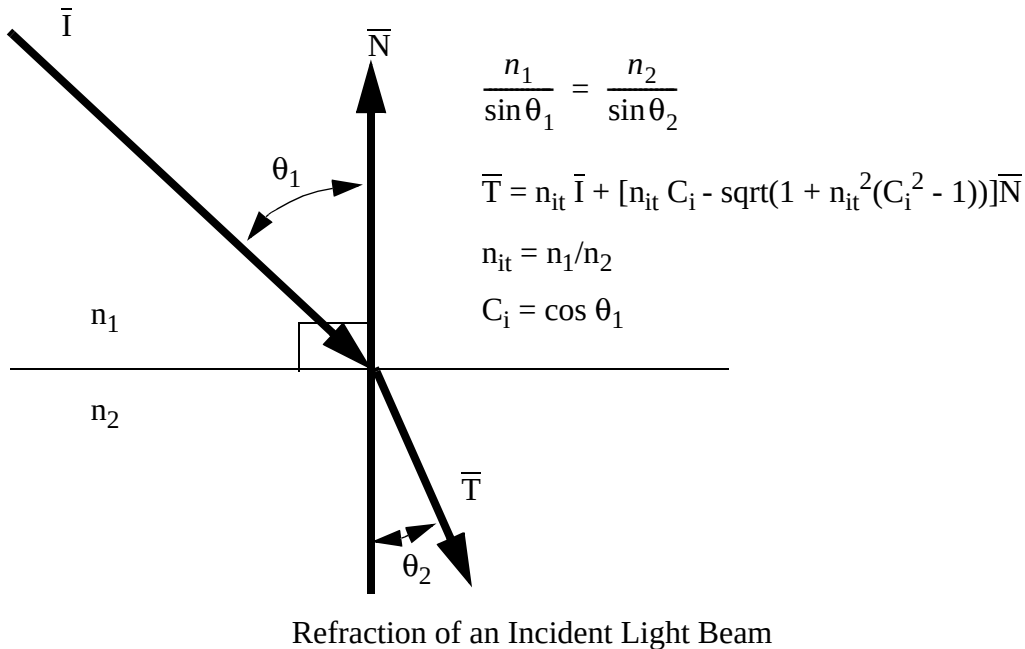
12.7.1 A Model for Light Ray Reflection

- When light strikes a surface it is often reflected. The reflection model is quite simple. In this case a simple fraction of the incoming light will be used. More sophisticated models may be constructed using information about particular materials.

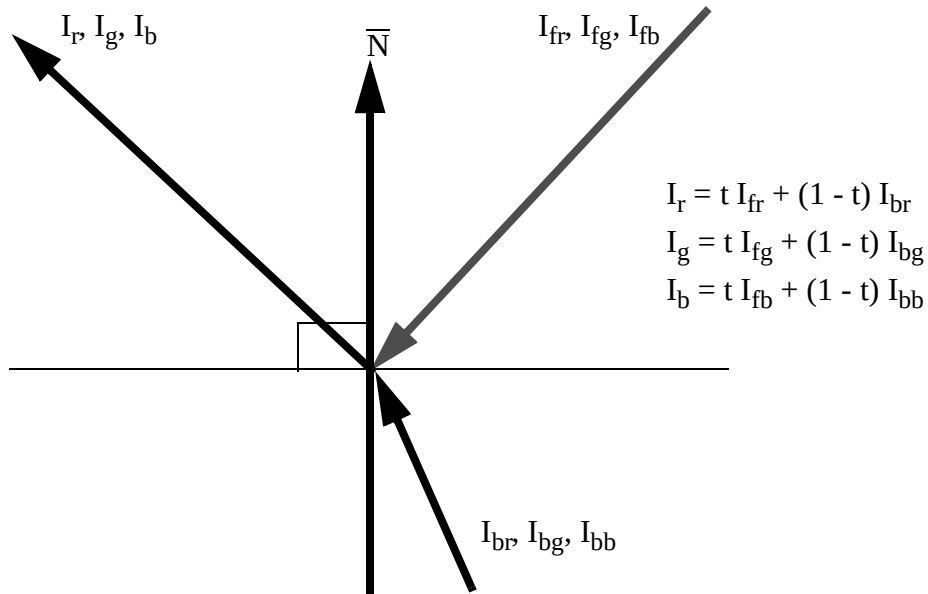


12.7.2 A Model for Light Ray Refraction:

- When light enters or exits material, its path changes. There are also optical adjustments based on the transparency of the specific material. The path change is determined by Snell's law, and the change in optical properties are considered by a simplified formula.



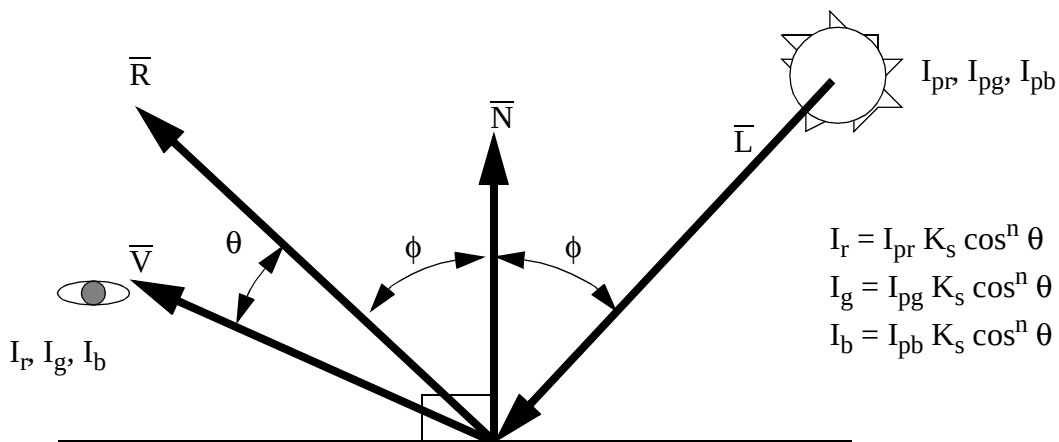
- After the collision vector has been calculated, the object's transparency must be taken into account. This is done by using a transparency coefficient 't'. When 't' has a value of 1 the object is opaque, and all light originates from the surface. If the object has a 't' value of 0, then all light is from refraction.



Light Transmission in the Presence of Transparency

12.7.3 A Model for Specular Reflection of Point Light

- The highlights which often appear on an illuminated object can be estimated also. In this case the Phong model is used. This model includes an estimate of how ‘fuzzy’ the light patch appears. This is not done if the collision point lies in a shadow.



Specular Reflection of Light

12.8 RAY TRACING

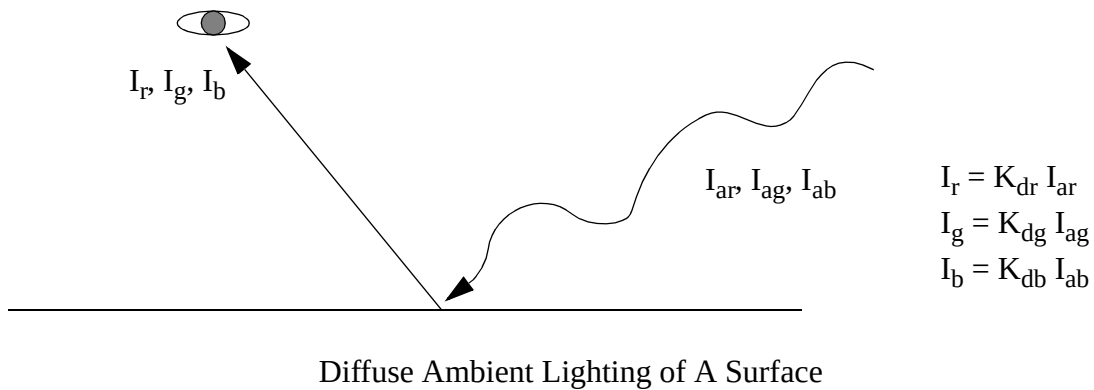
- This method no longer uses polygons, but relies directly on the geometrical model.
- The lines of light which make up the pixels of the picture, are back traced, and followed through reflection, refraction, and ambient light.
- This method generates a horrendous number of calculations.
- The figure on the next page shows how ray tracing is done for the first bounce, many other bounces may be performed for more realism.
- Advantages,
 - Realistic texture and appearances are easy to use
 - This is the only display method suited to rendering curved surfaces
 - The images give photograph quality
- Disadvantages,
 - Incredibly Slow due to large number of calculations
 - Requires a very powerful computer
- Support technology is being produced at a tremendous rate.
- Currently used for producing pictures and videos for sales marketing, etc. Many commercials on TV are produced with this method. Used in movies, like Terminator2, the Abyss, etc.

12.8.1 Basic Ray Tracing Theory

- The theory of ray tracing has two basic concerns. Firstly, the light path is of interest, secondly the light transmitted is also of interest. These both involve interactions with real objects, thus light-object interaction must also be covered in this section. Note that in most models discussed the geometric relations are not given. Almost all of these relations are given, derived from other sources, or easily obtained with dot products.

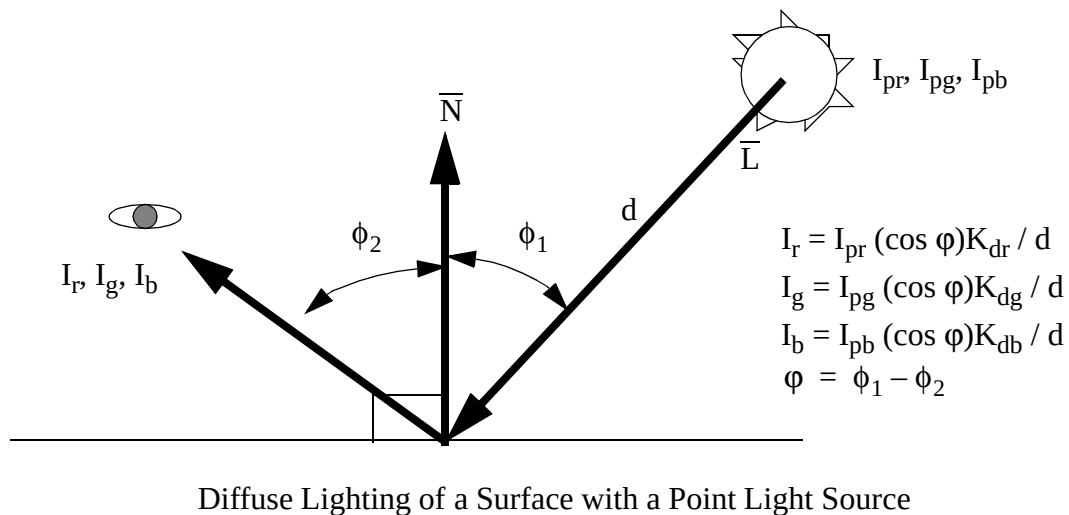
12.8.1.1 - A Model for Diffuse Reflection of Ambient Light

- Ambient light is assumed to be evenly distributed over the entire volume of space. This naturally occurring light will provide a basic level of illumination for all objects.



12.8.1.2 - A Model for Diffuse Reflection of Point Light:

- Point lighting is somewhat more sophisticated than Ambient lighting. In this case the direct light will illuminate the surface. The effect is approximated using the angle to the light source, and some constants (based on material properties) to estimate the 'brightness'. Please note that this calculation is not done if the collision point lies in a shadow.



12.8.1.3 - Collision of a Ray with a Sphere:

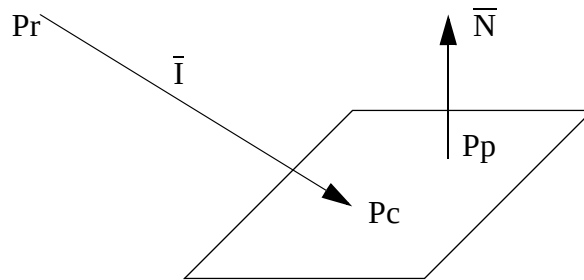
- The method for finding the intersection between a ray and a sphere has been covered in “An Introduction to Ray Tracing” by Glassner [1989]. Therefore, the method will only be reviewed briefly. The method is quite simple because the relationship may be explained explicitly. To state the description in the book:

- i) Find distance squared between ray origin and centre,
- ii) Calculate ray distance which is closest to centre,
- iii) Test if ray is outside and points away from sphere,
- iv) Find square of half chord intersection distance,
- v) Test if square is negative,
- vi) Calculate intersection distance,
- vii) Find intersection point,
- viii) Calculate normal at point.

- This algorithm gives at worst 16 additions, 13 multiples, 1 root, and 3 compares.

12.8.1.4 - Collision of a Ray With a Plane:

- A method for ray plane intersection was used to find collisions with the checkerboard. This method was devised from scratch. The proof will not be given, but it involves a parametric representation of the line, which is substituted into the plane equation. The parameter value was then found, and the collision point calculated. The algorithm for Line - Plane intersection is given below:



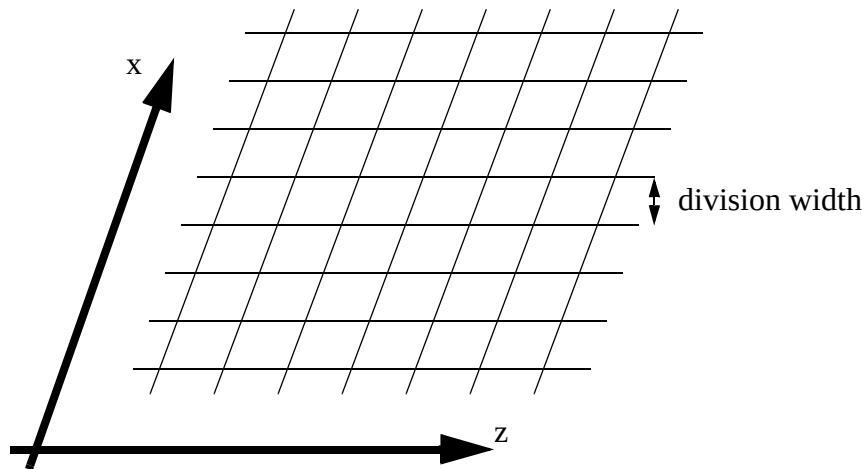
A Ray Colliding with a Plane

- i) The dot product between $\bar{I}$ and $\bar{N}$ is found,
 - ii) If the dot product is greater than, or equal to, zero then the line does not intersect the plane,
 - iii) The line-plane intersection parameter is calculated (the dot product may be used here),
 - iv) The intersection point is calculated.
- This algorithm is also based on an explicit relationship, therefore it is quite fast. In the best case there are 3 multiplies, 2 adds and 1 compare. In the worst case there are 12 multiplies, 1 divide, 10 adds/subtracts.

12.8.1.5 - Mapping a Pattern

- When we are rendering images we quite often will try to add a texture or pattern to the surface. This can be done by having a separate splines model of the object. While rendering the parametric model is interrogated as each surface point is being rendered.
- A simple example is the mapping of a checkerboard pattern on a plane using a function.

This function was simplified by aligning the checkerboard along the x-z plane. This avoided the problem of having to transform the point on the plane (thus speeding calculations). The only problem to be considered was how to map alternating squares onto the floor. This involved a calculation of a remainder value when dividing the x-z location by the grid spacing.



The remainder function will find the offset from an integer number of divisions. This value will be up to plus/minus half a division. Thus

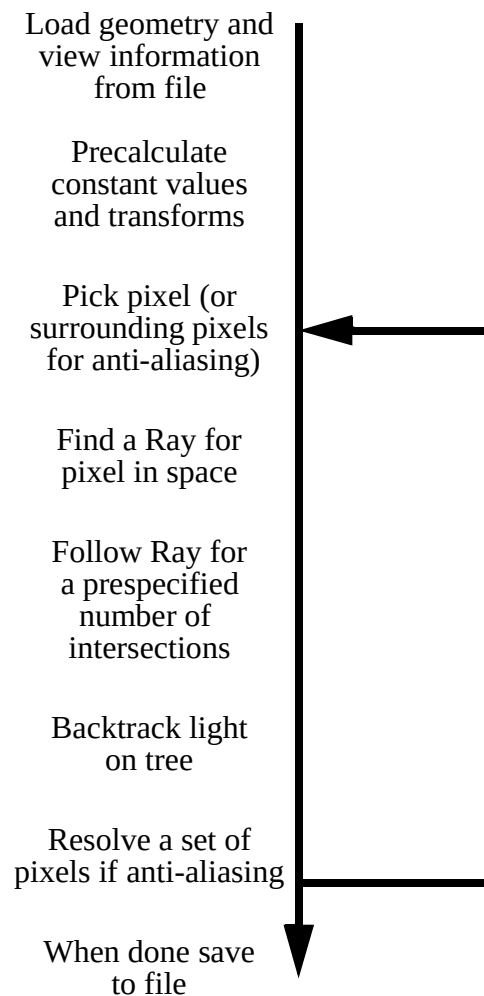
```
if((remainder(x, division) * remainder(y, division)) > 0.0) then
    assign color 1
else
    assign color 2
endif
```

A Surface with a CheckerBoard Pattern

12.8.2 Ray Tracer Algorithms

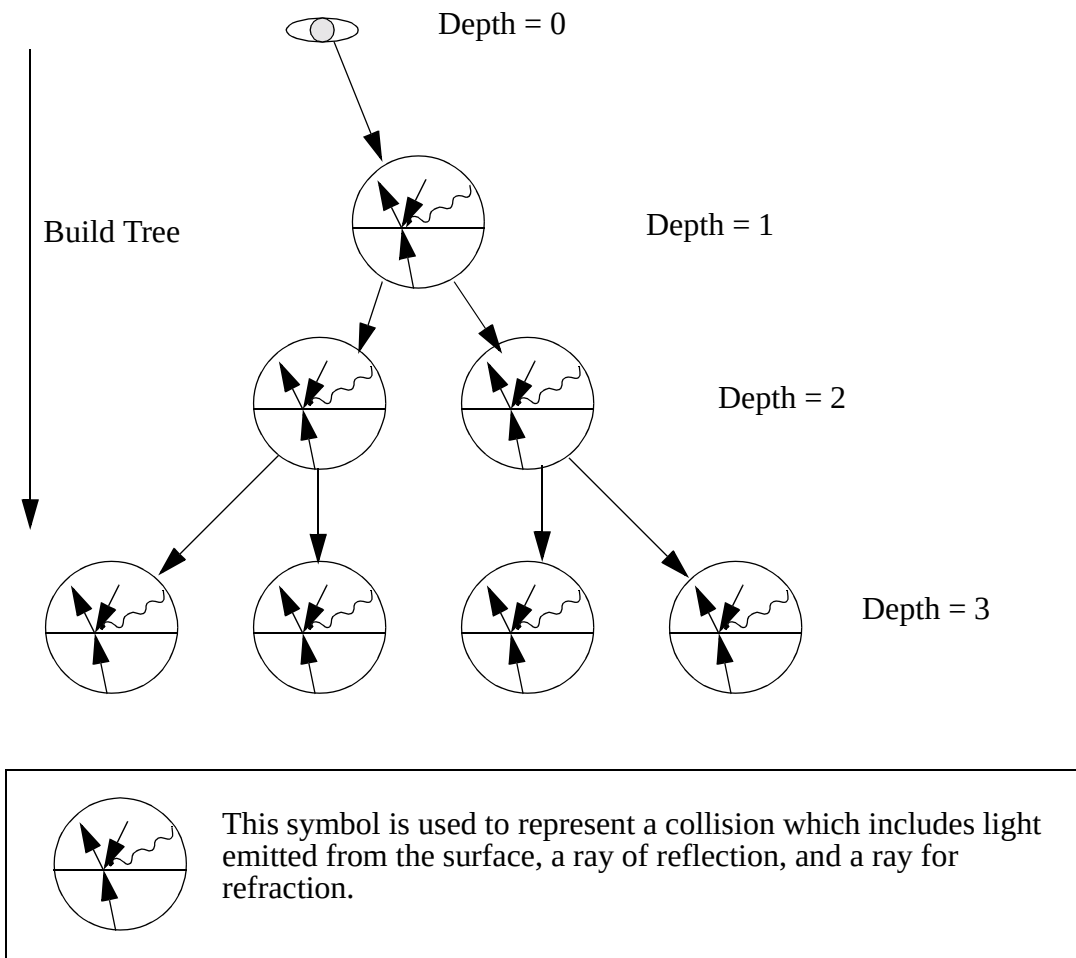
- This ray tracer will backtrack rays, and follow their paths through space. This involves a basic

set of functions. First the general algorithm description is given for ray tracing, in general,



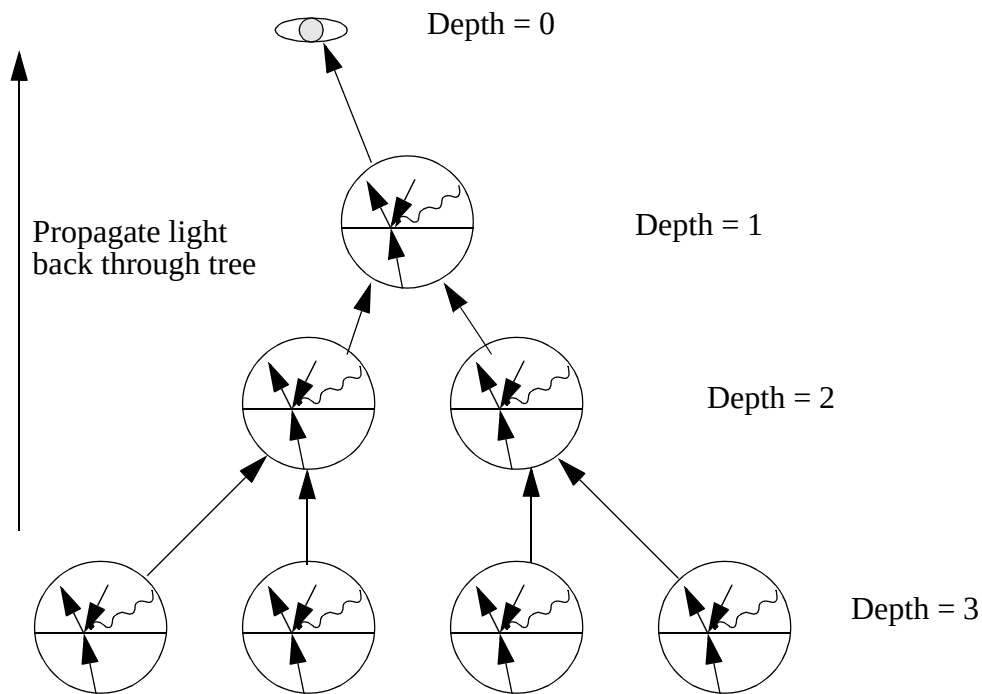
Basic Order of Higher Level Ray Tracer Features

- This structure involves performing as many calculations as possible before execution begins. Although not all calculations are done before, some tricks will be described below which greatly improve the speed of ray tracing.
- The process of filling up the image has multiple approaches. If a simple image is needed, then a simple scan of all the pixels can be made. An anti-aliasing technique requires oversampling of the pixels. When a pixel is chosen, the pixel must be mapped to a ray in space. This ray may be considered on a purely geometrical basis.
- When backtracking rays in space, they will either hit an object, or take on the background color. If an object has been hit by a light ray, it can either reflect or refract, and it will take on the optical properties of the surface. This leads to a tree data structure for the collisions.



The Basic Concept of Back Tracing Rays

- As seen in Figure above, the light rays may be followed, but the only colors (color explanation follows later) attached to any nodes in this tree are the diffuse lighting from ambient and point sources, and also the specular reflection. After the tree has been fully expanded, the light may be traced forward to the eye. The figure shows the light being propagated back up the tree to the eye (screen pixel).



The Basic Concept of Back Tracing Light Values in a Ray Tree

- This structure gives rise to a tree based on a linked list. Each node in the linked list has:
 - Forward and back pointers,
 - Surface normals, and optical properties for refraction (used for backtracking),
 - The lighting from diffuse and specular sources,
 - Vectors for before and after reflection and refraction,
 - Positions for ray origins and collision points,
 - The depth in the search tree,
 - The identification of which object the ray has come from and has hit.
- All of these properties are used in a dynamic fashion, and the list may be grown until all branches have been terminate (i.e. hit background) or are at the maximum search depth.
- This Ray tree structure is constructed with the data stored in a second structure. The second structure for scene data contains geometric information:
 - Objects for each ball including centers, radii, and ordered lists of the closest ball,
 - An object for the floor, eye, and light, including positions, orientations, and a list of balls, ordered from closest to farthest,
 - Transformation matrices,
 - The same perspective parameters used in the user interface,
 - A list of picture areas only filled with background light,

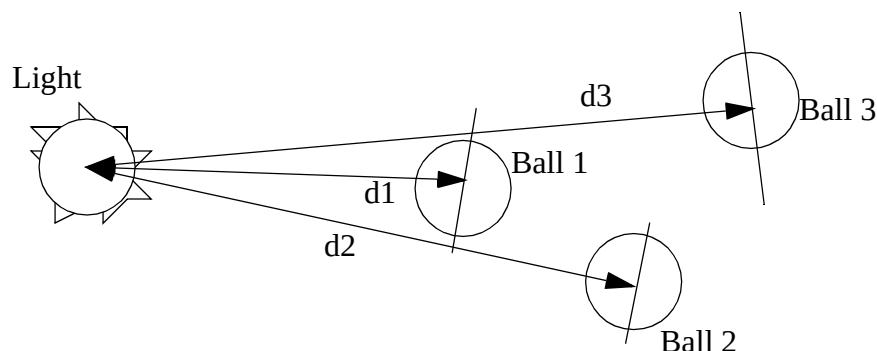
- Optical properties for balls, and floor: index of refraction, and shininess, transparency, diffuse, Phong, and specular constants,
 - The maximum ray tree depth,
 - Precalculated values for use in the program.
- Both of these groups of data items have been 'lumped' together in separate data structures. As mentioned before the rays are organized as a linked list. The geometric information is stored in a second structure. By using this approach it makes it very easy to pass the various classes of information around the system, without pushing large numbers of variables on the stack.

12.8.3 Bounding Volumes

- Bounding test volumes were set up, but these have proven to be ineffective in most cases where the scene is filled, their only benefit is in a nearly empty scene, where they save a ray calculation for every pixel. There are two main bounding volumes. The first volume is constructed around the set of balls. The second is constructed to represent the half of the screen which the floor will be displayed in (this could be all or none).

12.8.4 Shadows

- Within the image there are certain objects which will be in shadows. In these cases there is a reduction in the amount of light. To detect this case a ray is traced between the light, and the point of interest (the only objects which will be hit here are spheres). If the light hits any spheres, the shadow is confirmed. When a shadow is confirmed, the specular reflection, and diffuse lighting from a point source are ignored.
- To speed the checking process, the balls have been ordered by their distance from the light. The balls are checked in order, until one is hit, or the distance from the ball is greater than the distance between the point and the light.

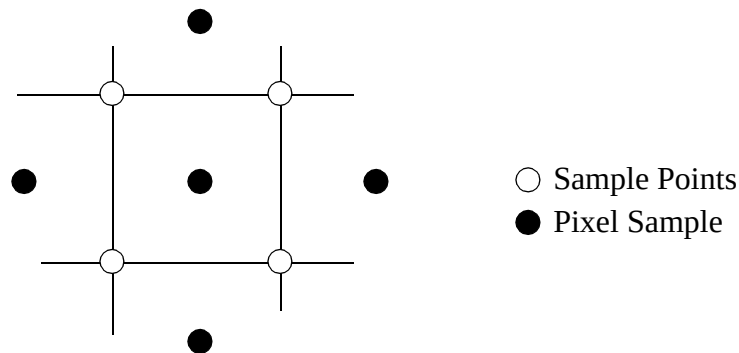


Depth Ordered Balls

- In the figure above there are a set of balls ordered by depth from the light source. If we are looking for a shadow on any point of Ball 2, we first find a vector from the point to the light. The search then begins backward from the light. If the light strikes Ball 1, then Ball 2 is in its shadow. If the light misses it then Ball 2 is not in a shadow. Say we are finding a shadow on the floor, and the floor is known to be some distance (from the light) between Ball 1 (d_1) and Ball 2 (d_2). The algorithm would check for collisions with Balls 1 and 2. A check would reveal that the distance for d_2 is greater than the distance to the floor, and since, no balls will have been hit, it may be concluded that the floor is not in a shadow, and the search is discontinued. This can save a great deal of search time when the light is close to the floor.

12.8.5 Aliasing

- A problem with ray tracing is that features smaller than a pixel can be missed. Another problem is that features not aligned to the pixel grid become choppy. A method for solving this problem is Supersampling Anti-aliasing. This uses an approach where the pixel map is sampled normally, as well as between pixels, and the pixel values are combined.



Sample Points for Anti-aliasing

The black points in Figure 18 are sampled, the four points around it are used to adjust the value. This will tend to give a local averaging effect which overcomes sudden discontinuities in image space.

12.8.6 Advanced topics

- Some advanced topics include,
 - Pattern and texture mapping,
 - Fog and lighting effects,

- Multiple light sources,
- Splines as primitives,
- Estimates of non-point light sources,
- New primitives,

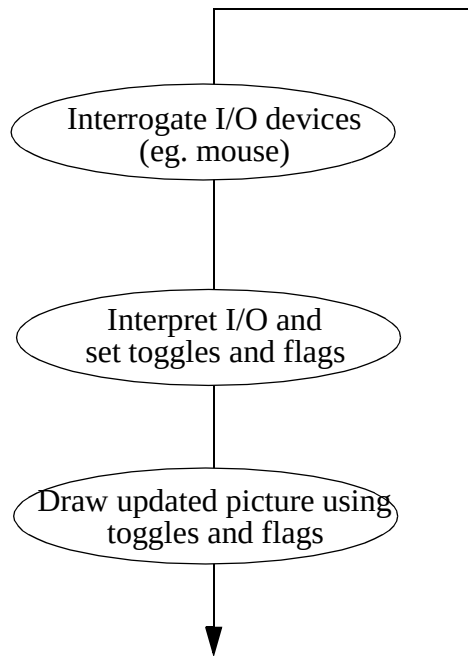
12.9 RADIOSITY

- An emerging technique for rendering ---XXXXXXXXXX

12.10 ADVANCED GRAPHICS TECHNIQUES

12.10.1 Animation

- To simulate motion we collect a set of still frames of a rendered object being translated and rotated. These are played back to simulate the motion.
- The paths of motion may be defined using splines, equations, functions, etc.
- Some tricks used for doing animation are,
 - Only drawing the part of the screen that is changing
 - Blurring images for fast moving objects reduced the flicker.
 - Alternating two pictures (double buffering). While one picture is being drawn, the previous picture is displayed.
- Buffer swapping gives smooth looking graphics by:
 - i) Swapping the screen with a memory block,
 - ii) Erasing the previous image (within a bounding box),
 - iii) Redrawing the image,
 - iv) Repeat the loop.
- When using interactive animations a program structure is required that separated user inputs from rendering.



- The first program loop examines inputs and user requests from the mouse, keyboard, etc. Based on these values we reset internal flags and registers.
- Based on the values of the internal registers the drawing is updated.
- During execution, the program loops through both of these operations, and thus provides a separation between simultaneous control, and display motion.
- With this architecture it is easy to add 'real time' control functions.

12.11 REFERENCES

Chang, T-C., Wysk, R.A. and Wang, H-P., Computer-Aided Manufacturing, Second Edition, Prentice Hall, 1998.

Foley, J., Van Dam, A., "Fundamentals of Interactive Computer Graphics", Addison Wesley, 1984.

Glassner, A.S., "An Introduction to Ray Tracing", Academic Press, 1989.

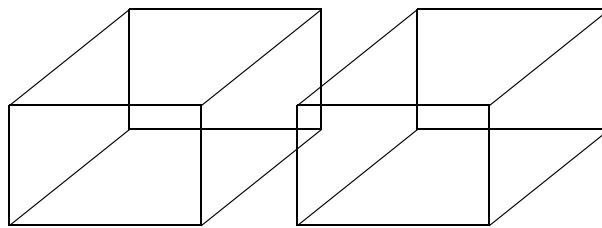
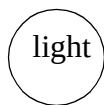
Glassner, A.S., "Graphics Gems", Academic Press, 1990.

Harrington, S., “Computer Graphics: A Programming Approach”, McGraw Hill, 1983.

Rogers, D.F., Adams, J.A., “Mathematical Elements for Computer Graphics”, McGraw Hill, 1990.

12.12 PRACTICE PROBLEMS

1 Given the geometry below, draw how it would appear with 5 different computer graphics techniques.



13. NEW TOPICS

13.1 VIRTUAL REALITY

- In some way a computer is used to replace, or augment reality for a user.

***** See attached articles

13.2 MULTIMEDIA

- The power of the computer has allowed computers to present information in a non-linear, single media manner.
- Computers can present documents with hypertext that allows the user to hop about, by clicking on words.
- Sounds, and moving pictures can also allow a user to get a more powerful understanding of information.

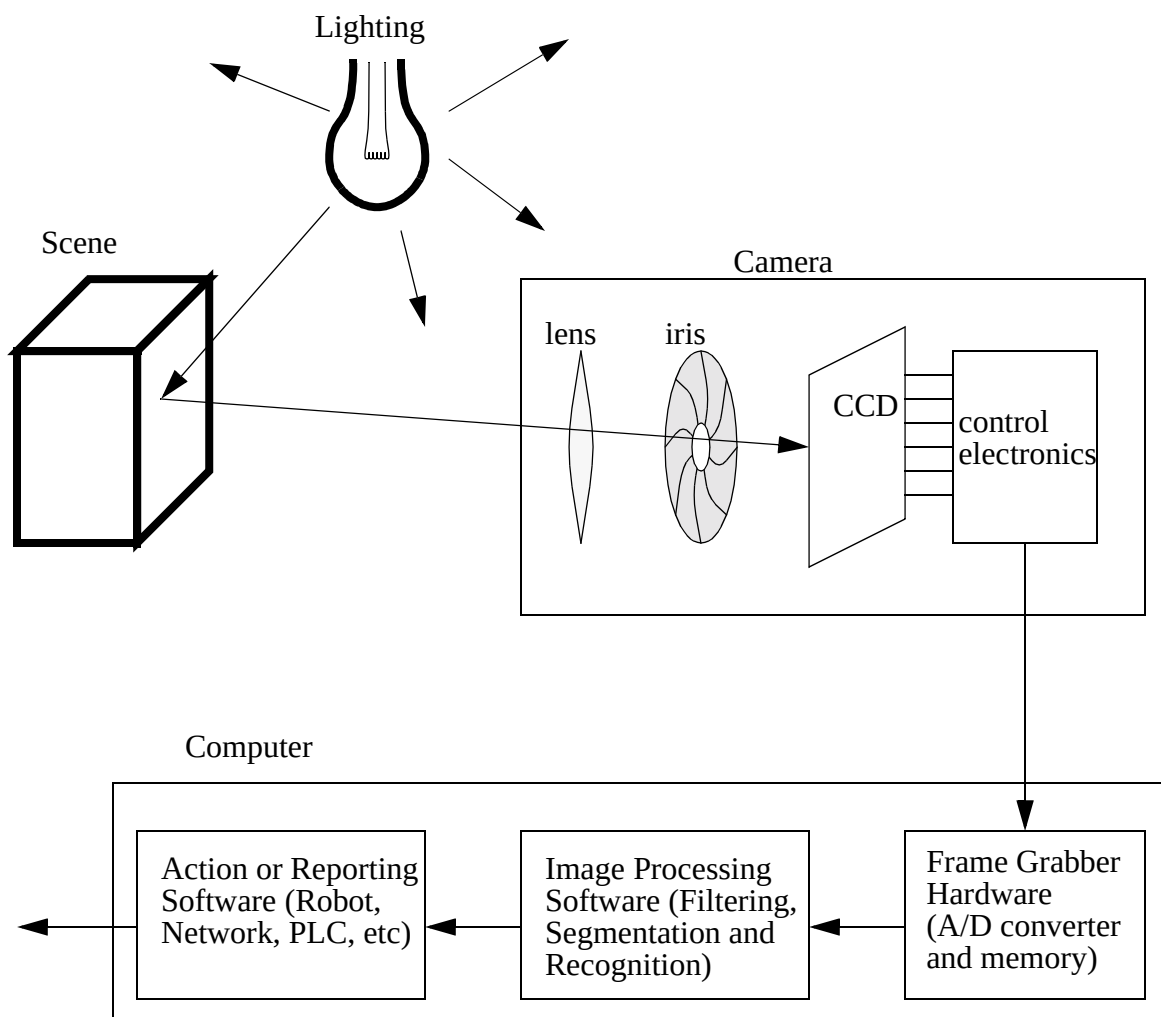
***** See attached article

14. VISIONS SYSTEMS

- Vision systems are suited to applications where simpler sensors do not work.

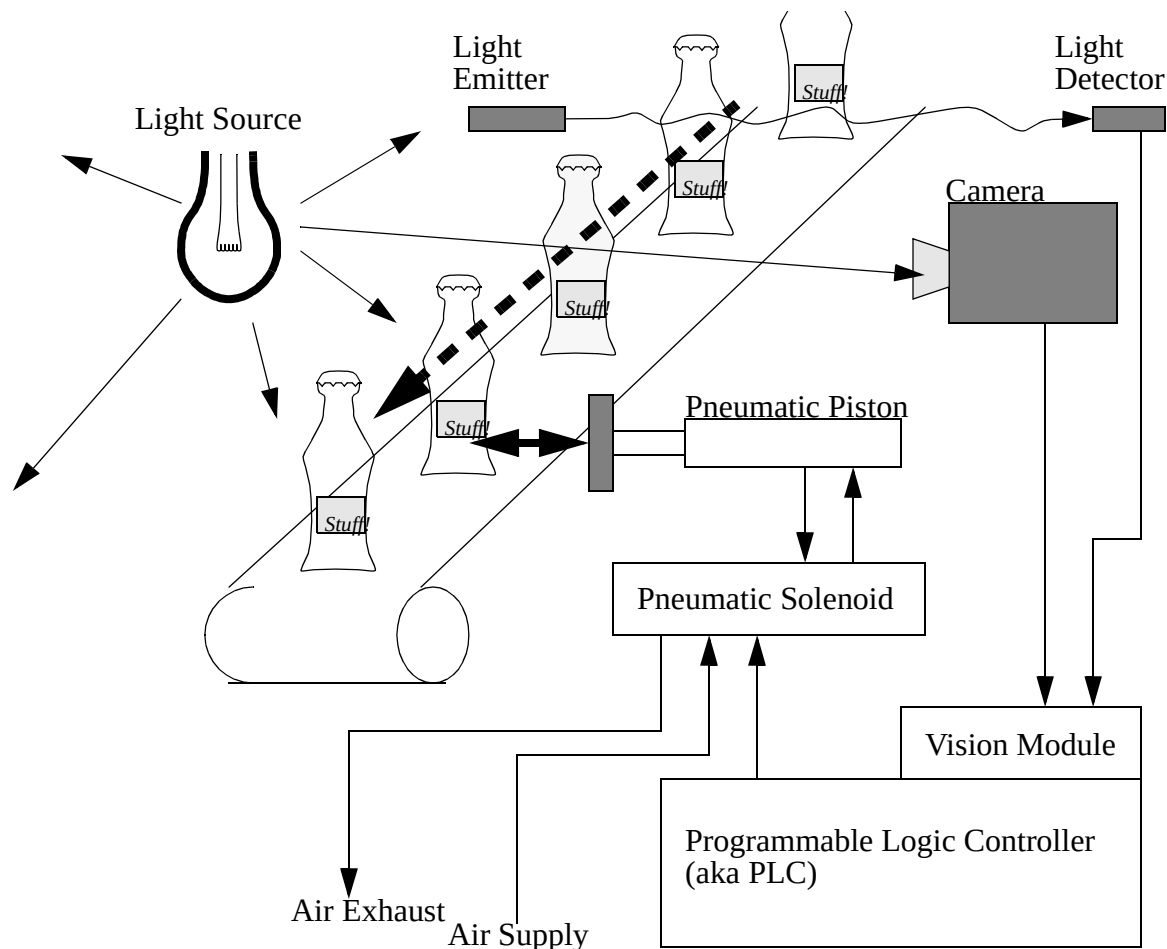
14.1 OVERVIEW

- Typical components in a modern vision system.



14.2 APPLICATIONS

- An example of a common vision system application is given below. The basic operation involves a belt that carries pop (soda) bottles along. As these bottles pass an optical sensor, it triggers a vision system to do a comparison. The system compares the captured image to stored images of acceptable bottles (with no foreign objects or cracks). If the bottle differs from the acceptable images beyond an acceptable margin, then a piston is fired to eject the bottle. (Note: without a separate sensor, timing for the piston firing is required). Here a PLC is used as a common industrial solution controller. - All of this equipment is available off-the-shelf (\$10K-\$20K). In this case the object lighting, backgrounds and contrast would be very important.



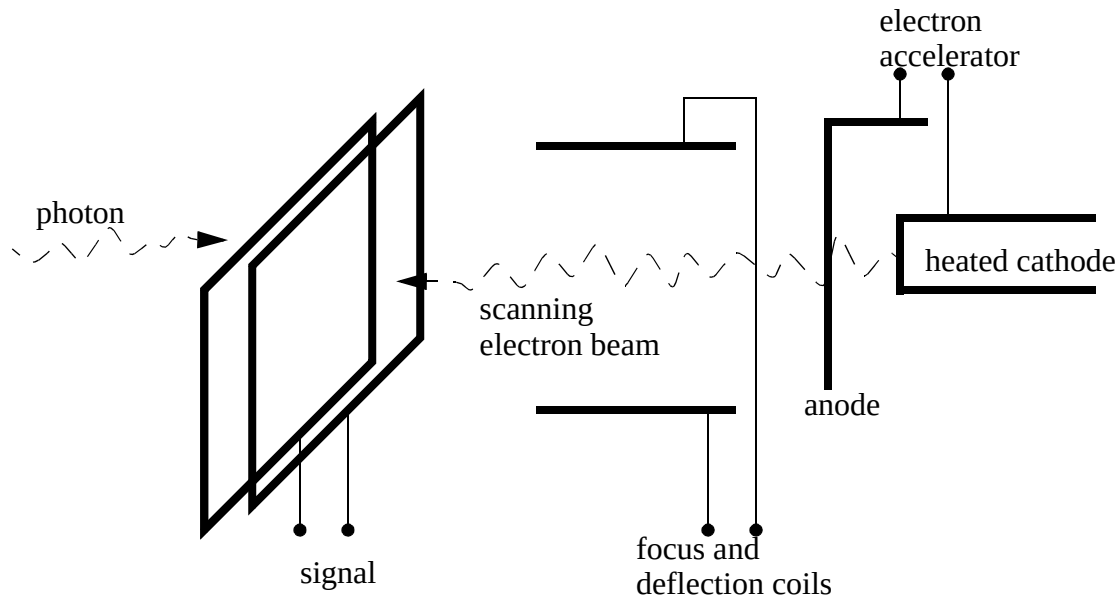
14.3 LIGHTING AND SCENE

- There are certain features that are considered important in images,
 - boundary edges
 - surface texture/pattern

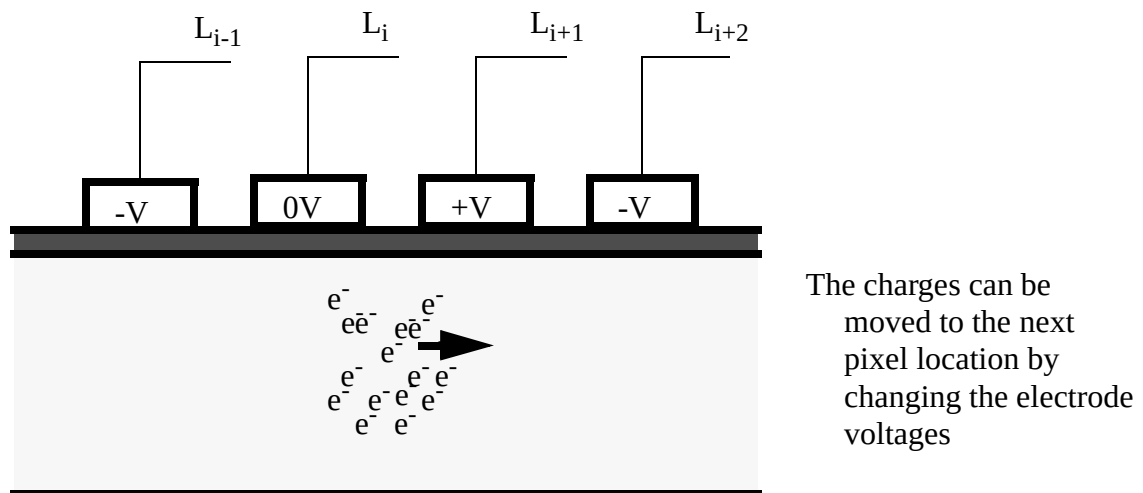
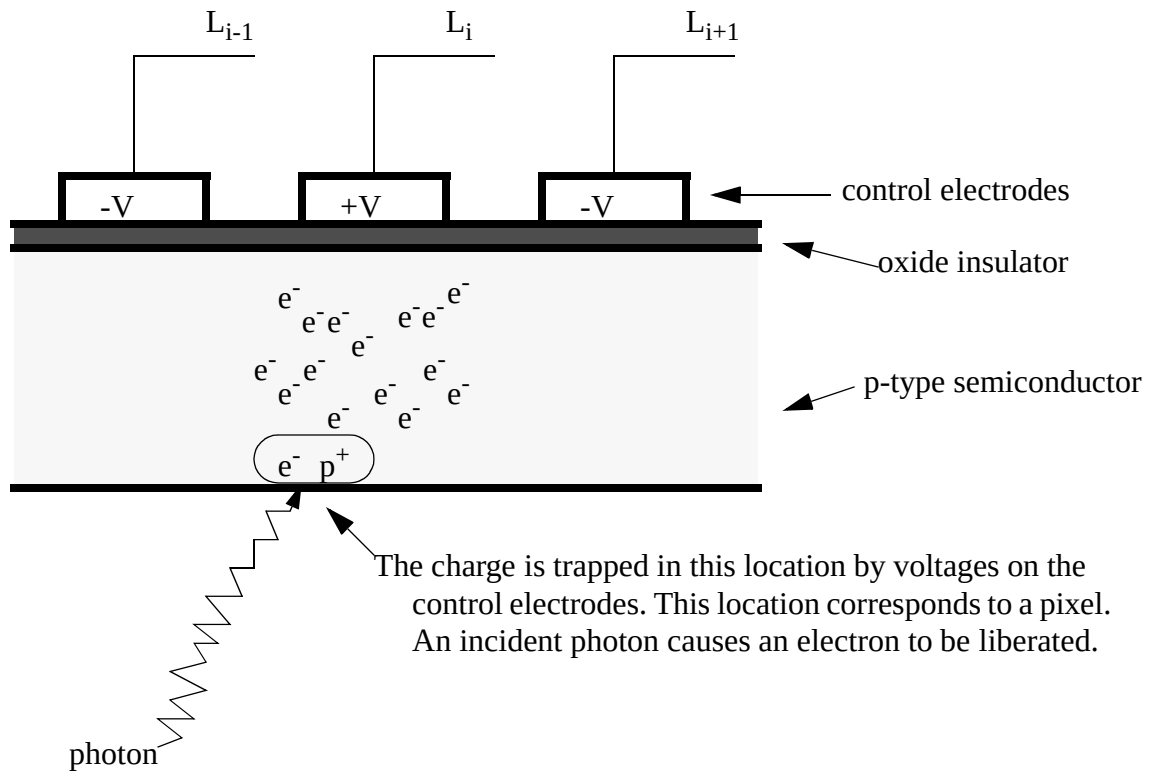
- colors
- etc
- Boundary edges are used when trying to determine object identity/location/orientation. This requires a high contrast between object and background so that the edges are obvious.
- Surface texture/pattern can be used to verify various features, for example - are numbered buttons in a telephone keypad in the correct positions? Some visually significant features must be present.
- Lighting,
 - multiple light sources can reduce shadows (structured lighting).
 - back lighting with luminescent screens can provide good contrast.
 - lighting positions can reduce specular reflections (light diffusers help).
 - artificial light sources provide repeatability required by vision systems that is not possible without natural light sources.

14.4 CAMERAS

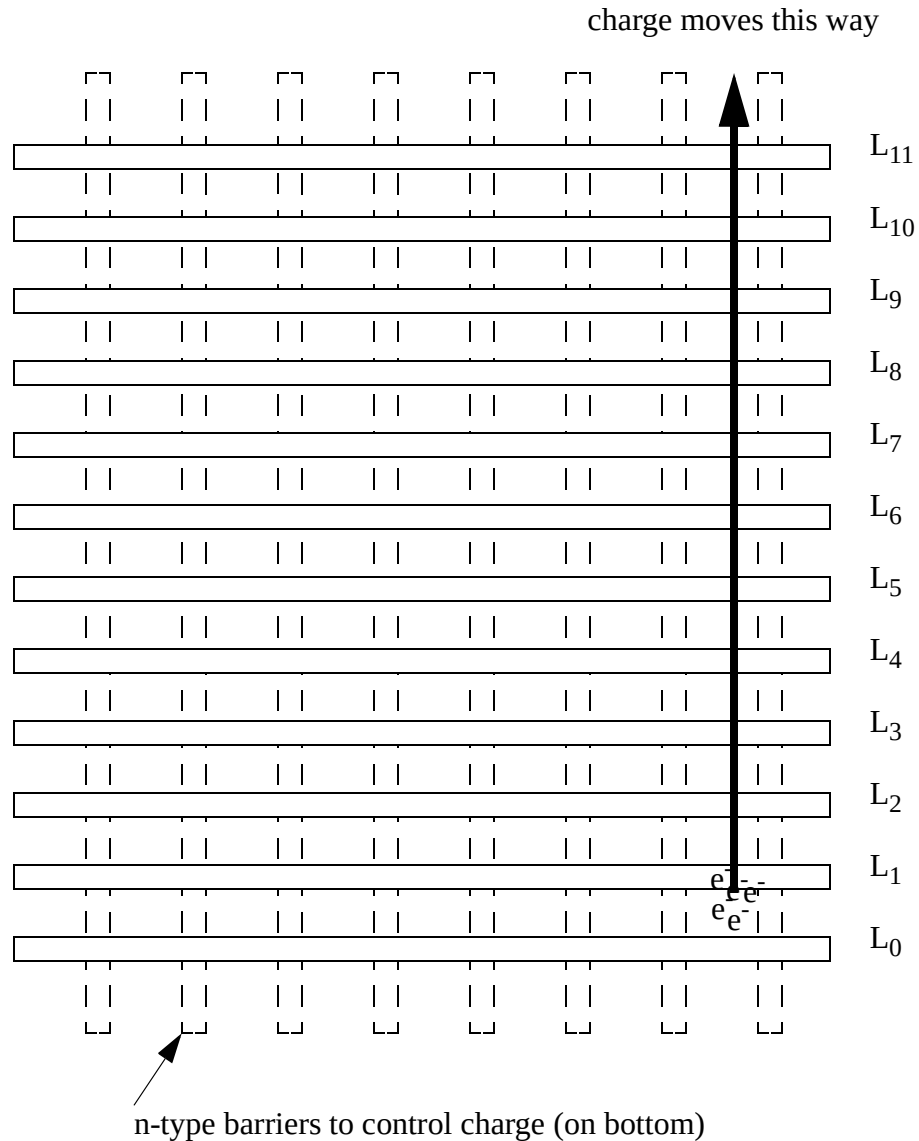
- Cameras use available light from a scene.
- The light passes through a lens that focuses the beams on a plane inside the camera. The focal distance of the lens can be moved toward/away from the plane in the camera as the scene is moved towards/away.
- An iris may also be used to mechanically reduce the amount of light when the intensity is too high.
- The plane inside the camera that the light is focussed on can read the light a number of ways, but basically the camera scans the plane in a raster pattern.
- An electron gun video camera is shown below. - The tube works like a standard CRT, the electron beam is generated by heating a cathode to eject electrons, and applying a potential between the anode and cathode to accelerate the electrons off of the cathode. The focussing/deflecting coils can focus the beam using a similar potential change, or deflect the beam using a differential potential. The significant effect occurs at the front of the tube. The beam is scanned over the front. Where the beam is incident it will cause electrons to jump between the plates proportional to the light intensity at that point. The scanning occurs in a raster pattern, scanning many lines left to right, top to bottom. The pattern is repeated some number of times a second - the typical refresh rate is on the order of 30Hz



- **Charge Coupled Device (CCD)** - This is a newer solid state video capture technique. An array of cells are laid out on a semiconductor chip. A grid like array of conductors and insulators is used to move a collection of charge through the device. As the charge moves, it sweeps across the picture. As photons strike the semiconductor they knock an electron out of orbit, creating a negative and positive charge. The positive charges are then accumulated to determine light intensity. The mechanism for a single scan line is seen below.



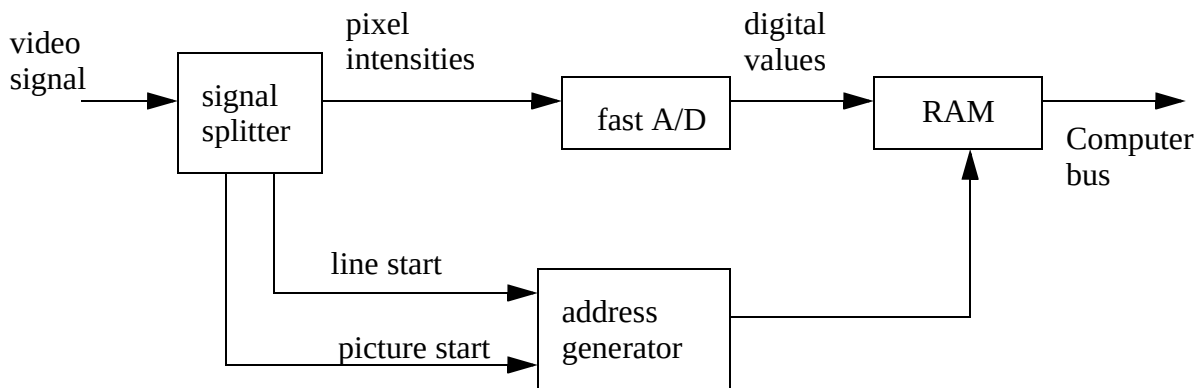
The description of moving the charge is for a single scan line, this can be expanded to consider the entire CCD.



- Color video cameras simply use colored filters to screen light before it strikes a pixel. For an RGB scan, each color is scanned 3 times.

14.5 FRAME GRABBER

- A simple frame grabber is pictured below,



- These items can be purchased for reasonable prices, and will become standard computer components in the near future.

14.6 IMAGE PREPROCESSING

- Images are basically a set of pixels that are often less than a perfect image representation. By preprocessing, some unwanted variations/noise can be reduced, and desired features enhanced.
- Some sources of image variation/noise,
 - electronic noise - this can be reduced by designing for a higher Signal to Noise Ratio (SNR).
 - lighting variations cause inconsistent lighting across an image.
 - equipment defects - these cause artifacts that are always present, such as stripes, or pixels stuck off or on.

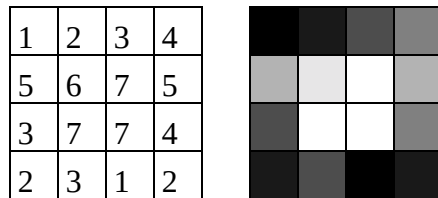
14.7 FILTERING

- Filtering techniques can be applied,
 - thresholding
 - laplace filtering
 - fourier filters
 - convolution
 - histograms
 - neighborhood averaging

14.7.1 Thresholding

- Thresholding basically sets a transition value. If a pixel is above the threshold, it is switched fully on, if it is below, it is turned fully off.

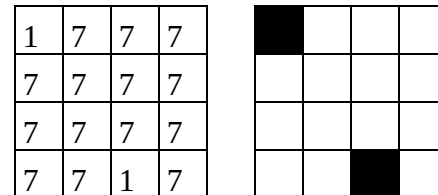
Original Image



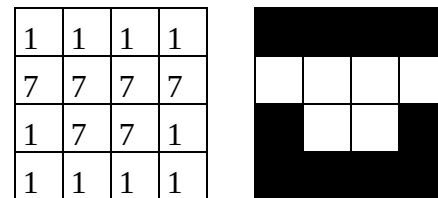
 an array of pixel brightness

It can be difficult to set a good threshold value, and the results are prone to noise/imperfections in the image.

e.g. Threshold = 2

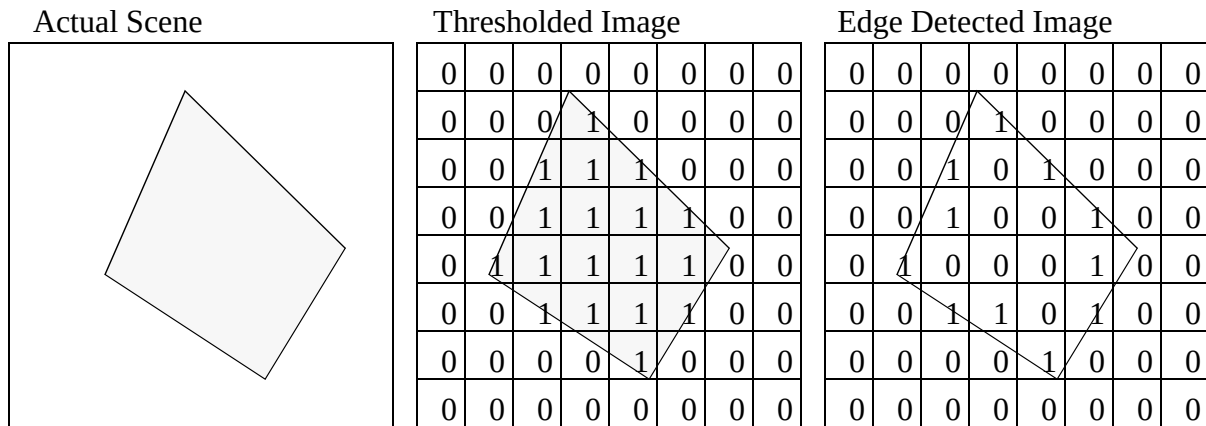


e.g. Threshold = 5



14.8 EDGE DETECTION

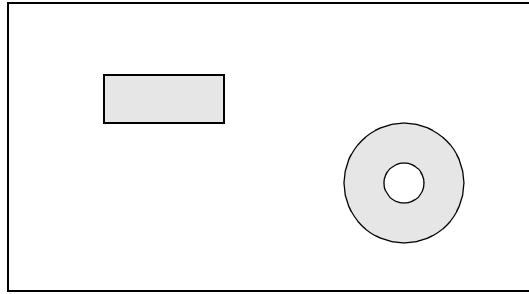
- An image (already filtered) can be checked to find a sharp edge between the foreground and background intensities.
- Let's assume that the image below has been prefiltered into foreground (1) and background (0). An edge detection step is then performed.



A simple algorithm might create a new image (array) filled with zeros, then look at the original image. If any pixel has a vertical or horizontal neighbor that is 0, then the value of 1 is copied across to the new image.

14.9 SEGMENTATION

- An image can be broken into regions that can then be used for later calculations. In effect this method looks for different self contained regions, and uses region numbers instead of pixel intensities.



Actual Scene

0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	1	1	0
0	0	0	0	0	0	0	1	1	0	1	0
0	0	0	0	0	0	0	0	1	1	1	0
0	0	0	0	0	0	0	0	0	0	0	0

Thresholded

1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1
1	1	2	2	2	1	1	1	1	1	1	1
1	1	2	2	2	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	3	3	3	1
1	1	1	1	1	1	1	3	3	4	3	1
1	1	1	1	1	1	1	1	3	3	3	1
1	1	1	1	1	1	1	1	1	1	1	1

Segmented

- A simple segmentation algorithm might be,
 1. Threshold image to have values of 1 and 0.
 2. Create a segmented image and fill it with zeros (set segment number variable to one).
 3. Scanning the old image left to right, top to bottom.
 4. If a pixel value of 1 is found, and the pixel is 0 in the segmented image, do a flood fill for the pixel onto the new image using segment number variable.
 5. Increment segment # and go back to step 3.
 6. Scan the segmented image left to right, top to bottom.
 7. If a pixel is found to be fully contained in any segment, flood fill it with a new segment as in steps 4 and 5.

14.9.1 Segment Mass Properties

- When objects are rotated in the vision plane it may become difficult to use simple measures to

tell them apart. At this point global attributes, such as perimeter lengths, length/width ratios, or areas can be used.

- The centroid of a mass can be determined with the expression for the x direction (y is identical)

$$\bar{x} = \frac{\sum M_i \tilde{x}_i}{\sum M_i} = \frac{1}{n} \sum_{i=1}^n \tilde{x}_i$$

where,

$\bar{x}$ = the x centroid from the left of the screen

n = the number of elements in the segment

$\tilde{x}_i$ = the distance from the left of the screen to the pixel centre

- Area is simply the sum of all pixels in the segment,

$$A = \sum p_i$$

where,

A = Area of image (in pixels)

p_i = 1 if the pixel is in the segment

- Perimeter is the number of pixels that can be counted around the outside of an object.

e.g.

0									8		Area = 21 Perimeter = 16 x Centroid = 3.5 y Centroid = 1.5
0		1	1	1	1	1	1	1	0		
		1	1	1	1	1	1	1	0		
		1	1	1	1	1	1	1	0		
		0	0	0	0	0	0	0	0		
4											

- Compactness can be a measure of mass distribution,

$$C = \frac{P^2}{A}$$

where,

C = compactness

P = perimeter

A = area

- Another measure of mass distribution is thickness,

$$T_{min} = \frac{D_{min}}{A} \quad T_{max} = \frac{D_{max}}{A}$$

where,

T = thickness

Dmin/Dmax = smallest/largest diameters

A = Area

14.10 RECOGNITION

14.10.1 Form Fitting

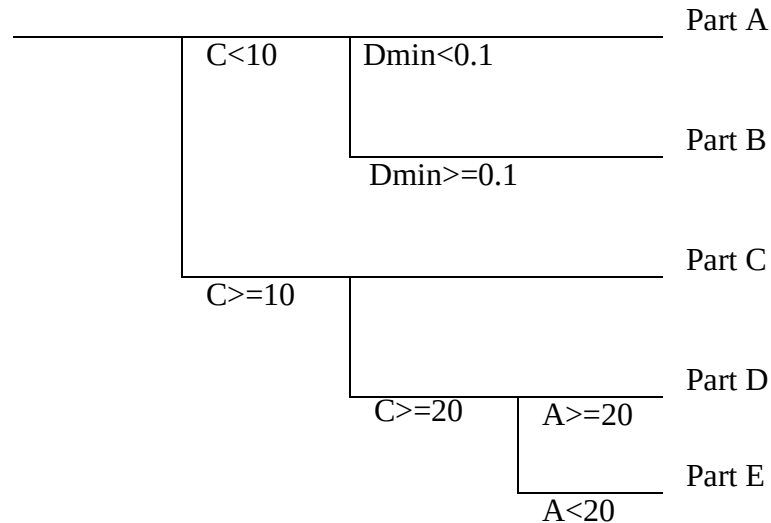
- It can sometimes help to relate a shape to some other geometric primitive using compactness, perimeter, area, etc.
 - ellipse
 - square
 - circle
 - rectangle

14.10.2 Decision Trees

- In the event that a very limited number of parts is considered, a decision tree can be used. The tree should start with the most significant features first, then eventually make decisions on the least significant. Typical factors considered are,
 - area
 - hole area

- perimeter
- maximum, minimum and average radius
- compactness

- An example of a decision tree is given below. (Note: this can be easily implemented with if-then rules or Boolean equations)



Bar Codes

- Bar codes are a common way to encode numbers, and sometimes letters.
- The code is sequential left to right, and is characterized by bars and spaces of varied widths. The bar widths corresponds to a numerical digits. These are then encoded into ASCII characters.
- To remain noise resistant there are unused codes in the numerical sequence. If any value scanned is one of the unused values the scan is determined to be invalid.
- There are different encoding schemes.
 - Code 39/Codabar - these use bars of two different widths for binary encoding
 - Code 128 - these use different bar widths uses proportional widths to encode a range of values
 - UPC (Universal Product Code) -
 - EAN (European Article Numbering) -
- The example below shows how a number is encoded with a bar code.

14.11 PRACTICE PROBLEMS

1. Consider a circle and an ellipse that might be viewed by a vision system. The circle has a 4" radius, whereas the ellipse has a minor and major radius of 2" and 4". Compare the two definitions using form factors (compactness and thickness) and show how they differ.

ans.

circle

$$R = 4$$

$$D_{min} = 8 \quad D_{max} = 8$$

$$A = \pi R^2 = 50.3$$

$$P = \pi(2R) = 25.1$$

Compactness values differ

$$C = \frac{P^2}{A} = 12.5$$

the min/max values are the same for the circle

$$T_{min} = T_{max} = \frac{D_{min}}{A} = 0.16$$

circle

$$R_1 = 2 \quad R_2 = 4$$

$$D_{min} = 4 \quad D_{max} = 8$$

$$A = \pi R_1 R_2 = 25.1$$

$$P \approx 2\pi \sqrt{\frac{R_1^2 + R_2^2}{2}} = 19.9$$

$$C = \frac{P^2}{A} = 15.8$$

$$T_{min} = \frac{D_{min}}{A} = 0.16$$

$$T_{max} = \frac{D_{max}}{A} = 0.32$$

2. Describe image resolution in vision systems.

ans. Resolution of a video image describes the number of rows and columns of pixels in a video image. A higher resolution means that there are more rows of pixels in the images, and therefore we can distinguish smaller details.

3. An image has been captured from a video camera, and stored in the matrix below.

64	87	54	64	12	35	22	36
36	57	76	24	84	26	63	74
84	187	201	234	195	222	198	25
54	78	197	198	34	75	218	74
25	9	84	202	194	213	192	79
37	25	57	98	93	95	91	89

a) Use a threshold of 100 to filter the image.

ANS.

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	0
0	0	1	1	0	0	1	0
0	0	0	1	1	1	1	0
0	0	0	0	0	0	0	0

b) Perform an edge detection on the thresholded image.

ANS.

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	0
0	0	1	1	0	0	1	0
0	0	0	1	1	1	1	0
0	0	0	0	0	0	0	0

c) Segment the image into distinct regions.

ANS.

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	0
0	0	1	1	2	2	1	0
0	0	0	1	1	1	1	0
0	0	0	0	0	0	0	0

d) Calculate the compactness and thickness for the region above the threshold.

ANS.

$$C = \frac{(22)^2}{13} \quad T = \frac{3}{13} \text{ OR } \frac{6}{13}$$

e) Calculate form factors including perimeter, area, centroid, compactness and minimum and maximum thickness.

4. We have four part shapes (as listed below) that will be arriving on a conveyor. We want to develop a decision tree for the vision system to tell them apart. We also need to find their centroids relative to the top left of the image so that a robot may pick them up.

Isosceles triangle 6" per side

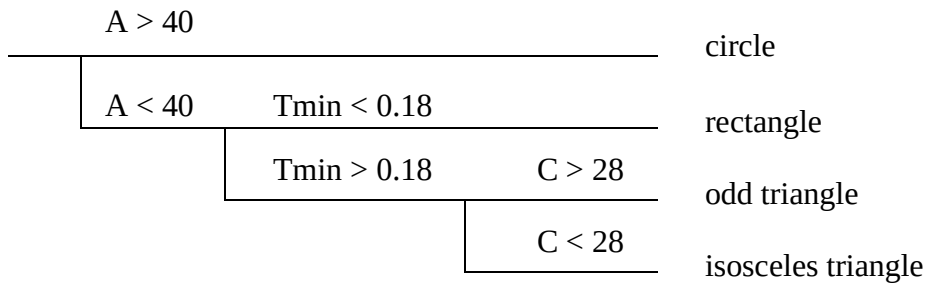
Rectangle 2" by 8"

Triangle with side lengths 8", 5" and 4"

Circle 5" Radius

ans. First, calculate the form factors

Form	Area	Perimeter	Compact	Tmin
isosceles triangle	15.59	18	20.78	0.3333
rectangle	16	20	25	0.125
odd triangle	8.18	17	35.33	0.25
circle	78.54	31.42	12.57	0.127



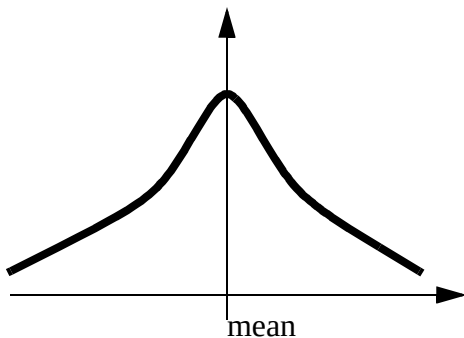
15. SIMULATION

- Some complex systems can't be modeled because of,
 - random events
 - changing operating conditions
 - too many interactions
 - exact solutions don't exist
- Simulation is used to determine how these systems will behave
- Simulation typically involves developing a model that includes discrete stations and events that occur with some probable distribution.
- We can then examine the simulation results to evaluate the modeled system. Examples include,
 - machine utilization
 - lead time
 - down time
 - etc.
- This is a very effective tool when considering the effect of a change, comparing decision options, or refining a design.
- Some simulation terms include,
 - System - the real collection of components
 - Model - a reasonable mathematically (simpler) representation of the system
 - State - the model undergoes discrete changes. A state is a 'snapshot' of the system
 - Entity - a part of the system (eg machine tool)
 - Attributes - the behavior of an entity
 - Event - something that changes the state of a machine
 - Activity - when an entity is going through some activity. (eg, press cycling)
 - Delay - a period of time with no activity
- Good approach to simulation,
 1. Determine what the problem is
 2. Set objectives for the simulation
 3. Build a model and collect data
 4. Enter the model into a simulation package
 5. Verify the model then check for validity
 6. Design experiments to achieve goals
 7. Run simulations and collect results
 8. Analyze and make decisions

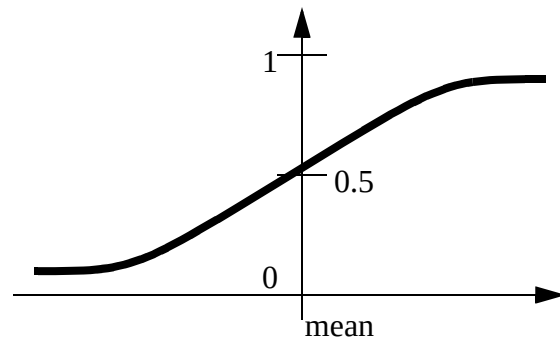
15.1 MODEL BUILDING

- If we are building a model for a plant floor layout, we will tend to have certain elements,
 - material handling paths (transfer)
 - buffers/waiting areas (delays)
 - stock rooms (source)
 - shipping rooms (destination)
 - machine tools (activities)
- Some of these actions can be stated as exact. For example, a transfer time can be approximated and random (manual labor), or exact (synchronous line), or proportional to a distance.
- Some events will occur based on availability. For example, if there are parts in a buffer, a machine tool can be loaded and activity occurs.
- Some activities and events will be subject to probabilities. Consider that the operation time in a press may vary, and there is probability of scrapping a part.
- The random variations can be modeled as,
 - discrete - for individual units
 - continuous for variations
- Well known distributions include,

Normal/Gaussian

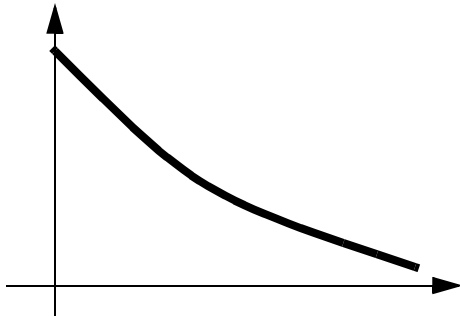


Probability Density

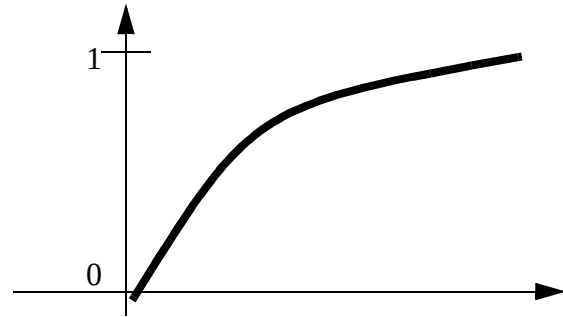


Cumulative Probability

Poisson/Exponential

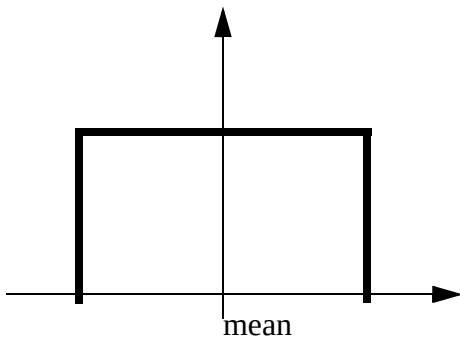


Probability Density

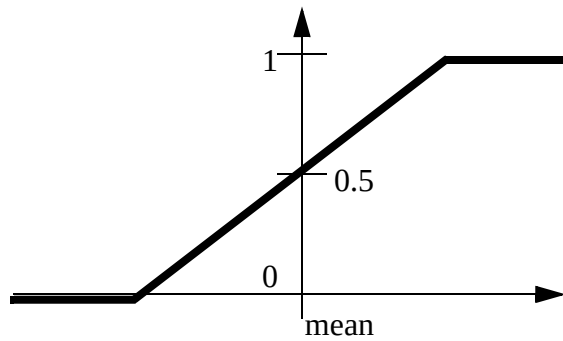


Cumulative Probability

Uniform

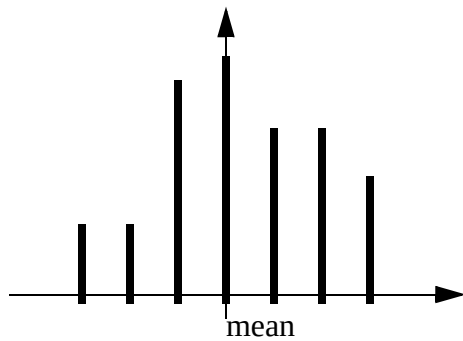


Probability Density

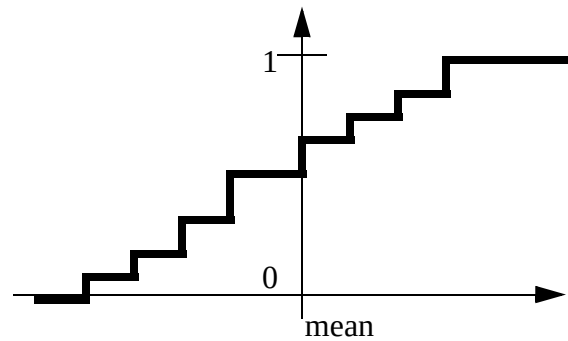


Cumulative Probability

Normal/Gaussian



Probability Density



Cumulative Probability

- This data may be found using data provided by the manufacturer, sampled in-house, etc.

15.2 ANALYSIS

- To meet goals, simple tests may be devised. These tests should be formulated as hypotheses. We can then relate these to the simulation results using correlation.

$$cov = \sum (x_i - \mu_x)(y_i - \mu_y) - \mu_x \mu_y \qquad corr = \frac{cov}{\sigma_x \sigma_y}$$

where,

cov = covariance of data sets x and y

corr = correlation of sets x and y

corr = 1 completely related

corr = 0 no relationship

corr = -1 inversely related

- Simulation software will provide information such as,

- production rates
- machine usage
- buffer size
- work in process

15.3 DESIGN OF EXPERIMENTS

- WHAT? combinations of individual parameters for process control are varied, and their effect on the output quality are measured. From this we determine the sensitivity of the process to each parameter.
- WHY? Because randomly varying individual parameters takes too long.
- e.g. A One-Factor-At-A-Time-Experiment

Effect: We are finding the causes of cracks in steel springs.

Causes:

1. Steel temperature before quenching 1450F or 1600F
2. Carbon Content .5% or .7%
3. Oil quench temperature 70F or 50F

Experiments 1 and 2:

Run 1:

1. 1450F	}	yield(%) 72 70 75 77, $\bar{X}=73.5\%$
2. 0.5%		
3. 70F		

Run 2:

1. **1600F	}	yield(%) 78 77 78 81, $\bar{X}=78.5\%$
2. 0.5%		
3. 70F		

Observation: 1600F before quench gives higher yield.

Run 3:

1. 1600F	}	yield(%) 77 78 75 80, $\bar{X}=77.5\%$
2. **0.7%		
3. 70F		

Observation: Adding more carbon has a small negative effect on yield.

Run 4:

1. 1600F	}	yield(%) 79 78 78 83, $\bar{X}=79.5\%$
2. 0.5%		
3. **50F		

Observation: We have improved the quality by 6%, but it has required 4 runs, and we could continue.

- The example shows how the number of samples grows quickly.
- A better approach is designed experiments
- e.g. DESIGNED EXPERIMENT for springs in last section

- set up orthogonal array

Run	1.	2.	3.	Yield%	Ri = $\bar{X}$
1	1450	0.5	50		
2	1600	0.5	50	79 78 78 83	79.5
3	1450	0.7	50		
4	1600	0.7	50		
5	1450	0.5	70	72 70 75 77	73.5
6	1600	0.5	70	78 77 78 81	78.5
7	1450	0.7	70		
8	1600	0.7	70	77 78 75 80	77.5

Note the binary sequence

- Find effects of each factor

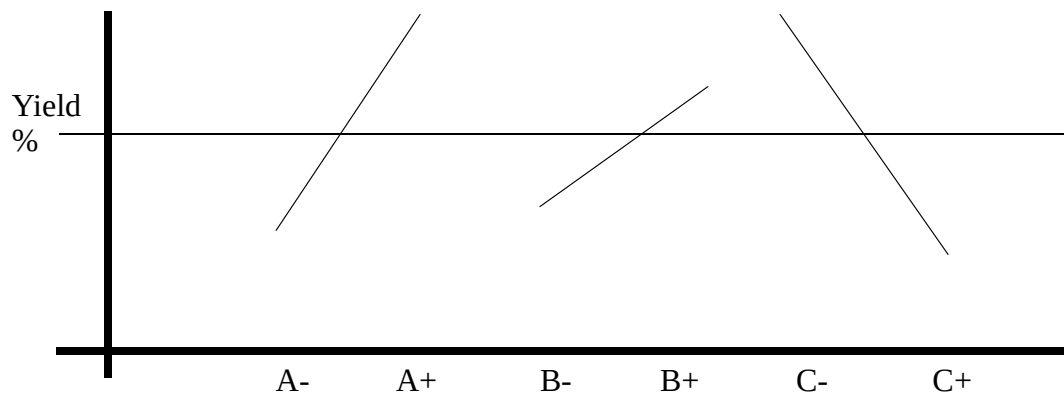
Main Effect = (Average at High) – (Average at Low)

$$\text{Main Effect of A} = \frac{(R_2 + R_4 + R_6 + R_8)}{4} - \frac{(R_1 + R_3 + R_5 + R_7)}{4}$$

$$\text{Main Effect of B} = \frac{(R_1 + R_2 + R_5 + R_6)}{4} - \frac{(R_3 + R_4 + R_7 + R_8)}{4}$$

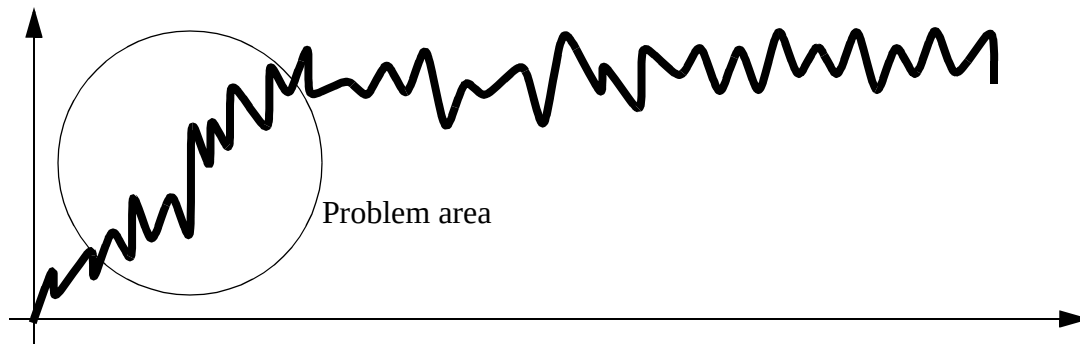
$$\text{Main Effect of C} = \frac{(R_1 + R_2 + R_3 + R_4)}{4} - \frac{(R_5 + R_6 + R_7 + R_8)}{4}$$

- these can be drawn on an effect graph



15.4 RUNNING THE SIMULATION

- When a simulation is first run it will be empty. If it is allowed to run for a while it will settle down to a steady state. We will typically want to,
 - run the simulation for a long time
 - or, delay the start of data collection
 - or, preload the system with parts



15.5 DECISION MAKING STRATEGY

- The general sequence of thought when making decisions is,
 - purpose
 - direction
 - plans
 - action
 - results
- General properties of strategy include,
 - time horizon
 - impact
 - concentration of effort
 - patterns of decisions
 - pervasiveness
- The levels of strategies include,
 - corporate
 - business
 - departmental/functional

- Decisions can be categorized,
 - hardware/fixed
 - capacity
 - facilities
 - technology
 - vertical integration
 - software/flexible
 - workforce
 - quality
 - production planning/material control
 - organization
- Typical criteria for making decisions might include,
 - consistency
 - harmony
 - contribution

15.6 PLANNING

A Historical Review of Artificial Intelligence Planning

by H. Jack

for
Prof. B. Mercer
Prof. C. Ling

1.0 Introduction

“The best laid schemes o’ mice and men
Gang aft a-gley” Robert Burns

One of the fundamental skills of intelligence is planning. Generally, planning is described as finding sequences of actions to satisfy one or more goals. Despite its importance, the process of planning is not well understood, and as a result is underdefined. This has lead to a number of attempts to mimic the planning process with computer based experiments. These Artificial Intelligence planners use various algorithms to approach a fundamental set of planning problems.

By their nature, planning problems are defined using goal states. If there is a single goal, the problem is often simple. Planning for multiple goals is also simple if the goals are completely independent. Some goals must be satisfied in a particular order; this occurs when they are independent but ordered. The most complex case arises when a number of goals must be satisfied simultaneously (this is non-linear planning; all other cases are linear).

To move between a start and goal state, operators are applied. The choices and orders of operators are both major concerns. A great deal of work has been done for selecting operators and choosing their order, but problems of complexity still tend to overwhelm most planning methods.

2.0 Planning Problems

Researchers have embodied the planning problems with a symbolic approach. Thus, the

problem is easily described in terms of lists, rules and expressions. This approach was first described by Green [1969] who discussed a planner based on resolution.

2.1 Representation of Problem Space

The representation of problem space is primarily split into two main subjects. Initially the state of the world must be described by a model. As a plan is executed, the world model will evolve through time. If a world model is not used (for simulation) then the operators must be applied to the world directly. This introduces potential problems when backtracking failed plans.

2.1.1 The World Model

A list of conditions (predicates) are stored in a list. As conditions in the world change, conditions are added and removed from the list.

2.1.2 The Solution (Plan) Model

Operators are applied which may update a world model. As the model changes, it will form a linear sequence of states over time. This is equivalent to a path through a graph [Korf, 1987]. As a result, the description of planning often takes on the terminology used for search.

2.2 Representation of Goals

At any time the world model contains a set of conditions. When some goals are introduced, they must describe the desired state of the world model. Thus, goals are described with a conjunctive expression of all the desired conditions in the world model. Each condition is considered a separate goal.

2.3 Operators

To describe actions, operators are used. Traditionally, an operator has a set of preconditions which must exist in the world model before an action can be performed. If the preconditions of an action are satisfied, the results of the action are simulated by deleting and adding

conditions to the world model.

It is important to note that this may result in one operator violating the preconditions of other operators.

2.4 Plans

The representation of plans is important (we will avoid the subject of planning until section 3). Ultimately, all plans must be stored as sequences of serial, concurrent or parallel preconditions and actions. It is also possible to store states after each operation. Including a state representation allows the ability to track plan progress, and to check for failures during execution. These advantages do not come without costs. It is easy to store sequential plans with all of their preconditions. Non-linear plans are much harder to store, because of the non-linear graph structure caused by inherent parallelism.

Korf [1987] described the various plan types and how they may be described with graph search theory. His work proved formally that sequential plan steps are preferable, although not always possible.

2.4.1 Sequential Plans

In its most simple form, a sequential plan is a list of operations. A more sophisticated plan also describes the state of the world model after each operation. One such method for doing this is triangle tables [Nilsson, 1980, pp. 282-286]. These tables store the result of each operation in (selected rows of) the column beneath. The rows selected are determined by which subsequent operations the results are preconditions for.

Figure 1 - A Triangle Table [Nilsson, 1980, pg. 285]

The plan's progress may be verified by examining the kernel. As shown in Figure 1, the kernel (outlined in double lines) for STACK(B, C) is a matrix of conditions. If these conditions match the world model, then the plan is progressing normally at STACK(B, C).

Nilsson also mentions the complete expansion of state space in a tree form [1980, pp. 282-284]. Each node becomes an intermediate plan state, and each arc becomes an operation. This approach is both memory intensive and inefficient, but recovery from plan execution failure is easier. An example of this approach applied to a blocks world problem is shown in Figure 2.

Figure 2 - A Search Tree in Plan Space [Nilsson, 1980, pg. 283]

2.4.2 Non-Sequential Plans

By treating all plans as sequential, we have assume that all operations are ordered, and two events may not happen simultaneously. In actuality, many operations are parallel in nature, and therefore it may be desirable to have plans which are concurrent and only partially ordered. The main disadvantages of unordered plans are that progress checking (and error recovery) will be more difficult, and the scheduling of events must still be specified.

An early planner called NOAH [Sacerdoti, 1975, 1977] used a representation called procedural nets. The nets had a starting point at the left hand side, from which plan execution would begin. During execution concurrent paths could branch and rejoin. Preconditions and operations were located at various positions along the paths. The only drawback to this representation is that conditions that were previously set can not be undone. Tate [1977] discussed an approach based on project networks, and claimed that he overcame the problem with procedural nets. These nets are similar to procedural nets except the goals and preconditions are not separated.

Figure 3 - A Procedural Net (for painting) [Sacerdoti, 1977, pg. 10]

Tate discussed finding an order for an unordered plan using Operations Research techniques on a project network. DEVISER [Vere, 1983] was able to generate plan representations with non-sequential operations, using an approach based on Tate's project networks.

2.5 Planning Objectives

A real problem rarely has a unique solution. Typically there are either multiple solutions, or none at all. When there are multiple solutions, a value analysis must be performed to find the greatest valued plan. If there are no solutions, goals must be compromised to reach a solution. A diagram by Gevarter [1985] which he derived from the book by Wilensky [1983] is shown below in Figure 4.

Figure 4 - Objective Evaluation [Gevarter 1985, pg. 71]

Although, this figure illustrates a comprehensive planning approach, planners commonly find the first solution possible, or quit if none is found.

3.0 Planners

Early planners used structured environments that could result in a single plan. Later plan-

ners started to deal with the complexity of real world problems. This section discusses a number of approaches to planning, but first, an outline of planning issues is covered.

3.1 Planning Issues

Exhaustive solutions to planning problems are quite infeasible for any large problem. For example, the large search tree for the simple blocks world problem makes this obvious in Figure 2. Thus, approaches have been formulated to exploit the nature of the planning process, and prune the search tree.

3.1.1 Means-End Analysis

A Search for a plan may be extensive when working from the start node. Starting at the goal state and reducing the difference between the goal and the start makes the problem more tractable. This technique is referred to as Means-End Analysis and was originally proposed for GPS (General Problem Solver) by Newell and Simon. This is the most common method used in planning systems.

3.1.2 Inductive Plans

The reversibility of an action is important when generating a plan. If actions can be undone, then plans may be forward searched in the real world. This is because reversible actions make it possible to backtrack from a failed solution. If actions are not reversible, a world model must be used to simulate the effects of actions. Planning in the real world is not desirable, but may become necessary when the world model has randomly changing conditions.

3.1.3 General Comments on Plan Completeness

A search for a plan may be an overwhelming task. Thus, a number of simplifying assumptions are often made in planning,

- there is only one operation to perform at each task,
- operators are costless,

- operator order is costless,
- the world models are limited to the problems only,
- time is not a factor,
- plans are sequential,
- operations are concurrent (not parallel).

These factors tend to simplify search space. But, even when searching the simplified problem spaces, there are a number of arbitrary decisions which have to be made. These usually arise because of the arbitrary sequence of operator execution, arbitrary choice of alternative operators, and lack of information. In the presence of arbitrary decisions the best plan can be found with the aid of cost estimates [e.g., Tate, 1977]. Lack of information is often dealt with by a least commitment strategy, where decisions are made as late as possible, or not at all [Sacerdoti, 1975, 1977 and Stefik, 1981 a, b].

3.2 Complexity

The complexity of planning is related to the number of viable plans in plan space. Alternate plans can be created by unordered operators, and equivalent operators (they cause branches in the plan search space). A number of approaches have been developed to deal with the complexity issue. In general, these methods focus on dividing or constraining the search, so that the problem becomes tractable.

3.2.1 Non-Hierarchical Planners

When goals are used to directly find operators, the system is referred to as non-hierarchical. This name definition is commonly used to describe all planners which do not fall into the other categories, namely hierarchical, skeletal, and opportunistic planning. One attribute of these planners is that since they consider all of the search space at once, the search must cover a larger number of possible solutions. The common approach is to use a linear search through space. Some descriptions of the more famous planners follow. This includes descriptions of the most popular non-hierarchical planners, NOAH and STRIPS.

3.2.1.1 STRIPS

STRIPS [described in Nilsson, 1980] is based on a goal stack. To begin, the goals are pushed on the goal stack. An Operator is found which satisfies the top goal on the stack, and it will also be pushed on the top of the stack. The preconditions for the operator are then also pushed on the stack. The basic process continues as the condition currently at the top of the stack is examined. If the condition of the top of the stack is satisfied (by the world model), it is removed. If an operator is at the top of the stack all its preconditions have been satisfied and therefore the operator is added to the plan, and used to update the world model. If the condition at the top of the stack is unsatisfied, it will be replaced with an operator which will satisfy it (then all of the operators preconditions will also be pushed on the stack). This process continues until the stack is empty, thus indicating a complete process plan.

It is possible for strips to become caught in certain situations that involve non-linear plans, and fail to produce a solution. In some cases STRIPS is able to find solutions to hard problems, but still results in inefficient solutions.

3.2.1.2 HACKER

Sussman's work on HACKER [1974] used a plan and repair approach. If an existing plan satisfied the goals presented, it was used. Otherwise, a plan was generated by subdividing the goals until they could be matched to operators. The plan could then be examined by a set of daemons which can detect known problem patterns, and fix them. Afterwards, the plan was simulated, and any existing errors were found. The problems which caused the errors were identified, and fixed with general debugging techniques. The problem patterns were then stored as new daemons for later problem pattern matching. This process continues until the plan is error free.

Sussman's examination of this method has become famous because of the failure of HACKER (a linear planner) to perform a certain three block problem. Dubbed the Sussman Anomaly, the problem involved blocks stacked in such a way that two goals had to be combined. This served as an excellent example of the limitations often encountered by the simpler linear planning schemes.

3.2.1.3 Goal Regression

Waldinger [1977] proposed a planner which would satisfy one goal at a time. The planner

would arbitrarily choose one goal as the first and find a plan for it. A second goal would be incorporated into the plan by trying to expand the second goal after the last step in the first plan. If the preconditions of the second goal plan were violated then the attempt would fail and the second goal would be reconsidered before the last step in the first plan. If the second goal could not be expanded there, then it would be moved though the first plan by another step. This regression will continue until the goal is successful, and is expanded, or it fails by reaching the start of the other plan.

Waldingers approach did not discuss many of the detailed issues, and was lacking an implementation to verify his ideas. The planner could fail in some situations because of its inability to violate the precondition constraints. But, he did discuss some of the anticipated side-effects which would occur if the preconditions could be violated (and goals expanded prematurely). He asserted that some previously unsolvable problems could be solved, but he also stated that some problems could arise. For example, it is possible to have a solution which repeatedly does an action and then undoes it. It is also possible to generate inefficient solutions having needless operations.

3.2.2 Hierarchical Planners

A more efficient approach to planning involves least commitment examination of the various hierarchies of detail. In other words, the planning should begin at an abstract high level [abstractions are discussed in Korf, 1987]. As planning progresses, the abstract goals should be expanded into more detailed sub-goals. Planning becomes a task of examining a small search space, pruning the search, then expanding the new pruned search space. This makes problems with a large search space tractable (e.g., non-linear plans).

3.2.2.1 ABSTRIPS

A simple approach to hierarchical planning was based on STRIPS [Nilsson, 1980]. This new planner was called ABSTRIPS [described in Cohen and Feigenbaum, 1982], and only involved the addition of priorities to the preconditions already used in STRIPS. The planning techniques were similar to STRIPS, except that all of the first stage of planning was done with the highest priority conditions. After a complete high level plan was pushed on the stack, it was expanded at a lower priority level. The effect was a greatly reduced search space which could allow solutions of much greater complexity.

In terms of abilities, ABSTRIPS has all the features of STRIPS, but is much more efficient at solving large problems.

3.2.2.2 NOAH, NONLIN and DEVISER

The procedural nets mentioned earlier are important to NOAH [Sacerdoti, 1975, 1977]. If the reader refers to Figure 3 they will note the square nodes. These are used to represent both goals and sub-goals. At each iteration of the search, the square nodes are expanded into sub-plans. After this, three constructive critics are applied to resolve conflicts, eliminate redundant preconditions, and deal with unbound variables. Resolving conflicts involves ordering unordered operations to avoid precondition violations. If preconditions are specified more than once, they should be recognized and the redundant ones eliminated. Unbound variables can occur because of the general form of operation definitions. This can lead to problems assigning them. The critics approach is to bind them to a formal variable or to an instance. The least commitment strategy dictates that the formal variables (or unbound variables) should only be bound to an instance when no other alternatives exist.

Tate [1977] indicates that Sacerdoti's approach has problems because it is unable to back-track and undo failed plan steps. Therefore, Tate developed NONLIN, which is like NOAH except that it keeps lists of all decisions made (and all goals achieved) so that backtracking is possible in planning (as well as execution checking). If a plan cannot be found, NONLIN backtracks and tries a new solution.

Vere [1983] states that both NOAH and NONLIN assume that tasks are concurrent, thus trivializing the time planning problem. DEVISER [Vere, 1983] was based upon NONLIN, and can consider parallel tasks with critical execution times. The planner was able to plan device operation plans for a fly by of an interplanetary satellite. The significant difference between NONLIN and DEVISER is the addition of time windows in the consideration of conflict resolution.

3.2.2.3 MOLGEN

MOLGEN [Stefik, 1981a, b] uses a less general approach to planning. Three layers of abstraction are used: the strategy, design and planning spaces. Goal relations are considered and manipulated in strategy space. Details of the plan are generated in the planning space. In the design space, constraints are considered. The interesting feature of MOLGEN is its use of

constraints. Details of each operation include constraints. As operations are specified, the constraints are formulated, and propagated to be considered against constraints from other parts of the plan. MOLGEN then chooses objects to satisfy the constraints of the operations. If a plan is formulated at the strategy level, but the constraints cannot be satisfied at the design level, then the strategy level will replan.

This planner was successful with the complex problem of planning molecular genetics experiments, and shows promise for other more sophisticated problems.

3.2.3 Skeletal Planners

The technique of skeletal planning is variant, not generative as the previous methods are. If a successful plan template is available from earlier planning sessions (or elsewhere) it is stored in a plan database. Before planning begins, the goals are compared against the skeletal plans. If plans are found to satisfy the goal pattern, they are then checked for their success with the current world model. Any remaining plans will be examined for suitability, and the best will be chosen. A successful implementation of this technique was demonstrated with MOLGEN [as described in Cohen and Feigenbaum, 1982, pp. 557-562].

3.2.4 Opportunistic Planning

In his book Gevarter [1985] describes an approach to planning which is opportunistic. This only refers to a technical report (by Hayes-Roth and Hayes-Roth), but suggests that plans may be developed in two stages. First, parts of a plan may be devised with backtracking. Secondly, the parts may be linked together, added to, and enlarged as opportunities become available. Gevarter asserts that this is a more human like approach to planning.

4.0 Conclusions

The literature makes it obvious that work needs to be done in planning. In particular, planners need to be developed which will deal with complex problems, which have process costs. It is the author's opinion that as existing planners are applied to more applications, the existing planning techniques will be refined to meet the needs of the real world problems. A few features are listed below which would be desired in a planning system.

- Be able to deal with linear and non-linear plans.
- Produce a plan in a shorter time with a slightly higher cost.
- Produce plans with low costs, having greater computation time.
- Deal with plans involving millions of steps.
- Deal with equivalent operators.
- Learn from experience.
- Be able to backtrack in the event of plan execution failures.
- Be able to perform inductive and deductive reasoning.
- Deal with probabilistic events.

The hierarchical planning approach has performed well on many of these points, suggesting that is an excellent method to consider for future planning research and application.

References

- Cohen, P.R., and Feigenbaum, E.A., 1982, *The Handbook of Artificial Intelligence*, William Kaufmann, Inc., Vol. 3, pp. 523-562.
- Gevarter, W.B., 1985, *Intelligent Machines: An Introductory Perspective of Artificial Intelligence and Robotics*, Prentice Hall, Inc., pp. 67-81
- Green, C., 1969, "Application of Theorem Proving to Problem Solving", IJCAI, 1969.
- Korf, R.E., 1987, "Planning as Search: A Quantitative Approach", Artificial Intelligence, pp. 65-88.
- Nilsson, N.J., 1980, *Principles of Artificial Intelligence*, Tioga Publishing Co., pp. 282-287.
- Rich, E., 1983, *Artificial Intelligence*, McGraw-Hill Book Company, pp. 247-277.
- Sacerdoti, E., 1975, "The Nonlinear Nature of Plans", International Joint Conference on Artificial Intelligence, pp. 206-214.
- Sacerdoti, E., 1977, *A Structure for Plans and Behavior*, Elsevier North-Holland Inc.
- Stefik, M., 1981a, "Planning with Constraints (MOLGEN: Part 1)", Artificial Intelligence, pp. 111-140.
- Stefik, M., 1981b, "Planning and Meta-Planning (MOLGEN: Part 2)", Artificial Intelligence, pp. 141-170.
- Sussman, G.J., 1974, "The Virtuous Nature of Bugs", Proceedings of the AISB Summer Conference, pp. 224-237.
- Tate, A., 1977, "Generating Project Networks", IJCAI, pp. 888-893
- Vere, S.A., 1983, "Planning in Time: Windows and Durations for Activities and Goals", IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. PAMI-5, No.3, pp. 246-267.

Waldinger, R., 1977, "Achieving Several Goals Simultaneously", Machine Intelligence.

Wilensky, R., 1983, Planning and Understanding, Addison-Wesley Publishing Co.

15.7 NEURAL NETWORK THEORY

Abstract

Robot inverse kinematics is a fundamental problem in robotic control. Past solutions for this problem have been through the use of various algebraic or algorithmic procedures, which may become involved and time consuming. In this paper, a neural network approach to the problem of robot inverse kinematics is evaluated. The neural network approach deserves examination because of the fundamental properties of computation speed, fault tolerance, the ability to learn from limited examples, and they can generalize to untrained solutions.

This paper begins with a brief introduction to neural networks, then describes the application of feedforward neural networks to the inverse kinematics of a typical three link manipulator. A comparison of two layer networks is made with and without compensation networks. Problems were encountered in the solution, these are described and discussed. The implications of the experimental results are then discussed. The approach described in this paper can be applied to other manipulator configurations.

I. Introduction

Over the past decade, the artificial intelligence community has undergone a resurgence of interest in the research and development of artificial neural networks. An artificial neural network is an attempt to simulate the manner in which the brain interprets information as determined by the current knowledge of biology, physiology, and psychology [14]. Artificial neural networks behave in much the same manner as biological neural networks, giving many of the same benefits. Artificial neural nets are fault tolerant, exhibit the ability to learn and adapt to new situations, and have the ability to generalize based on a limited set of data. This arises because of the structure which allows neural nets to process information simultaneously, as opposed to the serial nature of traditional digital computers. This parallel nature, inherent in neural networks, achieves the increase in speed by distributing the calculations among many neurons. The network structure provides a level of fault tolerance, which allows the network to withstand component failures without having the entire network fail.

This paper focuses on a popular feedforward model of neural networks. In this model a set of inputs are applied to the network, and multiplied by a set of connection weights. All of the weighted inputs to the neuron are then summed and an activation function is applied to the summed value. This activation level becomes the neuron's output and can be either an input for other neurons, or an output for the network. Learning in this network is done by adjusting the connection weights based upon training vectors (input and corresponding desired output). When a training vector is presented to a neural net, the connection weights are adjusted to minimize the difference between the desired and actual output. After a network is trained with a set of training vectors, the network should produce a good output match for the inputs.

Artificial neural networks are mainly used in two areas --- ***pattern recognition***, and ***pattern matching***. ***Pattern recognition*** is performed by classifying an unknown pattern through comparisons with previously learned pat-

terns. This ability is termed associative recall. An example of this is the use of neural networks to recognize hand-written digits [4]. In pattern recognition, when a particular pattern is noisy or distorted, the network can generalize and choose the closest match [3][4][10][13]. **Pattern matching** uses continuous input patterns to evoke continuous output patterns (i.e. one-to-one mappings). An example is the use of a neural network as a basic controller for a plant. The controller would accept the plant conditions as the inputs, and a set of control outputs would drive the current manufacturing process. This approach is discussed in more detail in [11]. This paper uses the pattern matching strategy for the inverse kinematics of a typical three link manipulator.

This paper focuses on the kinematic control of a three link manipulator working in three dimensional space, within a quarter of the robot workspace. When manipulators interact with the environment, the position of the end effector must be described. Human users are more suited to working in cartesian coordinates than the joint coordinates of robots; hence, there is a need for conversion between the two coordinate systems. The conversion from world coordinates to robot joint angles is called **inverse kinematics**. In the past, the problem of solving the inverse kinematics has been done through the application of various methods. These methods take one of two basic forms (i) **Closed form** equations or, (ii) **Iterative**. A **closed form** solution is found by decoupling the joints responsible for position and those responsible for orientation. These solution types are more desirable since explicit equations are available. **Iterative techniques**, such as, Newton-Raphson, Newton-Gauss and Linearization are available when no closed form solutions can be obtained. Many of these solutions require that the kinematics be performed off-line because of the computational demands of the algorithms. A brief survey of these various algorithms is given in [2] and [15]. Both of the forms are generally rigid and do not account for uncontrollable variables such as manufacture tolerances, calibration error, and wear. A neural network approach could adapt to changes in the robot due to wear from extended use over time. Such algebraic and algorithmic procedures cannot incorporate these situations.

Previous research in the application of neural networks to kinematic control was performed for a to a two-link planar manipulator (working in a plane), and a five degree of freedom robot working in three dimensional space. The manipulators were controlled with a high accuracy; however, the training region for the two link robot was a small square in the centre of the workspace, and the five link robot was limited to a small wedge volume within the robot workspace [5][6][7]. This paper explains research extended to investigate the performance throughout the entire workspace of the robot. Investigations of a more complete representation of inverse kinematics throughout the entire workspace is provided.

The paper begins with a general introduction to the artificial neuron, a brief description of feedforward neural networks, and a description of the backpropagation learning algorithm. The inverse kinematics problem is described, and the application of a two-layer neural network is discussed. The use of compensation networks is explored and the results of the experimental work are presented. The implications of the neural network approach to the kinematic control problem are presented at the conclusion of this paper.

II. The Basic Artificial Neuron

The basic building block of an artificial neural network is the neuron. In a popular model which will be

used in this paper, the connection weights between neurons are adjusted. The neuron receives inputs o_{pi} from neuron u_i while the network is exposed to input pattern p . Each input is multiplied by a connection weight w_{ij} , where w_{ij} is the connection between neurons u_i and u_j . The connection weights correspond to the strength of the influence of each of the preceding neurons. After the inputs have been multiplied by the connection weights for input pattern p , their values are summed, net_{pj} . Included in the summation is a bias value θ_j to offset the basic level of the input to the activation function, $f(net_{pj})$, which gives the output o_{pj} . Figure 1 shows the structure of the basic neuron.

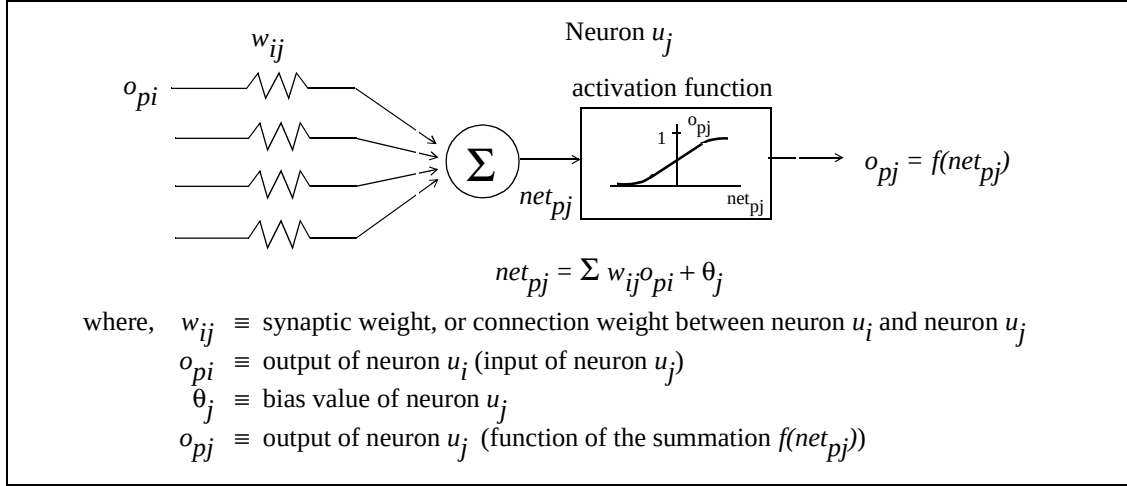


Figure 1 Basic structure of an artificial neuron.

In order to establish a bias value θ_j , the bias term can appear as an input from a separate neuron with a fixed value (a value of +1 is common). Each neuron requiring a bias value will be connected to the same bias neuron. The bias values are then self-adjusted as the other neurons learn, without the need for extra considerations.

In calculating the output of the neuron, the activation function may be in the form of a threshold function, in which the output of the neuron is +1 if a threshold level is reached and 0 otherwise. Squashing functions limit the linear output between a maximum and minimum value. These linear functions, however, do not take advantage of multi-layer networks [14]. Hyperbolic tangents and the sigmoidal functions are similar to real neural responses; however, the hyperbolic tangent is unbounded and hard to implement in hardware. In this paper, the Sigmoidal function is used because of its ability to produce continuous non-linear functions, which can be implemented in hardware in future research areas. Figure 2 shows some commonly used activation functions.

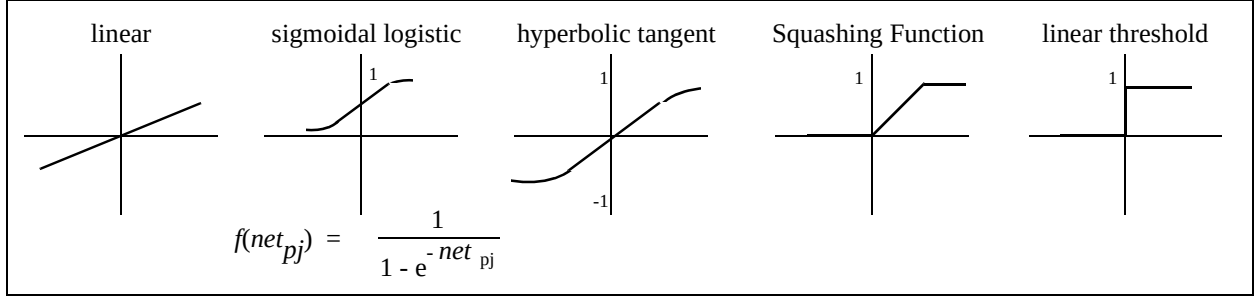


Figure 2 Common activation functions.

For more in depth information on neural networks, please refer to introductions to the subject in [3], [8], [14] and [16].

III. Neural Network Architectures in Feedforward Systems

A single neuron can only simulate 14 of the 16 basic boolean logic functions. It cannot emulate the X-OR, or the X-NOR gates [9]. These limitations require that more than a single neuron be used, and thus, the architecture becomes an important consideration. In Feedforward systems, networks propagate the inputs forward through the neural net. Each neuron has inputs that only come from neurons in preceding layers. A one-layer neural net consists of a layer of input neurons and a layer of output neurons. A multi-layer network consists of an input layer of neurons, one or more hidden layers, and an output layer.

The circular nodes in figure 3 represent basic neurons, as described in the previous section. The input neurons are shown as squares because they only act as terminal points (i.e. $o_{pi} = \text{input}$). The input layer does not process information; thus, it is not considered to be a part of the structure and is numbered layer 0. A simple one-layer network can effectively map many sets of inputs to outputs. In **pattern recognition** problems, this depends upon the linear separability of the problem domain. In **pattern matching** problems, this depends upon continuity, and topography of the function. If such a set of connection weights cannot be found using one-layered networks, then multi-layered networks must be considered.

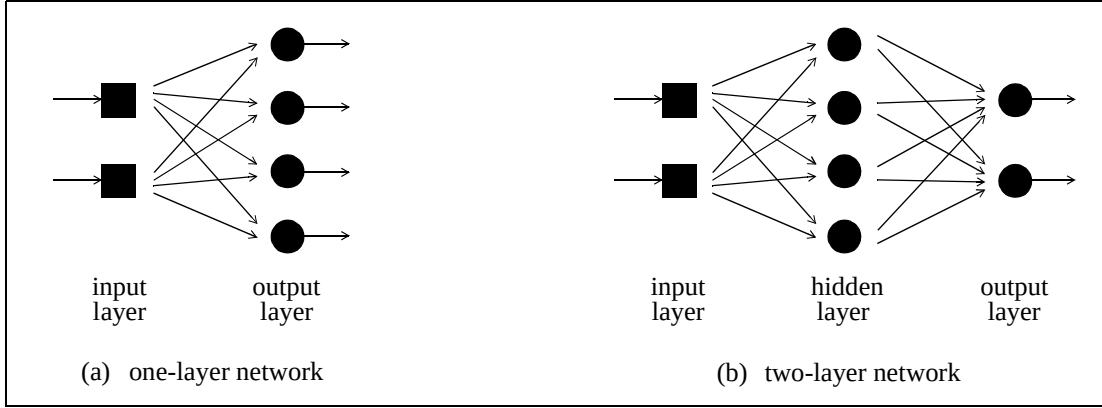


Figure 3 Simple feedforward artificial neural networks.

IV. Backpropagation of Errors

Learning methods may be either supervised, or unsupervised. Supervised learning is required for pattern matching, and backpropagation is the most popular of the supervised learning techniques. In the backpropagation Training Paradigm, the connection weights are adjusted to reduce the output error. In the initial state, the network has a random set of connection weights. When a system starts with all connection weights equal, the network begins at a sort of local optimum, and will not converge to the global solution. In order for the network to learn, a set of inputs are presented to the system and a set of outputs are calculated. A difference between the actual outputs and desired outputs is calculated and the connection weights are modified to reduce this difference [14].

After inputs have been applied and the network output solution has been calculated, the estimated error contribution, δ , by each neuron must be calculated. The calculations begin at the output layer of the network. The delta value for any output neuron is computed as,

$$\delta_{pj} = (t_{pj} - o_{pj}) f'(net_{pj}) \quad \text{—————} \quad (1)$$

where, t_{pj} is the desired output of output neuron u_j , and o_{pj} is the actual output, and $f'(net_{pj})$ is the first derivative of the activation function with respect to the total input (net_{pj}) for the given input pattern p evaluated at neuron u_j .

For the output layer, the change in connection weights can easily be calculated since the δ of the output neurons is easily determined. With the introduction of hidden layers, the desired outputs of these hidden neurons becomes more difficult to estimate. In order to estimate the delta of a hidden neuron, the error signal from the output

layer must be propagated backwards to all preceding layers. The delta, δ_{pj} , for the hidden neurons is calculated as,

$$\delta_{pj} = f'(net_{pj}) \sum_k \delta_{pk} w_{jk} \quad \text{--- (2)}$$

where, $f'(net_{pj})$ is the first derivative of the activation function with respect to net_{pj} at hidden neuron u_j , δ_{pk} is the delta value for the subsequent neuron u_k , and w_{jk} is the connection weight for the link between hidden neuron u_j and subsequent neuron u_k . This process of calculating the δ s is performed for all layers until the input layer is reached

After all the error terms have been calculated for all the neurons, the weights may be adjusted. The estimated weight change $\Delta_p w_{ij}(n+1)$ is calculated for each input connection to neuron u_j from neuron u_i by,

$$\Delta_p w_{ij}(n+1) = \underbrace{\eta}_{\text{new delta}} \delta_{pj} o_{pi} + \underbrace{\alpha}_{\text{momentum term}} \Delta w_{ij}(n) \quad \text{--- (3)}$$

where η is the learning rate, α is the smoothing term, δ_{pj} is the delta of neuron u_j , o_{pi} is the output of preceding neuron u_i , and $\Delta w_{ij}(n)$ is the weight change in the previous training interval. The estimated errors are used to estimate the weight changes with the least mean squares method.

The backpropagation method is essentially a gradient descent approach which minimizes the errors. In order to adjust the connection weights, the gradient descent approach requires that steps be taken in weight space. If the steps are too large, then the weights will overshoot their optimum values. If the steps are too small, it may take longer to converge and may become caught in local minima. The step size is then a function of the learning rate. The learning rate, which varies between 0 and 1, should be large enough to come up with a solution in a reasonable amount of time without having the solution oscillate within weight space. By introducing the *momentum* term, which varies between 0 and 1, the learning rate can be increased while preventing oscillation. The momentum term is introduced by adding to the weight change some percentage of the last weight change. Common values for η range from 0.1 to 0.9, and common values of α range from 0.2 to 0.8

The training set is comprised of points which have inputs and desired outputs, which may or may not be in a random order. In this paper, the training points were chosen at random from an ordered table. This allows a weight correction after each point is trained. This method helps avoid local optimum points in weight space, by creating a random walk type of weight optimization, and gives a much faster global convergence. This process is then repeatedly performed for all training examples until a satisfactory output error is achieved. For an in-depth derivation of the back propagation method, the reader is suggested to consult Rumelhart and McClelland [14].

V. A Typical Three Link Manipulator

In order for a robotic manipulator to perform tasks within space, a user must specify a location in three dimensional space. The robotic controller must then determine the correct joint coordinates to locate the manipulator. This area of robotics, Inverse Kinematics, has been well researched and many good solutions exist. In all cases the

* note: The subscript notation (i, j, k) means that neuron u_i precedes neuron u_j , which precedes u_k , etc.

resultant solutions are highly specific to a particular robot configuration, with exact dimensions. Such explicit solutions do not tolerate changes over time. These changes may be caused by poor tolerances in manufacture, wear over extended periods of operation, damage to the robot, and poor calibration. Computations are often complex, can be quite slow, and require a computer capable of performing complex mathematical functions.

An example of a robot which may be used with an artificial neural network control system is the three link manipulator as shown in figure 4. The forward and inverse kinematic equations, used by the authors in testing, are given in the equations (4) and (5).

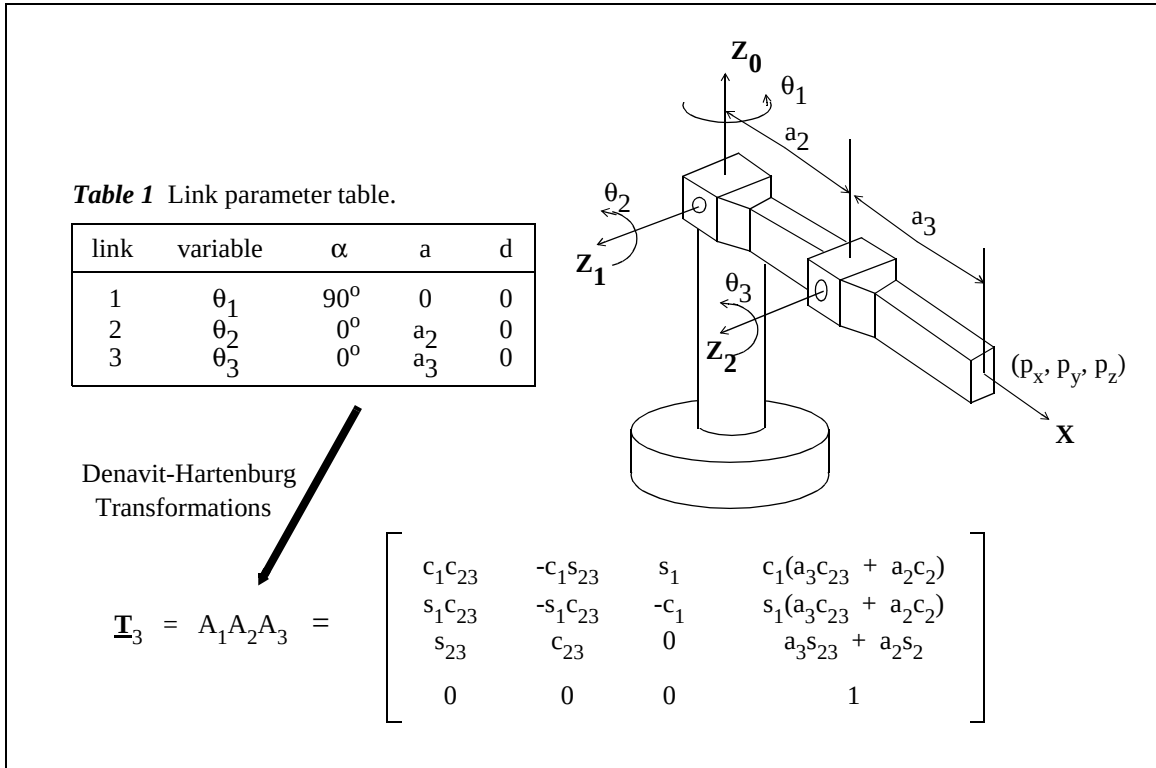


Figure 4 A typical three link manipulator.

The $\pm$ sign in θ_2 indicates the general configuration of the robot arm. The positive sign (+ve) corresponds to an elbow down solution, and the negative sign (-ve) corresponds to an elbow up solution.

$$\left. \begin{aligned} p_x &= c_1(a_3c_{23} + a_2c_2) \\ p_y &= s_1(a_3c_{23} + a_2c_2) \\ p_z &= a_3s_{23} + a_2s_2 \end{aligned} \right\} \text{forward kinematic equations} \quad (4)$$

$$\left. \begin{aligned} \theta_1 &= \tan^{-1} \left[\frac{p_y}{p_x} \right] \\ \theta_2 &= \tan^{-1} \left[\frac{\beta}{\pm \sqrt{p_z^2 + \alpha^2 - \beta^2}} \right] - \tan^{-1} \left[\frac{\alpha}{p_z} \right] \\ \theta_3 &= \tan^{-1} \left[\frac{p_z - a_2s_2}{\alpha - a_2c_2} \right] - \theta_2 \end{aligned} \right\} \text{inverse kinematic equations} \quad (5)$$

$$\begin{aligned} \text{where} \quad & s_1 = \sin \theta_1 \\ & c_1 = \cos \theta_1 \\ & c_{12} = \cos (\theta_1 + \theta_2) \\ & s_{12} = \sin (\theta_1 + \theta_2) \end{aligned} \quad \text{and} \quad \begin{aligned} \alpha &= + \sqrt{p_x^2 + p_y^2} \\ \beta &= \frac{\alpha^2 + p_z^2 + a_2^2 - a_3^2}{2a_2} \end{aligned}$$

It should be noted that the inverse kinematics is not determinate. The sources of indeterminacy for this manipulator occur:

- for Elbow up and Elbow Down Configurations, and
- when the end of the arm is directly over the origin, θ_1 is undefined.

Other examples of indeterminacy occur in PUMA style robots where left and right arm configurations would give indeterminacy. These sources of indeterminacy may all be overcome by constraining the inverse kinematics to a particular case.

VI. Training a Neural Network for Inverse Kinematics

Neural Network solutions have the benefit of having faster processing times since information is processed in parallel. The solutions may be adaptive and still be implemented in hardware by using specialized electronics. Neural systems can generalize to approximate solutions from small training sets. Neural systems are fault tolerant and robust. The network will not fail if a few neurons are damaged, and the solutions may still retain accuracy. When implementing a neural network approach, complex computers are not essential and robot controllers need not be spe-

cific to any one manipulator.

The nature of neural networks require that a set of training points be chosen which represent the nature of the inputs (x , y , z) and the corresponding outputs (θ_1 , θ_2 and θ_3). If the training points chosen tend to be clustered, then the network will be very accurate when dealing with points near the cluster. In this case, the robot should be familiar with points throughout the entire workspace; thus, points should be evenly distributed. The order in which points are presented to the network also affect the speed and quality of convergence. If the points are not presented in a random order, each training update will tend to train the network for the current section of space which the points are from.

The neural network architecture chosen for this problem is a simple network with one hidden layer. The network has three input neurons for the desired position, and three output neurons for the estimated joint angles. A bias neuron (with a value of +1) is attached to all of the neurons in the hidden layer, and to all of the output neurons. The number of neurons in the hidden layer varied from 10, 20, and 40. This network architecture is pictured in figure 5.

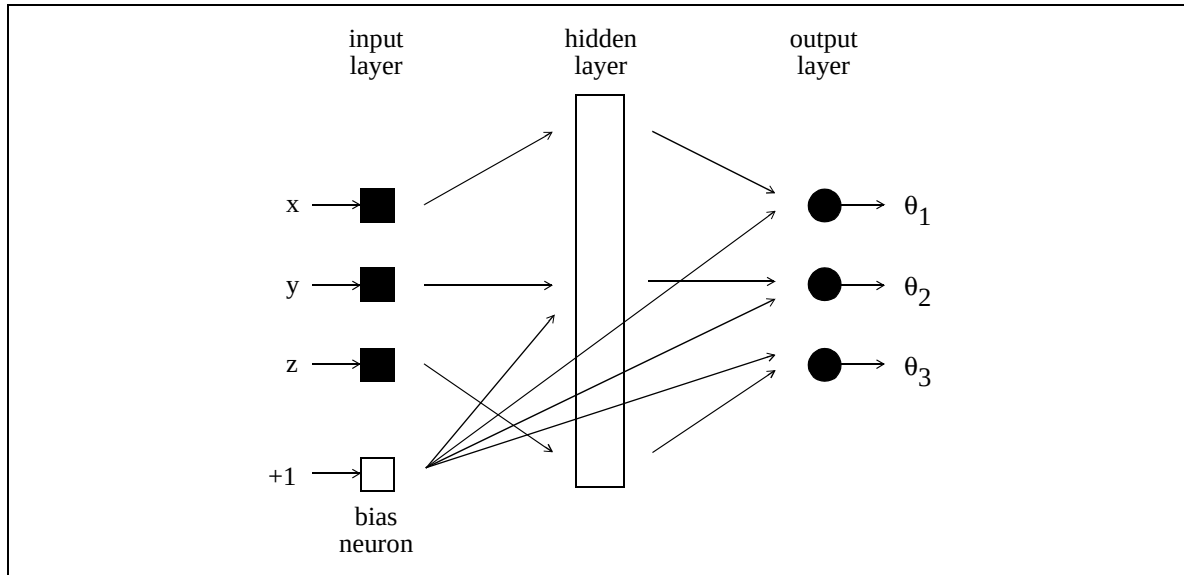


Figure 5 Neural architecture for solving the inverse kinematic problem. For simplicity, the network is shown with a small amount of arrows; however, the network is actually fully connected.

It is expected that because the network is estimating the joint angles, there will be some errors involved. The value of the errors will vary over the work space and can be corrected by implementing secondary compensation neural networks. To calculate the joint coordinates, the main neural network was trained. After training the main network, the discrepancy between the network results and the actual values were found and those errors were used to train a second network for correction. The second network is referred to as a compensation network, and had an identical architecture to the main network. If both the main and compensation networks calculate an output estimate and a correction estimate, there will be yet another smaller error. Another network can then be set up to estimate this smaller error, and the accuracy of the solution may be improved further. Accuracy does improve as more correction networks

are used. This paper tested a total of five correction networks. Please note that each of the networks was trained separately, one after another. Also note that the outputs from the networks are multiplied to scale the outputs up. The scaling factors on the outputs of each of the nets was made successively smaller.

Instead of gathering experimental information from a robot, the Inverse Kinematic equations were used. The use of simulated data provided a reliable training and comparison for the neural network. In order for the neural network to generalize to a one-to-one mapping, the training set consisted of a single solution set. To overcome the indeterminacy in the solution, a few assumptions were made:

- elbow up was assumed (for the elbow down solution, a separate network would need to be considered)
- the region above the origin was not used for training, to avoid the singularities at this location
- the robot was trained over a one quarter sphere volume of the entire workspace ($90^\circ > \theta_1 > 0^\circ$).
Training within a quarter of the workspace is valid because of the symmetry of the robot workspace

The decision to train over a quarter of the work sphere was based upon the premise that by limiting the scatter of the training points, the most accurate inverse kinematics mapping would be acquired. If the training set had covered the entire work space, then the solution may have been poorer with longer convergence times, because the network would have been forced to generalize over a volume which is four times larger.

VII. Performance of a Neural Network for Inverse Kinematics

All configurations tested were feedforward using the sigmoidal activation function. One hidden layer was used with the number of hidden neurons in the layer at 10, 20, and 40. The network converged slowly to a less than optimal solution when no bias was applied to the neurons in the hidden layer. Networks were tested with and without the bias neurons on the output neurons, with no noticeable difference, so, a bias was applied to every hidden neuron, and to every output neuron. The compensation networks had the exact same configuration as the original network.

For all of the data presented, the learning rate was set to 0.9, and the smoothing rate was set to 0.8. The number of iterations was set to be very high (the order of thousands) so that all of the network results should be at their optimum configuration. The training periods were performed on a SUN Sparcstation 1, taking times from 1 to 10 hours. All of the programs have been written in the 'C' programming language.

The errors of the solutions were measured in two ways: i) with an average error, and ii) with a deviation [1]. The actual error for a single joint, o_{error} , is the difference between the actual output, o_{actual} , and the desired output, o_{desired} . The actual error was found for each joint in each training point, and then used as the basis of the subsequent calculations. The average error was found by summing the absolute errors of all the joints for all the training points, and then dividing by the number of training points as seen in equation (6). The deviation was found as in equa-

tion (7).

$$o_{\text{error}} = o_{\text{actual}} - o_{\text{desired}}$$

$$o_{\text{average}} = \frac{\sum o_{\text{error}}}{3n} \quad (6)$$

$$S = \sqrt{\frac{\sum (o_{\text{error}})^2}{(3n-1)}} \quad (7)$$

where,

- o_{error} $\equiv$ the output error for a joint
- o_{actual} $\equiv$ the actual joint angle as determined by the network
- o_{desired} $\equiv$ the desired joint angle
- o_{average} $\equiv$ the absolute average of the errors for all the joints
- n $\equiv$ the number of training points
- $3n$ $\equiv$ the number of data points, for all three joints for the training set
- S $\equiv$ the deviation of the errors from the zero error point

The actual number of training points was 889, thus there were 2667 errors in total. The measure of average error gives a good indication of how close the network is to the ideal solution. The deviation gives an indication of how well grouped the points are. The results from the test runs are available in table 3.

An evenly distributed set of test points, that were not a part of the training set, were presented to the network to evaluate the uniformity of the solutions. Figures 6 displays the topology of the test points for the single plane (i.e. at a constant z level) in this test, z was equal to 0. Please note that the plots are only showing the x, y displacements of the points, but is not showing the z displacement (which is perpendicular to the page). The first plot shows the workspace with the test points evenly distributed. The points in the subsequent cases indicate the neural network's positioning of the robot arm. The joint coordinates as determined by the neural network where applied to the forward equations to obtain the cartesian locations.

Table 2 Performance of different neural architectures.

	Network Architecture	Number of Connections	average absolute error degrees per joint	standard deviation degrees per joint
10 hidden neurons	1 hidden layer 10 wide	73	3.02	4.22
	1 hidden layer 20 wide	143	2.22	3.39
	1 hidden layer 20 wide with 1 correction net	286	1.66	2.63
	1 hidden layer 20 wide with 2 Correction nets	429	1.27	2.06
20 hidden neurons	1 hidden layer 20 wide with 3 correction nets	572	1.19	1.92
	1 hidden layer 20 wide with 4 correction nets	715	1.04	1.61
	1 hidden layer 20 wide with 5 correction nets	858	1.01	1.55
	1 hidden layers 40 wide	283	1.72	2.59
40 hidden neurons	1 hidden layer 40 wide with 1 correction net	566	1.37	2.21
	1 hidden layer 40 wide with 2 Correction nets	849	1.20	1.90
	1 hidden layer 40 wide with 3 correction nets	1132	1.10	1.77
	1 hidden layer 40 wide with 4 correction nets	1415	1.10	1.76
	1 hidden layer 40 wide with 5 correction nets	1698	1.09	1.76

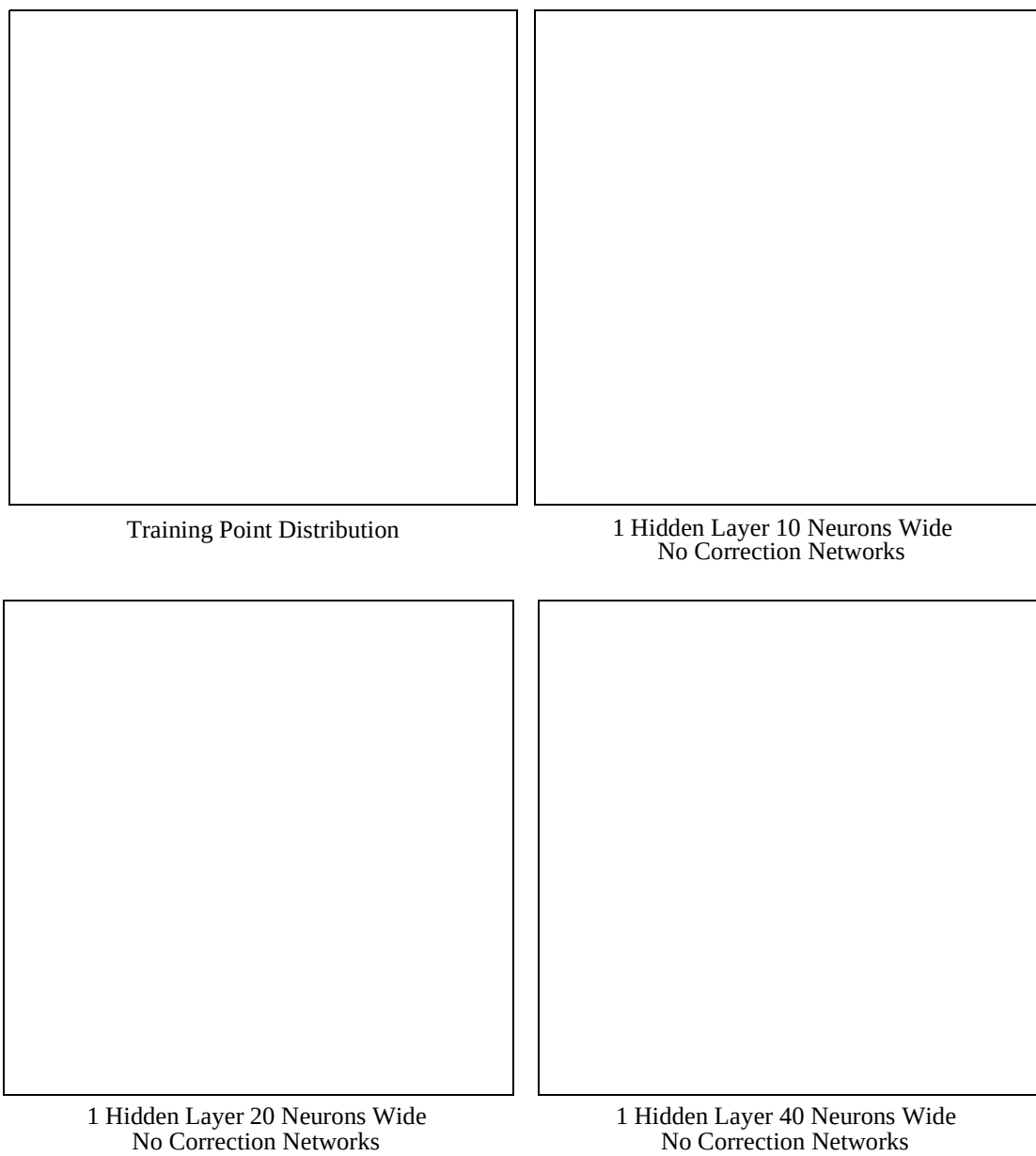


Figure 6 Plot of Inverse Kinematic Distribution

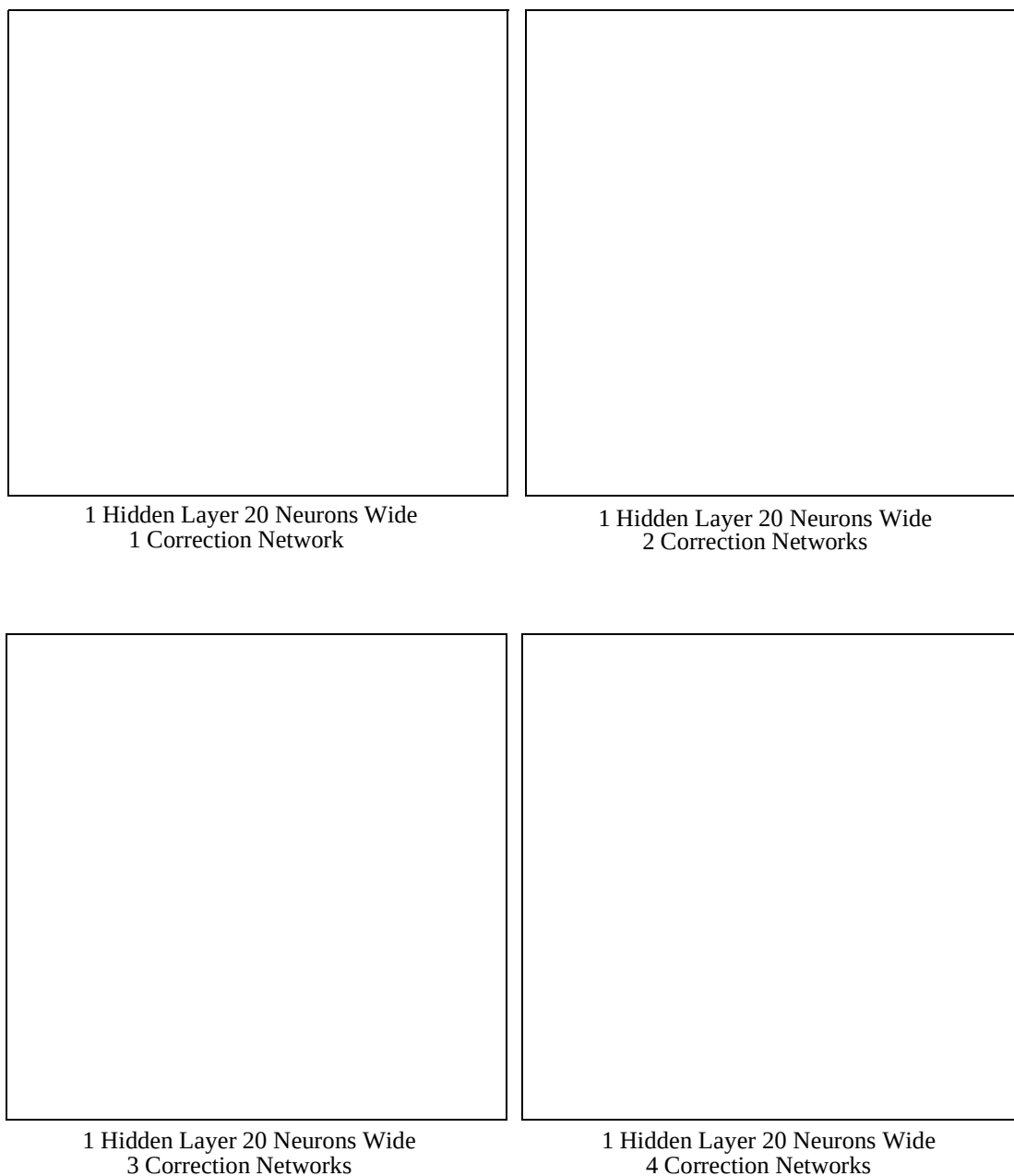


Figure 6(cont'd) Plot of Inverse Kinematic Distribution

In the diagrams, the lines indicate the boundaries of the training region. The points should lie inside and be evenly distributed, as in the first example of figure 6. The discontinuities are obvious on this diagram. The point of the cone is one discontinuity. The circular arc represents the second discontinuity point. The two lines which are along the sides represent the boundaries of the training region only.

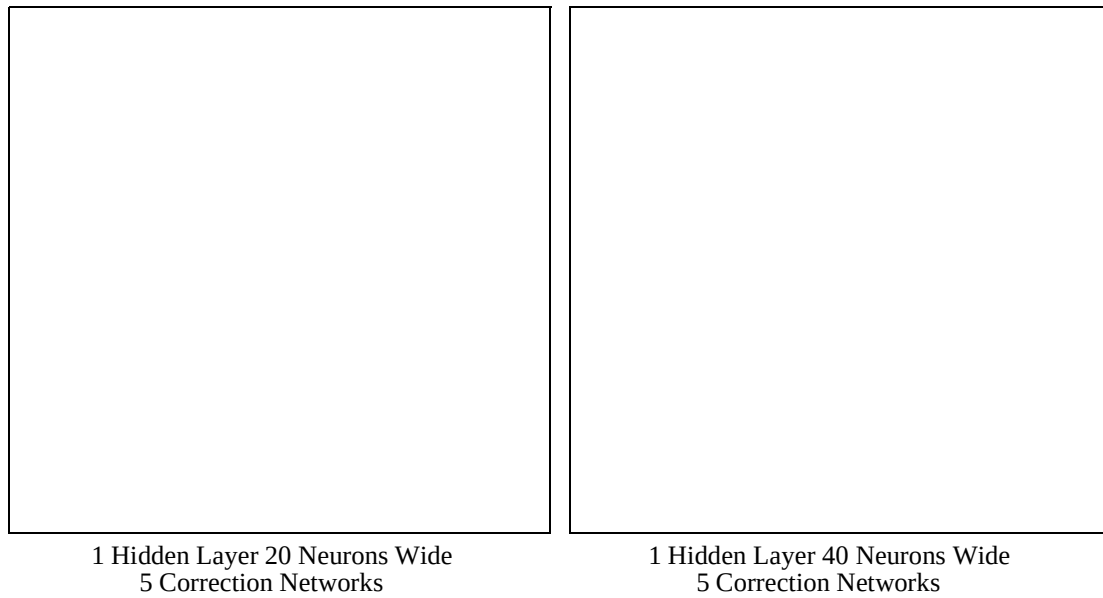


Figure 6(cont'd) Plot of Inverse Kinematic Distribution

VIII. Discussion

The results in table 3 present a good picture of the relationship between the number of neurons and the generalization capability. When the number of neurons is doubled, the average joint error is reduced by approximately 25%. The standard deviation of the error also decreases by over 20% in the same conditions. The use of correction networks appears to have a beneficial effect upon the network errors. The effect of correction nets was greatest on the networks with fewer neurons. The first three correction networks also tended to have the most effect, but subsequent networks were not very influential.

The points which were well within the training region were placed within a tolerance which was less than one percent of the workspace radius. As the manipulator approaches the discontinuities, the errors become close to 10 percent. The errors increase as the edge of the training region is approached. This suggests that some technique would have to be developed to train the network to deal with, or avoid, the singularities. To correct these problems it may be possible to train the network for the complete volume of the workspace, but not include the regions where singularities occur.

If the singularities are not dealt with, and the network is used as it has been trained here, some problems would be encountered.

- Switching from an elbow up to elbow down solution would not be possible, because the network does not approach the singularity very well.

- If the quarter of a sphere is mirrored, then a discontinuity is created at intersections of the edges of the training regions. This results because the neural network solution does not perform very well near the edges of the training sphere.
- Placing the end of the manipulator over the origin of the robot will result in a singularity configuration.

Figure 7 gives a graphic representation of the errors associated when the robot approaches singularity points.

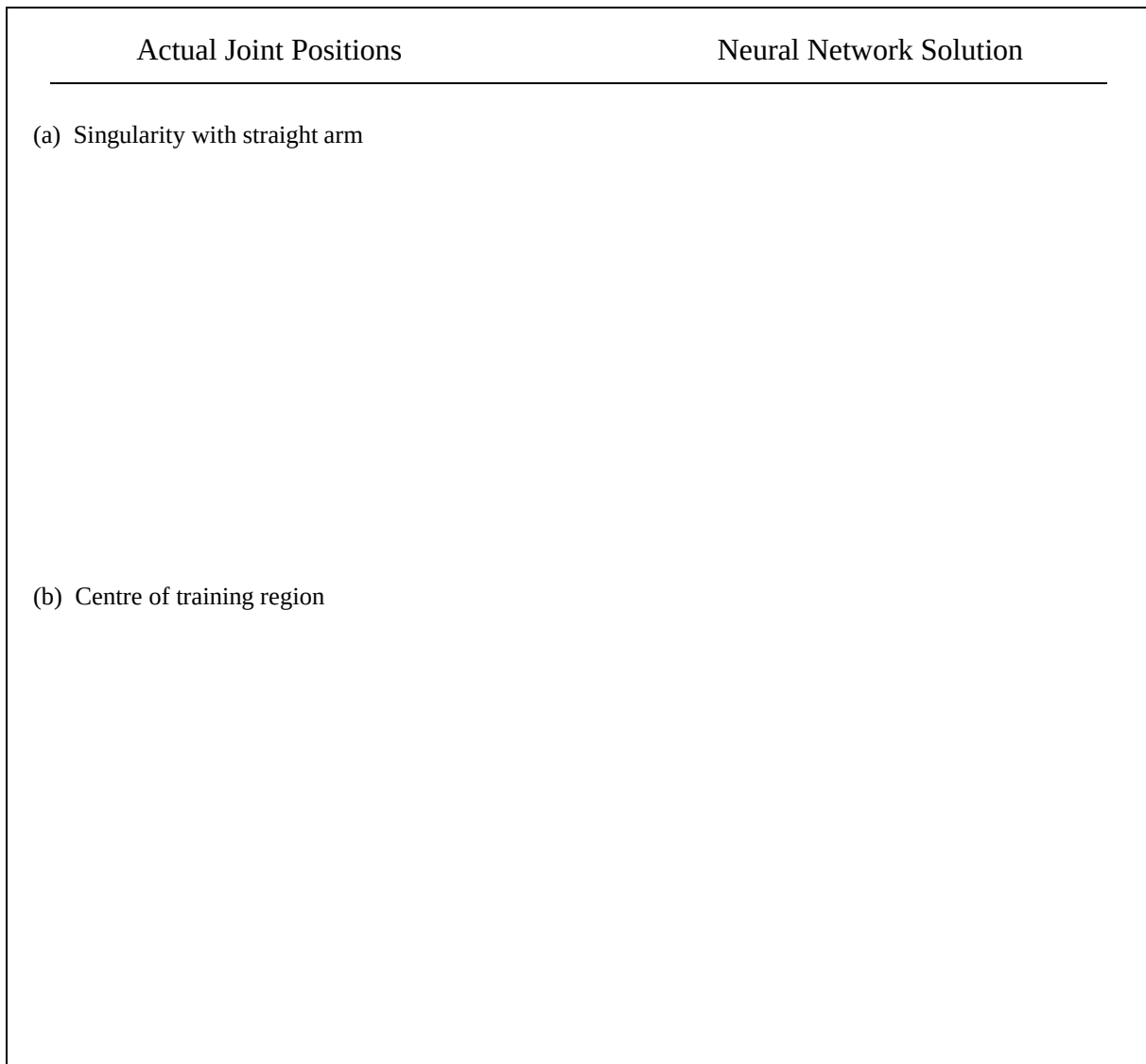


Figure 7 A Comparison of Actual and Neural Network Estimated Position

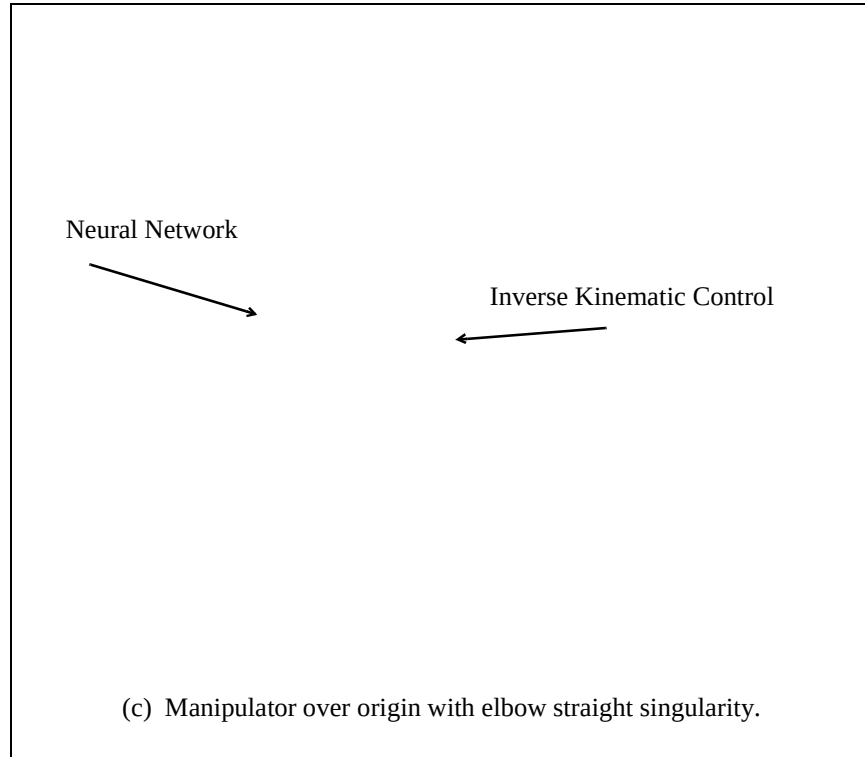


Figure 7(cont'd) A Comparison of Actual and Neural Network Estimated Position

IX. Conclusions

As the results indicate, this method of robotic control will not be very accurate in certain areas of the workspace; however, the method does work very well when the robot is restricted to small areas in the centre of the workspace. These results agree with the work done by Josin [5][6][7]. In his papers, for the planer manipulator, he uses a 2D square in the centre of the workspace, to avoid areas of discontinuity. Josin was able to obtain high accuracy by focusing on a subset of the inverse kinematics solution; however, if he were to approach the singularities within his workspace, then his accuracy would decrease. His research was also extended to a 3D wedge volume for the training region with a five link manipulator where the results were highly accurate in this volume. Again, if singular configurations were approached, the generalization capability would surely decrease.

The benefits of the networks are quite distinct. If implemented, the largest inverse kinematics network could be run on a standard Neural Network Accelerator (available for IBM PCs, SUNs, and other computers). Some of the accelerators available run at 10 million connections per second (A few run above 20 million connections per second). This would mean that the network which is 40 neurons wide with 5 correction nets could be processed every 0.17ms or 5900 times per second. For the software which was written for this research, it was possible to process the most complex network in this paper about 150 times per second. The network could have been trained for any manip-

ulator, which indicates that it is a good general technique.

If neural networks are to be implemented as robotic controllers, there are a few points to consider. If a Neural Network controller is trained for a specific manipulator, the network could be saved. When a new controller is attached to a manipulator a pretrained network could be loaded. Sample points may be used for occasional updates to the controller, and thus achieve adaptive control. The ability for such systems to adapt to changes within the environment is an important step towards artificially intelligent machines. In conclusion it must be said that this research must be re-examined when more is understood about Neural Networks.

X. Future Work

Even though the neural networks are not suitable for precise robotic control, they show potential. Neural Networks seem to provide errors similar to what a human would get if the feedback senses were not used for error correction. This suggests that such neural network solutions could be used in a robotic controller that models human flexibility, instead of attempting to obtain superhuman capabilities. This could be done by incorporating a network to estimate inverse kinematics into a feedback system. This system would first estimate the position, determine the error (possibly from a vision system) and then have a second network correct this error. Neural controllers may be applied to autonomous robots such as two-legged "walking" machines where high accuracy is unimportant. If this were to be attempted, then there is indeed a solution to the problem of robot coordination.

Some methods may be researched to help improve the use of neural networks in the inverse kinematics problem. The network could make use of the orientation of the end effector to determine whether an elbow up, elbow down solution or left arm, right arm solution is the best configuration. This network does not provide any estimate of the error. To help detect errors, a neural network model of the forward kinematics could be used to take the inverse kinematic output, and regenerate the x, y, z positions. From this, the error could be used as a confidence measure for the solution.

Neural network solutions are not suited for high precision robotic applications; however, this approach may be useful for generalized robot control such as dynamics, resolved rate control, path planning and task planning. This method could also be used in conjunction with existing methods. If an exact solution is required, and an iterative Inverse Kinematics method must be used, then the neural networks may be used to provide a good initial guess. Another implication of artificial neural networks is the ability to solve previously unknown or intractable problems. Algorithms that require complex computing power for robotic control could be simplified by implementing an artificial neural network controller.

References

- [1] Devore, J. L., **Probability & Statistics for Engineering and the Sciences**. Brooks/Cole

Monterey, California, USA, 1982.

- [2] Fu, K. S., R. C. Gonzalez, and C. S. G. Lee, **Robotics, Control, Sensing, Vision, and Intelligence**. McGraw-Hill Book Company, New York, NY, USA, 1987.
- [3] Guyon, I., **Neural Network Systems**. Proc. of the INME Symposium, Lausanne, 1989, *(to appear)*.
- [4] Guyon, I., I. Poujard, L. Personnaz, G. Dreyfus, J. Denker, Y. LeCun, **Comparing Different Neural Architectures for Classifying Handwritten Digits**. Proc. of Intern. Jt. Conf. on Neural Networks (IJCNN), Washington, DC, USA, 1989, *(to appear)*.
- [5] Josin, G., **A Neural Representation of an Unknown Inverse Kinematic Transformation**. Proceedings nEuro 88 - First European Conference on Neural Networks, Paris, France, June 1988.
- [6] Josin, G., **Neural-Space Generalization of a Topological Transformation**. Biological Cybernetics, vol.59, pp 283 - 290, 1988.
- [7] Josin, G., D. Charney, and D. White, **Robotic Control Using Neural Networks**. Proc. of the First European Conference on Neural Networks - nEuro88, 1988.
- [8] Lippman, R. P., **An Introduction to Computing with Neural Nets**. IEEE ASSP Magazine, vol. 4 no. 2, pp 4 - 22, April 1987.
- [9] Minsky, M. L., and Seymour A. Papert, **Perceptrons**. MIT Press, 1969.
- [10] _____, **Neural Networks**. Byte, pp 216 - 245, August 1989.
- [11] Psaltis, D., A. Sideris, A. Yamamura, **Neural Controllers**. IEEE First Intern. Conf. on Neural Networks, vol. 4, pp 551 - 558, 1987.
- [12] Ranky, P. G., and C. Y. Ho, **Robot Modelling: Control and Applications with Software**. IFS (Publications) Ltd., UK, 1985.
- [13] Reber, W. L., and John Lyman, **An Artificial Neural System Design For the Rotation and Scale Invariant Pattern Recognition**. IEEE First Intern. Conf. on Neural Networks, vol. 4, pp 277 - 283, 1987.
- [14] Rumelhart, D. E., and James L. McClelland, **Parallel Distributed Processing**. vol. 1, MIT Press, 1986.
- [15] Tourassis, V. D., and Marcelo H. Ang, Jr., **A Modular Architecture for Inverse Robot Kinematics**. IEEE Trans. on Robotics and Automation, vol. 5, no. 5, pp 555 - 568, October 1989.
- [16] Wasserman, P. D., **Neural Computing - Theory and Practice**. Van Nostrand Reinhold, New York, NY, USA, 1988.

16. ARTIFICIAL INTELLIGENCE (AI)

16.1 OVERVIEW

- What is Intelligence ? - Nobody knows, but there are lots of ideas.
- So if we don't know what intelligence is, how can we write programs that are intelligent ? - AI researchers guess at how parts of the brain work, then try to reproduce the effect in a program.
- A Myth is that artificial programs use intelligence to guide themselves, but they are not as stupid as traditional programs.
- Most AI methods take ONE aspect of human thinking, and then model it, such as,
 - Logical Reasoning
 - Recognition
 - Association
- There are many older, and newer AI topics of interest,
 - Expert Systems
 - Planning Systems
 - Search Techniques
 - Fuzzy Logic
 - Neural Networks
 - Genetic Algorithms
 - Symbol Manipulation
 - etc.
- Each of the AI techniques is distinct in which applications they are useful for, and how they are applied.
- In manufacturing there are many problems which are difficult to solve using computers, such as trying to get a computer to 'listen for a particular PING when doing inspection'. But, AI can help solve some of these problems.
- AI is not magic, you must still understand the problem before an AI system will help you solve it.
- The List below says a few words about some popular AI topics, and how they relate to applications,

Expert Systems - These systems use exact rules with true or false conditions, and results.
"If the engine stops, the car is out of gas"

Fuzzy Logic - This method still uses rules which do not need true or false conditions. "When I want the engine to go slower, I give it less gas" where slower, and less are somewhat arbitrary values.

Neural Networks - This method is equivalent to learning by example. "I don't know why, I just know to go faster, I push the accelerator that much".

- For the three methods above, how well the problem can be described determines which is to be used. In effect, if the problem is very logical, use expert systems, if you can still make rules, but nothing is true or false use fuzzy logic. Finally, if you can't define rules for solving a problem, but can do it by intuition, use neural networks.

16.2 EXPERT SYSTEMS

- To implement an expert system, a knowledge engineer will talk to 'experts' about how they solve a problem. The Knowledge engineer will then try to develop a set of rules for solving the problem. After the rules are done, they are entered into Expert System Software (\$0 - \$20,000). Expert system software will then ask questions (or check sensors, or look at data files) to compare rules to conditions, and see the results.
- The rules are in the form shown on the other page
- There are two ways to search rules,
 - Forward Chaining - Consider what you know now, then check rules to see if all conditions are satisfied. The Results of the rule give you a new conclusion. The rules are all checked again using the Results from the previous rule. This is often used for choosing an action.
 - Backward Chaining - This method will backtrack from a set of consequents to find which conditions caused them. This method is often used for determining how something was done.
-

16.3 FUZZY LOGIC

- Rules are created which make sense to humans, for example when driving a car some acceleration rules may be,
 - if LOUD_NOISE and FAST_SPEED then SLOW_SPEED
 - if QUIET_NOISE and FAST_SPEED then SAME_SPEED
- Each of the rule conditions, and results, can be represented with a 1 dimension matrix,

LOUD_NOISE = { 0.0 0.1 0.5 1.0 }

QUIET_NOISE = { 1.0 0.5 0.1 0.0 }

FAST_SPEED = { 0.0 0.2 0.7 1.0 }

SLOW_SPEED = { 1.0 0.5 0.2 0.0 }

SAME_SPEED = { 1.0 1.0 1.0 1.0 }

The sets are considered normalized from minimum to maximum. For example is the NOISE sets were from 20dB to 100dB, then a noise level of 80dB would result in a value of 0.5, or a 50% membership. This way you can say if the noise is 'absolutely loud', giving a value of 1.0, or 'a bit loud' giving a value of .5.

- The matrices are combined using the rules to get a result matrix
- Because the conditions (like LOUD_NOISE) are defined with a sort of weight, or membership rules are easier to make up.
- Fuzzy logic controllers have been very successful at solving control problems.

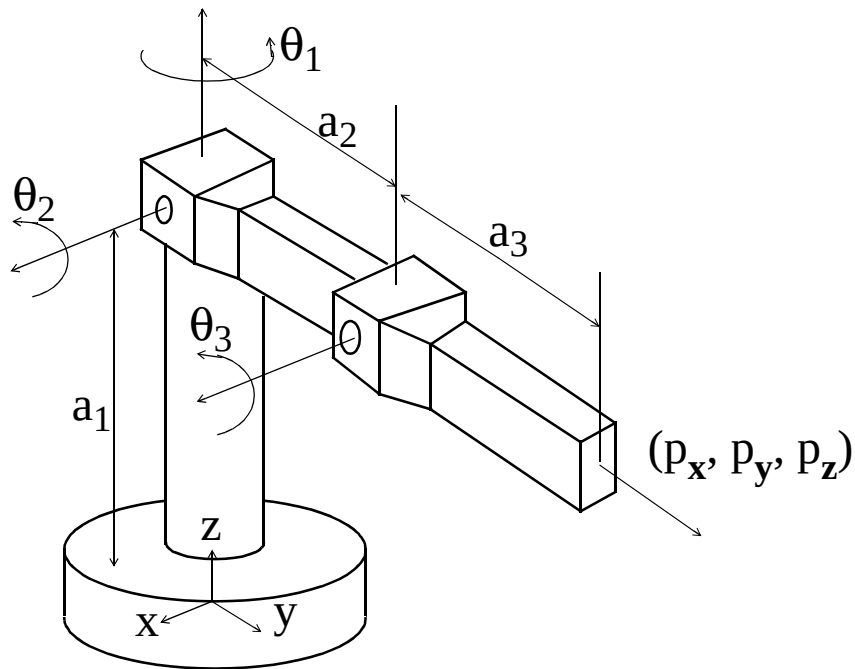
16.4 NEURAL NETWORKS

16.4.1 Neural Network Calculation of Inverse Kinematics

- Objectives: To give insight into the neural network solution of the inverse kinematics problem.

16.4.1.1 - Inverse Kinematics

- Forward ematics for a 3-link manipulator



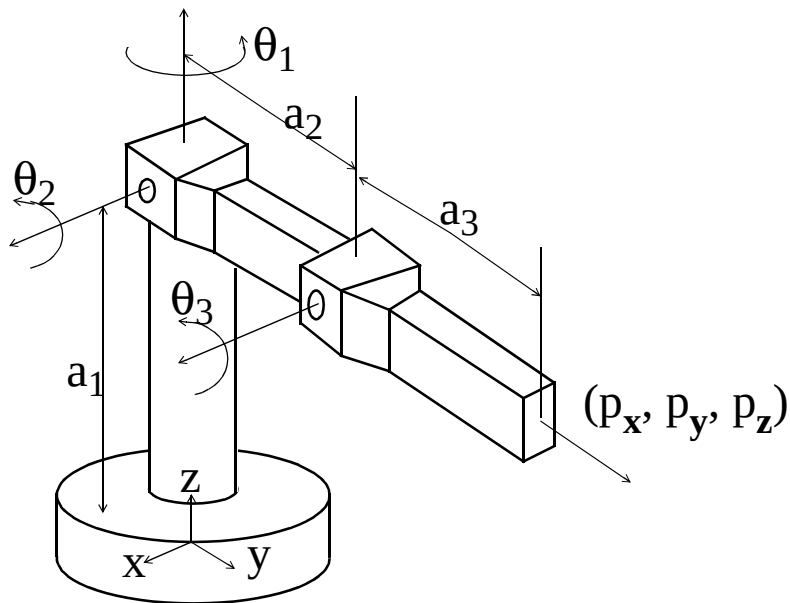
Allows the end of the robot to be located in space relative to the base of the robot. The calculations are done to convert joint coordinates (joint angles) to cartesian coordinates.

$$p_x = \cos\theta_1 (a_3 \cos(\theta_2 + \theta_3) + a_2 \cos\theta_2)$$

$$p_y = \sin\theta_1 (a_3 \cos(\theta_2 + \theta_3) + a_2 \cos\theta_2)$$

$$p_z = a_3 \sin(\theta_2 + \theta_3) + a_2 \sin\theta_2 + a_1$$

- Inverse Kinematics for a three link manipulator



Allows the joint coordinates of the robot to be determined by a point in space. In most cases there four possible ways to reach most points in space.

$$\alpha = \sqrt{p_x^2 + p_y^2} \quad \beta = \frac{\alpha^2 + a_2^2 - a_3^2 + p_z^2}{2a_2}$$

$$\theta_1 = \text{atan}(p_y / p_x)$$

$$\theta_2 = \text{atan} \left[\frac{\beta}{\pm \sqrt{p_x^2 + \alpha^2 - \beta^2}} \right] - \text{atan} \left[\frac{\alpha}{p_z} \right]$$

$$\theta_3 = \text{atan} \left[\frac{p_z - a_2 \sin \theta_2}{\alpha - a_2 \cos \theta_2} \right] - \theta_2$$

- Inverse Kinematics techniques
 - explicit, with exact solution (as for the 3 link manipulator)
 - iterative, for use when an infinite number of solutions exist.

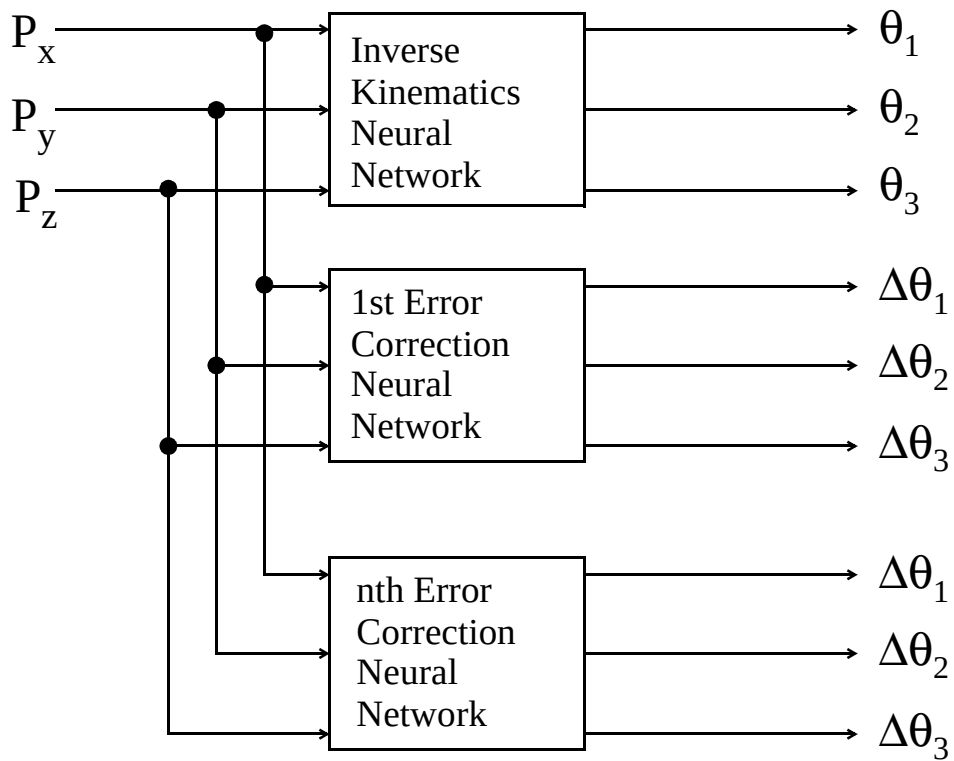
- Problems that occur when doing inverse kinematics with these methods are,
 - both methods require a computer capable of mathematical calculations
 - the methods do not adapt to compensate for damage, calibration errors, wear, etc.
 - solutions may be slow, especially for iterative solutions
 - solutions are valid only for a specific robot
- advantages of using these methods are,
 - both solutions will yield exact answers
 - properties of both of these methods are well known

16.4.1.2 - Feed Forward Neural Networks

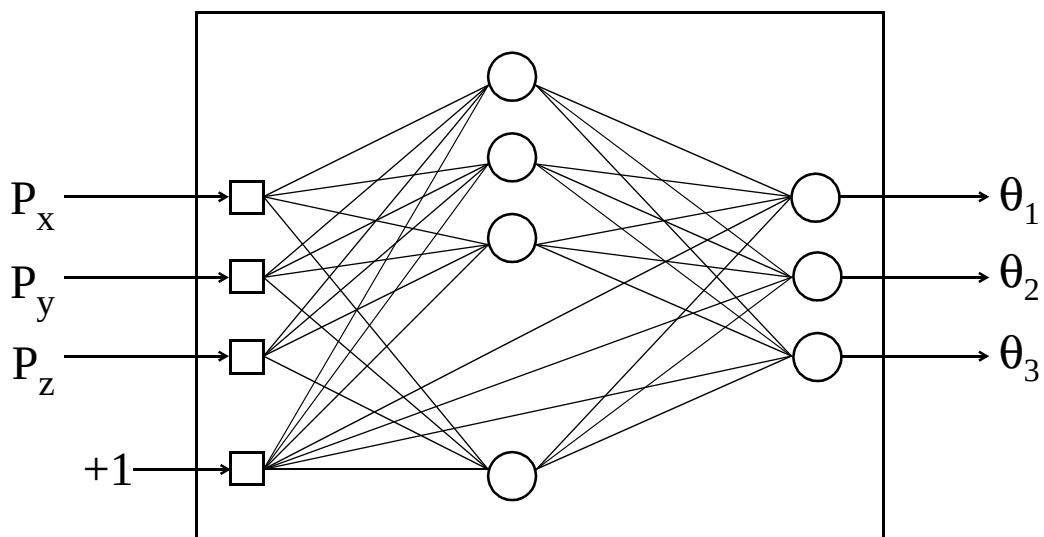
- A feed forward neural network was used with a sigmoidal activation function
- the back propagation learning technique was used.
- disadvantage of these network is
 - unpredictable errors occur in the solution
 - discontinuous problem spaces cause problems for the networks
 - training may be very slow
 - these networks are not well understood
 - neuro computers are not commonly available
- advantages
 - the architecture is fault tolerant
 - faster calculations
 - can be adjusted for changes in the robot configuration
 - the controllers are not specific to a single robot

16.4.1.3 - The Neural Network Setup

- The figure below shows how the neural network was configured to solve the problem



- The first neural network estimates the proper inverse kinematics. This will contain a small error, therefore a second net is used to estimate the errors. Additional correction networks can be added after this.



- The networks were generally connected with

- 10, 20 or 40 neurons in the hidden layer
- a bias input was connected to each neuron
- the layers were all fully connected
- there were runs with one, and two hidden layers

16.4.1.4 - The Training Set

- The problem is reduced using either left or right arm configurations, the solution is also constrained to elbow up or elbow down.
- Discontinuities were avoided by not training the neural network in the region above the origin. The elbow straight configuration is also a minor singularity problem.
- Training points were evenly distributed throughout the robot workspace
- Only a quarter of the robot workspace was used because of the robot symmetry.
- The general protocol for training was,
 - apply the desired position to the input, and train for the desired joint angles.
 - When accuracy was high enough, the first correction net was trained by comparing the actual errors, and the desired values. Additional correction networks were also trained in some cases.
 - the error was measured by using an RMS measure of the differences

$$\begin{aligned}
 o_{\text{error}} &= o_{\text{actual}} - o_{\text{desired}} \\
 o_{\text{average}} &= \frac{\sum o_{\text{error}}}{3n} \\
 S &= \sqrt{\frac{\sum (o_{\text{error}})^2}{3n}}
 \end{aligned}$$

- o_{actual} - Actual angle output of Neural Network
- o_{desired} - Desired angle output of Neural Network
- o_{error} - Error of angle output of neural Network
- o_{average} - Average angle error for all joints
- S - R.M.S. error for all joints
- n - Number of training points

- A list of results are provided below,

	Network Architecture	Number of Connections	average absolute error degrees per joint	standard deviation degrees per joint
10 hidden neurons	1 hidden layer 10 wide	73	3.02	4.22
	1 hidden layer 20 wide	143	2.22	3.39
	1 hidden layer 20 wide with 1 correction net	286	1.66	2.63
20 hidden neurons	1 hidden layer 20 wide with 2 Correction nets	429	1.27	2.06
	1 hidden layer 20 wide with 3 correction nets	572	1.19	1.92
	1 hidden layer 20 wide with 4 correction nets	715	1.04	1.61
	1 hidden layer 20 wide with 5 correction nets	858	1.01	1.55
40 hidden neurons	1 hidden layers 40 wide	283	1.72	2.59
	1 hidden layer 40 wide with 1 correction net	566	1.37	2.21
	1 hidden layer 40 wide with 2 Correction nets	849	1.20	1.90
	1 hidden layer 40 wide with 3 correction nets	1132	1.10	1.77
	1 hidden layer 40 wide with 4 correction nets	1415	1.10	1.76
	1 hidden layer 40 wide with 5 correction nets	1698	1.09	1.76

- The results in the table were obtained for a variety of network configurations
- A visual picture of the network configurations is shown below, and on subsequent pages. These are based on a set of test points that lie in a plane of the workspace.

***** Add figure of network point locations, and test conditions

- As seen in the experimental results, there are distortions that occur near the origin, and the edges of the workspace, as would be expected with the singularities found there.
- The errors also increased near the training boundaries

***** Add in more of the results figures

16.4.1.5 - Results

- The mathematical singularities caused by cartesian coordinates, and the +/- 180 degrees singularity could be eliminated by selecting another set of coordinates for space and the arm.
- The best results were about 1 degree RMS.
-